
复杂系统架构多视图统一建模关键技术研究

马君达

2023 年 5 月

中图分类号：N945.23

UDC 分类号：621

复杂系统架构多视图统一建模关键技术研究

作者姓名	马君达
学院名称	机械与车辆学院
指导教师	阎艳教授 王国新教授
答辩委员会主席	卢继平教授
申请学位	工学博士
学科专业	机械工程
学位授予单位	北京理工大学
论文答辩日期	2023 年 5 月

Research on Key Technologies of Multi-View Unified Modeling for Complex System Architecture

Candidate Name:	<u>Junda Ma</u>
School or Department:	<u>Mechanical Engineering</u>
Faculty Mentor:	<u>Prof. Yan Yan</u> <u>Prof. Guoxin Wang</u>
Chair, Thesis Committee:	<u>Prof. JiPing Lu</u>
Degree Applied:	<u>Doctor of Philosophy</u>
Major:	<u>Mechanical Engineering</u>
Degree by:	<u>Beijing Institute of Technology</u>
The Date of Defence:	<u>May, 2023</u>

复杂系统架构多视图统一建模关键技术研究

北京理工大学

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下进行的研究工作获得的研究成果。尽我所知，文中除特别标注和致谢的地方外，学位论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京理工大学或其它教育机构的学位或证书所使用过的材料。与我一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此申明。

签 名：马君达 日期：2023.05.29

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名：马君达 日期：2023.05.29
导师签名：何艳 日期：2023.05.29

摘要

随着装备系统复杂度的提高，基于模型的系统工程（MBSE, Model-Based Systems Engineering）能够有效支持复杂系统全生命周期的需求、设计、分析、验证和确认活动，提高装备设计效率。然而面向装备的复杂系统架构建模往往涉及多阶段多领域，建模语言规范的图类型有限，导致缺乏对复杂系统全生命周期的描述；不同建模语言语义的不一致，导致模型的歧义和质量下降；异构的建模工具构建的模型底层数据不一致，导致模型集成和交互困难，从而降低了装备整体数字化研发效率。因此，针对以上问题，论文主要研究内容如下：

（1）复杂系统架构多视图统一建模框架。为研究复杂系统多视图统一建模，构建面向复杂装备的基于霍尔模型的多视图统一建模框架，从系统维、逻辑维、学科领域维三个维度实现装备的生命周期过程，提出了多视图统一建模的分层抽象模型构建、基于图论有向图的抽象模型形式化表征、多视图统一建模语言表达三个层次递进的技术路线。

（2）基于 GOPPRR-E（Graph, Object, Point, Property, Relationship, Role, Extension）元素的分层抽象模型研究。针对装备复杂系统生命周期多阶段架构模型缺少通用抽象元素描述，不足以支持复杂系统统一表达的问题，进行统一建模分层抽象模型研究。首先将结合系统工程实践扩展的 GOPPRR-E 元元模型，作为本文语言的底层模型；然后结合 MOF（Meta-Object Facility）框架定义分层抽象模型，表达出各层的建模元素以及使用这些元素的规则，该分层抽象模型是定义复杂系统架构模型形式化表征的基础。

（3）基于图论有向图的抽象模型统一形式化表征研究。针对装备复杂系统架构模型的表达缺少完整的形式化语义，导致模型的理论研究缺失的问题，进行模型统一形式化表征研究。首先介绍图论的基本概念，其次基于元模型层类型图和模型层实例图开展元模型及模型形式化表征，再次形式化表征基于扩展约束的模型满足性约束，最后形式化表征项目和语言。以此实现数学逻辑对模型本质的精准定义，支持多视图统一建模语言的语法构建。

（4）多视图统一建模语言表达研究。针对支持复杂系统生命周期多阶段架构建模的多种建模语言底层数据异构导致模型表达不一致、集成困难等问题，开展多视

图统一建模语言表达研究。首先建立语言构建方法，然后指定元模型及模型的抽象语法，明确各元素的类及组合关系；最后定义具体语法，指定抽象语法中的元模型及模型等的显式表达，最终实现多视图统一建模语言规范的构建。

（5）复杂系统架构多视图集成表达验证平台设计与验证。通过搭建基于模型系统工程的多视图集成表达验证平台进行多视图统一建模关键技术的验证。首先在平台中分析 BPMN、DMN、OPM、UPDM 和 SysML 等 5 种建模语言，表明多视图统一建模语言满足各语言抽象能力，并通过可视化元模型规范技术、连接器类型规范技术等 4 种技术的对比分析，表明多视图统一建模语言的适用能力；然后验证多视图统一建模的性能指标：集成性、完整性和一致性。通过构建 11 种元模型进行智能电动汽车架构设计，验证多视图统一建模语言对复杂系统多阶段架构建模集成性的支持；通过对建模过程中 28 个模型、459 个对象等的形式化表达，验证对复杂系统生命周期架构建模完整性和表达一致性的支持。

关键词：多视图统一建模语言；GOPPRR-E；MBSE；元元模型；元模型；图论

Abstract

With the increase of system complexity, MBSE (Model-Based Systems Engineering) can effectively support the requirements, design, analysis, verification and validation of complex systems throughout the life cycle, and improve the efficiency of system design and development. However, complex system architecture modeling often involves multi-stage and multi-domain, heterogeneous architecture model leads to inconsistent model data, model integration and interaction is difficult, reducing the overall efficiency of digital research and development. At present, the representation of complex system architecture lacks a complete and self-consistent abstract model, which limits the stage or domain in which the model can be described. at the same time, the lack of accurate, consistent and clear formal representation between elements leads to the logic and rigor of the complex system architecture model. In view of the above problems, this thesis considers the integration of product life cycle process architecture model. The main research contents are as follows:

(1) *Multi-view unified modeling framework for complex system architecture*. In order to study the multi-view unified modeling of complex system, a multi-view unified modeling framework based on Hall model for complex equipment is constructed, and the life cycle process of equipment is realized from three dimensions: system dimension, logical dimension and discipline domain dimension. it also puts forward three progressive technical routes: hierarchical abstract model construction supporting multi-view unified modeling, formal representation of abstract model based on graph theory, and multi-view unified modeling language expression.

(2) *Research on hierarchical abstract model based on the concept of GOPPRR-E (Graph, Object, Point, Property, Relationship, Role, Extension)*. In order to solve the problem that the multi-stage architecture model of complex system life cycle lacks the description of general abstract elements and is not enough to support the unified expression of complex system, this studies the hierarchical abstract model of unified modeling. Firstly, the GOPPRR-E meta-metamodel, which is extended according to the practice of system engineering, is used as the underlying model of this language, and then the hierarchical abstract model is defined with MOF (Meta-Object Facility) framework to express the modeling elements of each layer and the rules for using these elements. The hierarchical abstract model is the basis for defining the formal representation of complex system architecture models.

(3) *Research on unified formal representation of abstract model based on graph theory.*

In view of the lack of complete formal semantics to express the complex system architecture model, which leads to the lack of theoretical research on the model, the unified formal representation of the model is studied. Firstly, based on the basic concepts of graph theory, the mathematical logic relations such as metamodel layer type diagram and model layer example diagram are expressed, and the model satisfaction constraint set is constructed. Then carry out the formal representation of meta-metamodel, metamodel and model respectively, and finally describe the satisfaction constraint set of the model by extending the constraint formal representation model. This research realizes the precise definition of the essence of the model by mathematical logic and supports the abstract syntax construction of multi-view unified modeling language.

(4) *Research on the expression of multi-view unified modeling language.* In order to solve the problems of inconsistent model expression and difficult integration caused by heterogeneous underlying data of multiple modeling languages that support multi-stage architecture modeling of complex system life cycle, the research on multi-view unified modeling language expression is carried out. First of all, specify the abstract syntax of meta-metamodel, metamodel and model, clarify the composition of the elements of the language and the relationship between inclusion and inheritance, and then define the specific syntax and specify the explicit expression of metamodel and model in abstract grammar. finally, the construction of multi-view unified modeling language is realized.

(5) *Design and verification of multi-view integrated representation verification platform for complex system architecture.* The key technologies of multi-view unified modeling are verified by building a multi-view integrated expression verification platform based on MBSE. Firstly, five modeling languages, BPMN, DMN, OPM, UPDM and SysML, are analyzed in the platform, which shows that the multi-view unified modeling language meets the abstract capabilities of each language. Through the comparative analysis of four technologies, such as visual metamodel specification technology and connector type specification technology, it shows the applicability of multi-view unified modeling language. Then verify the performance indicators of multi-view unified modeling: integration, integrity and consistency. Through the construction of 11 metamodels for intelligent electric vehicle architecture design, the support of multi-view unified modeling language for the integration of complex system multi-stage architecture modeling is verified; through the formal expression of 28 models and 459 objects in the modeling process, the support for the integrity and expression

consistency of complex system life cycle architecture model is verified.

Key Words: Multi-view unified modeling language; GOPRR-E; MBSE; meta-metamodel; metamodel; Graph Theory.

目录

摘要	I
ABSTRACT	III
目录	VI
图目录	X
表目录	XIII
缩略词	XV
第 1 章 绪论	1
1.1 研究背景	1
1.2 研究现状	2
1.2.1 基于模型的装备复杂系统设计过程抽象	2
1.2.2 规范建模技术及建模环境	8
1.2.3 建模语言形式化表征方法	16
1.3 存在的问题	19
1.4 课题来源与研究意义	19
1.4.1 课题来源	19
1.4.2 研究意义	20
1.5 论文内容及组织结构	21
第 2 章 复杂系统架构多视图统一建模框架	24
2.1 引言	24
2.2 基于模型的复杂装备系统设计流程	24
2.2.1 复杂装备系统研制过程	24
2.2.2 基于 MBSE 的复杂装备系统架构设计流程	25
2.3 复杂系统架构多视图统一建模框架	27
2.3.1 基于霍尔模型的复杂系统生命周期过程	27
2.3.2 复杂系统多视图建模需求	29
2.3.3 基于霍尔模型的多视图统一建模框架	32
2.4 多视图统一建模关键技术	33
2.5 多视图统一建模关键技术实现过程	35

2.6 本章小结	36
第 3 章 基于 GOPPRR-E 元素的系统架构分层抽象模型研究	37
3.1 引言	37
3.2 基于 MOF 建模机制的分层抽象模型	37
3.3 元元模型分析	39
3.4 基于 GOPPRR-E 及 MOF 的分层抽象模型	42
3.4.1 基于 MOF 框架的统一建模框架	42
3.4.2 M3 层 GOPPRR-E 元元模型	43
3.4.3 M2 层 GOPPRR-E 元模型	45
3.4.4 M1 层 GOPPRR-E 模型	49
3.5 本章小结	50
第 4 章 基于图论有向图的抽象模型统一形式化表征	51
4.1 引言	51
4.2 图论有向图基本概念介绍	51
4.3 元模型形式化表征	52
4.3.1 基于图论有向图的元模型层类型图	52
4.3.2 元模型形式化表征	53
4.3.3 元模型形式化表征示例	57
4.4 模型形式化表征	59
4.4.1 模型层实例图	59
4.4.2 模型形式化表征	62
4.4.3 模型表征示例	63
4.5 扩展约束形式化表征	64
4.5.1 抽象性约束	65
4.5.2 双向性约束	65
4.5.3 多重性约束	66
4.5.4 索引约束	67
4.5.5 唯一性约束	68
4.5.6 分解及剖视约束	69
4.5.7 绑定约束	69

4.6 项目与语言	70
4.7 本章小结	71
第 5 章 多视图统一建模语言表达研究	72
5.1 引言	72
5.2 多视图统一建模语言构建方法	72
5.3 多视图统一建模语言抽象语法	73
5.3.1 类	74
5.3.1 组合关系	76
5.4 多视图统一建模语言具体语法	79
5.4.1 属性元模型及模型具体语法	79
5.4.2 角色元模型及模型具体语法	81
5.4.3 关系元模型及模型具体语法	83
5.4.4 点元模型及模型具体语法	85
5.4.5 对象元模型及模型具体语法	87
5.4.6 图元模型具体语法	89
5.4.7 模型具体语法	91
5.4.8 项目及语言类具体语法	93
5.5 本章小结	95
第 6 章 复杂系统架构多视图集成表达验证平台设计与验证	96
6.1 引言	96
6.2 多视图集成表达验证平台设计	96
6.2.1 平台应用场景	96
6.2.2 平台框架设计	97
6.2.3 平台实现的技术基础	99
6.2.4 多视图统一建模语言解释器及代码生成器	100
6.2.5 平台开发与运行环境	103
6.2.6 平台主界面	104
6.3 分层抽象模型表达能力验证	105
6.3.1 定量分析	105
6.3.2 定性分析	108

6.4 基于案例的形式化表征及多视图统一建模语言能力验证	112
6.4.1 智能电动汽车架构设计方法	112
6.4.2 架构设计问题及需解决的指标	113
6.4.3 智能电动汽车架构设计方案	113
6.4.4 基于统一建模语言智能电动汽车架构设计	121
6.4.5 案例分析	131
6.5 本章小结	135
第 7 章 结论与展望	136
7.1 总结	136
7.2 主要创新点	137
7.3 展望	138
参考文献	140
附录 A 属性单位表	152
附录 B PLUG-IN 具体语法	155
附录 C 多视图统一建模语言 G4 语法文件	157
附录 D 多视图统一建模语言描述 SYSML 活动图	162
攻读学位期间发表论文与研究成果清单	170
攻读学位期间参与的科研项目	171
攻读学位期间参与的竞赛	172
致谢	173

图目录

图 1.1 GOPPRR 元元模型.....	7
图 1.2 UML 与 SysML 关系.....	10
图 1.3 论文总体结构图	23
图 2.1 基于 MBSE 的装备研制过程	25
图 2.2 复杂装备架构设计流程	26
图 2.3 系统工程霍尔模型	28
图 2.4 基于模型的复杂系统霍尔模型	28
图 2.5 基于霍尔模型的多视图统一建模框架	32
图 2.6 建模语言术语	35
图 2.7 关键技术实现过程	36
图 3.1 MOF 4 级层次结构.....	37
图 3.2 基于 MOF 框架的元建模流程.....	42
图 3.3 统一建模语言框架	43
图 3.4 元元模型元素	44
图 3.5 元元模型与可视化图符	44
图 3.6 GOPPRR-E 元模型	45
图 3.7 GOPPRR-E 图元模型	46
图 3.8 元数据类型	48
图 3.9 三层实例化示例	49
图 4.1 元模型层类型图	53
图 4.2 图元模型的表征组成	54
图 4.3 基于图论有向图的元模型层类型图示例	58
图 4.4 模型层实例图的示例图形表示	61
图 4.5 模型层实例图中属性的不同表示	62
图 4.6 基于 GOPPRR-E 的图及可视化模型映射实例	64
图 4.7 具有抽象节点类型的类型和实例图	65
图 4.8 双向边的类型图建模	66
图 4.9 多重性约束示例	67

图 4.10 索引约束示例	68
图 4.11 唯一性约束示例	68
图 5.1 抽象语法到具体语法的过程	73
图 5.2 多视图统一建模语言总体抽象语法	74
图 5.3 多视图统一建模语言模型的抽象语法	75
图 5.4 多视图统一建模语言对象模型的抽象语法	77
图 5.5 多视图统一建模语言关系模型的抽象语法	77
图 5.6 多视图统一建模语言点模型的抽象语法	78
图 5.7 多视图统一建模语言角色模型的抽象语法	79
图 5.8 形式化表征与具体语法的映射	81
图 6.1 平台技术框架	98
图 6.2 语言解释器和代码生成器	100
图 6.3 多视图统一建模语言解释器	101
图 6.4 语法分析树	102
图 6.5 多视图统一建模语言代码生成过程	103
图 6.6 建模环境主界面	105
图 6.7 系统性思维方法	112
图 6.8 智能电动汽车	113
图 6.9 智能交通系统	114
图 6.10 智能电动汽车特定域建模语言元模型集合	117
图 6.11 多视图统一建模语言资源	121
图 6.12 智能电动汽车架构模型构建流程	122
图 6.13 使命任务	123
图 6.14 运行概念模型	123
图 6.15 利益相关者运行分析	124
图 6.16 驾驶运行分析	125
图 6.17 特征和功能关系	126
图 6.18 电动汽车安全和保障相关特征	126
图 6.19 智能电动车功能分析模型	127
图 6.20 具有 ABS 功能的刹车时序图	128

图 6.21 智能电动汽车物理架构	129
图 6.22 故障树建模	130
图 6.23 元模型元素	133

表目录

表 1.1 元元模型对比分析	8
表 1.2 建模语言分析	14
表 2.1 基于模型系统工程的时间维和逻辑维过程矩阵	29
表 2.2 建模流程中的多视图应用示例	32
表 3.1 元元模型的比较表	40
表 3.2 关系的比较	41
表 3.3 属性的比较	41
表 3.4 继承的比较	42
表 3.5 GOPPRR-E 元元模型的扩展元素	49
表 4.1 元模型形式化表征模型	57
表 4.2 节点	61
表 4.3 边统计	61
表 4.4 扩展约束的形式化表征	70
表 5.1 基础数据类型	80
表 5.2 角色方向	82
表 5.3 角色形状固定语法	82
表 5.4 关系形状固定语法	84
表 5.5 对象形状具体语法	88
表 5.6 图模式具体语法	90
表 5.7 主要类之间的二元关系	92
表 6.1 平台开发环境	104
表 6.2 平台运行环境	104
表 6.3 建模语言信息	106
表 6.4 活动图抽象元模型	107
表 6.5 建模语言中抽象元模型数量	107
表 6.6 可视化元模型规范技术的使用	109
表 6.7 连接器类型分析	109
表 6.8 符号规范技术	110

表 6.9 建模语言符号可视化	111
表 6.10 系统顶层概念图说明	118
表 6.11 元模型集成	131
表 6.12 智能电动汽车架构设计总体介绍	132
表 6.13 特定域形式化元模型统计	133
表 6.14 形式化架构模型统计	134

缩略词

MBSE	基于模型的系统工程 (Model-Based Systems Engineering)
GOPPRR-E	一种基于图、对象、点、属性、关系、角色、扩展的元元模型 (Graph, Object, Point, Property, Relationship, Role, Extension)
MOF	元对象机制 (Meta-Object Facility)
ARIS	集成信息系统体系结构 (The Architecture of Integrated Information Systems)
GME	通用建模环境 (The Generic Modeling Environment)
UML	统一建模语言 (Unified Modeling Language)
SysML	系统建模语言 (Systems Modeling Language)
UPDM	基于 DoDAF 和 MODAF 的统一配置文件 (Unified Profile for DoDAF and MoDAF)
BPMN	业务流程建模语言 (Business Process Model and Notation)
DSML	特定领域建模语言 (Domain-Specific Modeling Language)
EAST-ADL	基于开放汽车系统架构标准的描述汽车机电系统架构的特定域建模语言 (Architecture Description Language (ADL) for automotive embedded systems)
ARCADIA	架构分析与设计集成方法 (ARChitecture Analysis and Design Integrated Approach)
OPM	对象过程方法 (Object-Process Methodology)
FTA	故障树分析 (Fault Tree Analysis)
MMTG	元模型层类型图 (Meta Model Type Graph)
MIG	模型层实例图 (Model Instance Graph)
MOFLP	使命-运行-特征与功能-逻辑-物理架构建模流程 (Mission-Operation-Feature/Function-Logic-Physical Architecture)

第 1 章 绪论

1.1 研究背景

随着装备性能指标要求的提高,参与的学科、系统数量不断增多,装备系统复杂度不断提升,导致研制过程产生大量的涉及各子系统、部件和组件的要求、规格、描述及接口等的复杂关联关系。传统过程中,设计人员通过文档的形式收集装备需求、规范、技术描述,跟踪设计决策,显示系统从最初的概念到实现的演变过程。然而,这种基于文档的方法具有局限性和缺点,如可追溯性差、歧义、误解、缺乏清晰度等。随着计算机技术的快速发展,以面向对象的图形化、可视化的模型描述装备系统的方式越来越成熟,基于模型的研制工作在装备复杂系统开发过程中所占的比重越来越大,国际系统工程学会(INCOSE)在 2007 年发布的《系统工程 2020 年愿景》中提出了一种解决装备全生命周期活动建模的系统性方法——基于模型的系统工程^[1],被 NASA、美国国防部、欧空局等政府组织和相关机构积极地应用于航天器、飞机等复杂装备系统的研制过程中^{[(2).[3)]}。目前 MBSE 已是工业界和学术界比较关注的热点技术,其通过可视化的建模方法可以有效支持复杂系统全生命周期需求、设计、分析、验证和确认等活动,MBSE 具有如下特点:(1)使用系统工程方法来执行基于模型的设计和研发;(2)支持全生命周期中的需求分析,体系及系统、子系统等架构设计,运维服务等模型的管理;(3)通过相关技术实现需求追溯、系统架构驱动、行为及性能表达、仿真及测试等^[4]。

架构是一组概念的图形表示,这些概念是架构的一部分,包括它们的规范、元素和组件^[5]。在复杂系统设计过程中,系统设计人员需要在适当的抽象层次上以图形描述各抽象单元(如:用例、状态、需求等),以及他们之间的相互关系(如继承、泛化、对象流、控制流等),这些抽象的单元以及它们之间的关系系统称为系统的架构^[6]。在复杂系统架构设计过程中,系统生命周期不同阶段包含了任务、需求、活动、状态、时序、模块、流程等多种架构视图的建模。然而不同阶段架构的抽象能力决定了视图的表现形式,影响利益相关者对系统不同视图的认知识别效率,如当车辆系统与车载系统由不同结构的视图描述时,不同的描述环境和语义影响到车辆数据信息的准确和快速传递。因此在复杂系统架构设计过程中,多视图能否统一建模将严重影响设计的效率。

在上述背景下，面向复杂系统生命周期多阶段架构建模问题，论文考虑多阶段模型进行统一抽象，以提高多视图统一描述能力及模型一致性为目标，开展复杂系统架构多视图统一建模关键技术研究，支持装备复杂系统生命周期过程架构模型统一构建，为增强系统设计人员的集成设计能力和设计效率提供了一种有效手段。

1.2 研究现状

基于上述研究背景，针对面向装备的复杂系统多视图统一建模，本文将开展基于模型的复杂系统设计过程抽象、建模规范及建模环境、模型形式化表征方法等支持多视图统一建模方面的现状调研。

1.2.1 基于模型的装备复杂系统设计过程抽象

1.2.1.1 基于模型的装备复杂系统设计过程抽象需求

近些年，随着装备研制任务数量的大幅增加，系统复杂性不断增长，增加了不可预测交互的可能性，如在飞机研发过程中，顶层需求的数量已经增长到数千个，但只有 18% 在研发期间保持稳定^[8]，因此导致多次重新设计活动，增加了成本，延长了进度，并降低了性能，其中，很大一部分成本是在设计的非常早期阶段承担的^[9]。

此外，当特定领域需要的设计服务是由分布在不同组织或机构之间的不同种类的专家团队提供时，可能受到不同组织单位或合作伙伴所拥有的工作份额的影响，增加了额外的边界和接口，装备的协作开发需要在整个生命周期中进行高效和有效的信息交换。特别是在人和人共同工作的工具之间，这样的交换需要是正确、一致和有意义的^[10]。如在车辆设计过程中，由于对约束条件和需求的理解不同，跨职能团队之间很容易产生冲突，导致混淆和缺乏理解。因此，需要在多个层面上保证信息的一致交换和解释。首先在利益相关方方面，工程师、客户、运营商和供应商的认知必须保持一致。在工具方面，开发中部署的异构“设计生态系统”之间的互操作性需要由所有相关方保证。

数字技术可以在减少信息交换的分离程度、促进许多参与者之间的知识共享和简化集成工作方面发挥关键作用。在支持装备数字化设计工程转型的使能器中，MBSE 在过去十几年中越来越受欢迎。MBSE 是建模方法的形式化应用，以使建模方法支持系统需求、设计、分析、验证和确认等活动，这些活动从概念性设计阶段开始，持续贯穿到设计开发以及之后的生命周期阶段^[11]。自从 2007 年初 INCOSE 面向

工业界学术界发起 MBSE 倡议开始，此定义逐渐被业界普遍接受为 MBSE 标准定义。MBSE 是向以模型为中心的一系列方法转变这一长期趋势的一部分，这些方法被应用于机械、电子和软件等工程领域，以期望取代原来系统工程师们所擅长的以文档为中心的方法，并通过完全融入系统工程过程来影响未来系统工程的实践。

抽象模型是将现实的对象简化成与其相似的替代物，在基于模型的复杂系统生命周期建模过程中，从装备的任务分析到系统过程架构设计过程，体现出复杂系统设计不同阶段系统模型的抽象能力的价值^[12]。

基于模型的任务分析以装备在某一系统场景中的使用方式为背景、以不同用户需求为输入、明确系统需求为目标，按照任务运行流程开展任务级模型的构建，明确任务要素与系统参数的耦合关系。在此任务分析过程中，存在信息、数据、指南规则等信息与数据要素的抽象；测量组、类型等约束概念、环境概念等高层概念的要素抽象；块、值类型、执行者等定义模块要素内容的抽象。

基于模型的系统过程架构设计实现复杂系统需求、功能、逻辑、物理架构设计过程^[13]，复杂系统研制流程与 MBSE 研制流程的深度融合，模型数据在各专业建模工具间的流转与集成，并综合权衡指标进行整体系统评估^[14]。该过程包含有需求要素抽象，活动、动作、决断等功能要素抽象，状态、序列、始点、终点等逻辑要素抽象，以及模块、执行者、约束等物理要素抽象。

以上过程中各抽象层级、抽象规则不一致将导致系统架构设计过程面临协同研发受阻、信息理解分歧、数据传递困难等问题^[15]，实现同构一致的抽象要素表述，能够支持系统需求、功能、逻辑、物理架构及评估过程的统一建模，统一模型的抽象要素是提高装备系统设计效率的有利手段。

1.2.1.2 基于元元模型的抽象模型

元元模型定义装备复杂系统架构的抽象模型要素，通过元元模型展示不同的抽象要素组成方式，本文调研 ARIS 元元模型、ECore 元元模型、GME 元元模型、MS DSL Tools 元元模型、GOPRR 元元模型并进行了分析：

(1) ARIS 元元模型

ARIS (The Architecture of Integrated Information Systems, 集成信息系统体系结构) 是德国萨尔大学的 A.W.Scheer 教授提出的一种基于过程的元元模型结构^[16]。ARIS 分为五个不同的视图：组织、数据、功能、流程和产品/服务，三个不同的抽象层：需求、设计和实现。在描述企业模型过程中，把与企业有关的对象，按照对象间存在

的各类关系关联起来，并采用直观简洁的图形化方式，将企业流程的每一活动过程分层次的、由粗到细的描述出来。ARIS 建模的核心思想就是多视图、多层次、多关联，全生命周期地描述企业信息系统的各方面，并提供各建模信息之间的关联关系，为多种描述方法之间的自动转换和联合分析提供基础。ARIS 有一套标准的软件工具，包括 ARIS Toolset、ARIS Easy Design、ARIS for mySAP、ARIS Simulation、ARIS ABC、ARIS Publisher 等。这些产品可相互集成，可描述企业的结构、流程、应用系统，对流程进行设计、分析及优化^[17]。

ARIS 方法提供了一组建模概念如模型类型、对象定义类型、关联定义类型等。它是以基于特定 ARIS 的元元模型的通用方式定义的。基于该模型，ARIS 方法也可以通过特定的插件扩展到特定的建模上下文，但 ARIS 元元模型更加关注信息的转变，舍弃与信息过程无关的因素，主要应用于集成信息系统领域，缺乏对于装备的复杂要素的抽象。

（2）ECore 元元模型

ECore 是 EMF（Eclipse Modeling Framework，Eclipse 为基础的代码生成工具和建模框架）中使用的元元模型，该模型是 EMOF（MOF 的一个子集）的一个实现，它是一个自解释的模型^[18]。Ecore 允许通过利用其构造表达其他模型，同时也是它自己的元模型（即：Ecore 是根据其本身来定义的）。将 Ecore 用作基础元模型允许建模者充分利用整个 EMF 生态系统和工具，尽可能多地将应用程序级模型映射回 Ecore。Ecore 是目前基于 Eclipse 平台的模型驱动架构建模工具开发和使用的核心和基础。

Ecore 核心元素主要有 E 类（EClass）、E 属性（EAttribute）、E 引用（EReference）以及 E 数据类型（EDatatype）四个^[19]：

1) E 类：用于对包括“E 类”自身在内的“类”元素进行建模。类由唯一的名称属性进行标识。类可具有多个属性以及多个与其他类之间的关系。同时一个类可以有多个父类。

2) E 属性：用于对类元素所拥有的属性进行建模。

3) E 引用：用于对类元素与类元素之间的关系进行建模。通过 String 类型的名称属性和“类”类型的 e 引用类型进行唯一标识。如果两个类之间具有互逆关系，则这两个类之间有两条方向相反，分别表征上述两种关系的关系。

4) E 数据类型：用于对属性的数据类型进行建模。数据类型可以是原始类型如 float 或 int 或者诸如 Java.util.Date 之类的对象类型。

Ecore 是 MOF 的 Java 实现，根据 Java 语言特性对 MOF 适当简化并增加 Java 语言特性，可实现 UML 建模语言，用于软件模型的定义，但该元元模型通过 E 引用的方式构建类和类之间的关系，缺乏对复杂关系的约束建立，E 类又对装备的接口表达及接口关联方式缺乏描述，因此对于复杂装备生命周期的要素抽象能力有限。

（3）GME 元元模型

GME (The Generic Modeling Environment, 通用建模环境) 是 Vanderbilt 大学软件集成系统研究所开发的可配置的工具包，主要是一个用于电气工程领域特定域建模的工具^[20]。它支持创建特定域建模和程序合成环境，图形化的特定域模型可以表示应用程序及其环境，包括硬件资源及其关系。配置是通过元模型指定应用程序域的建模范式来完成的。

GME 元元模型中，模型、原子、集、关联和引用都是第一类对象^[21]。原子是基本的对象，不能包含其他部分，每种原子都与一个图标相关联，并且可以拥有一组预定义的属性，属性值是用户可以更改的。模型是框架中的复合对象，可以有部件和内部结构，模型中的一个部件始终具有一个角色。建模范式决定了模型中允许哪些部件扮演哪些角色，但是建模者决定给定模型包含的特定实例和部件数量（显式的约束可以限制设计空间）。关联只能表示同一模型包含的对象之间的关系，关联可以是有向的，也可以是无向的，可以有属性。引用类似于面向对象编程语言中的指针，引用不是一个“真正的”对象，它只是指向一个对象，在 GME 中，引用必须作为模型中的一部分出现，在包含引用和被引用对象的模型之间建立一种关系。

GME 的第一类对象对于系统要素的抽象表达完整，但被设计用于电气工程领域，其中的原子和集在其他领域较难理解，且内含的控制规则不具有通用性，对于面向装备的架构建模、流程建模，缺乏表达能力，目前尚未用于复杂装备生命周期的完整验证。

（4）MS DSL Tools 元元模型

MS DSL Tools (The Microsoft Domain-Specific Language Tools) 允许在 Visual Studio 中定义语言和生成器。MS DSL Tools 是 Visual Studio SDK 的一部分，并扩展了 Microsoft Visual Studio，以支持特定域建模^[22]。MS DSL Tools 工具，允许开发人员自行设计专属的图形化工具，它内置了模型的相关支持，以及模型与图形之间的支持，还包括对模型的验证，规则、事务的支持。

MS DSL Tools 工具的核心是一组称为内存存储的应用编程接口，简称为存储；

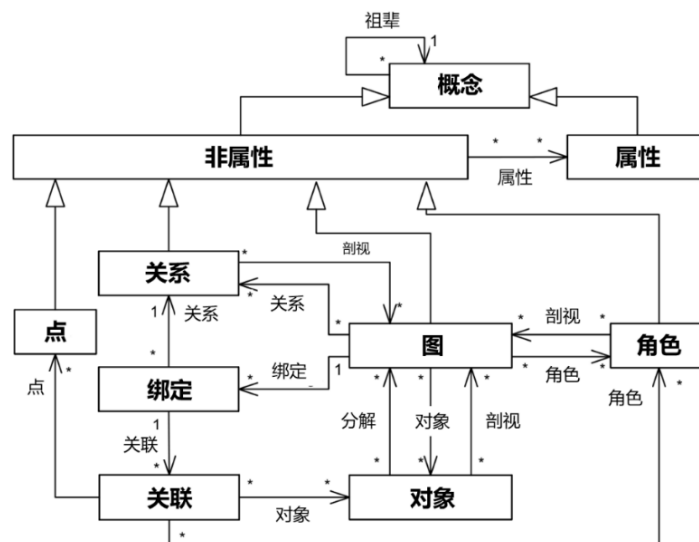
它是通过一个名为存储的类实现的，存储提供了一套支持 DSL 工具行为的基本设施：模型元素和链接的创建、操作和删除；撤消/重做、规则和事件等。在 DSL Tools API 中，元元模型的抽象类 `ModelElement` 和 `ElementLink` 提供了对所有可用于创建、操作和删除存储中的模型元素和链接的功能的访问。在存储中，每个领域模型元素都是一个 `c#` 类的实例，该类直接或间接地是 `ModelElement` 的子类。这些类是由 DSL 工具从 DSL 作者定义的特定域模型生成的。事实上，这些 `c#` 类中的每一个都是由 DSL 定义中的一个元素生成的，称之为领域类。类似地，每个链接都是一个 `c#` 类的实例，该类直接或间接地是 `ElementLink` 的子类。这些类是由领域关系的元素生成的。并且 `ElementLink` 本身就是 `ModelElement` 的一个子类，所以每个链接都可包含领域类的操作，比如拥有属性^[23]。

MS DSL Tools 中元元模型由领域类、领域关系、领域角色、命名领域元素、领域属性和语言组成。但因其专用性和代码的封闭性导致扩展性差，仅针对特定域开发，不支持现有的常用建模语言，因此集成能力差，且很难对 MS DSL Tools 元元模型进行独立研究。

（5）GOPRR 元元模型

GOPRR 是由六种元素构成的元元模型，分别为图、对象、点、属性、关系、角色。如图 1.1 所示，元元模型支持元模型设计；元模型支持建模；模型用于描述体系视点。GOPRR 相关元元模型说明如下^[24]：

- 图（Graph）：图元素包含其余几种元素，对系统的某个视点进行分析，在一个框图内对系统进行描述。
- 对象（Object）：模型中的主要元素，用于表示一种实体，可以单独存在，也可以与其他对象进行关系链接。
- 点（Port）：通常附属到对象上，用于表示对象和角色连接的端口。
- 属性（Property）：用于定义及描述一些元类型（对象、角色及关系）的特性。一般而言，属性具备四种数据类型：String, double, Integer, Boolean。属性不能单独存在，附属到其他元元模型上用于表示其特性。
- 关系（Relationship）：通过角色与对象连接，表示对象之间以何种方式进行连接。
- 角色（Role）：在关系的两端，表示对象以何种方式进行连接。



以上不同的元元模型表达不同的模型要素抽象形式，进而描述不同复杂系统的设计过程及领域，表 1.1 中总结了以上五种元元模型，其中：

通过统计以及对比分析，从抽象元素个数方面，ARIS、MS DSL Tools、GOPPRR 都为 6 个；领域方面，不限制领域的 Ecore 是最优的选择，GOPPRR 及 MS DSL Tools 支持特定域，则有开发的空间；工具方面，5 种元元模型都有各自的建模工具，但并不通用；扩展性方面，出 GME 之外，其他元元模型均有不同程度的扩展能力；开放性方面，ARIS、GOPPRR 均具备良好的开放性。在以上五个方面中，领

域及开放性对于装备抽象能力的影响较大，其他方面影响次之，因此本文对以上元元模型有个初步的评估，ARIS 及 GOPRR 元元模型更适合用于模型的抽象，但仍不能明确能够满足对装备复杂系统生命周期各阶段要素的描述，仍需具体的分析，本文将在文中第 3 章中进一步分析。

表 1.1 元元模型对比分析

元元模型	抽象元素个数	领域	工具	扩展性	开放性
ARIS	6	企业信息系统	ARIS Toolset 等	插件扩展	具备
ECore	4	不限制	Eclipse 平台	EMF 生态系统内可扩展	不具备
GME	5	电气工程领域	领域特定域建模的工具	不具备	不具备
MS DSL Tools	6	特定域	MS DSL Tools 工具	Visual Studio 内部扩展	不具备
GOPRR	6	特定域	MetaEdit+	工具内扩展	具备

1.2.2 规范建模技术及建模环境

1.2.2.1 规范建模技术

针对基于模型的装备复杂系统设计，规范建模技术是对目标系统做出表征的手段^[25]。目前，主要应用的 MBSE 规范建模技术有以下几种^{[(26],[27])}。

(1) 基于统一建模语言 UML 的规范建模

从历史上看，建模语言已经以许多不同的形式存在了很多年。但是，由于存在多种建模语言，使用建模语言的主要核心——通信能力丢失和破坏，结果使设计师之间存在多种建模语言。因此，工业企业迫切需要将所有建模技术整合为一种语言，这种语言称为统一建模语言^{[(28)-[30)]}。UML (Unified Modeling Language) 的主要目标是对系统进行建模，解决复杂系统中固有的规模问题以及创建人与机器都可以理解的建模语言。UML 是用于说明、可视化、构建和编写一个正在开发的、面向对象的、软件密集系统的制品的开放方法。最新的 UML 2.2 中一共定义了 14 种图，从不同侧面对系统进行描述，UML 的目标是以面向对象图的方式来描述任何类型的系统，具有很宽的应用领域，但其中最常用的是建立软件系统的模型。

建模方面, Rambe 等^[31]使用 UML 方法建立学校付款信息系统, 并使用黑箱测试方法对软件进行评估, 满足了行政人员和学生对系统的需求。Wesley 等^[32]在进行系统设计时, 使用 UML 类图提取生产线体系结构和特性模型, 从需求上允许从业者参与软件产品线的开发。以上两位学者验证了 UML 在软件系统建模的优势, 但 UML 不应该仅仅局限于软件, 根据 Holt^[33]的调研, UML 是一种灵活的系统, 可以在任何开发环境、任何流程模型、所有生命周期阶段和任何应用程序领域中使用。Schütz 等^[34]提出了一种用于制造系统的 UML 建模方法。该模型采用了扩展的 UML 元模型, 通过描述机器、机器的组件、组件的状态和组件的操作对制造系统进行标准说明, 但进行了大量的扩展。

UML 在软件领域的建模具有很强的表达能力^[35], 但是在其他领域表达能力较弱, 特别是在本文所研究的复杂装备, 除了可能包含的软件组件, 可能还包括硬件、人员、设施、过程等更多的系统元素。系统工程师基于 UML 进行建模工作, 并不能很好的描述系统。也就是说, 针对装备整体研发, UML 存在“盲点”, 基于当前已有的 UML 元素不足以对装备的复杂系统进行充分的、有效的表达。因此, 基于 UML 的扩展, 系统工程领域更为广泛应用的建模语言 SysML 应运而生。

(2) 基于系统建模语言 SysML 的规范建模

SysML^[(36)-(38)] (Systems Modeling Language) 是针对系统工程应用的通用建模语言。系统建模语言可以分析系统及超系统的规格、分析、设计及验证及确认。SysML 是在对 UML2.0 子集进行重用的基础上, 面向系统工程进行扩展, 加入系统工程应用所需的建模元素。UML 与 SysML 的关系如图 1.2 所示, SysML 完全继承了 UML 中的包图、状态机图、序列图和用例图, 基于 UML 扩展活动图、模块定义图、内部模块图, 并根据系统工程特性新增需求图和参数图。SysML 提出之后, 在学术界和工业界都得到广泛关注。作为标准的系统建模语言, SysML 支持从需求到验证的整个设计过程。

SysML 重点解决了系统建模问题。在系统需求方面, Soares 等^[39]在道路交通管理系统开发中提出了一种基于 SysML 的需求工程方法, 对用户需求进行图形化建模, 明确它们之间的映射关系, 并且使用 SysML 需求表增强需求的可追溯性。在装备系统建模方面, Capasso 等^[40]使用 SysML 建模语言, 通过 RFLP (Requirements engineering, Functional design, Logical design and Physical design) 建模方法, 实现 MBSE 的电动汽车推进系统建模。在系统验证方面, Marcos^[41]等使用 SysML 进行架

构建模，使用 Petri 网对系统进行行为建模，实现对需求的初步验证。在数据管理方面，面对设计过程中涉及的大量数据，必须使用高层抽象来管理此数据并“整体”分析系统，且设计人员还需要保留系统的所有底层信息，以便能够在以后进行优化和修改^[42]，如石福丽等^[43]基于 SysML 对无人机顶层需求及其与设计数据之间进行的映射，基于模型的层次结构优势，通过模型实现对系统数据的管理。同时 SysML 还可进行安全性需求验证^[44]，活动图验证^[45]，进行可靠性分析^[46]，提供验证环境接口^{[[47],[48]]}等。

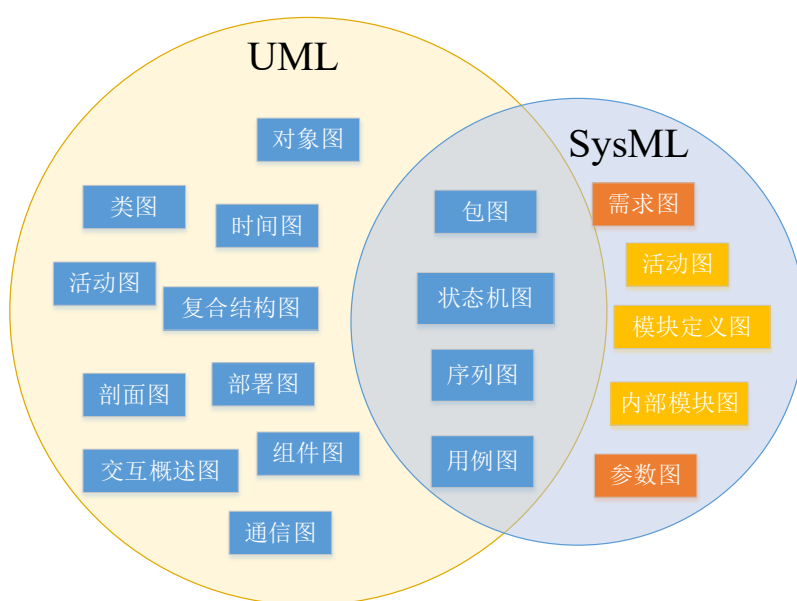


图 1.2 UML 与 SysML 关系

SysML 语言在系统建模中具有比较完善的表达能力，通过 9 张图可以形成产品的系统架构模型，同时通过扩展实现需求管理及性能分析，但装备的系统生命周期包含多个阶段，该建模语言的表达能力还不足以完全支撑装备全生命周期的架构设计，对体系、流程等涉及的要素和过程缺乏表达。

（3）基于体系建模语言 UPDM 的规范建模

UPDM^{[[49]-[51]]}（Unified Profile for DoDAF and MoDAF）是对象管理组织（OMG, Object Management Group）建立的一个基于 SysML，针对美国国防部体系架构框架（DoDAF）和英国国防体系架构框架（MoDAF）的行业建模标准，以满足对需求、参数、分配和其他关键特性的描述。UPDM 将 DoDAF2.0 的 8 个视点（Viewpoint）和 52 个模型（Model）标准化，目标是显著提高与企业相关的系统体系结构建模的质量、生产力和有效性，促进体系结构模型的重用和可维护性，改进工具的互操作

性，加强与利益相关者之间的沟通，并减少由于不同类型的工具实现和不同时期的语义培训而造成的影响^[52]。

国内对作战体系的论证有多年经验，吴骞华^[53]通过分析使命的方式将与任务对应的场景进行确认，并建立任务描述模型，建立武器装备体系作战能力中所需要的多个不同场景。孙岩等^[54]基于 UPDM 建立基于时序的能力与任务描述模型，实现武器装备体系作战能力所需的指控过程与多个杀伤链场景，并通过验证体系模型能力，为武器装备体系的作战性能和作战效能评估提供了支持。国外除了在国防体系中应用 UPDM，在其他方面也有研究，如系统架构方面，Papke^[55]尝试在系统安全性设计时，参考 UPDM 体系结构，设计时域和操作时域中的敏捷系统体系结构的特定元素，以支持具有弹性和安全能力不断发展的系统的定义和设计，应用范围局限；如流程方面，Sanduka 等^[56]基于 UPDM，在早期设计阶段中引入时间规范和时序分析避免在后期设计阶段体系结构无法满足其时序要求时进行更改，但效果并不优于 SysML 的时序图验证。

因此，基于 UPDM 建模语言能力，可以规范的定义作战场景，规划体系设计，确定系统需求，为体系工程的发展奠定了良好的基础。但该建模语言针对 DoDAF 和 MODAF 框架开发而来，仅限于体系建模，对装备生命周期的其他模型表达能力不足。

（4）基于业务流程模型和标记法 BPMN 的规范建模

BPMN^[(57)-(59)] (Business Process Model and Notation) 是业务流程建模的标准，它创建在与 UML 的活动图非常相似的流程图法 (flowcharting) 基础上^[60]，为业务流程图 (BPD, Business Process Diagram)^[61]中的特定业务流程提供一套图形化标记法。BPMN 1.0 规范于 2004 年 5 月向公众发布，2.0 版本则由 OMG 在 2011 年 1 月发布，在其最新版本中提供了五个基本类别的元素：流对象、数据、连接对象、泳道和器物。BPMN 的主要目标是要提供被所有业务用户理解的一套标记语言，包括业务分析者、软件开发者以及业务管理者与监察者，特点是语义丰富、表达能力强和易于解释，从而降低了错误知识转移的风险^[62]。

BPMN 已经应用于工业设计、制造、产品维护，医疗等多个领域和生命周期阶段。如王刚等^[63]基于 BPMN 规范及微服务架构设计设计轻量级工作流引擎，使流程的可复用性和灵活性得到了极大的提高，为业务系统提供良好的支持。Kozma 等^[64]将 BPMN 应用在工业物联网框架中，可动态、灵活地支持遵循工业 4.0 预期的自动化制造流程。Fernandes 等^[65]通过使用流程图定义每个相关的工作人员必须遵循的维护

标准，将 BPMN 应用于雷诺汽车曲轴箱机构基于状态和预测性的维护过程中。Bazhenova 等^[66]引用真实世界的医疗保健流程展示了结合 BPMN 流程建模与 DMN (Decision Model and Notation) 决策建模结合的适用性。此外，安全性对于业务性能很重要，Rodriguez 等^[67]对 BPMN 进行扩展，集成安全需求，使在业务流程级别的客户、最终用户及业务分析师等能够表达其安全需求。

然而，BPMN 在支持特定领域或非功能属性方面的能力有限，因为它只提供了通用元素并且只关注业务流程的功能需求。

(5) 基于特定域建模语言 DSML 的规范建模

特定领域语言 (domain-specific language, DSL) 提供了一种抽象机制来处理特定领域的复杂性。将主要领域概念映射到建模语言中的对象，其他概念映射到对象的属性、关系、子模型及其模型链接中^[68]。DSL 被定义为相关符号表示相关工具支持的元模型，元模型描述了 DSL 的概念、性质、建模概念之间的语义关系和模型层次结构等等。在复杂产品的整个生命周期过程中，系统的开发基于系统工程的不同视角，不同的系统视角包含不同的领域建模，当标准的建模语言无法满足领域特性的时候，需要开发特定的领域模型，特定领域建模 (DSM, domain-specific modeling) 的出现，为系统设计提供了一种便利的方法，它能够实现不同领域及工业标准的细化，大幅提高设计效率并简化系统的开发过程。DSM 模型开发的基石是语言的定义，也就是特定域建模语言。特定域建模语言是在 DSM 方法体系中，为强调模型驱动开发而从 DSL 中划分出来的分支，是一种专门针对某一目标领域精心设计并用以解决特定领域应用建模问题的建模语言^[69]。如 AADL (Architecture Analysis & Design Language) 是一个在嵌入式系统中使用 SAE 标准，对嵌入式系统软硬件架构进行描述的特定域建模规范；EAST-ADL (Architecture Description Language (ADL) for automotive embedded systems) 是基于开放汽车系统架构标准的描述汽车机电系统架构的特定域建模规范。

1) 架构分析和设计语言 AADL

AADL 是在 UML 的基础上提出的设计标准，是一个支持 MBE (Model Based Engineering) 的建模基础架构。其目的是提供一个标准的、精确的方式，设计与分析嵌入式实时系统的软件、硬件体系结构及功能和非功能性质，采用单一模型支持多种分析的方式，将系统设计、分析、验证、自动代码生成等关键关节融合于统一框架之下。

AADL 具有语法简单、功能强大、可扩展的优点,得到了欧美工业界,特别是航空航天领域(如 Airbus, Rockwell Collins, Boeing, Honeywell, Lockheed Martin)的支持,卡内基梅隆大学、麻省理工、伊利诺伊大学厄巴纳分校、宾夕法尼亚大学,美国 NASA、法国 IRIT (Toulouse Institute of Computer Science Research)、NRIA、Verimag 等研究机构对 AADL 展开了深入研究与扩展^[70]。SPICES^[71]、TOPCASED^[72]、AVSI-SAVI^[73]是国际工业界和学术界合作的 AADL 重大研究项目,涉及 AADL 标准扩展、建模工具、语法语义、验证与分析、可靠性、可调度分析及代码生成等方面的研究,这些也是目前 AADL 的研究热点。

AADL 建模语言在领域内具有很高的认可度,为嵌入式系统的研发提供很大支持,但也因嵌入式实时系统的特性使该语言的应用领域受到限制,与其他语言的数据交互或集成 AADL 建模语言发展需要考虑的因素。

2) 面向汽车嵌入式系统的架构描述语言 EAST-ADL

在汽车行业,MBSE 带来了嵌入式系统的设计,建模和验证的解决方案。法国项目 AEE (嵌入式电子架构) 首先进行了尝试,该项目的成果之一是创建了一种名为 AIL Transport (Architecture Implementation Language for Transport) 的语言,它允许在多个抽象级别的嵌入式架构的同一框架中进行规范。之后,一种为设计异步反应式实时系统的 CLARA ADL 通用语言被开发,该语言特别注意控制流。随后,ITEA 欧洲项目 EAST-EEA (2001 年 7 月 - 2004 年 6 月),涉及到汽车制造商、供应商和研究机构,研究汽车嵌入式架构和开发,它开始提出统一汽车软件开发的观念,并朝着通用符号迈进,整个方法需要得到合适的验证和验证 (V&V) 工具的支持,EAST-ADL 的概念在这样的背景下提出,它被指定并用于验证和验证活动,并得到工具 GME2000 支持架构模型的表达^[74]。

EAST-ADL 是一种使用元建模结构(如类、属性、关系)的特定域语言,基于 UML、SysML 和 AADL 的概念扩展,但仅为了适应汽车需求并符合 AUTOSAR (汽车开放系统架构)。EAST-ADL 涵盖的方面包括车辆特性、功能、需求、可变性、软件组件和通信^[75]。

(6) 其他规范建模技术

MBSE 方法中可用于规范建模的建模语言还有很多,如面向对象过程的方法 OPM (Object-Process Methodology) (^{[76],[77]}),采用 ARCADIA 方法的 Capella 建模框架(^{[78],[79]})等,这些建模语言在各自的领域内为系统工程建模做出贡献。

通过以上文献的综述，总结建模规范技术的建模语言的基本情况如表 1.2 所示，从视图数量方面，各建模语言存在较大差异，其受到抽象能力、涉及领域生命周期阶段等多方面的影响；从生命周期阶段方面，各个建模语言涉及的生命周期阶段不同，体系、系统、子系统都有涉及；从生命周期过程方面，SysML 涉及的过程最多；扩展性方面 UML、SysML、AADL 具备扩展能力，但扩展能力有限，并不能实现面向特定域的扩展。因此综合以上分析，未有单一建模语言能够满足全生命周期阶段如体系、系统、子系统等，生命周期过程如需求、分析等的建模，当复杂装备研发过程中使用多种建模语言时，不同建模语言产生的异构数据将导致集成困难，这将大大增加设计成本，降低设计效率。

因此，统一描述各建模语言底层数据规范，开发具备集成各种建模语言规范能力的多视图统一建模语言，是解决该问题的可行手段。

表 1.2 建模语言分析

建模语言	视图数量	生命周期阶段	生命周期过程	扩展性
UML	14	软件系统设计	需求、设计、分析	具备
SysML	9	系统设计	需求、设计、分析、验证及确认	具备
UPDM	52	体系设计	设计、分析、验证及确认	不具备
BPMN	3	流程设计	设计	不具备
AADL	3	嵌入式子系统设计	设计	具备
EAST-ADL	10	面向汽车嵌入式子系统 设计	设计、分析	不具备
OPM	1	系统设计	设计	不具备
Capella	28	系统设计	设计	不具备

1.2.2.2 建模环境

建模语言是为面向对象系统的产品进行说明、可视化和编制文档的一种标准语言，建模环境提供了装备模型可视化的环境，验证语言的性能，建模语言和环境是相互依存的关系，通过调研^[80]，目前应用较多的建模环境有 MagicDraw、Rhapsody、Capella、OPCAT 等：

MagicDraw 是 No Magic 公司开发的架构建模工具，该工具提供对 UML、SysML、

BPMN、UAF 等语言的完整支持，且功能全面。MagicDraw 提供了代码工程机制（具有对 J2EE、C#、C++、CORBA IDL 编程语言，.NET、XML Schema、WSDL 的完整支持），以及数据库模式建模，DDL 生成和逆向工程工具。MagicDraw 应用在 NASA JPL 提出的基于模型的集成系统环境 OpenMBEE^[81]，欧洲航天局（ESA）提出的欧洲航天局架构集成环境 ESA-AF^[82]等之中，以该软件为中心的插件开发和软件接口定义，开展以系统建模软件为中心的生态软件布局，并被 Odita^[83]、Lindsey^[84]、Shah^[85]、Godart^[86]等学者应用于航天、航空等多领域，控制系统、嵌入式系统等多系统的研究和实践应用中。

Rhapsody 是 IBM 公司的一款商业化的 MBSE 建模工具，提供 UML、SysML、UPDM、AUTOSAR 等建模语言的建模，支持 Harmony SE 方法论，支持需求分析、架构设计、嵌入式代码生成、中文文档生成等。Rhapsody 提供与市场上优秀的配置管理工具、需求管理跟踪工具、测试分析工具的接口，支持数据的交互和协同^[87]。在欧洲多集团组织的 DANSE（Designing for adaptability and evolution in system of systems engineering）（2011-2015 年）项目中，Rhapsody 使用 UPDM 进行了体系架构建模，并将 SysML 状态图生成为 FMU 模型进行联合仿真^[88]。Kalawsky^[89]、海晓航^[90]、张绍杰^[91]等学者在车辆、航空等领域，飞机控制系统、安全系统等多系统中实践应用。

Capella 是法国 Thales 公司开发的开源建模工具，底层是自定义的一套特定域建模语言，通过 ARCADIA 方法论的引导，指导用户进行系统架构设计^[92]。Capella 在 Thales 集团内部有着广泛应用，从航空电子、轨道系统、防务系统、卫星系统、地面站到通信系统等都采用该方法开展系统工程的实践，并服务对象包含运载火箭制造商阿丽亚娜集团、航空发动机制造公司 Rolls Royce、法国核电巨头法马通等。

OPCAT 是 Dori 团队开发的一款基于 Java 的建模软件。基于 OPL 语言及对象过程方法，将系统的结构和行为整合在一个统一的模型中。OPCAT 通过并发、同步和离散时间执行实现可视化建模和模型仿真^[93]。Törmänen^[94]、Hause^[95]、Casebolt^[96]等学者使用 OPCAT 开展了基于 OPM（Object-Process Methodology）方法的业务流程、语言替代、集成优化等方面的研究。

除了以上国外的建模环境之外，国内华中科技大学、国防科技大学等高校，航天科工三院三部，华如科技、烜翊科技、迪捷软件、杉石科技等科技公司都在积极开展自主可控的 MBSE 建模工具的研发，并已有部分建模软件实现了推广应用。

以上建模环境，支持不同的建模语言进行模型构建，为不同产品和生命周期阶段提供服务，在国防、科研的多个领域广泛应用，随着数字化进程的推进，更多领域更多装备的设计将使用基于模型的技术手段。但当生命周期过程模型需要集成与交互时，各工具固化的结构、支持的有限建模语言和软件封闭性导致集成困难，并且当面对特定的复杂装备，现有的建模环境支持的建模语言并不能完全支持其要素的描述，通过开发特定域建模语言以满足要素的表达，又会消耗大量的人力和时间及资源成本。因此开发支持多建模语言，并可支持语言自定义的建模环境是解决集成问题、特定域建模问题的必要手段。

1.2.3 建模语言形式化表征方法

一个模糊的、不一致的或不清楚的模型不能被期望产生一个品质好的最终产品，特别是当模型到复杂系统详细设计的转换在很大程度上是自动的。装备复杂系统模型的质量由许多方面决定，其中精确度、一致性和完整性最为重要。模型的精确度对应模型的模糊性，换句话说，在使用建模语言进行系统建模过程中，不同的人 and 工具以完全相同的方式理解模型的程度。形式上的一致性意味着模型至少一个实例或实现的存在，而完整性意味着包含所有相关方面，或者表达在所有情况下预测系统行为的能力。

上述“质量标准”在数学界有一个明确的、普遍认可的解释。然而，为了使数学解释的好处可用于装备复杂系统建模领域，必须对建模进行数学解释。形式化方法是一种严格以数学为基础的方法，能够清晰、精确、抽象、简明地规范和验证系统及其性质，帮助发现其他方法不容易发现的系统描述的不明确、不一致或不完整，有助于增强人们对装备系统的理解，形式化的建模方法能够极大地提高系统的非功能属性。

形式化建模的理论基础是形式化规格说明语言，这种语言采用数学的形式描述系统将要“做什么”，用带有精确定义的形式语言来描述系统的功能，它是设计与实现系统的出发点，也作为验证装备复杂系统模型是否正确的依据^[97]。关于形式化规格说明语言的研究主要分为以下两类。

(1) 代数方法：它为目标复杂系统的规格说明提供了一些特殊的机制，包括描述抽象概念并进行进程间连接和推理的方法。如 CSP、LNint-e、CCS 等都是代数方法。

CSP 是一种描述并发系统的表示法，它的组件称为进程，通过通信相互作用，并与环境相互作用。UML 状态图是一种图形化语言，它包含大量具有良好结构语义的构造，但缺乏正式的行为语义。Muan 等^[98]使用了 Hoare 的 CSP 来形式化 UML 状态图的行为。其形式化过程是：每个 UML 状态映射到一个 CSP 过程，每个 UML 事件映射到一个 CSP 事件。当一个状态变为活动状态时，它将等待下一个事件的发生，该事件将触发将系统带到下一个状态的转换，然后通过模型检查器使状态图的系统设计能够在设计阶段得到正式的检查。

LNint-e 是一种扩展到包含区间表达式的瞬时时间逻辑的语言，通过事件执行的开始和结束瞬间以及它们的过程来描述事件执行。UML 状态机图为系统的动态行为建模，因此时间方面具有关键作用，Carlos 等^[99]使用时间逻辑来形式化 UML 状态机图语义，表达状态和转换相关的不同操作，以及与复合状态相关的转换。通过使用点表达式定义，使其具有很强的表达能力，降低了计算成本。

CCS 是描述并发系统的表达过程代数语言，使用简单的结构描述动态行为，适用于评估系统属性的定性正确性。LEI 等^[100]提出了一种由 UML 序列图表示的场景生成 CCS 模型的方法，用一种形式化符号作为序列图的场景表达式，然后在此基础上提出一套将场景表达式转换为 CCS 模型的规则。通过场景语言来描述序列图，为电气工程师的需求建模提供了一种严格的方法，减少开发人员的难度和工作量。

以上代数方法仅针对部分建模语言视图的形式化，而且对开发人员的要求很高，对于架构的描述往往难以理解，导致架构只能为“小众”服务，同时代数方法可扩展性差，对架构进行修改后需要重新进行代数计算和验证，代价很大。

(2) 基于模型的方法：其基本思想是利用一些已知特性的数学抽象来为目标系统的状态特征和行为特征等构造模型。如 Z 语言、B 语言、Petri 网、图论等都是基于模型的方法。每一种基于模型的方法都会提出一套自己的规范，然后采用数学方法来说明规范。

Z 语言是用于编写规范的符号，以类型化集合理论和一阶逻辑为基础，此外还包含许多用于构造规范的构造，特别是模式，模式的使用提供了一种规范演算，通过这种演算，大型系统的规范可以从较小的部分构建^[101]。Z 语言作底层形式化符号，Z 的模式结构可以直接链接到类和类结构的概念，在面向对象的结构和形式表达的解释之间提供了一个清晰的链接。France 等^[102]将 UML 类图进行了 Z 语言的正式表达，定义了类、类实例和类关联的概念，并提供了集合和泛化结构的形式化形式。通过

类图的形式化，对模型结构有更深入的理解，并对建模的属性进行严格的推理。但是从语义模型派生的 Z 规范的生成非常繁琐。

B 语言^[103]是基于集合论，广义替换语言和一阶逻辑的语言，是一种正式的软件开发方法，涵盖了从规范到实现的软件过程。由一组变量、与这些变量相关的不变性属性和操作组成，系统的状态，即变量值集，只能通过操作进行修改。Lano 等^[104]定义了一个从 UML 类图到 B 语言的转换，用于验证 UML 模型的一致性，并验证这些模型的预期属性。由于 B 语言针对软件开发的属性，对于复杂系统建模缺少通用性。

Petri 网用于从体系结构到详细设计的系统生命周期的各个阶段的行为建模。例如，Levis 等^[105]使用 Petri 网对离散事件分布式智能系统的体系结构建模。Wang 等^[106]提出了一种使用 Petri 网建模制造控制系统的方法。SysML 活动图提供了系统行为的相对容易理解的描述，但是不直接进行形式正确性分析，如果没有正式的验证方法，系统可能直到实现阶段才会被验证，Petri 网为行为建模提供了形式化的语法和语义，支持系统模型的形式化分析。Edward 等^[107]将 SysML 活动图中的一组建模元素定义了形式化的数学符号，给出了 Petri 网与活动图之间的映射规则，将活动图中由一组行为模型形式化为等价的 Petri 网模型，应用 Petri 网的分析能力验证系统行为。Petri 网的形式化主要用于活动、行为图的分析。

图论是数学的一个分支，以图为研究对象。图论中的图是由若干给定的点及连接两点的线所构成的图形，这种图形通常用来描述某些事物之间的某种特定关系，用点代表事物，用连接两点的线表示相应两个事物间具有这种关系。图是众所周知、易于理解且常用的表示系统状态、复杂对象、图和网络的方法，如流程图、实体关系图等，学者们在形式化建模语言方面早有研究，在计算机领域常用图转换理论研究模式识别、编程语言的语义、编译器描述、函数式编程语言的实现、数据库系统的规范、抽象数据类型的规范、分布式系统的规范等^[108]。在复杂系统架构设计领域，图和图转换的理论用来为建模语言提供明确的、形式化的语义，支持语言的定义、转换等。如 Anneke 等^[109]使用图论的概念提出了 UML 类图和对象图形式化语义，并定义了约束类型和评价的形式化表达，支持 UML 严格的数学定义。Kang 等^[110]等使用图转换规则实现了序列图到活动图的转换。Roswitha 等^[111]通过用节点类型继承的概念扩展图语法，集成元建模方法与图转换，实现高效的可视化语言定义和模型操作。

综上，在装备复杂系统设计方面，模型视图与图论在形式上，本质都是带有连接边的节点，使用图论的数学理论定义建模语言的形式化表征，可更准确的捕获建模语言的语义。

1.3 存在的问题

根据国内外研究现状可知，学者和科研人员等对基于模型的装备复杂系统设计过程抽象模型、模型形式化表征，建模语言及工具等方面展开了广泛而深入的研究，已经取得了很多有价值的研究成果，但依旧存在一些缺点或不足，具体表现在以下几方面。

(1) 面向装备的复杂系统设计包含基于模型的任务分析，需求分解与追溯，系统设计与评估等生命周期过程，目前的抽象模型在完备性和自洽性等方面很难兼备，导致装备复杂系统生命周期一致性描述困难，缺少一套统一的完备且自洽的抽象模型，能够抽取共性的建模要素，以实现分层次递进的模型统一表达过程。

(2) 在装备复杂系统架构建模过程中，系统生命周期多阶段、多领域建模要素的统一描述及相互关系的建立，要有严格的逻辑性和严谨性，才能避免人类认知的歧义性和保证计算机执行的一致性，图论使用数学理论能够精确定义系统功能的形式化，但目前仍缺少能够解决装备复杂系统生命周期对统一抽象模型进行准确、清晰、一致的形式化表征方法。

(3) 目前 MBSE 有多种建模语言，每一种建模语言都有适用的生命周期阶段或适用的领域，但在装备复杂系统设计过程中，视图类型的建模语言缺乏统一的底层语义规范，单一的建模语言很难满足系统生命周期的设计需求，多建模语言的联合应用又面临语义不一致及工具之间接口问题，因此装备复杂系统生命周期的建模缺少具备集成各建模语言规范能力的多视图统一建模语言及环境，以提高设计效率和模型准确性。

1.4 课题来源与研究意义

1.4.1 课题来源

本课题来源于“十四五”装备预研共用技术项目“XXX 多系统建模与设计空间智能决策技术”，项目编号：50923010101。国家重点研发计划项目“复杂产品设计

制造服务一体化系统建模理论”，项目编号：2020YFB1708100。

1.4.2 研究意义

为支持装备复杂系统生命周期架构的统一描述，本文力图寻找支持复杂系统架构多视图统一建模的方法：以更高抽象层次的模型支持对生命周期中多阶段模型的描述和表达，将不同阶段、不同领域的系统模型用一种简单的标准化方法进行描述。为系统设计人员在计算机环境下进行装备模型架构设计、模型集成和交互、模型计算等提供技术支撑，具有一定的理论意义和应用价值。

(1) 对装备复杂系统生命周期架构设计的意义。基于模型的系统工程以模型的形式展示了装备系统生命周期多阶段的视图模型，系统设计人员的关注点是将原有基于文档的设计方案通过固有的建模语言转换为模型，便于数据的追溯和重用，对于系统生命周期的元素抽象和模型的准确性少有关注。本文提出了三层结构的抽象模型，从更高抽象层次描述各个领域（机械、控制、液压等），系统生命周期各个阶段（体系、系统、子系统等）、各个过程（任务分析、功能分析、架构设计等）三维一体的建模要素，并通过统一形式化方法，对抽象模型的各要素数量、关系等进行数学逻辑的表征，扩展了装备复杂系统生命周期架构设计的描述深度和模型基于数学方法的统计和分析能力。

(2) 对基于模型的系统工程领域的意义。长期以来，基于模型的系统工程的研究始终围绕着建模语言、建模工具和方法论三个重要支柱上，如基于 SysML 语言的建模规范，基于 MagicDraw 工具的架构建模，基于 MagicGrid 方法的建模过程，系统设计人员的思维没有跳出三大支柱的影响，只是在三大支柱的作用下完成产品的架构建模，而忽略了装备研发本身的业务需求。本文提出了复杂系统架构多视图统一建模思想，回归企业的业务本质，根据装备复杂系统的研制流程和领域特点，基于多视图统一建模语言，设计符合该装备系统的建模规范及建模流程，真正的实现装备的正向研发。

(3) 对装备系统架构建模平台的意义。系统架构建模平台的作用是提供装备系统设计人员架构建模环境，通常情况下的系统架构建模平台，能够提供如 SysML、UPDM 等一个或少数的建模语言库，支持在固定建模语言库下进行建模，但应用领域的不同导致建模语言难以满足对装备领域特定要素的描述，平台需要具备根据差异进行定制的能力。本文提出的复杂系统架构多视图集成表达验证平台，提供元模

型构建环境，既可以满足规范的建模语言，也可以根据具体装备领域的业务需求设计建模语言。这种方法提升了建模平台的适应能力和模型集成能力，更适合企业的实际建模需求，能够有效的为装备系统设计人员提供系统多领域多阶段建模支持。

1.5 论文内容及组织结构

本文围绕基于模型系统工程和复杂系统生命周期多阶段多领域建模的相关理论和技术基础，对复杂系统抽象模型、复杂系统架构模型统一表达形式化方法和多视图统一建模语言进行深入研究，并基于本文开发的建模语言实现多视图集成表达验证平台。本文共划分为七章，其总体结构如图 1.3 所示，各章具体内容安排如下。

第1章 绪论。阐述本文的研究背景，综述国内外研究现状，指出现有研究的不足和存在的问题，介绍本文的研究意义、课题来源和主要研究内容。

第2章 复杂系统架构多视图统一建模框架。为解决装备复杂系统生命周期多阶段架构模型的统一描述和集成，保证唯一权威数据源，制定了基于霍尔模型的多视图统一建模框架，从系统维、逻辑维、领域维概括了装备的研制过程，并提出分层抽象模型构建技术、基于图论有向图的抽象模型形式化表征技术、多视图统一建模语言表达技术三个层次递进的技术路线。

第3章 基于 GOPPRR-E 元素的系统架构分层抽象模型研究。针对装备复杂系统生命周期多阶段架构模型缺少通用抽象元素描述，不足以支持复杂系统多架构模型统一表达的问题，进行复杂系统分层抽象模型研究。首先确认使用 GOPPRR 元元模型，并结合系统工程实践扩展为 GOPPRR-E 元元模型，作为本文语言的底层抽象模型；然后结合 MOF 框架定义本文的分层抽象模型，表达出各层的建模元素以及使用这些元素的规则，该分层抽象模型是定义装备复杂系统架构模型形式化表征的基础。

第4章 基于图论有向图的抽象模型统一形式化表征。针对复杂系统架构模型缺少完整的形式化语义，导致架构模型的理论研究缺失的问题，进行抽象模型形式化表征研究。首先分析图论的基本概念，然后基于元模型层类型图和模型层实例图等数学逻辑关系分别开展元模型形式化、模型形式化，以及针对模型满足性约束集的扩展约束形式化表征。实现了数学逻辑对模型本质的精准定义，支持多视图统一建模语言语法的构建。

第5章 多视图统一建模语言表达研究。针对支持装备复杂系统生命周期多阶段架构建模的多种建模语言底层数据异构导致模型表达不一致、集成和交互困难等问

题，开展多视图统一建模语言表达研究。首先研究多视图统一建模语言构建方法，然后定义抽象语法，指定元元模型、元模型及模型的抽象类和语言的包含、继承等关系，形成文本语言的框架；最后定义具体语法，指定抽象语法框架中的项目、语言、元模型及模型等的具体表达，每个可表示的抽象语法概念都扩展为一个具体的语法规范结构，最终形成多视图统一建模语言的语言规范。

第6章 复杂系统架构多视图集成表达验证平台设计与验证。为验证多视图统一建模关键技术，搭建基于模型系统工程的多视图集成表达验证平台，开发建模环境。然后基于验证平台开展多视图统一建模关键技术验证，通过对比 BPMN、DMN、OPM、UML 和 SysML 五种建模语言，验证多视图统一建模的分层抽象模型对于各建模语言的模型要素、各元模型及模型规范技术的满足性；通过智能电动汽车的架构设计，验证多视图统一建模语言对于装备复杂系统多阶段架构建模的支持，提高模型集成能力和设计效率；通过对建模过程的形式化表征分析，验证对装备复杂系统生命周期架构模型完整性和一致性表达的支持。

第7章 结论与展望。总结全文，归纳主要研究成果，展望研究前景。

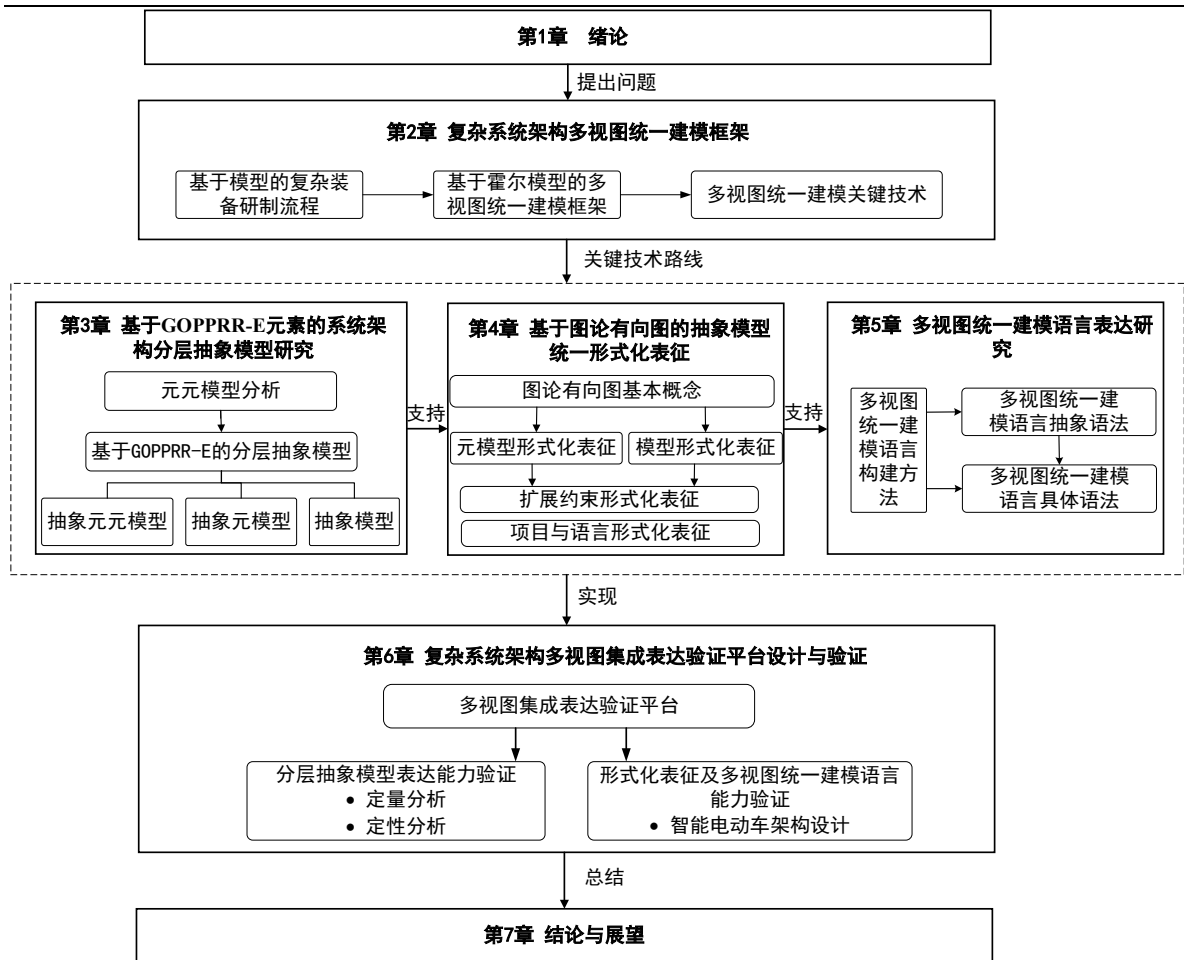


图 1.3 论文总体结构图

第 2 章 复杂系统架构多视图统一建模框架

2.1 引言

复杂装备的研制过程涉及方案设计、详细设计等多个设计阶段，机械、电气等多个学科领域，功能分析、综合设计等多个流程。在针对复杂装备设计时，面临着装备系统规模庞大、模型表达力不足、技术状态控制困难等问题，影响设计质量和研发效率。本章在基于模型的系统工程建模过程中，通过构建支持复杂装备建模的多视图统一建模框架，规范装备生命周期多阶段架构设计流程，明确多视图统一建模关键技术。

2.2 基于模型的复杂装备系统设计流程

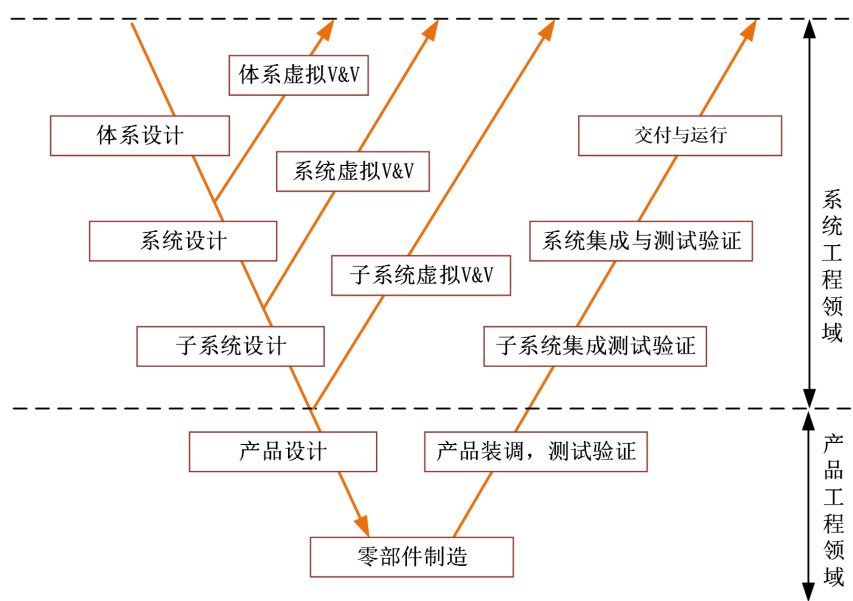
智能驾驶平台主要由车辆底盘、车载系统、感知系统、通讯系统等系统组成，涉及机械、电气、控制、液压等学科领域，是多学科交叉集成的先进复杂装备。智能驾驶平台以道路安全行驶为主要目的，配备具有高清晰、高频率、远距离探测等特点，对车辆稳定性、操控性、安全性等提出了更高的要求，同时其面临城市交通的复杂环境，对平台的环境适应性、灵活机动性和避障能力等提出了更多的挑战。

以智能驾驶平台为代表的车辆复杂装备在系统规模、技术难度、可靠性安全性要求、研制周期与成本要求等方面都对车辆的研制能力提出了更高的要求，传统的以文档为核心的系统工程方法存在的问题主要体现在：众多信息分散于各个文档，难以保证完整性和一致性；对复杂的、动态交互性强的活动难以描述，表达力不足，有时会产生歧义；技术状态控制困难，工作量大，维护困难；缺少早期验证手段，主要依靠后期的实物验证，代价大、周期长。

为应对上述挑战，结合以智能驾驶平台为代表的装备研制流程及特点，研究基于 MBSE 的车辆系统工程设计过程，建立符合装备研发的 MBSE 设计流程体系，实现智能驾驶平台等装备系统设计流程的规范化描述，更高效的支撑装备的研制工作。

2.2.1 复杂装备系统研制过程

复杂装备的研制过程 V 模型如图 2.1 所示，通过体系设计、系统设计、子系统设计等过程，构建基于模型的设计方案，如针对智能驾驶平台的研制过程，体系设计



2.2.2 基于 MBSE 的复杂装备系统架构设计流程

(1) 使命分析

以确定需要解决的问题以及潜在的解决方法。使命分析的主要步骤是：1) 识别与问题空间表征有关的全部实体；2) 定义这些实体的使命以及期望得到的结果；3) 确定或表征完整使命的潜在方案。如从智能驾驶平台的工况、交互及场景等构建智能驾驶平台所在交通系统的顶层概念场景，分析其内部功能和对外接口，获取潜在的设计需求。

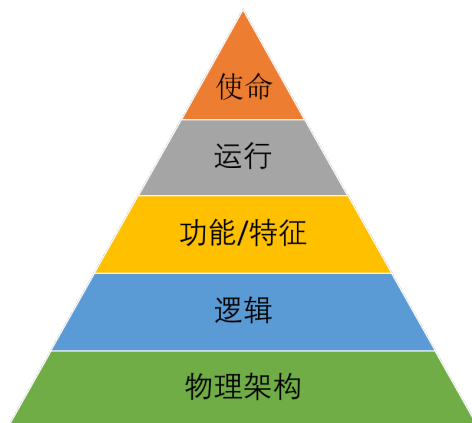


图 2.2 复杂装备架构设计流程

（2）运行分析

运行分析的目的是从运行角度回答如何实施智能驾驶平台使命任务，深入了解系统交互。通过深入理解外部实体（用户、外部系统等）如何与使命概念中定义的系统交互，用户需求被形式化为涵盖所有可能运行条件的运行任务实现。首先需指定系统和上下文，如复杂交通环境系统，识别使命概念运行系统中的目标智能驾驶平台系统，并对其进行运行分析：如分析智能驾驶平台运行状态，获取驾驶员、软硬件设计师、供应商等利益相关者需求，并分析各利益相关者在各个运行阶段系统的预期行为。

（3）特征和功能分析

特征分析是从使命场景、运行过程、用例中提炼出的装备特征，将装备特征扩展为装备功能，功能架构设计主要关注功能的定义，需覆盖系统特征，其关系和交换基于已识别出的运行分析（运行分析的输出 $\Rightarrow$ 功能架构的输入）中的参与者用例。可以将功能视为离散的“服务”，它必须实现目标系统的一个或多个目标。如首先进行车辆特征分析，包含动力控制、感知、自动驾驶等特征。再根据特征映射为相应的功能分析，如动力总成控制功能、全自动驾驶控制功能、生成全局路径功能等分析。

（4）逻辑分析

逻辑架构定义了目标装备系统的组成。系统逻辑架构实现如智能驾驶平台的所有分配功能，并考虑功能流和交互。目标系统的逻辑分解定义系统执行时所需的子系统/逻辑项目。逻辑视点相对于物理视点仍保持了一定的抽象层级，定义了所需的子系统类型和逻辑项目，但仍然为具体的技术实现（例如软件/硬件）保留一定的余地。比如，定义了实现智能驾驶平台刹车功能所需的刹车逻辑项目，但没有定义所需的刹车装置技术方案。

（5）物理架构分析

物理架构模型是对物理元素的管理，提供目标系统的解决方案，旨在满足逻辑架构元素和系统需求，物理架构是架构和技术选择以及设计准则的选择的结果，如通过明确实现逻辑模型中的智能驾驶平台对应的车架、底盘、车轮、转向器、电机等物理组件的及个数，确定物理组件间的连接关系及物理组件的属性等，最终形成物理架构。物理架构支持参数研究，以便进行分析、权衡和整体优化。物理架构一旦得到验证，就可以视为详细设计的基础。

面向系统工程的复杂装备的架构设计流程是一个产品生命周期多阶段表达的过程，该流程解决装备系统方案设计问题，通过基于模型的过程，实现各阶段的数据传递和一致性表达。

2.3 复杂系统架构多视图统一建模框架

2.3.1 基于霍尔模型的复杂系统生命周期过程

随着系统工程的广泛实践应用，1962 年霍尔出版了《系统工程方法论》，强调要把系统工程作为一种解决实际问题的程序，用形态分析法将系统生命周期阶段和问题求解的逻辑过程分成两个维度，用时间维（事物演化成熟的维度）和逻辑维（人和组织分析问题解决问题的维度）的二维形态分析矩阵定义和组织复杂系统过程^[112]。1969 年，霍尔增加了知识维，提出系统工程的三维形态分析模型如图 2.3 所示。知识维列举了应用的学科领域：工程、医学、建筑、商业、法律、管理、社会科学和艺术等各学科知识和技能。时间维表示复杂装备研制从开始到结束按时间顺序排列的全阶段：发展研究阶段、论证研究阶段、方案决策阶段、工程研制阶段、成果转化阶段、运行保障阶段、更新阶段。逻辑维表示时间维的每一个阶段内所要进行的工

作内容和应该遵循的思维程序，包括：界定任务范围、任务分析、系统功能分析、系统综合、系统权衡优化、系统决策、系统实施共 7 个逻辑步骤。这样就形成了由时间维、逻辑维和知识维所构成的三维空间结构。

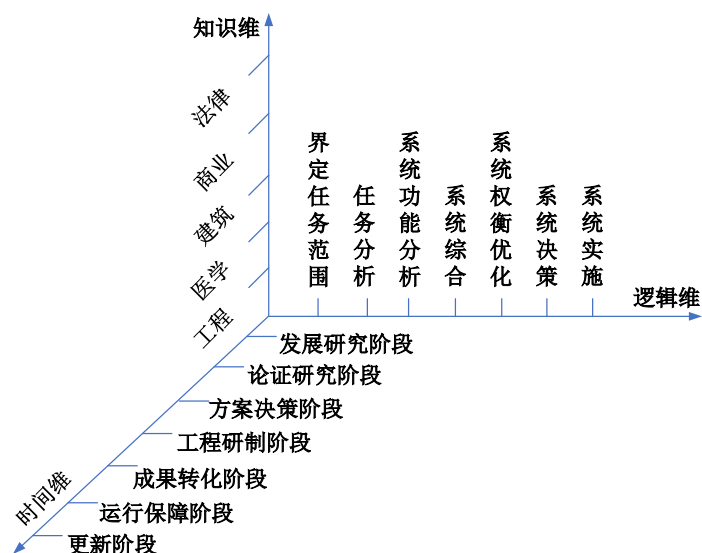


图 2.3 系统工程霍尔模型

霍尔模型从多个维度定义系统工程的设计过程，是重要的参考模型，本文基于对霍尔模型的理解，结合复杂装备的研制过程，提出面向装备研发的基于模型的复杂系统霍尔模型。

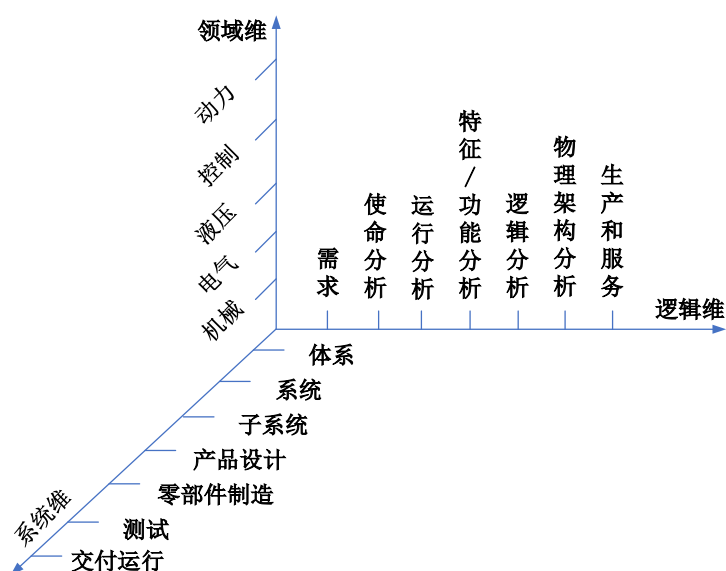


图 2.4 基于模型的复杂系统霍尔模型

如图 2.4 所示，基于模型的复杂系统霍尔模型分为系统、逻辑、领域三个维度。

复杂装备的研制涉及到机械、电气、液压、控制多个学科领域的交叉融合。装备如车辆的生命周期研制过程中，其发展研究阶段、论证研究阶段、方案决策阶段可实例化为体系级、系统级、子系统层级的装备设计阶段，工程研制阶段可实例化为装备产品设计阶段，成果转化阶段可实例化为装备的零部件制造、测试阶段，运行保障阶段可实例化为装备的交付运行阶段。逻辑维按照装备研制流程，分为需求、使命分析、运行分析、特征/功能分析、逻辑分析、物理架构分析、生产和服务。

霍尔模型的最大优点是将系统工程过程按逻辑维和系统维严格分开，把装备研制的一维线性过程，增加了一个维度，变成了二维平面。系统维和逻辑维将各系统阶段和逻辑步骤综合起来，形成基于模型系统工程过程矩阵，是系统分析和设计的有效工具，为解决复杂系统的规划、组织和管理问题提供一种统一的思想方法。如表 2.1 为针对车辆为代表的复杂装备形成的系统工程过程矩阵，在体系、系统及子系统设计阶段参与了更多的需求、使命、运行等过程，通过精细的系统模型构建，更早期的模型分析、验证和确认增加了设计方案的准确性、减少后期错误的不可逆。每个系统维度对应的逻辑维度过程根据具体的装备特征有所不同，该过程经过 A_{11} — A_{77} 共 23 个步骤实现系统的生命周期。

表 2.1 基于模型系统工程的时间维和逻辑维过程矩阵

逻辑维 \ 系统维	需求	使命分析	运行分析	特征/功能分析	逻辑分析	物理架构分析	生产和服务
体系	A_{11}	A_{12}	A_{13}				
系统	A_{21}		A_{23}	A_{24}	A_{25}	A_{26}	
子系统	A_{31}		A_{33}	A_{34}	A_{35}	A_{36}	
产品设计	A_{41}				A_{45}	A_{46}	A_{47}
零部件制造	A_{51}						A_{57}
测试	A_{61}						A_{67}
交付运行	A_{71}						A_{77}

2.3.2 复杂系统多视图建模需求

2.3.2.1 基于建模语言的架构设计流程

基于 MBSE 的复杂装备设计流程关注复杂产品的总体方案设计过程。该流程通过架构设计活动使得解决方案/系统在功能、物理和质量方面结构化。目前基于建模语言视图的常用的设计流程有以下几种：

（1）基于 SysML 的架构设计流程

Harmony-SE^{[113],[114]}方法是 IBM 推出的一种基于 SysML 建模语言的服务请求驱动的设计流程，该流程可以反映经典的系统生命周期 V 模型，包括系统需求分析、功能分析、综合设计三个阶段，首先分析系统利益相关者的全部需求并将其转换为（功能和非功能）系统需求，然后将其转化为系统操作的功能描述，并确保系统功能可以覆盖全部需求，最后设计开发满足功能要求和性能约束的系统物理架构。OOSEM（Object-Oriented System Engineering Method）是“自上而下的场景驱动的过程，它使用 SysML 语言（自 2006 年以来 OOSEM 采用 SYSML 规范）来支持系统的分析、规范、设计和验证。”OOSEM 包括以下开发活动：分析利益相关者需求、定义系统要求、定义逻辑架构、综合候选分配架构、优化及评估备选方案、验证和确认系统^[115]。这些活动以迭代方式执行，可以应用于系统的每个层次结构^[116]。MagicGrid 是基于 SysML 语言的适用于复杂产品开发的设计流程^[117]。该流程按照产品不同的研制阶段需要关注的问题将设计过程分为问题域、解决方案域及实现域。在不同域内部，将分析过程细分为需求、行为、结构、度量指标及专业工程五个支柱，以便全面表达产品应具备的属性。涵盖了业务和任务分析、利益相关者需求及需求分析、系统需求定义、系统架构设计、方案权衡、分析验证及协同设计管理等内容，通过不断的设计迭代，实现模型元素追溯、便于设计重用及变更管理和影响分析^[118]。

（2）基于 DSML 的架构设计流程

ARCADIA（ARChitecture Analysis and Design Integrated Approach）是一种面向硬件和软件架构设计的基于特定域建模语言的设计流程，建模流程为运营分析-系统分析-逻辑架构-物理架构-产品结构分解。ARCADIA 方法具有简洁明了的实现流程，用户只需关注系统本身，便可以通过对应的模型组件实现对系统的描述^[119]。

（3）基于 OPL 语言的架构设计流程

OPM（Object-Process Methodology）对象过程方法流程基于 OPL 语言定义为系统开发，生命周期支持和演化的正式模式，它将简单、正式的可视化模型对象过程示意图与带有约束条件的自然语句对象过程语言相结合，实现为一个表现系统功能、结构和行为的集成的单个模型^[120]。

以上装备架构设计流程以 SysML、DSML、OPL 等建模语言视图为标准规范开展，对于系统层级的需求、功能、逻辑、物理架构等视角的表达能力较强。复杂装备基于模型的正向研制流程需考虑生命周期如体系、系统、子系统的全流程建模，仅仅

依靠上述设计流程难以实现对生命周期的描述。

2.3.2.2 复杂装备多视图建模需求

在复杂装备系统研制过程中，为基于模型的系统工程实现体系、系统、子系统架构模型的设计^[121]，并使使命分析、运行分析等逻辑活动贯穿于此过程中，本节对过程要素进行抽象分析以形成架构的统一描述。

使命分析的抽象如高层概念定义视图，构想视图，状态转移视图，描述车辆的任務及其分类或者场景车辆想定等要素，包含了参与者的思想和行动意图，该抽象模型用于制定任务，如车辆行驶、避障、自动驾驶、互联网通信等，并强调与环境和其他外部系统的相互作用。通常使用 UPDM、OPM 建模语言进行装备体系的建模，对装备进行顶层需求分析。

运行分析的抽象如装备运行状态视图，描述系统动态行为、状态、转换等要素，关注系统如何随时间的推移响应发生的事件而改变其状态，如行驶车辆的刹车过程，是探测、信息处理、液压推进、刹车片抱死等一个连续动作及状态转换得到的刹车结果，该过程将系统功能性需求转化为一个可执行模型，通常使用 SysML、OPM、DSML 等建模语言支持运行的分析。

特征/功能分析的抽象如车辆特征视图、功能组成视图、用例视图、活动视图，描述车辆动力控制、感知、自动驾驶等特征要素，车辆的加速、减速、转向等功能要素，驾驶员操作车辆、乘客乘坐车辆等用例要素，刹车、探测感知、避障行驶等活动要素。该过程包含了领域特点，如针对车辆的特征使用 EAST-ADL 建模语言进行表达，通用的抽象特征及功能使用 SysML、DSML 等建模语言支持表达。

逻辑分析的抽象如车辆的组件执行时序视图、部件间接口顺序视图，描述车载雷达、中央处理器、踏板、刹车等要素的时间顺序，关注车辆内部的逻辑关系，该抽象过程使用包含时序的 SysML、DSML 等建模语言支持表达。

物理架构分析的抽象如模块定义视图、模块组成视图、故障分析视图，描述车辆部件结构组成及关系要素，定义其结构属性和行为属性，包含有值类型、组成、约束、端口、操作、接受等，通过定义物理参数可进行数值分析、一阶逻辑仿真验证、故障分析等。通常可使用 SysML、DSML、FTA 等建模语言支持表达。

在以上建模流程中存在着多阶段的抽象，如表 2.2 的建模语言应用示例，各语言的视图展示的要素不同，规则各异，难以进行统一的表征。为实现霍尔模型三个维度的统一描述，减少语义歧义和交互成本，对于装备复杂系统生命周期各阶段的使

命分析、运行分析、特征/功能分析等逻辑过程需进一步进行抽象，以统一的抽象进行多视图的统一建模。

表 2.2 建模流程中的多视图应用示例

	使命任务	运行分析	特征/功能分析	逻辑分析	物理架构分析
UPDM	√				
OPM	√	√			
SysML		√	√	√	√
DSML		√	√	√	√
EAST-ADL			√		
FTA					√

2.3.3 基于霍尔模型的多视图统一建模框架

车辆等复杂装备的研制是一项多生命周期、多学科参与的复杂系统工程过程，在此过程中会产生大量模型及数据，这些模型及数据结构不同、形式迥异，难以保证研制中的数据一致性。结合智能驾驶平台的研制流程，参考霍尔模型三维结构模式，按照系统级次、研制流程、涉及学科领域三个维度可以给出复杂系统生命周期多视图统一建模框架，如图 2.5 所示，进而为设计人员提供一种系统建模的标准范式。

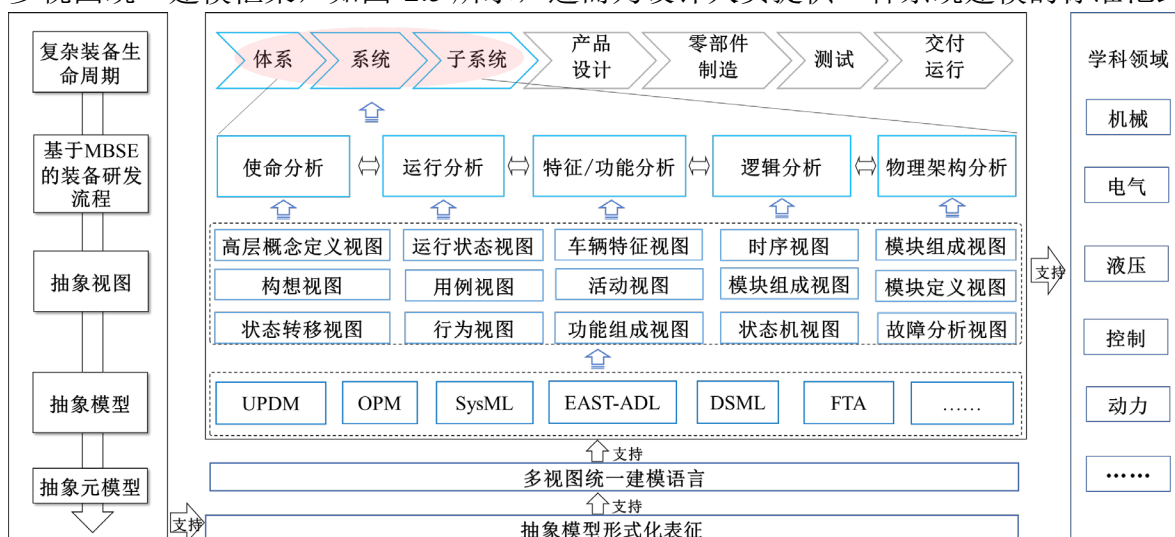


图 2.5 基于霍尔模型的多视图统一建模框架

复杂装备的研制过程包含方案设计到交付运行的多个阶段，基于 MBSE 的装备设计周期的边界主要集中在体系、系统及子系统的过程。该过程通过使命分析、运行分析、特征/功能分析、逻辑分析、物理架构分析的流程实现，支持装备在机械、电气、液压、控制、动力等领域的架构建模。构建的使命分析、运行分析等不同流

程阶段配置不同的视图，如使命分析包含高层概念定义视图、构想视图、状态转移视图等，用以描述以复杂装备如车辆的环境构成、行驶场景、车辆状态等。不同的视图由不同的建模标准规范所定义，为使车辆架构设计构建的多视图模型符合标准规范，在生命周期过程中集成 UPDM、SysML、EAST-ADL 等多种建模语言的视图。多建模语言之间存在着语义歧义和异构的问题，为实现复杂装备的统一建模，多建模语言由基于统一抽象模型的统一建模语言支持，从而实现语义和结构的统一。该统一建模语言通过装备设计流程的层层抽象，提取统一的抽象元模型和抽象模型，并将抽象元模型和抽象模型进行形式化表征来实现。

因此，为实现复杂系统架构多视图模型统一建模，本文共提出以下三项关键技术：

- (1) 系统架构分层抽象模型构建；
- (2) 抽象模型统一形式化表征；
- (3) 多视图统一建模语言表达。

2.4 多视图统一建模关键技术

本文提出三个相互递进的关键技术，支持复杂装备的生命周期多阶段架构建模。

(1) 系统架构分层抽象模型构建

为支持基于模型的复杂系统架构模型设计，需要将生命周期多阶段的建模要素进行抽象，提取共性，形成描述复杂系统设计各阶段的抽象模型。

1) 模型的抽象表示

元语言是一种指定其他语言的语法结构和词汇的语言。由元语言描述的语言称为“目标语言”。一种对象语言可以用来描述另一种语言，从而使其对象语言依次成为元语言。在使用模型描述系统工程方面，使用元语言来表示不同层级模型之间的关系，架构模型的元语言是用于创建模型的模型元素和关系，如 SysML 中模块定义图模型中的模块、执行者组成了架构模型的元语言，该元语言是提供模块、执行者的描述语言，是进一步的抽象。

“元”表示了一种抽象能力，将一种对象语言用来描述另一种语言，从而使对象语言成为其元语言。在支持复杂系统的建模过程中，模型的抽象称为元模型，元模型的抽象称为元元模型，本文要实现的是基于元元模型的统一抽象。

2) 基于 MOF 框架的抽象模型

MOF 起源于统一建模语言 (UML) [122]。基于对“元”的定义, OMG 使用一种元模型结构来定义 UML, 因此开发了 MOF, 作为一种编写模型和元模型 (模型的模型) 的半正式方法 [123] 广泛应用。MOF 的开发实现了系统的模型/元模型交换和集成。MOF 中的元建模是根据 4 级层次结构完成的, M0 层由模型实例组成, 可以是数据值、实例化的类对象、实例化的数据库表、算法、XML 代码或函数定义等, 它描述真实世界的物体。M1 层由模型组成, 这些模型也可以被视为是元模型的实例, 该层包括诸如 UML 图、类、模块和类型声明、数据库表声明或 XML 模式等元素。M2 层由元模型描述组成, 也可以认为是 MOF 的实例, 定义了 M1 层的语法和语义, 包括诸如 UML、XML、Java 等语言。M3 层是 MOF 语言本身, 用于定义 M2 级别元素。

MOF 是严格的元模型架构, 每一层的每个模型元素都严格对应的上层模型元素 [124], 非常适合作为建模语言的框架基础。UML 风格的类、类关联或类对象可以在 MOF 层次结构中的任何层进行定义, 以服务于不同的目的。例如, M2 层的类键入建模语言, M1 层的类用于建模语言的类型模型。然后通过面向对象风格的类/对象实例化关系将这些级别关联起来。 M_{i+1} 层的类元素提供了 M_i 层对象的类型描述, 反之, M_i 层对象实例化 M_{i+1} 层类。因此可基于 MOF 提供的分层架构, 开展本文的装备分层抽象模型构建。

(2) 基于图论有向图的抽象模型形式化表征

抽象的模型并不能精确地定义装备复杂系统设计过程, 基于模型系统工程提供了统一的建模语言实现复杂系统模型的形式化表达。抽象模型向建模语言的实现需将抽象模型信息表征为数学模型。图论的数学解释在精确度、一致性和完整性方面的优势可用于建模领域, 用图论解释建模抽象要素, 可进一步解释为建模语言的形式化语义, 因此, 在形成建模语言的过程中, 关键的一步是形成建模语言的形式化表征模型, 才能有效、准确的支持多视图统一建模的研究。

基于图论有向图的概念, 本文将针对分层抽象模型, 对装备架构建模过程抽象的要素、关系、约束等进行形式化语义的表达, 针对清晰的层次模型实例化过程, 开展不同层次的形式化表征, 及层级之间的继承、规则规范, 最终实现构建支持多视图统一建模语言的形式化表征模型。

(3) 多视图统一建模语言表达

如图 2.6 所示, 建模语言通过语法 (如何表示它) 和语义 (它意味着什么) 两个要素描述模型, 语法分为具体句法和抽象语法, 具体语法描述模型是如何被表示的

（例如，以 2D 矢量图形或文本形式），可以用于模型输入和模型可视化；抽象语法包含模型的“本质”，定义领域建模语言的组成及他们之间的构成关系。语义包括语义定义域（是什么含义）及语义映射（如何赋予模型含义），即由完整、全面且唯一的语义映射函数定义，该函数将每个抽象语法模型映射到语义域中的单个元素上。系统建模语言以用面向对象的、图形化、可视化的方式描述系统的底层元素，进而逐层向上组成集成化、具体化、可视化的系统架构模型，增加对系统描述的全面性、准确性和一致性^[26]。在 MBSE 方法中，用系统建模语言来描述装备系统架构模型，作为系统开发全过程中首要的工件（Artifact），并且对它进行管理、控制，并与仿真验证等其它部分进行集成。

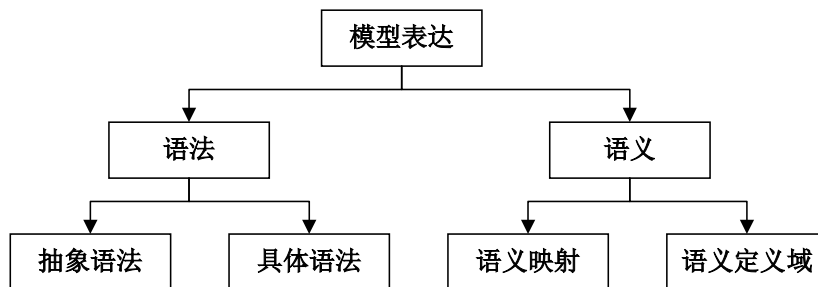


图 2.6 建模语言术语

建模语言的形式化表征模型提供了元元模型、元模型及模型的表达要素、关系，及存在的约束，通过抽象语法映射为统一建模语言的类和类之间的关系，然后针对抽象语言的每一个类进行文本形式化描述，每个可表示的抽象语法概念都有一个具体的语法结构。具体语法规则规范了统一建模语言的语言结构，提供了模型表达的模板。至于模型语义是根据具体领域问题构建的具体元模型及模型，本文将通过验证部分进行展示。多视图统一建模语言表达将主要提出统一建模语言的抽象语法及具体语法。

2.5 多视图统一建模关键技术实现过程

在支持多视图统一建模的分层抽象模型构建、基于图论有向图的抽象模型形式化表征、多视图统一建模语言表达三个关键技术的基础上，形成复杂系统架构多视图统一建模关键技术路线如图 2.7 所示。分层抽象模型是研究复杂系统设计过程中，为满足生命周期多阶段架构模型实现统一描述、唯一权威数据源，以及一致性数据传递，提出的模型要素全覆盖的分层抽象模型，为形式化表征技术提供抽象模型元

素及约束等；抽象模型形式化表征是以图论为基础，将具有规则和约束的数学模型应用在建模语言的开发中，针对所提供的抽象模型元素，形式化表征的严谨的数学逻辑模型，为统一建模语言表达提供形式化的表达要素、关系及约束，以支持准确、一致、清晰的进行复杂系统建模；统一建模语言表达是通过抽象语法和具体语法的定义，构建的针对多视图统一建模语言描述模型过程的元模型及模型规范，为复杂系统架构多视图统一描述提供基础支撑。基于以上三个关键技术的研究，构建装备复杂系统架构多视图集成表达验证平台，通过定量分析、定性分析及案例构建，验证对复杂系统生命周期多阶段要素的表达能力，验证准确形式化复杂系统生命周期的能力及架构统一建模能力，满足集成性、模型过程完整性和一致性。

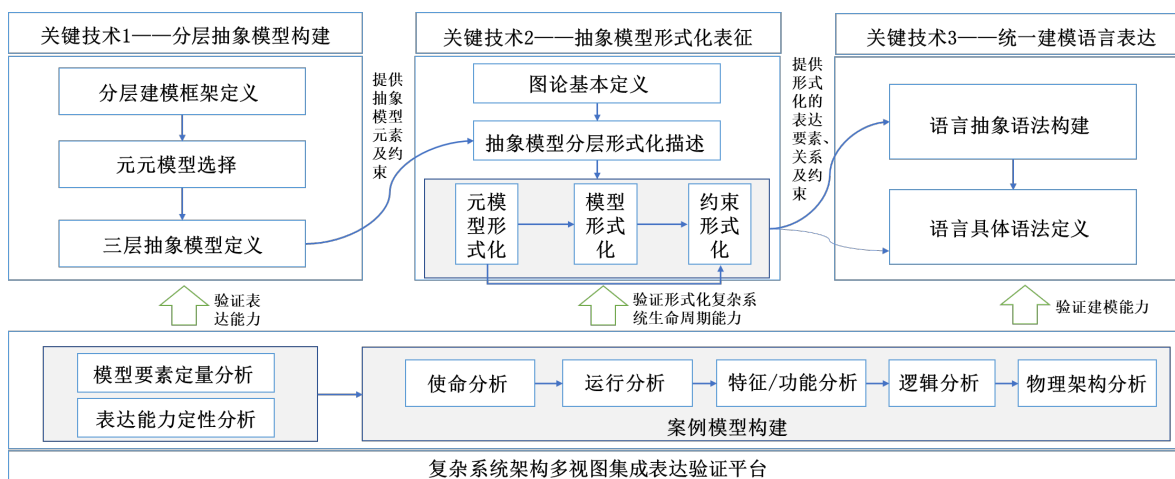


图 2.7 关键技术实现过程

2.6 本章小结

本章首先介绍了复杂装备的研制过程 V 模型，以及基于 MBSE 的复杂装备架构设计流程。然后构建面向复杂装备的基于模型系统工程霍尔模型，通过复杂系统多视图建模需求提出装备设计过程对多建模语言中多视图支持设计流程的需求，并构建基于霍尔模型的多视图统一建模框架，该框架表达了复杂装备研制过程中通过体系、系统、子系统设计实现装备方案设计，应用使命分析、运行分析、特征/功能分析、逻辑分析、物理架构分析等的设计流程，从流程体现了多建模语言视图的支持。最后从多视图统一建模框架中提炼出多视图统一建模关键技术，并制定了系统架构分成抽象模型构建、基于图论有向图的抽象模型形式化表征、多视图统一建模语言表达三个层次递进的关键技术路线。

第 3 章 基于 GOPRRR-E 元素的系统架构分层抽象模型研究

3.1 引言

面向装备复杂系统生命周期多阶段架构建模，统一的抽象模型可为建模提供一致的语义环境。虽然目前的 MOF 建模框架提供了严格的模型分层抽象架构，但核心的元元模型来自 UML 规范，并不能满足对生命周期多阶段模型的抽象元素的统一描述，通用性较差。将生命周期多领域、多阶段的建模元素进行抽象，提取共性，形成统一分层抽象模型是必要手段。统一分层抽象模型从元元模型的选择出发，本章将通过分析并提出元元模型 GOPRRR-E；然后，结合 MOF 建模框架，构建统一的分层的抽象模型。该抽象模型通过自底向上的层次映射关系实现装备生命周期模型一致性表达，为论文语言的研究奠定基础。

3.2 基于 MOF 建模机制的分层抽象模型

如图 3.1 所示的 MOF 四层结构，本文主要研究元元模型、元模型、模型三个层次的抽象模型。

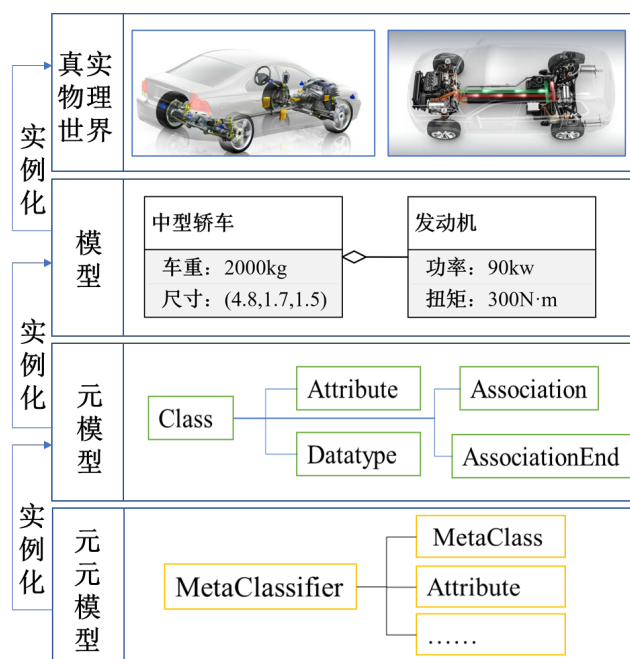


图 3.1 MOF 4 级层次结构

(1) 面向 MOF 类的抽象元元模型

M3 层 MOF 模型由一组关联的 M3 层类组成“元-元类”，本文称之为元元模型，是描述元模型的元语言，元元模型作为 M3 层是描述模型底层数据的通用结构。元元模型定义了一组关联的 M3 层的类^[125]，其中最重要的如：MetaClassifier（所有元模型分类器的通用超类型）、MetaClass（对所有非基本数据类型的元模型分类器进行分类）、Datatype（一种数据类型）、Attribute（与元模型分类器相关联的一种属性）以及 Association 和 AssociationEnd（在元模型分类器之间建立的类型关联）。该元元模型是所有元模型分类器的通用超类型：例如，UML 元模型中 UML 类的定义和 XML 元模型中 XML 模式的定义都可以编码为 MetaClassifier 的 M2 级对象实例。

M3 层的类 MetaClassifier 是一个抽象的元元模型类，用于对一个由其子类继承的通用结构进行分类，如 MetaClass、Attribute 和 Association。它的结构包括以下元素：一个字符串类型的名称属性，用于标识分类器实例的名称；一个布尔值 isAbstract 属性，用于确定分类器实例是否在继承层次结构中是抽象的；Constraint 类型元素的关联集合约束，定义 M2 MetaClassifier 元对象的 M1 级实例所需的行为。

（2）基于 MOF 类的抽象元模型及模型

元元模型对构成 M2 层元模型的元素种类进行分类。抽象元模型是这些元元模型的相关 M2 实例的集合，是描述抽象模型的元语言。MOF 规范^[126]定义了 MOF 抽象元模型的结构，包括角色和关系，以及包含必须应用于类型结构的任何实例约束的语义。

M2 层的 MetaClassifier 实例必须满足以下约束：1）分类器不能是它自己的直接或间接的超类型；2）分类器的任何属性的名称不应与任何直接或间接超类型的属性名称冲突。

MOF 允许约束与元模型的任何元素进行关联，可以用非正式语言（例如英语）或正式语言（例如对象约束语言）编写。MOF 抽象模型以两种不同的方式使用约束：1）MOF 抽象模型本身具有为其每个类定义的一组约束。这些约束定义了模型的语义，指定了 M2 抽象元模型必须如何表现。2）抽象模型包含一个名为 Constraint 的类，该类与模型的所有其他类相关联，此类的实例用于为 M2 抽象元模型编写语义，进而用于指定 M1 实例化模型的行为方式。然后通过关联的元元模型对象实例的集合在 MOF 中表示元模型，即这些实例是 M2 层的对象。

元模型是一组 MetaClassifier、MetaClass、Datatype、Attribute、Association、AssociationEnd 和 Constraint M2 层对象的集合，其中的对象只能相互引用。

M1 层是模型层，是 M2 层的实例化，根据 M2 层不同的语义组合，定制不同的模型表现形式，如 UML 中的对象图实例、类图实例，以及实例化的数据类型、枚举类型、属性等。

MOF 提供了严格的模型分层架构，但 MOF 的核心规范源自于 UML 规范，UML 提供了形成 MOF 元模型的结构，从 UML 2.4 版本开始，UML 和 MOF 共享共同的元模型和元元模型^[126]，不符合本文通过多视图统一建模实现复杂系统架构模型统一表达的愿景。因此本文将研究更为通用，具备高层级抽象能力的元元模型，结合 MOF 的框架实现支持统一建模的抽象模型。

3.3 元元模型分析

在 MBSE 领域，架构模型是具体实现的抽象，模型开发的基石是语言的定义。元建模方法是一种广泛使用的借助元模型对语言进行形式化的方法。一般来说，元模型定义了建模语言的概念及其关系以及约束和建模规则。为了定义元模型，需要一种元建模语言，即本文的元元模型。通过文献综述，本文调研了 ARIS、Ecore、GME、MS DSL Tools、GOPRR 五种元元模型，首先实现对元元模型元素的统一分类，然后进行了多种指标的对比分析，以此为依据确定本文的基础元元模型。

将元元模型抽象为比较标准的第一类元建模概念，如对象类型、关系类型、角色类型、端口类型、模型类型、属性类型，以及元模型元素的结构、重用和标识的概念抽象为对比标准^[127]如下：

(1) 对象类型：定义具有相等特征（如关系类型或属性）的对象集或类。通常，对象类型表示建模域的概念或实体类型。

(2) 关系类型：关系类型连接对象类型，并被定义为参与对象类型的（ n 元）笛卡尔乘积的子集。

(3) 角色类型：角色类型定义了对象类型如何参与到关系类型中。如果元元模型支持角色概念，那么关系在角色之上间接引用对象。因此，关系类型的一些特性被转移到角色类型。

(4) 端口类型：端口类型定义一组或一类端口。

(5) 模型类型：模型类型是具有相等属性的一组或一类模型。模型类型对可用的元模型元素进行分组并实例化。

(6) 属性类型：指元模型类型的属性，在模型级别，属性可以保存具体的值。

(7) 连接到模型类型：一些元元模型支持模型类型和其他元素类型之间的连接，例如，模型类型的细化或集成不同的模型类型，与子模型的关系是二元的，其中：

1) 参与元素：定义子模型关系的源和目标，确定目标是一个模型类型，而源是元模型类型，比如对象、关系或角色类型。

2) 多重性：定义与子模型连接的元素实例的数量，取值范围为一对一和一对多。

(8) 分组：这个概念允许在定义的部件或模块中构造元模型元素。

(9) 约束语言：约束允许为元模型元素定义附加的条件，这些条件必须在建模期间或建模之后完成。

在表 3.1 中，对象类型、关系类型和属性是基本的元建模概念，因为每个元元模型都支持这些概念，但所有元元模型中有一半支持概念模型类型和角色类型，模型类型允许实现视图概念，角色类型允许实现复杂的关系。根据表中支持的第一类元建模概念，可以推断出以下元元模型的顺序：GOPPRR = GME > MS DSL Tools > ARIS > Ecore。

表 3.1 元元模型比较表

	ARIS	Ecore	GME	MS DSL Tools	GOPPRR
第一类概念					
对象	对象定义 对象出现	E 类	原子、模型、集	领域类	对象
关系	连接定义 连接出现	E 引用	关联（集，引用）	领域关系	关系
角色	无	无	角色	领域角色	角色
端口	无	无	端口	无	端口
模型	无	无	模型	无	图
属性	属性	E 属性	属性	领域属性	属性
连接到模型	支持	无	支持	无	支持
分组	方法，模型	E 包	文件夹，范式	语言，命名空间	项目，图
约束语言	无	OCL	OCL	编程语言	自有，OCL
统计	5	5	9	6	9

在表 3.2 中，关系的元数表示关系类型中可以包含的对象类型或角色类型的数量，多重性定义在关系类型可以与另一个对象类型相关联的对象的数量，逆表示关系类型的方向，组合表示容器对象和以关系保存的被包含对象之间的生命周期依赖关系，依赖表示关系对其他元素的依赖关系，对象集表示关系类型为两端分配一个对象类集。根据表中的支持情况，可以推断出元元模型中关系的顺序：Ecore = GME = MS DSL Tools = GOPPRR > ARIS。

表 3.2 关系的比较

关系	ARIS	Ecore	GME	MS DSL Tools	GOPPRR
数量	二元	二元	二元	二元	N 元
多重性	默认值(0..n)	支持	通过角色	通过角色	通过角色
逆	无	支持	通过角色	通过角色	通过角色
组合	无	支持	支持	支持	支持
依赖性	方法	E 类	范式	领域类	图
对象集	支持	无	无	无	无
统计	4	5	5	5	5

在表 3.3 中, 属性类型的依赖表示属性依赖于元模型中的其他类概念, 多重性表示允许定义多值属性, 默认值表示实例化的属性可定义默认值, 唯一性表示该属性值不可以多次出现。这里 GOPPRR 元元模型方法不仅支持依赖性、多重性、默认值及唯一性的定义, 而且支持其他所有第一类概念的配置。通过属性的对比, 可以推断出元元模型中的属性支持程度顺序为: GOPPRR > GME > Ecore = MS DSL Tools = ARIS。

表 3.3 属性的比较

属性	ARIS	Ecore	GME	MS DSL Tools	GOPPRR
依赖性	方法	E 类	范式	领域类, 领域关系	项目
多重性	单值	多值	单值	单值	多值
默认值	无	支持	支持	支持	支持
唯一性	无	支持	无	无	支持
归属对象	支持	支持	支持	支持	支持
归属关系	支持	无	支持	支持	支持
归属角色	无	无	无	无	支持
归属端口	无	无	无	无	支持
归属模型	支持	无	支持	无	支持
简单数据类型	支持	支持	支持	支持	支持
统计	6	6	7	6	10

此外, 在表 3.4 中, Ecore、GOPPRR、GME 和 MS DSL Tools 支持继承, 而 ARIS 不支持继承。继承主要是指属性和建模的重用。通常, 这四个元元模型支持对象类型之间的继承。而 GOPPRR 和 GME 还实现了关系或模型类型等元素之间的继承。元元模型的继承能力排序为: GOPPRR > GME > Ecore > MS DSL Tools > ARIS。

综上分析, GOPPRR 元元模型具有较强的抽象能力和表达能力, 能够更清晰的表达类, 以及描述类和类之间的关系, 是最为全面的元元模型。

表 3.4 继承的比较

继承	ARIS	Ecore	GME	MS DSL Tools	GOPRR
单继承/多继承	无	多继承	多继承	单继承	单继承
对象可继承	无	支持	支持	支持	支持
关系可继承	无	无	支持	无	支持
角色可继承	无	无	无	无	支持
端口可继承	无	无	无	无	支持
模型可继承	无	无	支持	无	支持
统计	0	2	4	2	6

3.4 基于 GOPRR-E 及 MOF 的分层抽象模型

3.4.1 基于 MOF 框架的统一建模框架

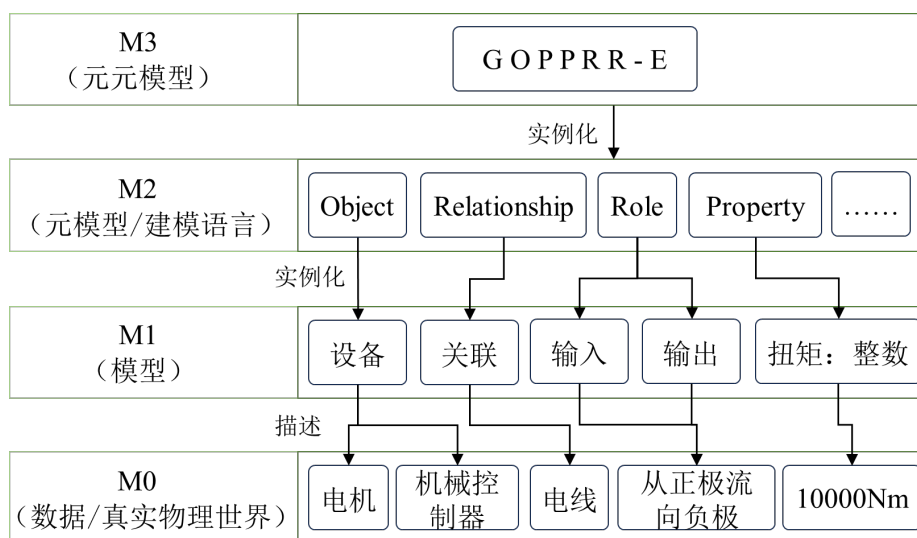


图 3.2 基于 MOF 框架的元建模流程

在上一章节的分析中，GOPRR 元元模型能够支持更为复杂的模型概念，因此基于 GOPRR 元素的元元模型可作为本文的研究基础，并根据系统工程实践，本文创新性的将 GOPRR 元元模型扩展为 GOPRR-E 元元模型，其中的 E (Extension) 表示扩展。通过 MOF 严格的元模型架构，以及总结的 GOPRR-E 元元模型，本文提出了基于 MOF 框架的模型统一建模语言框架。如图 3.2 所示，该框架规范了领域模型构建过程，在 M3 层使用 GOPRR-E 元元模型作为顶层概念，以实现不同领域装备架构模型的统一描述。在 M2 层，领域系统工程师根据领域相关知识、规范、约束及标准，利用 GOPRR-E 元元模型设计元模型，也就是 MBSE 常用的如 SysML、UPMD 等建模语言，以及特定领域自定义的建模语言。在 M1 层，系统工程师利用设

计好的建模语言构建模型，从而实现不同的建模需求。由此实现基于 GOPPRR-E 元模型的元建模方法支持对异构模型的统一描述。

上文从文本语言的角度，介绍了本文研究的基于 MOF 的建模框架，而从可视化角度，建模语言对应特定的符号表示，构成了建模的基础。如图 3.3 展示了 GOPPRR-E 可视化在 M3-M0 框架中的层次分布。在 M3 层定义 GOPPRR-E 元元模型，在 M2 层，将 GOPPRR 元元模型进行组合并添加扩展约束 E，形成标准建模语言，最后在 M1 层进行模型的构建，在可视化的图模型的底层，是一套支持多视图统一表达的文本式建模语言。基于此，在本文的后面章节，将研究基于 GOPPRR-E 元元模型的元模型，应具备怎样的规则以及约束关系，才能支撑本文开展形式化表征的研究。

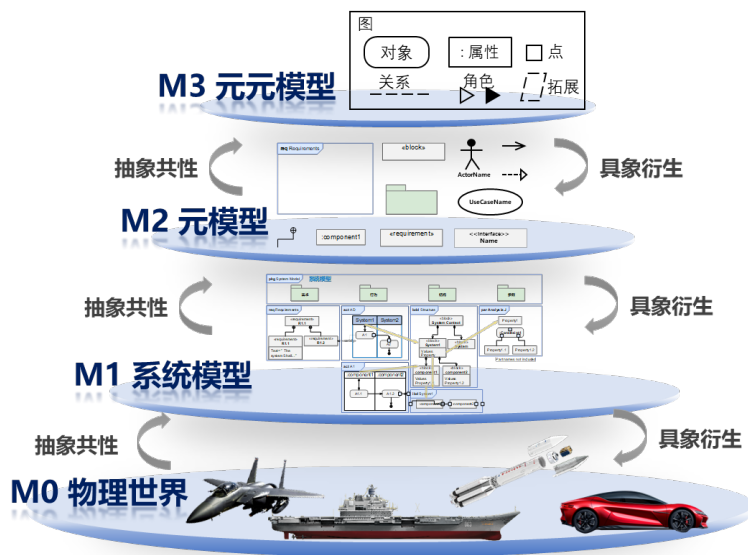


图 3.3 统一建模语言框架

3.4.2 M3 层 GOPPRR-E 元元模型

作为一种抽象概念，元元模型明确了元素的分类，通过提取共性，每一种元元模型代表了一类可视化类型的抽象，是一个“共性值”为空的集合。如图 3.4 所示，GOPPRR-E 元元模型包含：G (MetaMetaGraph)、O (MetaMetaObject)、P (MetaMetaPoint)、P (MetaMetaProperty)、R (MetaMetaRelationship)、R (MetaMetaRole)、E (MetaMetaExtension)，分别对应图元元模型、对象元元模型、属性元元模型、点元元模型、关系元元模型、角色元元模型和扩展元元模型，严格的说，扩展与其他元元模型元素存在本质的区别，它并没有像其他元元模型元素具有实体的属性，而是作为一种衍生的约束概念，通过制定规则、建立约束对其他元

元模型进行关联和限制。扩展促进了开发建模语言的生产力，并有助于在建模语言“家族”中实现更高级别的一致性和连贯性。

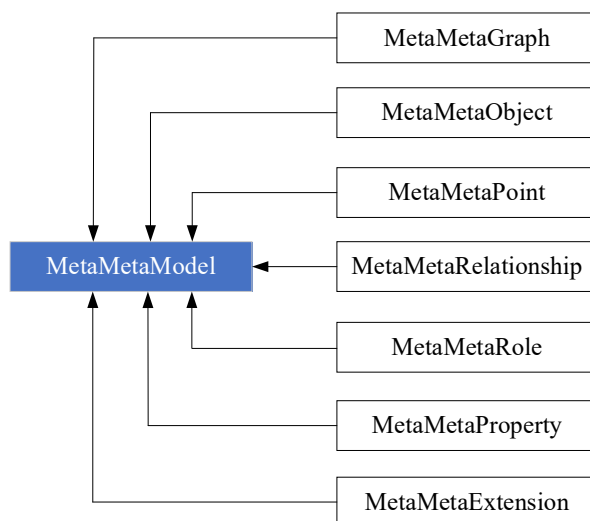


图 3.4 元元模型元素

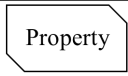
元元模型名称	类名	可视化图符	元元模型名称	类名	可视化图符
图元元模型	MetaMetaGraph		属性元元模型	MetaMetaProperty	
对象元元模型	MetaMetaObject		角色元元模型	MetaMetaRole	
关系元元模型	MetaMetaRelationship		点元元模型	MetaMetaPoint	
扩展元元模型	MetaMetaExtension	无			

图 3.5 元元模型与可视化图符

如图 3.5 所示，M3 层展示了元元模型层的基本元素，图元元模型是一个容器的概念，用来包含显示其他元素；对象元元模型定义一组或一类具有相同特征的对象，可由不同的语义实例化为具备不同含义和表现形式的对象元模型；关系元元模型定义一种连接方式，是可视化线形图符的抽象，可由不同的语义实例化为具备不同含义和表现形式的关系元模型；属性元元模型提供用于支持其他元素实例化的条件，可被定义为通用或特有的属性元模型；角色元元模型表示关系的两端，目的通过实例化适配关系元模型完善关系的表达；点元元模型定义一组或一类端口，目的是实例化配合对象元模型实现输入和输出，并需关注端口的位置特性；扩展元元模型表示一种约束的集合，遵循一定的语法以约束六种元素之间的关联关系。通过元元模

型，对建模要素进行顶层的抽象，为元模型的构建提供支持。

3.4.3 M2 层 GOPPRR-E 元模型

M2 层元模型实现了对元元模型元素的集合，是建模语言的构建过程，如图 3.6 所示，展示了 GOPPRR-E 元模型之间的关系^[128]。

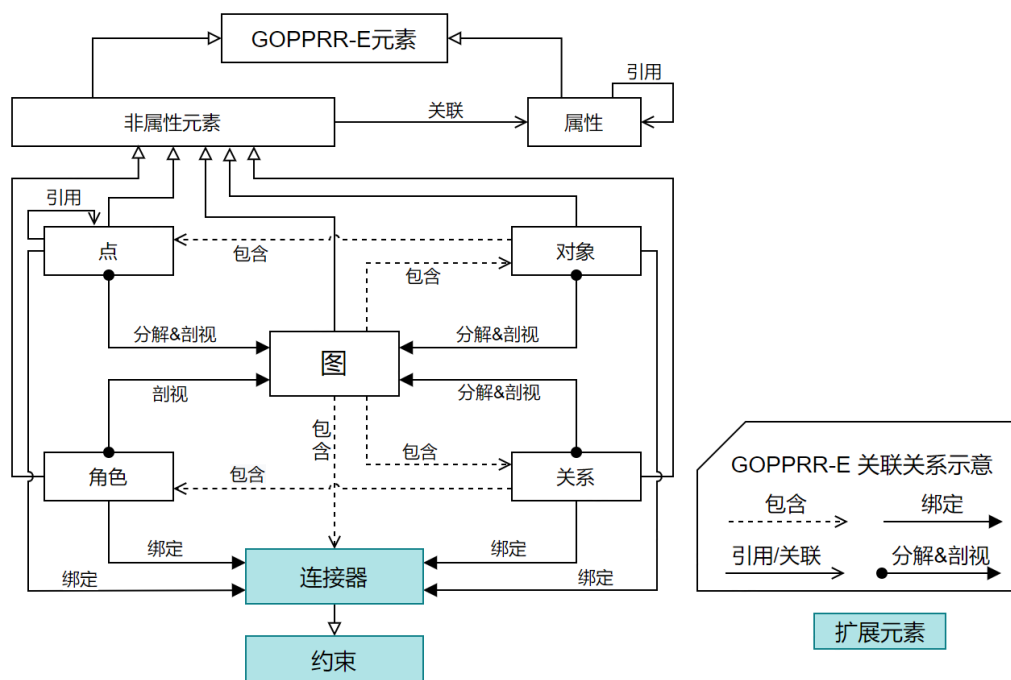


图 3.6 GOPPRR-E 元模型

元模型包含非属性元模型和属性元模型两种，非属性元模型包含图、对象、点、关系、角色，通过组合描述建模语言，属性元模型则用于描述非属性元模型的特性、指标等。

(1) 图元模型 (MetaGraph)

图元模型表示模型的某一视图，是构建建模语言的过程。如图 3.6 中，图元模型包含对象和关系元模型，同时图元模型可以具有多个属性元模型。图元模型通过对对象元模型及关系元模型的最佳配置，来实现模型中对象和关系的调用。重要的是，通过在图元模型中配置对象和关系的连接关系，达到绑定约束的目的，图 3.7 中定义了两个对象元模型以及三个关系元模型。

(2) 对象元模型 (MetaObject)

对象元模型实例化对象元元模型，通过定义自身可视化的形状、颜色，及添加不同的属性和点元模型等，实现具有特定语义的模型元素。对象元模型是图元模型

中的主要组成元素，一般来说，对象元模型最终实例化建模域的概念或实体类型，且是可独立于关系模型单独存在的实体，在可视化模型中由矩形等具有语义的符号表示。如图 3.7 如中的 MetaObject1 及 MetaObject2 表示两个不同的对象元模型，MetaObject1 形状为长方形，包含了 ID、名称、体积三个属性以及绿色和黄色两个点元模型，MetaObject2 形状为圆角矩形，包含了 ID、名称、重量三个属性，两个对象通过不同的定义表示不同的语义。

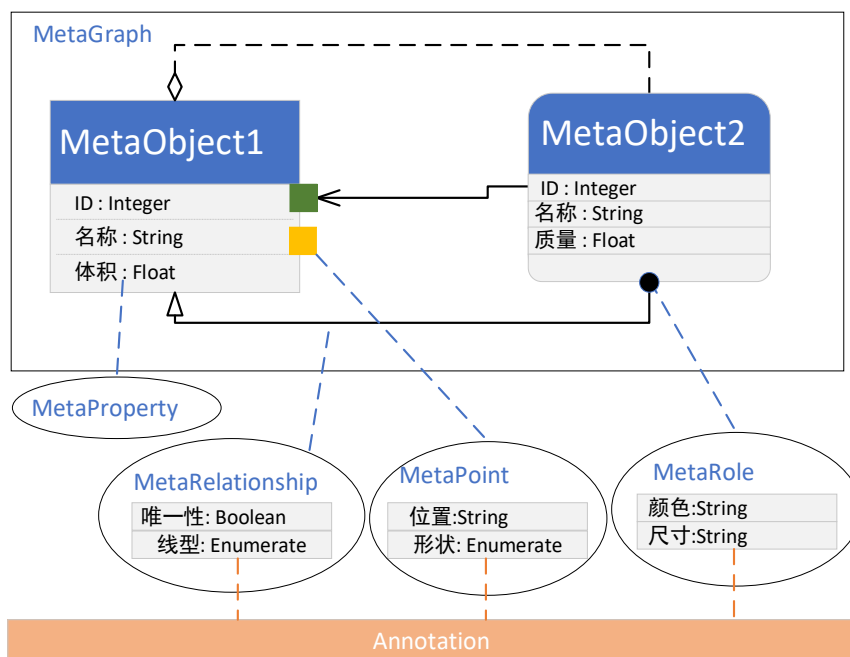


图 3.7 GOPRR-E 图元模型

(3) 关系元模型 (MetaRelationship)

关系元模型在建模域中连接对象模型，包含角色元模型和属性元模型，也是图元模型中的重要组成元素，在图 3.7 中，展示了三种关系，两种实线型和一种虚线型。关系元模型的定义中，可视化的是线型，除此之外，隐式的包含有唯一性、属性等配置。

关系元模型可以具有以下性质：

- 1) 数量：指定关系元模型中可以涉及的角色元模型的数量，值可能是两个或 n 个。
- 2) 逆：二元关系类型可以定义为另一种关系类型的逆。反向关系以相反的顺序包含原始关系的元组。
- 3) 组合：组合影响图元模型中关联在关系元模型上的对象之间的生命周期依赖

关系。一般来说，如果图元模型中容器被删除，则相关/包含的关系元模型和对象元模型也会被删除。

4) 依赖：依赖影响关系模型的生命周期。通常，关系模型的存在取决于对象模型。

5) 角色：关系元模型支持角色元模型，角色必须配置在关系中才具备使用含义。

6) 属性：关系元模型可配置多个属性元模型。

(4) 角色元模型 (MetaRole)

角色元模型定义对象元模型如何参与关系元模型，与属性元模型建立包含关系。由于元模型支持角色概念，因此关系通过角色间接关联对象，在 GOPRR-E 元素中，关系元模型的一些特征被转移到角色元模型中。在元模型中，考虑一个实用的原因：有时，不可能明确地确定关联的特定端点，这可能需要指定约束。在这种情况下，可以将角色分配给构成关联结尾的实体类型。角色只能支持关联端标识，前提是它的标识符在元模型对应实例的关联中是唯一的。在图 3.7 中，三种关系元模型配置了六种类型的角色元模型，可视化显示为不同的形状箭头，如菱型、空心箭头、普通箭头、实心圆点和无，除此之外，还可隐式的包含颜色、属性等配置。

角色元模型可以具有以下性质：

1) 多重性：这个特征取自关系元模型。多重性是在角色元模型而不是关系元模型中定义的。通过角色定义了与关系元模型中可以与另一个对象相连的对象的数量。多重性可以用具体的值/间隔来定义（例如 n 、 $0..1$ 、 $0..n$ 、 $1..n$ ）。如果不支持多重性，则默认情况下该值通常为 $0..n$ ($n \in \mathbb{N}$)。

2) 依赖：此功能类似于关系元模型。角色元模型的存在取决于关系元模型。

3) 对象集：每个角色元模型都与一个对象元模型相关。此功能支持为每一端连接对象元模型集。

4) 属性：角色元模型可配置多个属性元模型。

(5) 点元模型 (MetaPoint)

点元模型实例化点元模型端口，与属性元模型建立包含关系，在图元模型中允许匹配对象通过角色连接的附加语义或约束，限定某个对象可以被连接的关系的角色元模型。在图 3.7 中，点配置在 MetaObject1 中，通过颜色和形状展示不同可视化，图中配置了黄色和绿色两种端口。点元模型在建模语言中非必要，但点的存在必须配置在对象元模型之中，点元模型除了定义端口位置，也定义端口类型，控制

对象的输入输出。

(6) 属性元模型 (MetaProperty)

属性元模型是其他元模型语义的重要组成部分，不同的元模型指定不同数据类型的属性。如图 3.8 我们可以区分两种数据类型：常规数据类型和元模型类数据类型。

1) 常规的数据类型，包含字符串、布尔类型、日期、时间、整数、实数等。2) 其他元模型元素，如对象数据类型、图数据类型等。在图 3.7 中，ID 属性的数据类型为整数 (Integer)，名称属性的数据类型为字符串 (String)，体积属性的数据类型为浮点型 (Float)，在建模域，属性模型可以包含具体值。属性元模型不能单独存在，必须配置在其他五种元模型上才有意义。

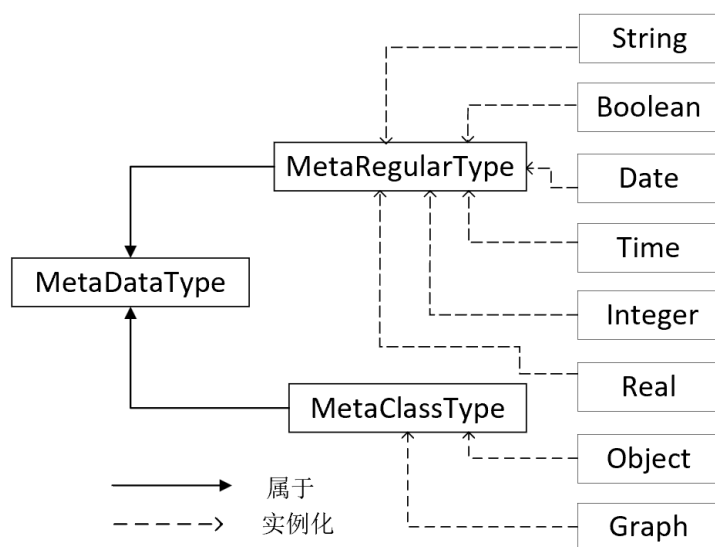


图 3.8 元数据类型

属性元模型可以具有以下性质：

- 1) 依赖：属性依赖于元模型中的其他元素。如果一个属性独立于其他元模型，那么它们可以在其他元模型中重用。
- 2) 多重性：如果多重性大于 1，这个特性允许定义多值属性。
- 3) 唯一性：如果多重性大于 1 且属性被标记为唯一，则任何值都不能出现多次。
- 4) 默认值：如果元模型元素被实例化，则允许定义默认值。
- 5) 可归属性：指定可归属的元模型元素，如对象、关系、端口、角色或模型类型。

(7) 扩展元元模型 (MetaExtension)

GOPRR-E 元模型作为构建领域建模语言，遵循一定的语法以约束六种元素之

间的关联关系，在 GOPRR-E 元素中，这些规则和约束是通过扩充概念描述的，服务于元实体实例之间关联的规范。扩展的衍生概念主要有如表 3.5 所示几种^[130]。

表 3.5 GOPRR-E 元元模型的扩展元素

关联规则	描述
包含	描述元模型之间的存在的组合关系，如图可以包含对象、关系，对象可以包含点。
绑定	描述对象、点、角色、关系元模型之间存在的约束方式，通过绑定的方式可以定义元建模语言的规则。其中，一个对象的端点对应多个角色，每个角色至少对应一个对象的端点，每个关系绑定两个角色。
分解	约束一个对象（或关系、点）元模型在建模域可以分解为一个图实例。
剖视	约束一个对象（或关系、点、角色）元模型在建模域可以从不同视角映射为多个图实例。
引用/关联	定义点或属性元模型的引用，以及对象、关系等与属性的连接。

3.4.4 M1 层 GOPRR-E 模型

M1 模型层是对元模型层的实例化，继承元模型的所有配置和约束，进行基于元模型的建模，如图 3.9 所示，ObjectInstance1 是 MetaObject1 的实例化，除继承元模型的形状、颜色等固有特征外，对于属性进行了赋值，ID 的数据类型为整数，实例化为 129，名称的数据类型为字符串，实例化为“摩托车油箱”，体积的数据类型为浮点型，实例化为 10.5，其单位隐式表达，ObjectInstance1 的端口 PointInstance1 为油箱的出口，ObjectInstance2 表示“发动机”模块其 ID 为 1294473，名称为“发动机”，重量为 20.5，通过 PointInstance1 与 PointInstance1 建立关联关系。

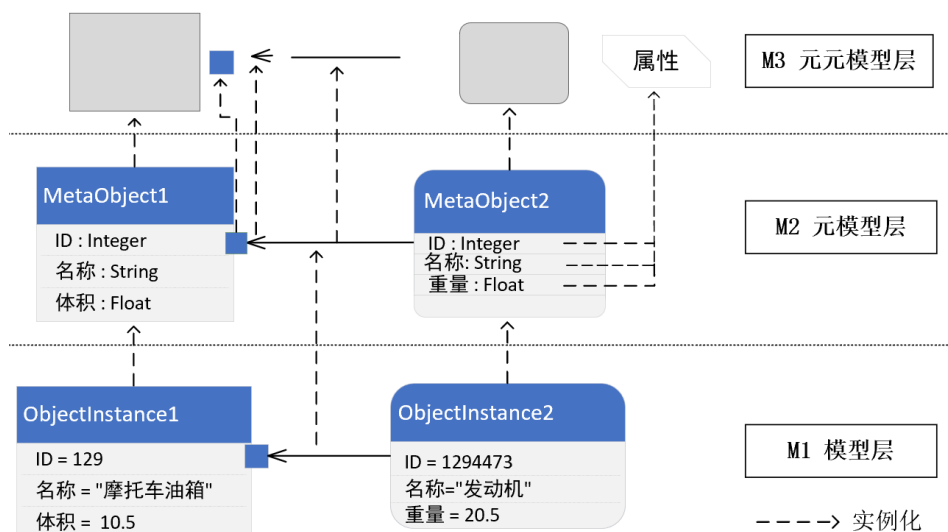


图 3.9 三层实例化示例

M1 模型层模型具备的性质如下：

- (1) 图元模型实例化为模型，是其他模型内容的集合。
- (2) 对象元模型实例化为对象模型，点元模型实例化为对象模型配置的点模型，点模型不能够单独存在，必须通过配置在对象模型的形式存在。
- (3) 关系元模型实例化为关系模型，关系模型依赖于对象模型，当对象模型数量 $n \leq 1$ 时，关系模型的数量 $m = 0$ ；当对象模型数量 $n \geq 2$ 时，关系模型的数量 $m \geq 0$ ；角色元模型实例化为关系模型配置的角色模型，角色模型不能够单独存在，必须通过配置在关系模型的形式存在，且角色模型的数量 $r = 2$ 。
- (4) 属性元模型实例化为属性模型，属性模型依赖于其他模型存在。

扩展约束如分解、剖视、绑定等根据元模型的定义，约束模型的建模过程。分层抽象模型表达能力的验证将在 6.3 节开展。

3.5 本章小结

鉴于 MOF 基本框架中 M3 元元模型层表达的不足，本章首先分析现有的元元模型，确认了使用 GOPPRR 元元模型作为本文语言框架的 M3 层，并结合系统工程实践扩展为 GOPPRR-E 元元模型；然后通过结合 GOPPRR-E 元元模型与 MOF 框架形成统一建模语言框架，并在 M3-M1 层框架中定义一组新的抽象模型，表达各层的建模元素以及使用这些元素的规则。该分层抽象模型是定义一个复杂系统架构模型建模语言形式化表征的基础。

第 4 章 基于图论有向图的抽象模型统一形式化表征

4.1 引言

装备复杂系统模型的精确度、一致性和完整性是模型的“质量标准”，该标准在数学界有一个明确的、普遍认可的解释。为了使数学解释的优势可用于建模领域，须用数学模型表示系统模型，即构建建模语言的形式化语义。基于研究现状对建模语言的形式化表征方法的调研，更多分析认为图（如 UML 类图）的自然解释是基于图论有向图的——本质上就是只有连接边的节点。因此，本章开展基于图论有向图的形式化研究，通过对基于 GOPPRR-E 的各层抽象模型的元素及逻辑关系的研究，用数学表达式来构建各层级抽象模型，为系统建模提供更清晰的理论定义，形成一套支持多视图统一建模语言构建的形式化表征模型。

4.2 图论有向图基本概念介绍

建模语言如 UML，是一种视图语言，它的“句子”是图表，但任何图表的主要部分仍然是文本，因此需要在图表中约定可视化文本，任何一段文字都是来自某个字母表的一串字母，所以本文使用 *String* 来可视化需要表达的字符串。

为了完整的使用图论符号表达，首先定义唯一标识符、数据属性和代数的基本形式概念。

定义 1 唯一标识符

唯一标识符是一个元组：

$$ID = \langle Str \cup Under \cup Num \rangle \quad (4.1)$$

其中：

- *Str* 是一组无空格字符串， $Str \subseteq String, \forall str \notin Spa$ ，*Spa* 表示空格；
- *Under* 是一组下划线；
- *Num* 是一组数字。

在建模语言中，唯一标识符是对不同实体在所有空间和时间上被视为唯一的标识，通过唯一标识符保证实体的存在和可查询。

本文的核心概念之一是图。因此接下来定义一个有向多排序的图。

定义 2 图

图是一个元组：

$$G = \langle Node, Edge, src, tgt \rangle \quad (4.2)$$

其中：

- $Node$ 是一组节点的组合， $Edge$ 是一组有向边的集合， $src: Node \rightarrow Edge, tgt: Edge \rightarrow Node$ ，分别表示源函数和目标函数^[131]；
- 图 G 中一个连续序列 $e_1 \dots e_n \in Edge$ ，表示路径中的连续边在图中连接，例如，当 $1 \leq i < n$ 时， $tgt(e_i) = src(e_{i+1})$ ；
- 如果 G 中不存在连续序列 $e_1 \dots e_n$ ，则一组边 $E \in Edge$ 是非循环的，这里当 $e_i \in E, 1 \leq i \leq n$ 时， $tgt(e_n) = src(e_1)$ ，表示该边无向。

需要注意的是，虽然这个定义还没有指定节点或边的标签，但节点和边可进行标识。在某些情况下，存在 $Node, Edge \subseteq ID$ ， ID 是标识符的集合，这些标识符对于研究来说是有意义的；在模型图中，将标识符包含在图形的可视化中作为唯一标识符。

4.3 元模型形式化表征

4.3.1 基于图论有向图的元模型层类型图

图论中图的语义概念表达了节点和边的一种基本组合，为将图论应用于建模语言，实现结构化描述和正确性的目的，元模型层不仅体现的是节点和边，也包括连接施加的局部约束等，以实现一些规则，对模型层产生约束。因此在图概念的基础上本节定义元模型层类型图。

定义 3 元模型层类型图

元模型层类型图是一个元组：

$$MMTG = \langle NType, EType, src, tgt, ext \rangle \quad (4.3)$$

其中：

- $NType \subseteq ID$ 是一组由唯一标识符确认身份的节点类型， $EType \subseteq ID$ 是一组由唯一标识符确认身份的边类型；
- $\langle NType, EType, src, tgt \rangle$ 表示一个图 G ， $NType$ 是节点集合， $EType$ 是边集合，对于任意的 $e \in EType, src(e) = id(e)$ ；
- $ext \subseteq NType \times NType$ 是一种自反偏序关系，表示某些节点类型从其他节点

类型继承，这里的自反性 $T \text{ ext } T$ 适用于所有节点类型 $T \in EType$ ， T 是根标识符。

通常使用大写字母(T, E)来表示节点类型和边类型。使用 $fer \in ext$ 来表示扩展。定义 $X \text{ fer } Y$ ，指当且仅当 (1) $X \text{ ext } Y$, (2) 不存在 Z ，使 $X \text{ ext } Z \text{ ext } Y$ ，其中 $Z \notin \{X, Y\}$ 。形式上， fer 是 ext 内部的直接前导关系，也就是生成 ext 的最小关系。

定义 3 中，边的源函数的条件表明，边的源类型同时是它的标识符，边类型由其源类型或标识符唯一确定。这种设置允许在图中使用具有相同名称的边类型，仅使标识符保持不同。

此外，由于 ext 是偏序的，这种排序不需要面向全部节点：这意味着节点类型可以扩展多个其他节点类型，即在常用术语中，元模型层类型图支持多重继承，如元素层次间和元素之间的继承。同时， ext 的偏序性质意味着不能存在继承循环。

元模型层类型图的可视化表示可以通过以下方式给出：将每个节点类型绘制为带有类型标识符的框，将每个边类型绘制为带有边名的箭头，将每个继承关系绘制为未标记的三角形箭头。图 4.1 显示了一个元模型层类型图示例，它与传统的类图视图非常接近。

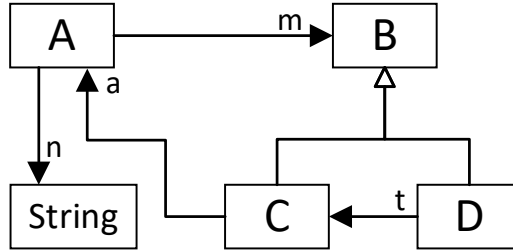


图 4.1 元模型层类型图

4.3.2 元模型形式化表征

(1) 图元模型形式化表征

根据图论的定义，元模型层类型图是一个元组 $MMTG = \langle NType, Etype, src, tgt, ext \rangle$ ，根据抽象元元模型和抽象元模型的描述，将语义表达式进行扩展定义。则图元模型是一个元组：

$$V_{mm}^{iv} = \langle Mtype, V_{mObject}, V_{mPoint}, V_{mRole}, E_{mRelationship}, P_{mProperty}, A_{mm} \rangle \quad (4.4)$$

因为在 GOPRR-E 中定义了角色，因此函数 $src: Node \rightarrow Edge$ 及函数 $tgt: Edge \rightarrow Node$ 作为 V_{mRole} 及 E_{mRela} 中存在的一种关系隐式的表达：

- $iv \in ID$ 是为每一个模型赋予唯一的标识符;
- mm 表示该元组是元模型;
- V_{mm} 表示图元模型, 是多个其他函数的集合;
- $Mtype$ 表示元模型的类型, 如构建 SysML 的需求图, $Mtype \rightarrow Requirement$;
- $V_{mObject}$ 、 V_{mPoint} 、 $V_{mRole} \subseteq NType$, 是图论中的节点, 如图 4.2 所示, 通过不同的颜色分别表示对象元模型节点、点元模型节点、角色元模型节点, 图中 * 表示节点数量可以是多个;
- $E_{mRelationship} \subseteq EType$, 是图论中的边, 表示作为边的关系元模型, 对于 $r \in V_{mRole}, \forall e \in E_{mRela}$, 使 $src(e) = r$;
- $P_{mProperty}$ 表示属性元模型, 如图 4.2 中的节点和边都可以包含属性;
- A_{mm} 表示注释, 在元模型中隐式的包含一些辅助描述信息, 如描述、形状等, 需增加元模型的表达, 以完善形式化表征。 A_{mm} 动态可调整, 根据不同的元模型元素, 所包含的辅助语义不同, 图元模型中 $A_{mm} = \langle lab, description \rangle$, 其中的 $description$ 表示对该元模型进行解释的一个字符串。

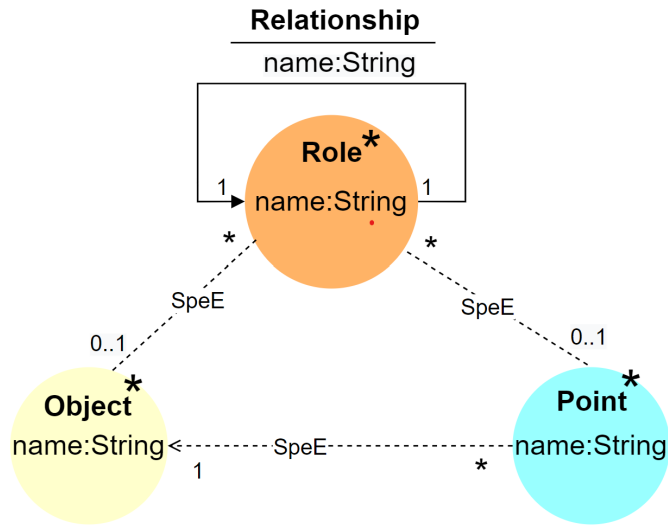


图 4.2 图元模型的表征组成

(2) 属性元模型形式化表征

基于 GOPRR-E 抽象模型的 M2 层属性元模型, 将属性元模型定义为一种特殊的节点, 每个属性元模型是由属性类型和注释组成, 属性元模型形式化表征为:

$$P_{mProperty}^{ip} = \langle Type, A_{mm} \rangle \quad (4.5)$$

根据图论定义, 属性元模型可通过边关联其他节点, 但该边不能具有任何的属性:

- $ip \in ID$ 是为每一个属性赋予唯一的标识符;
- $mProperty$ 表示该键值对是属性元模型;
- $P_{mProperty}$ 表示属性元模型的集合, 且 $P_{mProperty}^{ip} \in P_{mProperty}$;
- $Type$ 表示 3.4.3 章节中属性元模型的数据类型, 如 `String`、`Integer`、`boolean` 等;
- $A_{mm} = \langle lab, unit, description \rangle$, 可标识属性的标签, 单位及解释等。

在建模语言的使用中往往不直接使用属性元模型, 而是使用属性元模型的继承, 以此保持属性元模型的可重用性。因此定义属性元模型引用为一个元组 $P_{mProExt}^{ipe} = \langle value, ext, A_{mm} \rangle$, 其中:

- $\forall p' \in P_{mProExt}^{ipe}, \exists p \in P_{mProperty}^{ip} \Rightarrow p' ext p$;
- 其中, $ipe \in ID$, 是为每一个属性元模型引用赋予唯一的标识符;
- $mProExt$ 是引用属性元模型的标识;
- $value$, 是对属性赋初始值, 且对于 $n \in P_{mProExt}^{ipe}, lab(n) \in P_{mProperty}^{ip}$;
- $A_{mm} = \langle lab \rangle$ 。

(3) 点元模型形式化表征

基于 GOPRR-E 抽象模型的 M2 层点元模型, 点元模型表示一类端口, 是一种可单独定义, 但不能够单独存在的节点。因此点元模型形式化表征为一个元组:

$$V_{mPoint}^{it} = \langle P_{mProExt}, A_{mm} \rangle \quad (4.6)$$

其中:

- $mPoint$ 是点元模型的标识;
- $P_{mProExt}$ 是点元模型引用的属性元模型;
- $P_{mProExt} ext P_{mProperty}$, 点元模型中的属性来自于属性元模型的继承;
- $it \in ID$ 是为每一个点元模型赋予唯一的标识符;
- $A_{mm} = \langle lab, description, shape, image \rangle$, 其中, $shape$ 表示端口可能展现的形状, $image$ 表示端口可能替换的展示图片。

在图 4.2 中, 点元模型通过 `SpeE` 边与对象进行绑定, `SpeE` 边是一种特殊的绑定关系, 且点节点与对象节点是一对多的绑定关系, 表明点在对象中才有含义。在建模语言中往往是对对象配置端口的形式展示语义, 因此使用点元模型的继承。点元模型的继承是一个元组:

$$V_{mPoExt}^{ite} = \langle PType, ext, A_{mm} \rangle \quad (4.7)$$

其中：

- $ite \in ID$ ，是为每一个点元模型引用赋予唯一的标识符；
- $mPoExt$ 是点元模型引用的标识；
- $\forall a \in V_{mPoExt}^{ite}, \exists b \in V_{mPoint}^{ite} \Rightarrow a \text{ ext } b$ ；
- $A_{mm} = \langle lab \rangle$ 。

(4) 角色元模型形式化表征

角色元模型控制关系的流入流出，并具有标识关系的作用，角色在关系（边）中的隐式表达，根据 3.4.3 节基于 GOPRR-E 抽象元模型中角色的定义，本节将角色元模型形式化表征为具有属性的一个元组：

$$V_{mRole}^{ir} = \langle RType, P_{mProExt}, A_{mm} \rangle \quad (4.8)$$

其中：

- $ir \in ID$ 是为每一个角色元模型赋予唯一的标识符；
- $mRole$ 是角色元模型的标识；
- V_{mRole} 表示角色元模型的集合；
- $RType$ 是角色的输入或输出方向， $RType = Import | Export$ ；
- $A_{mm} = \langle lab, description, shape, Mult \rangle$ 。

同样的，角色元模型在关系中才有含义，在建模语言中以关系配置角色的形式展示语义，但在关系中配置的角色已被定义，因此使用角色元模型的引用。

(5) 关系元模型形式化表征

基于 GOPRR-E 抽象模型的 M2 层关系元模型，该关系在建模域中连接对象类型，在图论中包含角色元模型的关系元模型表示为边，因此其形式化表征是对边 $Etype$ 的扩展，本文中关系元模型形式化表征为一个元组：

$$E_{mRelationship}^{ie} = \langle EType, P_{mProExt}, V_{mRole}, A_{mm} \rangle \quad (4.9)$$

其中：

- $ie \in ID$ 是为每一个关系元模型赋予唯一的标识符；
- $mRelationship$ 是关系元模型的标识；
- $E_{mRelationship}$ 表示关系元模型的集合；
- $EType$ 表示关系的类型，如 $EType \rightarrow include$ 表示包含关系；
- 对于任意的 $e \in E_{mRelationship}$ ， $src(e) = V_{mRole}(e).Import, tgt(e) = V_{mRole}(e).Export$ ；

- $A_{mm} = \langle ix, lab, description, shape, icon \rangle$, $ix: Edge \rightarrow IN$ 是为每条边赋值一个自然数的索引函数；将索引函数 ix 放在集合 A_{mm} 中，与其他函数共同形成关系元模型的函数集，函数 $icon$ 标识可视化的关系元模型。

(6) 对象元模型形式化表征

基于 GOPRR-E 抽象模型的 M2 层对象元模型，是对一组或一类具有相同特征的节点类型的抽象，图论中对象元模型是对节点 $NType$ 的扩展，并通过配置不同的属性表达不同的语义，因此包含了属性的引用，对象元模型形式化表征为一个元组：

$$V_{mObject}^{io} = \langle OType, P_{mProExt}, V_{mPoExt}, A_{mm} \rangle \quad (4.10)$$

其中：

- $io \in ID$ 是为每一个对象元模型赋予唯一的标识符；
- $mObject$ 是对象元模型的标识；
- $V_{mObject}$ 表示对象元模型的集合；
- $OType$ 表示对象的类型，如 $OType \rightarrow block$ 表示该对象元模型构建了对象—模块；
- V_{mPoExt} 是点继承节点的集合；
- $A_{mm} = \langle lab, description, icon, shape, mode \rangle$ ，其中 $mode$ 表示对象的形式化模式，如容器模式。

表 4.1 总结了元模型的形式化表征模型。

表 4.1 元模型形式化表征模型

GOPRR 元素	形式化表征模型
图元模型	$V_{mm}^{iv} = \langle Mtype, V_{mObject}, V_{mPoint}, V_{mRole}, E_{mRelationship}, P_{mProperty}, A_{mm} \rangle$
属性元模型	$P_{mProperty}^{ip} = \langle Type, A_{mm} \rangle$
点元模型	$V_{mPoint}^{it} = \langle P_{mProExt}, A_{mm} \rangle$
角色元模型	$V_{mRole}^{ir} = \langle RType, P_{mProExt}, A_{mm} \rangle$
关系元模型	$E_{mRelationship}^{ie} = \langle EType, P_{mProExt}, V_{mRole}, A_{mm} \rangle$
对象元模型	$V_{mObject}^{io} = \langle OType, P_{mProExt}, V_{mPoExt}, A_{mm} \rangle$

4.3.3 元模型形式化表征示例

如图 4.3 展示了模块定义图的元模型表征。

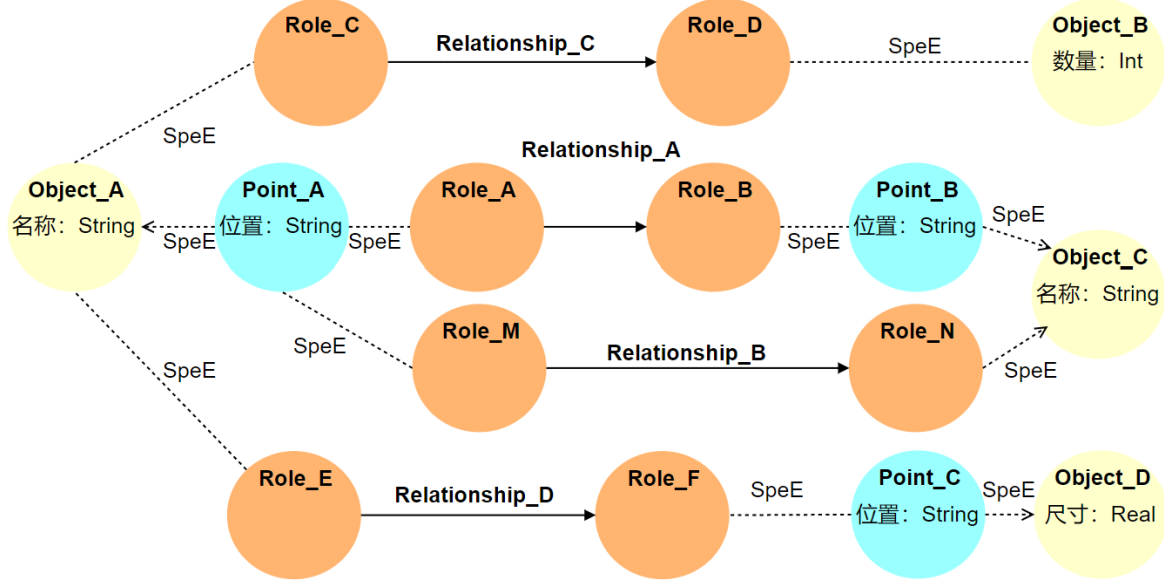


图 4.3 基于图论有向图的元模型层类型图示例

$V_{mm}^{iv} = \langle BDD, V_{mObject}, V_{mPoint}, V_{mRole}, E_{mRelationship}, P_{mProperty}, A_{mm} \rangle$, 其中, BDD 指模块定义图, 对象集合 $V_{mObject} = \{V_{mObject}^{Object_A}, V_{mObject}^{Object_B}, V_{mObject}^{Object_C}, V_{mObject}^{Object_D}\}$, 关系集合 $E_{mRelationship} = \{E_{mRelationship}^{Relationship_A}, E_{mRelationship}^{Relationship_B}, E_{mRelationship}^{Relationship_C}, E_{mRelationship}^{Relationship_D}\}$, 角色集合 $V_{mRole} = \{V_{mRole}^{Role_A}, V_{mRole}^{Role_B}, V_{mRole}^{Role_C}, V_{mRole}^{Role_D}, V_{mRole}^{Role_E}, V_{mRole}^{Role_F}, V_{mRole}^{Role_M}, V_{mRole}^{Role_N}\}$, 点的集合 $V_{mPoint} = \{V_{mPoint}^{Point_A}, V_{mPoint}^{Point_B}, V_{mPoint}^{Point_C}\}$ 。

将对象元模型 Object_A 进行形式化展开, $V_{mObject}^{Object_A} = \langle block, P_{mProExt}, V_{mPoExt}, A_{mm} \rangle$, 其中 $P_{mProExt}^{ipe1} = \langle String, \text{“名称”} \rangle \in P_{mProExt}$, 这里点依赖对象存在, 如 $\forall P \in V_{mObject}^{Object_A} \cdot V_{mPoExt}$, 有 $P \text{ ext } V_{mPoint}^{Point_A}$, $A_{mm} = \langle \text{“对象 A”}, \text{“对象 A 的描述”}, \text{“/image/icon1”}, rectangle, propertyMode \rangle$, 表示该对象标签为“对象 A”, 可添加描述为“对象 A 的描述”, 可视化图标地址为“/image/icon1”, *rectangle* 表示对象的可视化形状为矩形, *propertyMode* 表示对象模式为属性模式。

将关系元模型 Relationship_A 进行形式化展开, $E_{mRelationship}^{Relationship_A} = \langle Association, P_{mProExt}, V_{mRole}, A_{mm} \rangle$, 其中 $P_{mProExt}^{ipe2} = \langle String, \text{“名字”} \rangle \in P_{mProExt}$, 且角色与关系存在依赖关系, 其中 $V_{mRole} = \{V_{mRole}^{Role_A}, V_{mRole}^{Role_B}\}$, $V_{mRole}^{Role_A} = sre(E_{mRelationship}^{Relationship_A})$, $V_{mRole}^{Role_B} = tgt(E_{mRelationship}^{Relationship_A})$, 这里的 $A_{mm} = \langle ix, \text{“关联边”}, \text{“表示关联关系”}, straight, \text{“/image/icon2”} \rangle$, *ix* 表示自然数, 边的标签为“关联边”, 可添加描述“表示关联关系”, *straight* 表示类型为直线, 可视化图标地址为“/image/icon2”。

将点元模型 $Poiny_A$ 进行形式化展开, $V_{mPoint}^{Point_A} = \langle P_{mProExt}, A_{mm} \rangle$, 其中 $P_{mProExt}^{ipe3} = \langle String, \text{“位置”} \rangle \in P_{mProExt}$, $A_{mm} = \langle \text{“端口”} \rangle$ 。

将角色元模型 $Role_A$ 进行形式化展开, $V_{mRole}^{Role_A} = \langle Import, P_{mProExt}, A_{mm} \rangle$, $Import$ 表示该角色是输入端角色, $P_{mProExt}^{ipe4} = \langle null, null \rangle$, 表示未对角色配置属性信息, $A_{mm} = \langle \text{“输入角色”}, \text{“输入端角色描述”}, None, Mult \rangle$, 角色的标签为“输入角色”, 可添加描述“输入端角色描述”, “None”表示角色端类型为空无, “Mult”为自然数, 表示该角色的多重性。

通过以上描述, 基于 GOPRR-E 的图形式化表征了抽象元模型的各个元素以及元素之间的关系, 表明形式化表征模型在理论上支持建模语言的表达。

4.4 模型形式化表征

4.4.1 模型层实例图

定义 4 模型层实例图

在图论中, 模型层实例图是一个元组:

$$MIG = \langle G, ix, lab \rangle \quad (4.11)$$

其中:

- G 是一个图, $G = \langle Node, Edge, src, tgt \rangle$;
- $ix: Edge \rightarrow IN$ 是为每条边赋值一个自然数的索引函数;
- $lab: (Node \cup Edge) \rightarrow name$ 是为节点和边赋予标签的函数;
- 对于 $e_1, e_2 \in Edge$, 如果 $src(e_1) = src(e_2), ix(e_1) = ix(e_2), lab(e_1) = lab(e_2)$, 则 $e_1 = e_2$;
- 对于一个给定的节点 $n \in Node$, 标识符 $id \in ID$, 则出方向边的集合表示为 $out(n, id) = \{e \in Edge \mid tgt(e) = n, lab(e) = id\}$, 入方向边的集合表示为 $in(n, id) = \{e \in Edge \mid src(e) = n, lab(e) = id\}$ 。

函数 ix 表示的索引有以下两个用途:

(1) 区分边。图中可能存在从同一个节点出的不同的边, 甚至是这些边到达同一个节点, 称为平行边。本文通过索引显示的区分这些边, 并通过赋值赋予其各自的身份。事实上, 从定义中可以看出, 一条边的确定完全是由它的源节点、索引和标签决定的。

(2) 排序边。建模语言概念之一是有序关联，不仅定义了一种类型的对象与另一种类型的对象之间的一对多关系，而且还建立了与单个源对象相关的目标对象集的局部排序。

与边相比，节点的建立不适用上述定义。但应该注意到，节点除了标记功能之外，还需要存在一些区分机制。例如，在现实层面上，这种机制通常基于内存地址，或者对于 $Node \cap Data$ 中带有数据值的节点，可基于数据值；在建模层面上，可通过图中节点的位置作为区分机制。另一方面，为了便于直观的查看，通常为节点使用符号名称。

因此，我们基于以下约定得出了模型层实例图的图形表示：

(1) 节点被绘制为带有标签的框。在框中可添加唯一标识符，但它对图形的形式意义不起任何作用，实际其在定义 4 中也没有对应表达式；或者，在节点包含数据值的情况下，可以用显示数据值的字符串表示，我们在下面看到的标签是典型类型，对于数据值 $v \in Data$ 由 $type(v)$ 隐式给出。

如果一个节点有一个符号名称或一个表示其身份的数据值，那么可以让这个节点在图形表示中多次出现：即“克隆”表达，这些多次出现仍然代表同一个没有歧义

的节点。

(2) 边被绘制为带有叠加标签的箭头。标签前面可以有一个代表边缘索引的数字，用冒号与标签分隔；特别的，如果有多个带有该标签的边可通过编号来确定排序。

(3) 作为一种特殊情况，通过数据值或符号名称明确标识指向节点的边，可以用“ $label = id$ ”或“ $label: Type = id$ ”形式的内接方程而不是箭头来表示。其中， $label$ 是边标签， $Type$ 是目标节点的标签， id 是目标节点标识符。在任何一种情况下， id 都可唯一标识图的一个节点。

这种特殊的表示边的等式仅在存在带有给定标签的单一输出边时使用，即没有清晰的 e_1, e_2 ，使 $src(e_1) = src(e_2)$ ， $lab(e_1) = lab(e_2)$ ，此时不需要在方程中包含边索引。

模型层实例图用于表示具体系统，如 UML 图中用在单个程序图或对象图的级别上。图 4.4 给出了一个显示所有图形表示特征的示例。这里， y 和是在图中没有正式含义的标识符，而 9 和“hi”分别是 Int 和 $String$ 类型的数据值，22、55 等是边索引。该图表示的图形形式化内含由表 4.2 所示，定义了节点及其标签和标识符。需注意的

是，这里的节点标识 n1 到 n4 不是图形表示的一部分，节点不由其唯一确定，它们可以随机选择或动态分配。节点标识 9 和 “hi” 是数据值，由图表固定，节点标签可始终显示在图表中。

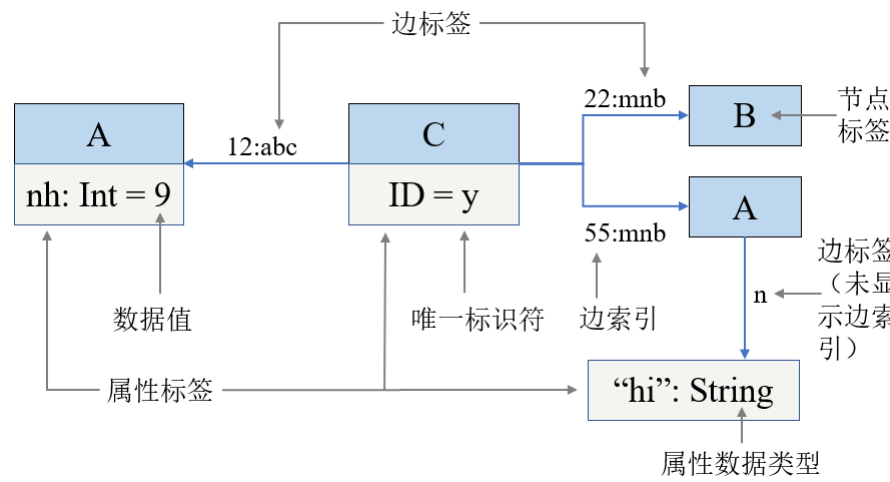


图 4.4 模型层实例图的示例图形表示

表 4.2 节点

Node	lab	id
n ₁	A	
n ₂	C	
n ₃	B	y
n ₄	A	
“hi”	String	

表 4.3 通过将每条边列为源节点、标签和索引的三元组来定义边。通过定义 4 的第三条，该三元组确定唯一地标识每条边。源节点和标签始终是图形表示的一部分；另一方面，可以省略边索引。该表还列出了定义 4 中边所需的函数。显然，前三条边是从边结构中选择适当的元素，而第四条边是从图中隐式的提取。

表 4.3 边统计

Edge	src	lab	ix	tgt
(n ₂ , abc, 12)	n ₂	abc	12	n ₁
(n ₂ , mn _b , 22)	n ₂	mn _b	22	n ₃
(n ₂ , mn _b , 55)	n ₂	mn _b	55	n ₄
(n ₄ , n, 1)	n ₄	n	1	“hi”

图 4.5 展示了两种不同的方式来描述同一个对象，该对象是一个具有单个字符串类型属性的非数据型节点，第一种表示法与常见可视化表示一致，在实践中更多使用，第二种表示法清晰的将属性分离，其数据值在形式上可作为一种图节点。

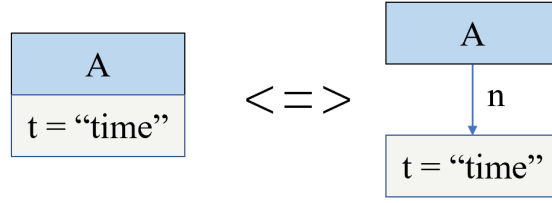


图 4.5 模型层实例图中属性的不同表示

4.4.2 模型形式化表征

根据基于 M1 层 GOPRR-E 抽象模型的定义、图论中模型层实例图，以及元模型的形式化表征，定义架构模型的形式化表征为一个元组：

$$M_{modIns}^j = \langle Mtype, V_{object}, E_{rela}, P_{property}, A_m, ix, lab, ref \rangle \quad (4.12)$$

其中：

- $j \in ID$ 是为每一个模型赋予唯一的标识符；
- $modIns$ 表明该元组是一个架构模型；
- 对于任意的 $j \in ID$, $\exists iv \in ID$, 使得 $M_{modIns}^j \text{ instance } V_{mm}^{iv}$, 对于任意一个实例化的架构模型，都是对一个对应图元模型的继承；
- $Mtype$ 表示架构模型的类型；
- ref 表示存在继承关系；
- $V_{object} \subseteq V_{mObject}$, 是对象模型表示的节点集合，表示在一个模型中会有多个对象， $V_{mObject}$ 是 V_{object} 的超类；对象模型中的 A_m 扩展了 A_{mm} , $A_m = \langle A_{mm}, location, color, text, size \rangle$, 可视化模型中需要元组 $\langle location, color, text, size \rangle$ 对视图中的对象位置、颜色、文字、大小等进行标识；
- 对于任意一个对象模型 V_{object}^{jo} , 可存在点模型 $V_{point} \subseteq V_{object}^{jo}$, 对于 $\forall n \in V_{object}$, $\exists m \in V_{mObject} \Rightarrow n \text{ ref } m, n.V_{point} \text{ ref } m.V_{mProExt}$, 点模型中的 $A_m = \langle A_{mm}, location, color, text \rangle$;
- $E_{rela} \subseteq E_{mRelationship}$, 是关系模型表示的边集合，表示在一个模型中会有多个关系， $E_{mRelationship}$ 是 E_{rela} 的超类；关系模型中的 A_m 扩展了 A_{mm} , $A_m = \langle A_{mm}, location, text, jump, smooth, winding \rangle$, 其中的元组 $\langle location, text, jump, smooth, winding \rangle$ 对视图中关系的位置、文字、跳线、平滑度、绕线等进行标识；

- 对于任意一个关系模型 E_{rela}^{jr} ，可存在角色模型 $V_{role} \subseteq E_{rela}^{jr}$ ，且 $|V_{role}| = 2, V_{role}^1.Rtype = Import, V_{role}^2.Rtype = Export$ ，对于 $\forall r \in V_{role}, \exists k \in V_{mRoleExt} \Rightarrow r \text{ ref } k$ ，角色模型中的 $A_m \text{ ref } A_{mm}$ ；
- $P_{property}$ 是属性的集合，在 GOPPRR-E 元素中了，属性是包含在其他元素中，作为对其他元素的一种扩展， $P_{property} \subseteq P_{mProExt}$ ，是属性元模型引用的子集合；
- 模型信息中注释集合 $A_m \text{ ref } A_{mm}$ 。

4.4.3 模型表征示例

如图 4.6 中，GOPPRR-E 模型 $M_{modIns}^{testModel} \text{ instance } V_{mm}^{iv}$ ，是图 4.3 元模型的实例化，表征为 $M_{modIns}^{testModel} = \langle BDD, V_{object}, E_{rela}, P_{property}, A_m, null, \text{"实例图"}, \text{ref} \rangle$ ，其中，BDD 表征图 4.3 中定义的模块定义图， $V_{object} = \{V_{object_obj0}^{object_obj0}, V_{object_obj1}^{object_obj1}, V_{object_obj2}^{object_obj2}, V_{object_obj3}^{object_obj3}, V_{object_obj4}^{object_obj4}\}$ ， $E_{rela} = \{E_{rela_rela1}^{rela_rela1}, E_{rela_rela2}^{rela_rela2}, E_{rela_rela3}^{rela_rela3}, E_{rela_rela4}^{rela_rela4}\}$ ， $P_{property} = null$ ，表示未对图本身添加属性。对于对象如 $V_{object_obj0}^{object_obj0}$ ，其包含两个点， $\{V_{point_po3}^{point_po3}, V_{point_po1}^{point_po1}\} \subseteq V_{object_obj0}^{object_obj0}$ ，对于关系 $E_{rela_rela1}^{rela_rela1}$ ，只含有两个角色 $\{V_{role_ro1}^{role_ro1}, V_{role_ro2}^{role_ro2}\} \subseteq E_{rela_rela1}^{rela_rela1}$ 。

图中 b 所表达的模型层实例图，是与针对图论表征实现映射的可视化模型，图中的对象节点如 $V_{object_obj0}^{object_obj0}$ 映射为模型对象 `object_obj0`，其包含了四类信息：`Object_A` 是元模型的 ID，表明对象的继承关系 $V_{object_obj0}^{object_obj0} \text{ ref } V_{mObject}^{object_A}$ ；属性名称的赋值为“车辆”，则 $V_{object_obj0}^{object_obj0}.P_{property}^{pro1}.A_m.lab = \text{"名称"}, V_{object_obj0}^{object_obj0}.P_{property}^{pro1}.value = \text{"名称"}$ ；ID=`object_obj0` 是以显式属性展示该对象实例化之后作为实体的唯一标识符；点节点映射为名称是 P1 和 P2 的两个点，则 $V_{point_po3}^{point_po3}.A_m.lab = \text{"P1"}$ ， $V_{point_po1}^{point_po1}.A_m.lab = \text{"P2"}$ 。关系边及角色节点映射为模型关系，模型关系包含四类：泛化、依赖、包含、关联，如 $E_{rela_rela1}^{rela_rela1}.A_m.lab = \text{"泛化"}$ ，其可视化结构分别映射为一端有空心箭头的实线、一端有单箭头的虚线、一端有单箭头的虚线及无箭头的实线等。属性映射为对象、关系等显式或隐式表达的属性。

通过映射关系的完全实现，基于 GOPPRR-E 的模型图形式化表征包含了可视化模型的所有内含表达。为基于该形式化表征模型进行建模语言的建设提供了理论基础。

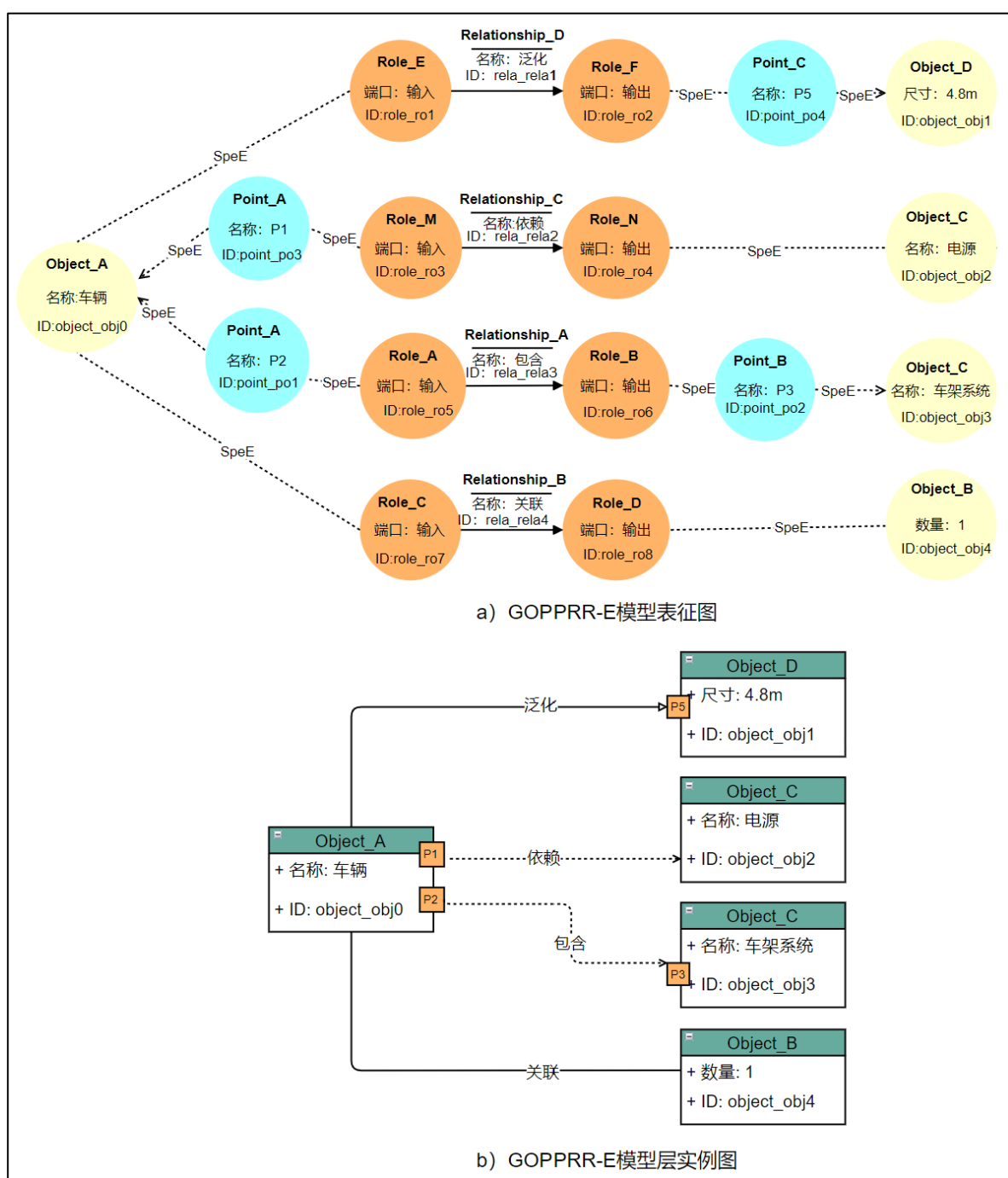


图 4.6 基于 GOPPRR-E 的图及可视化模型映射实例

4.5 扩展约束形式化表征

除了以上基于图论有向图的形式化表征的基本定义，本节介绍一种丰富元模型层类型图的方法，从而约束模型层实例。首先，本节给出图上约束集的一般定义，

然后定义了一系列针对 GOPPRR-E 元素关注的特殊类型的约束。

设 MMTG 为元模型层类型图。MMTG 上的约束集是一个元组 $\langle Con, sat \rangle$ ，其中：

- Con 是一组图约束；
- $sat \subseteq Inst[MMTG] \times Con$ ，是对 MMTG 实例的满意关系。

记 $MIG sat c$ 表示实例图 MIG 满足约束 c 。

以上图约束通过满意度关系 sat 进行了约束的图表示，但仍需进一步对 sat 进行定义。本文通过将元模型层类型图与约束集结合起来，以丰富模型的概念。

对于一个 MIG 图 G ，如果 $G \in Inst[MMTG]$ ，则对于所有 $c \in Con$ ， $G sat c$ 。本节总结有以下几种约束类型。

4.5.1 抽象性约束

一个非常普遍的约束是建模语言图中的给定类是抽象的，这意味着它可能没有直接的实例，而只能实例化其子类。对于类型和实例图而言，这意味着节点类型本身不能用作标签。

设 MMTG 是一个类型图，MMTG 的抽象性约束是一个谓词 $abstract(T)$ ，其中 $T \in NType$ 是 MMTG 中的一个节点，对所有 $G \in Inst[MMTG]$ ，定义满足性：

$$G sat abstract(T) :\Leftrightarrow \exists n \in Node_G. lab(n) = T \quad (4.13)$$

图 4.7 显示了抽象性约束的一个示例。类型图有一个抽象约束，通过在类型节点上的注释 $\{abstract\}$ 显示出来。A 不能直接进行实例化，只能关联成为 B 之后进行实例化。

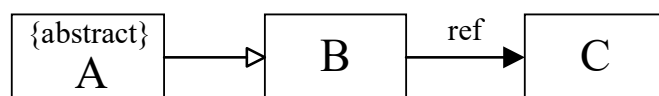


图 4.7 具有抽象节点类型的类型和实例图

在元模型中，存在属性的继承和点元模型的继承，只有 $P_{mProExt}^{ipe} ext P_{mProperty}^{ip}$ ， $V_{mPoint}^{it} ext V_{mPoExt}^{ite}$ 时，在模型中存在 $\exists n \in P_{mProperty}, P_{mProperty}^{ip}.lab(n) = V_{object}^{jo}.P_{mProExt}^{ip}.lab(n), \exists m \in V_{mPoint}, V_{mPoint}^{it}.lab(m) = V_{object}^{jo}.P_{mPoExt}^{ite}.lab(m)$ 。

4.5.2 双向性约束

在 UML 类图中，关联关系具备沿任意方向传递的属性，且关联的两端可以有自

己的名称。因此，图论中使用两条边对双向关联进行建模，每条用于其中一个方向，它们彼此相对。

设 MMTG 是一个类型图，MMTG 的双向约束表示为 $oppose(D, E)$ ， $D, E \in Etype$ 是 MMTG 中的边，并且 $src(D) = tgt(E)$ ， $src(E) = tgt(D)$ 。对所有 $G \in Inst[MMTG]$ ，定义满足性：

$$G \text{ sat } oppose(D, E) :\Leftrightarrow \forall n_1:src(D), n_2:tgt(D). |\{d \in out(n_1, D) | tgt(d) = n_2\}| = |\{e \in out(n_2, E) | tgt(e) = n_1\}| \quad (4.14)$$

图 4.8 给出了一个双向性约束的示例。元模型层类型图 (1) 有一个相关的约束 $oppose(B.c, C.b)$ ，形式化为两个箭头。图 (2) 中 “:” 表示实例化，表示图类型图 (1) 的一个有效模型实例，其中，C 和 B 实例包含了 C.b 和 B.c 两个方向类型的边。

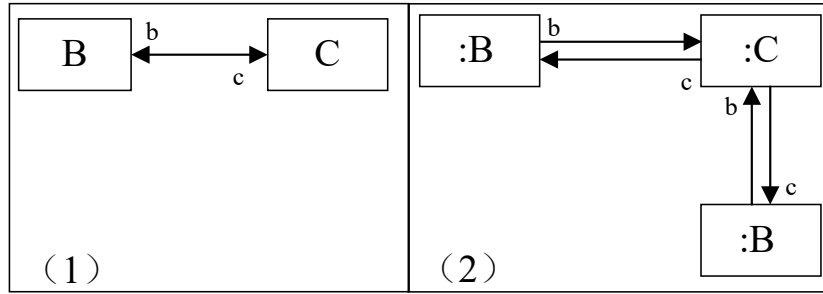


图 4.8 双向边的类型图建模

在 GOPRR-E 元素中，表现为如元模型中的角色元模型，边方向的输入与输出由角色元模型定义，当关系元模型的两端配置同一种方向的角色时候满足双向性约束，因此双向性约束表示为： $\exists e \in E_{mRelationship}, src(e) = tgt(e) = E_{mRelationship}^{ie} \cdot V_{mRole}^{ir} \cdot RType.Export$ ，即关系两端的角色都为输出角色。

4.5.3 多重性约束

多重性约束规定每个节点类型的实例允许一个关联边类型的实例的数量。在 GOPRR-E 元素中，多重性是视图的重要组成部分。为了形式化其语义，本节首先定义多重性，然后引入相应的图约束类型。

在形式上，多重性被定义为一个自然数的有序对，其中第二个（较高的）数可以取特殊值*指示无界性。如果一个数大于或等于下界，小于或等于上界，则被认为是在多重性范围内。

多重性表示为 $[i, j] \in \mathbb{N} \times (\mathbb{N} \cup \{*\})$ ，其中， $i \leq j$ 或者 $j = *$ ，多重性的集合用

$Mult$ 表示，特殊值 $*$ 表示不限制最大数目。

为了检索多重性的下界和上界，本节使用映射 $low, up: Mult \rightarrow (\mathbb{N} \cup \{*\})$ ，例如，如果 $\mu = [i, j]$ ，则有 $low(\mu) = i, up(\mu) = j$ 。对于一个任意的数字 n 和多样性 $\mu \in Mult$ ，如果 $low(\mu) \leq n$ ，且 $up(\mu) = *$ 或者 $n \leq up(\mu)$ ，则有 $n \in \mu$ 。

多重性可以附加到关系的源端和目标端；源多重性表示一个节点潜在的共享程度，而目标多重性则可简单理解为关系的多重性。

设 MMTG 是一个类型图，MMTG 中的多重性约束表示为 $mult(E, \mu)$ ，其中 $E \in EType, \mu \in Mult$ ，对所有 $G \in Inst[MMTG]$ ，定义满足性：

$$G \text{ sat } mult(E, \mu) :\Leftrightarrow \forall n: src(E). |out(n, E)| \in \mu \quad (4.15)$$

在图 4.9 中给出了一个多重约束的示例，元模型层类型图（1）有一个关联的约束 $mult(C.b, [3, 10])$ ，在箭头的头部附近写 $[3...10]$ （无中括号），图（2）是满足这个约束的实例，C类型的模型实例节点有4条C.b类型的边（即 $out(n, C.b) = 4 \in [3, 10]$ ）。

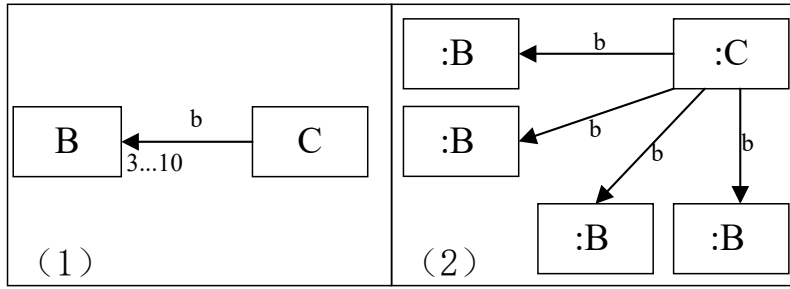


图 4.9 多重性约束示例

在 GOPRR-E 元素中，角色是关系的组成，多重性附加在角色之上，在角色元模型中多重性表示为： $V_{mRole}^{ir}.A_{mm}.Mult = [i, j]$ ， $[i, j] \in \mathbb{N} \times (\mathbb{N} \cup \{*\})$ ，其中 $i \leq j$ 或者 $j = *$ 。如 $V_{mRole}^1.Mult = [1, 2]$ ， $V_{mRole}^2.Mult = [3, 6]$ ，则在模型中，关系的源端的可重性为 $[1, 2]$ ，目标端的可重性为 $[3, 6]$ 。

4.5.4 索引约束

为了从模型图中捕捉有序的节点集合，需要形式化一组进行排序的图节点。虽然概念上很简单，但正确地捕捉到这一点实际上非常复杂。本节使用作为实例图一部分的边索引来解决：如果两个节点之间对边类型声明索引，则必须从1向上的连续范围中选择边索引的序列号。

设 MMTG 是一个类型图，MMTG 的索引约束表示为 $indexed(E)$ ， $E \in EType$ 。对所有 $G \in Inst[MMTG]$ ，定义满足性：

$$G \text{ sat indexed}(E) :\Leftrightarrow \forall n: \text{src}(E), \forall e \in \text{out}(n, E). 1 \leq ix(e) \leq |\text{out}(n, E)| \quad (4.16)$$

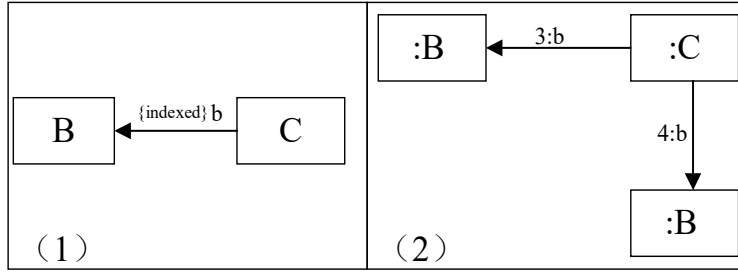


图 4.10 索引约束示例

索引约束表达有序的节点集合，当模型中需要使用索引约束对对象模型进行排序时，可定义 $\forall e \in E_{mRelationship}, E_{mRelationship}^{ie} \cdot A_{mm} \cdot ix(e) = \{a, b\}$ ，其中 $a, b \in \mathbb{N}, 1 \leq a \leq b$ 。图 4.10 是一个索引约束的例子。元模型层类型图（1）有一个相关的约束索引（C.b），由箭头附近的注释 {indexed} 进行可视化展示。图（2）满足此约束，因为两个带索引 {3, 4} 的 C.b 类型边展示了连续编号。

4.5.5 唯一性约束

一般的，在相同的元模型节点之间允许有多条相同类型的模型边实例，可通过索引来区分。但是在实际的多数情况下，模型构建者更希望确保目标节点最多出现一次，即在源节点和边类型的上下文中是唯一的。

设 MMTG 是一个类型图，MMTG 中的唯一性约束是 $unique(E, \mu)$ ，其中 $E \in EType$ 是一个边类型，对所有 $G \in Inst[MMTG]$ ，定义满足性：

$$\begin{aligned} G \text{ sat unique}(E) :\Leftrightarrow & \forall n: \text{src}(E). \forall e_1, e_2 \in \text{out}(n, E). \text{tgt}(e_1) \\ & = \text{tgt}(e_2) \Rightarrow e_1 = e_2 \end{aligned} \quad (4.17)$$

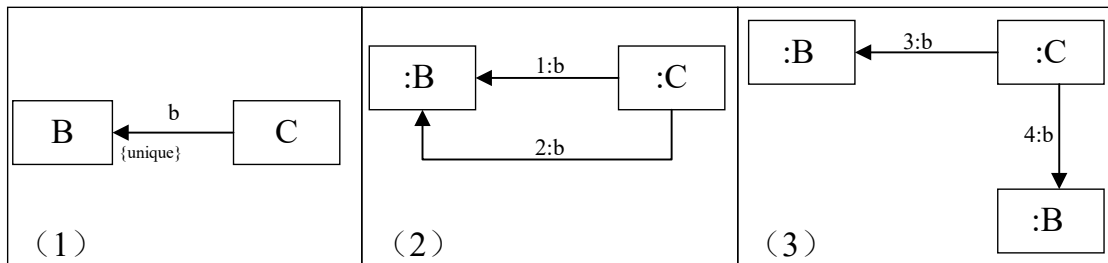


图 4.11 唯一性约束示例

在 GOPRR-E 元素中，唯一性约束表达目标节点的唯一，即输出端的角色连接唯一的对象，因此需保证 $\forall e \in E_{mRelationship}, E_{mRelationship}^{ie} \cdot A_{mm} \cdot ix(e) = 1$ ，图 4.11

显示了唯一性约束的示例。元模型层类型图（1）有一个关联的约束 $\text{unique}(C.b)$ ，由箭头附近的注释 $\{\text{unique}\}$ 可视化展示。图（2）从右侧节点到左侧节点有两条不同的 $C.b$ 标记边，因此不满足此约束。在图（3）中展示了正确的实例示范。

4.5.6 分解及剖视约束

分解和剖视约束表达对象模型与图的层次关系，是跨图的一种表达方式，分解通过对对象展开一个全新的视图，是对该对象的解释和进一步细化；剖视通过对对象选择性展开多张视图，通过多张视图从不同角度剖析该对象。

设 MMTG 是一个类型图， MMTG 中的分解约束是 $\text{decompose}(N, V)$ ，其中 $N \in \text{NType}$ 是一个节点类型， $V \in V_{mm}$ 是整个图，对所有 $G \in \text{Inst}[\text{MMTG}]$ ，定义满足性：

$$G \text{ sat } \text{decompose}(N, V) :\Leftrightarrow \exists v \in V, \exists n \in N, v_G.\text{Node} \subseteq n \quad (4.18)$$

设 MMTG 是一个类型图， MMTG 中的剖视约束是 $\text{explode}(N, V)$ ，其中 $N \in \text{NType}$ 是一个节点类型， $V \subseteq V_{mm}$ 是图的集合，对所有 $G \in \text{Inst}[\text{MMTG}]$ ，定义满足性：

$$G \text{ sat } \text{explode}(N, V) :\Leftrightarrow \forall v \in V, \exists n \in N, v_G.\text{Node} \subseteq n \quad (4.19)$$

在分解约束下，存在分解约束的对象，可以将该对象展开为一个图，即，对于任意的对象元模型 $\forall n \in V_{mObject}, \forall g \in V_{mm}$ ，存在 $n \in V_{mm}, g.V_{mObject} \subseteq n$ ；剖视约束下，存在剖视约束的对象，可以将该对象展开为多个图，即，对于任意的对象元模型 $\forall m \in V_{mObject}, \forall Q \subseteq V_{mm}$ ，存在 $m \in V_{mm}, \forall q \in Q, q.V_{mObject} \subseteq m$ 。

4.5.7 绑定约束

绑定约束指定了建模语言中给定类与类之间的连接约束，如 SysML 模块定义图中模块和约束模块之间可建立依赖关系，但不能够指定泛化关系。因此在图论中，使用对象节点、点节点、角色节点、边进行建模，节点和边的连接顺序体现绑定约束规则。

设 MMTG 是一个类型图， MMTG 的绑定约束表示为 $\text{bind}(N, E)$ ， $N \in \text{NType}$ 是 MMTG 中的节点， $E \in \text{Etype}$ 是 MMTG 中的边，对所有 $G \in \text{Inst}[\text{MMTG}]$ ，定义满足性：

$$G \text{ sat } \text{bind}(D, E) :\Leftrightarrow \exists n_1, n_2 \in N, \exists e \in E, \\ n_1 \text{ bind out}(n, E).tgt(e), n_2 \text{ bind out}(n, E).src(e) \quad (4.20)$$

在 GOPRR-E 的扩展元素中, 对象或点与关系的角色建立关系, 是元素间的主要约束关系, 满足绑定约束的模型对对象之间的连接起到限制作用, 对象和对象之间建立唯一或共享的表达关系, 因此, 存在 $\forall e \in E_{mRelationship}, \exists n_1, n_2 \in V_{mObject}, n_1(.V_{mPoint}^{it1})bind\ e.V_{mRole}^{ir1}, n_2(.V_{mPoint}^{it2})bind\ e.V_{mRole}^{ir2}$, 使模型按照元模型的规则相互匹配。

本章不仅选择了适当的元模型层类型图和模型层实例图以表征元模型和模型的语义表达, 形成 GOPRR-E 元素的抽象关系网, 更重要的贡献在于定义了扩展约束的语义, 以上约束总结在表 4.4 中, 通过建立约束的语义模型, 梳理模型中存在的约束关系, 形成对统一建模语言形式化表征的规范, 为后续的语言设计奠定基础。

表 4.4 扩展约束的形式化表征

扩展约束类型	形式化表征满足性约束集
抽象性约束	$\exists n \in P_{mProperty}, P_{mProperty}^{ip}.lab(n) = V_{object}^{jo}.P_{mProExt}^{ip}.lab(n);$ $\exists m \in V_{mPoint}, V_{mPoint}^{it}.lab(m) = V_{object}^{jo}.P_{mPoExt}^{ite}.lab(m)$
双向性约束	$\exists e \in E_{mRelationship},$ $src(e) = tgt(e) = E_{mRelationship}^{ie}.V_{mRole}^{ir}.RType.Export$
多重性约束	$V_{mRole}^{ir}.A_{mm}.Mult = [i, j]$
索引约束	$\forall e \in E_{mRelationship}, E_{mRelationship}^{ie}.A_{mm}.ix(e) = \{a, b\}$
唯一性约束	$\forall e \in E_{mRelationship}, E_{mRelationship}^{ie}.A_{mm}.ix(e) = 1$
分解约束	$\forall n \in V_{mObject}, \forall g \in V_{mm}, \exists n \in V_{mm}, g.V_{mObject} \subseteq n$
剖视约束	$\forall m \in V_{mObject}, \forall Q \subseteq V_{mm}, \exists m \in V_{mm}, \forall q \in Q, q.V_{mObject} \subseteq m$
绑定约束	$\forall e \in E_{mRelationship}, \exists n_1, n_2 \in V_{mObject},$ $n_1(.V_{mPoint}^{it1})bind\ e.V_{mRole}^{ir1}, n_2(.V_{mPoint}^{it2})bind\ e.V_{mRole}^{ir2}$

4.6 项目与语言

基于 GOPRR-E 元素及 MOF 框架的语义框架, 主要将模型语义的描述体现在元模型和模型的表达, 而为了体现形式化语义的整体性和相关性, 将建模语言统一形式化语义归于统一一个根节点, 实现自顶向下的语义解析, 和自底向上的语义递归。本文定义统一根节点为项目, 则项目是一个元组 $T_{project}^{it} = \langle L, lab \rangle$, 其中, T 表示项目, $it \in ID$ 是项目唯一的标识符, L 表示语言的集合。

L 作为多语言的集合, 形式化模型和元模型, 其定义是对图论的扩展, 定义一个

建模语言 $L^{il} = \langle Ltype, V_{mm}, M_{modIns}, M_{package}, E_{ext}, lab \rangle$, 其中, $L^{il} \in L$, $il \in ID$ 是项目唯一的标识符; $Ltype$ 表示语言的类型, 如 $Ltype \rightarrow SysML$; V_{mm} 表示元模型的集合, 是元模型层类型图的扩展; M_{modIns} 表示模型的集合, 是模型层实例图的扩展; $M_{package}$ 表示包的集合, $M_{modIns} \subseteq M_{package}$, E_{ext} 表示 GOPPRR-E 元素中扩展要素的集合。 mm 是 GOPPRR 代表的六种元模型元素, 如对象元模型、关系元模型、点元模型等, $modIns$ 是 GOPPRR 代表的模型元素如对象模型、关系模型、点模型等, ext 表示元元模型的扩展, 如包含、绑定、分解等, $lab: (Node \cup Edge) \rightarrow name$ 是为项目和语言赋予标签的函数。

项目和语言的形式化表征弥补了形式化元模型和模型语义的离散性, 使语义表达能够形成一个整体, 但对于元模型和模型本身没有产生影响。

4.7 本章小结

本章介绍了基于图论有向图对满足 GOPPRR-E 元素的建模语言分层抽象模型进行形式化表征, 为面向 MBSE 的多视图统一建模语言的研究提供了坚实的数学基础。首先分析了图论的基本概念, 其次分别基于元模型层类型图和模型层实例图开展元模型及模型形式化表征, 然后通过扩展约束形式化表征模型描述模型满足性约束集, 最后形式化表征顶层的项目与语言。至此, 本章完成了多视图统一建模语言的形式化表征, 实现了数学逻辑对语言本质的精准定义, 支持多视图统一建模语言的语法构建。

第 5 章 多视图统一建模语言表达研究

5.1 引言

装备复杂系统生命周期多阶段的架构建模，是多阶段建模语言实例化的过程，不同建模语言的抽象模型不同，数据异构，导致装备模型集成困难，影响设计效率。本文基于统一分层抽象模型及严格的数学模型形式化表征，实现一套完整的、抽象的、面向对象的建模语言，以实现对装备复杂系统生命周期多领域、多阶段模型的集成表达，并可实现在特定系统的建模过程中减少对建模语言进行二次开发和添加底层细节表达的需求。根据语言学定义，语言的设计需要先设计该语言的语法，本文首先根据抽象模型的形式化表征研究多视图统一建模语言的抽象语法，再通过抽象语法定义具体语法，从而实现系统模型统一建模语言的表达。

5.2 多视图统一建模语言构建方法

定义语言离不开对于抽象语法及具体语法的定义。语言的抽象语法指定了它的结构及其允许的組合，并且可以与指定解析规则的语法进行比较。这样的定义映射在基于图论有向图的抽象模型统一形式化表征中，抽象语法本身就是元模型和模型形式化表征的一种表现形式，是语言由抽象到具象的过程。

抽象语法表示了语言允许的结构，但没有说明是如何呈现给用户的，仅是说明在该领域中可用的概念。因此，语言的具体语法指定了形式化表征中的元素如何具体化的表示，每个可表示的函数概念都有一个具体的语法结构。目前存在多种类型的具体语法，但主要区别在于文本语言和图形语言^[129]。两者各有优缺点：文本语言更类似于编程语言，使程序员更容易使用特定域建模语言。另一方面，由于使用标准化符号，图形语言可以更好地表示问题域。不同的工具对具体语法有不同的选择，具体取决于工具的预期目标受众。例如，标准解析器使用文本语言，因为它们的目标受众包括在（文本）代码中指定系统的计算机程序员。如前所述，单个模型可以有不同的具体语法表示，一个是文本的，另一个是图形的。本文支持基于模型系统工程的多视图统一表达，即需要文本语言具体语法作为底层数据格式及编程，也需要图形化具体语法支持利益相关者进行建模。这里的图形化具体语法在 6.4 节案例介绍中展开，本章只针对文本具体语法进行分析。

如图 5.1 所示，本章节将按照如下路径开展形式化表征模型的具象过程，首先将形式化表征进行树状展开，其次其中的主要节点映射为语言的抽象语法的关键类，然后将每个抽象类扩展为一段详细的描述该类特性的语法，最后由具体语法形成多视图统一建模语言的语言规范，支持多种建模语言的表达。

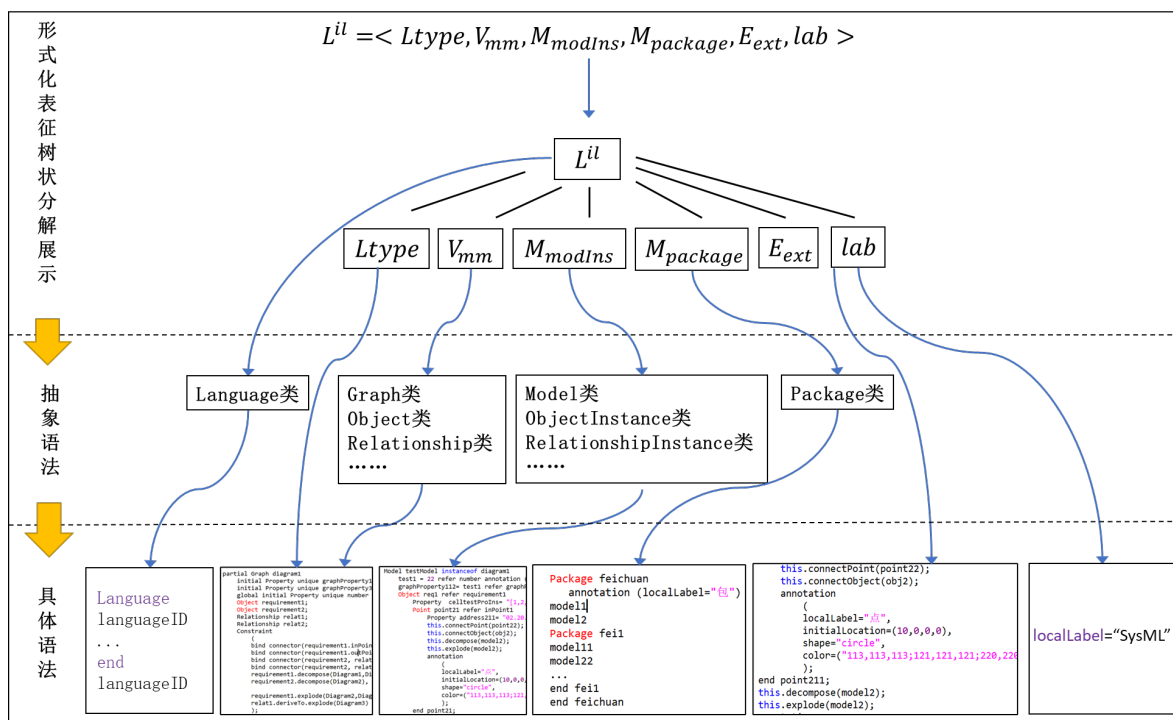


图 5.1 抽象语法到具体语法的过程

5.3 多视图统一建模语言抽象语法

元模型和模型的形式化表征映射明确了基于 GOPPRR-E 方法的元素类和类的关系，支持多视图统一建模语言抽象语法结构的映射。

根据 $T_{project}^{it} = \langle L, lab \rangle$, $L^{il} = \langle Ltype, V_{mm}, M_{modIns}, M_{package}, E_{ext}, lab \rangle$ 等项目自顶向下的形式化表征及扩展约束形式化约束的展开，多视图统一建模语言总体的抽象语法如图 5.2 所示，其中图中用 UML 类图表示语言抽象语法，主要使用了三种关系类型，分别是：1) 依赖 (Dependency): 表示的是类之间的调用关系，UML 中用带箭头的虚线表示依赖关系，而箭头所指的则是被依赖的类；2) 泛化 (Generalization): 表示的是类之间的继承关系，注意是子类指向父类，UML 中用带空心三角箭头的实线表示泛化关系，箭头指向的是一般个体；3) 聚合 (Aggregation): 是关联关系的一种特例，表示的是整体与部分之间的关系，部分不

能离开整体单独存在。UML 中用空心菱形头的实线表示聚合关系，菱形头指向整体。

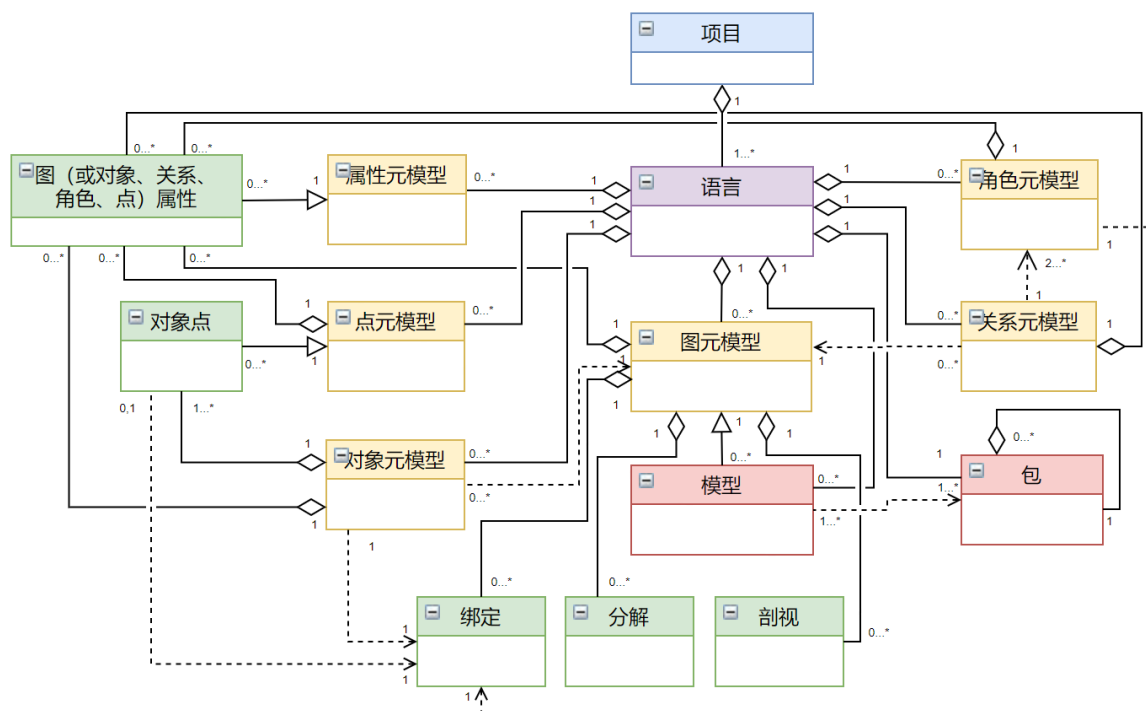


图 5.2 多视图统一建模语言总体抽象语法

图中，在 M3-M0 层架构的基础上，本文引入树形数据结构的思想来完善语言的整体性，树形数据结构是一类重要的非线性数据结构，树是以分支关系定义的层次结构。

5.3.1 类

首先根据项目 $T_{project}^{it}$ 的定义，多视图统一建模语言的起始类：项目类（ProjectClass），作为语言的根节点，一个项目可以包含多个语言类（LanguageClass）。

语言类（LanguageClass）定义该语言实现的语法，不同的语言类表示不同的建模语言，如命名为 SysML 的语言类需根据 SysML 的规范创建 SysML 建模语言。语言类包含多类要素，其中 $\{V_{mObject}, V_{mPoint}, V_{mRole}, E_{mRelationship}, P_{mProperty}\} \in V_{mm}^{iv}$ ，如图 5.2 中，黄色类块表示的元模型包含图元模型类（GraphClass）、对象元模型类（ObjectClass）、关系元模型类（RelationshipClass）、角色元模型类（RoleClass）、点元模型类（PointClass）、属性元模型类（PropertyClass）共六类，绿色类块中的图（对象、关系、角色、点）属性类（ElementPropertyClass），对象点类

(ObjectPointClass) 以及红色类块 $M_{package}$ 对应的包类 (PackageClass) 和 M_{modIns} 对应的模型类 (ModelClass), 此处包类唯一, 其他类的多重性为 $(0...*)$ [128]。

由模型的形式化表征 $M_{modIns}^j = \langle Mtype, V_{object}, E_{rela}, P_{property}, A_m, ix, lab \rangle$ 及对扩展约束的继承, 多视图统一建模语言模型的抽象语法如图 5.3 所示。

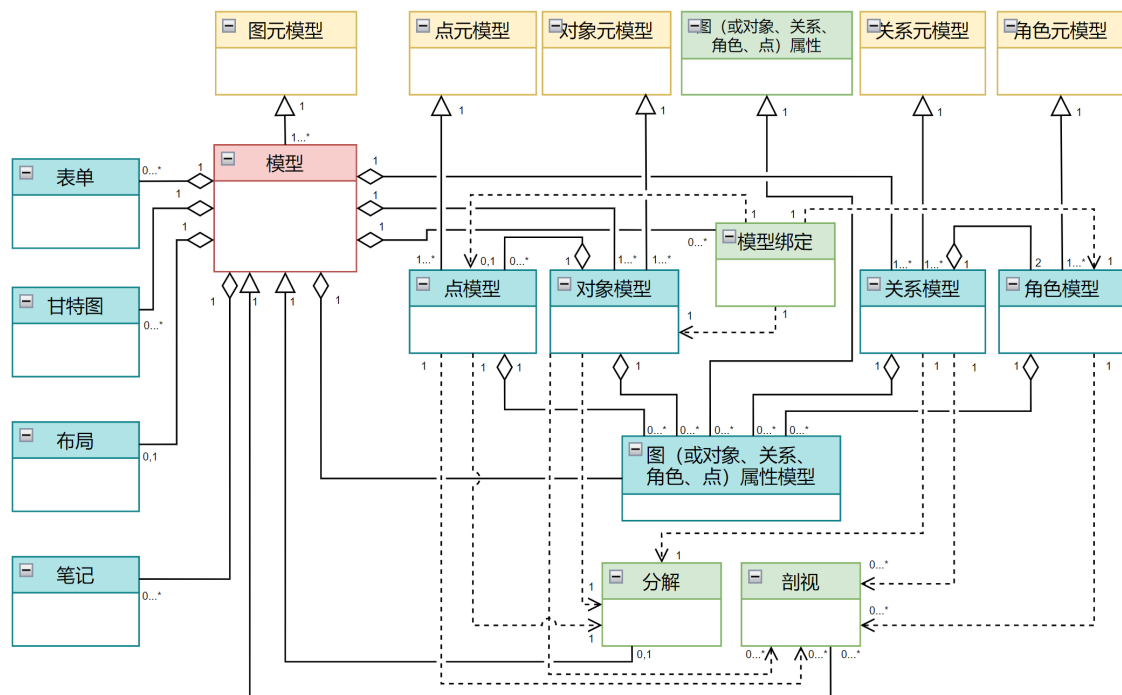


图 5.3 多视图统一建模语言模型的抽象语法

模型是对元模型的实例化, 因此主要元素和约束继承自元模型, 模型中包含更多形式化的信息要素, 是工程设计人员直接面对模型视图, 因此模型的语言的抽象语法将分为多个层次分别展开。

在模型的抽象语法中, 模型类 (ModelClass) 本身继承自图元模型类, 每个图元模型类可实例化为多个模型类。模型类是所有形式化元素的集合, 包含对象模型类 (ObjectInstanceClass)、关系模型类 (RelationshipInstanceClass)、属性模型类 (PropertyInstanceClass), 以及对象下实例化的点模型类 (PointInstanceClass)、关系下的实例化的角色模型类 (RoleInstanceClass), 并根据建模语言及工程应用增加了表单类 (TableClass)、甘特图类 (GanttClass)、布局类 (LayoutClass)、笔记类 (NoteClass) 等插件类。

5.3.1 组合关系

(1) 元模型类组合关系

$P_{mProExt}^{ipe}$ 映射为元素（图或对象、关系、角色、点）属性类，是对属性元模型类的继承，在原有属性的基础上添加对应元素的特征，以构建特定的元素属性，如图属性。

V_{mPoExt}^{it} 映射为对象点类是对点元模型的继承，点本身没有方向性，在集成于对象元模型之后表示对象输入输出的流端口。

双向性约束映射为角色元模型类对关系元模型类的依赖，通过固定在关系两端来实现关系流的流入和流出。

绑定约束及唯一性约束是针对对象与关系组合的强约束，依赖对象元模型（或对象点）以实现对象元模型和对象元模型之间约束关系的建立。每个绑定只针对一个对象元模型或对象点，与一个关系中的角色元模型。

多重性约束映射为图元模型中关系元模型的多重性，表示对象与对象连接的数量关系定义，如图中连接两个对象的关系元模型定义其多重性为[1,5]，则在实例化中，该关系连接一端的对象数量必须为 1~5 之间。

分解及剖视约束映射为图元模型中声明的多个分解或剖视功能，其中对象元模型、点元模型、关系元模型都可进行分解，对象元模型、关系元模型、角色元模型、点元模型可进行剖视。

(2) 模型类组合关系

抽象性约束映射为图（或对象、关系、角色、点）属性模型对图（或对象、关系、角色、点）属性元模型的继承，在对象模型、关系模型、角色模型、点模型、以及图模型本身中进行实例化，其中属性值是所有元素实例化的关键要素。

由模型形式化表征可得出，对象模型类、关系模型类、角色模型类、点模型类通过 *ref* 分别继承其对应元模型类，同时，对象模型包含多个点模型，用来确定端口的输入输出以及端口的位置，关系模型只能包含两个角色模型，因为一个实例化的关系表示一个确定的流，只能有一个输入端和一个输出端。

1) 对于对象模型形式化表征 V_{object}^{jo} ，其中的 $A_m = \langle A_{mm}, location, color, text, size \rangle$ ，提取并扩展，多视图统一建模语言对象模型的抽象语法包含如下关系如图 5.4 所示。

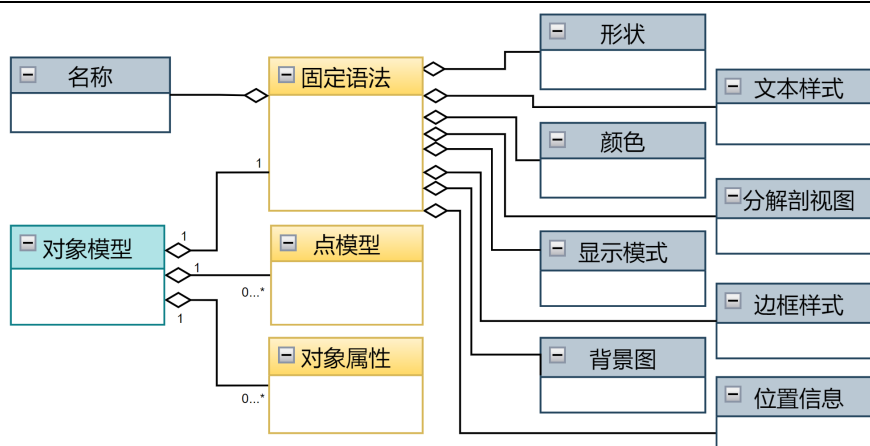


图 5.4 多视图统一建模语言对象模型的抽象语法

对象模型类的抽象语法主要包括对象属性模型、点模型类以及固定语法。文本语言需要具备可视化模型展示的所有特征，才能完全表达可视化模型，因此在固定语法中，将描述对象模型所必备的特有属性，包含名称、形状（节点标识）、颜色、显示模式、背景图、文本格式、分解剖视图、边框形式、位置信息等。对象属性需具备可扩展性，当系统建模需要展示多个平行属性时，对象属性具备同源属性复制能力。同样，对于点模型，作为对象的接口，满足对象输入输出功能，当系统建模需要使用多个接口时，点模型具备同源点复制的能力。

2) 对于关系模型的形式化语义 E_{rela}^{jr} ，其中的 $A_m = \langle A_{mm}, location, text, jump, smooth \rangle$ ，提取并扩展，多视图统一建模语言关系模型的抽象语法关系如图 5.5 所示。

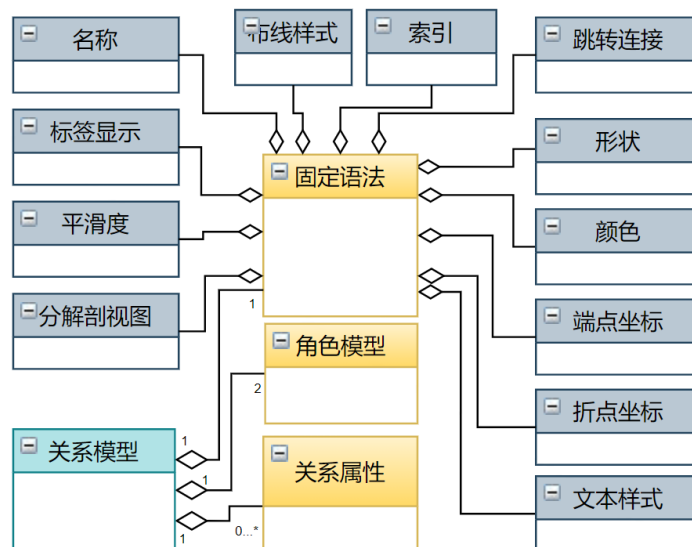


图 5.5 多视图统一建模语言关系模型的抽象语法

关系模型类的抽象语法主要包括关系属性模型、角色模型类以及固定语法。固定语法描述关系模型的状态信息，如表示关系可视化的形状和颜色，表示关系位置坐标的端点坐标和折点坐标，表示关系标识的名称、索引、文本样式和标签显示，表示关系绕线规则的布线样式、跳转连接、平滑度等，以及对关系进一步解释进行标识的分解和剖视图。本文语言设计了丰富的线形固定语法，可以满足不同使用习惯和使用需求的系统工程设计师的使用。

其中，索引约束映射为对关系模型类中索引的描述，通过在索引对多个对象间关系进行识别和排序。

3) 对于点模型形式化表征 V_{point}^{jt} ，其中的 $A_m = \langle A_{mm}, location, color, text \rangle$ ，提取并扩展，多视图统一建模语言点模型的抽象语法关系如图 5.6 所示。

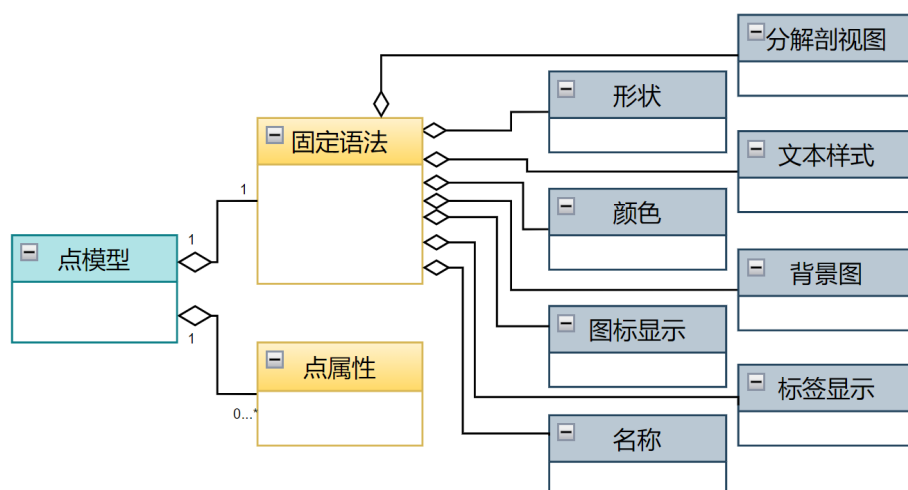


图 5.6 多视图统一建模语言点模型的抽象语法

点模型类的抽象语法主要包括点属性模型以及固定语法。点模型在 SysML、BPMN、Capella 等建模语言中作为端口有着十分重要的作用。因此点模型的固定语法也具备丰富的内容，如包含有名称、形状、颜色、文本样式、标签显示、图标显示、背景图等基本特性，以及可以标识点分解和点剖视图的能力。点依赖于对象存在，不能单独出现在模型中，但点具备独立的属性和固定语法。

4) 对于角色模型形式化表征 V_{role}^{jr} ，其中的 $A_m = A_{mm}$ ，提取并扩展，多视图统一建模语言角色模型的抽象语法关系如图 5.7 所示。

角色模型类的抽象语法主要包括角色属性模型以及固定语法。角色的固定语法较为简单，主要是通过形状被识别为具有不同规则的关系类型，使系统设计人员能

够更直观的识别和建立对象和对象之间的关系。除了名称和形状的基本特性，角色模型类的只能进行剖视，所以在固定语法中包含了标识剖视图的子类。

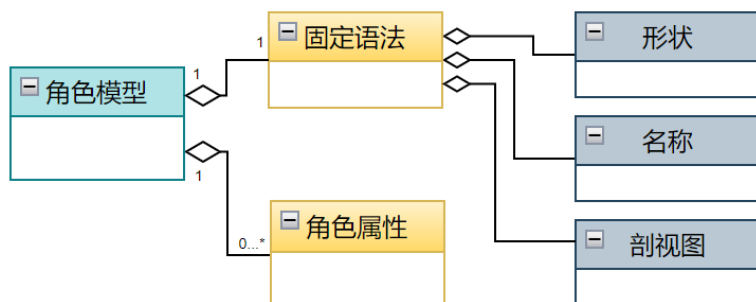


图 5.7 多视图统一建模语言角色模型的抽象语法

此外，包类在语言中起到导航作用，通过包可以将所建的模型进行分类，包可以包含多个模型的声明和新的包。如可以将系统建模过程模型分为需求模型包、功能模型包、逻辑模型包、架构模型包，实现对模型流程的分类构建。

通过对多视图统一建模语言的抽象语法进行映射，我们可以明确出语言的整体类结构和相互之间的关系，然后再通过具体语法的设计，实现语言的规范表达。

5.4 多视图统一建模语言具体语法

5.4.1 属性元模型及模型具体语法

通过抽象语法类及类组成关系的梳理，属性类在不同的层级中具备不同的解释，首先是定义属性的元模型，语言中用关键词 **Property** 标识，属性元模型经过被继承，成为其他元模型元素的属性，即元模型属性，则在语言中增加 **extends** 标识；在模型级别，属性被实例化为模型属性，语言中增加 **refer** 标识，模型属性可以包含具体的值。通过形式化表征中属性的上下文关系，定义属性应具备以下性质：

（1）依赖性

属性依赖于元模型中的其他元素，不能单独实例化，可在其他元素中被继承，称为元模型属性，如图元模型属性、对象元模型属性、点元模型属性、关系元模型属性、角色元模型属性。

（2）多样性

同一个属性元模型可被不同的元模型元素继承，以表达不同的属性含义。

(3) 唯一性

当元模型属性中包含关键词 **unique** 时，在包含该元模型的实例化模型中该属性的值是唯一的。

(4) 有序性

当同一元模型的元模型属性或模型的模型属性数量大于 1 时，可对属性的先后顺序做出要求。

(5) 初始值

元模型属性中包含关键词 **initial** 时，可对元模型属性赋初始值，初始值可以是整数型、实数型或字符串等。

元模型属性重要的作用是定义属性类型，通过对常用数据类型的总结，我们在语言中加入以表 5.1 下基础数据类型，以满足系统建模的基本需求。同时添加了附录 A 中的单位。

表 5.1 基础数据类型

类型	关键词
算数类型	整型 Int
	实数型 Real
字符串类型	String
布尔类型	Boolean
一维数组类型	Array
矩阵类型	Matrix
元胞数组类型	Cell
	无序、无重复集合 Set
	有序、无重复集合 OrderedSet
集合类型	有序、有重复集合 List
	键-值映射 Map

属性元模型类，图（或对象、关系、角色、点）属性类，属性模型类的具体语法如语法 5.1，语法 5.2，语法 5.3，其中 **unit** 是单位的关键词；属性值 **value** 的类型由属性的数据类型决定，如 **Int** 型的 **100**，**String** 型的“校名”；**description** 是对元模型的文字描述的关键词，如添加该属性的作用和使用范围；在元模型中的继承用关键词 *extends*，在模型中的引用关键词 **refer**；**propertySystemName**、**graphPropertySystemName**、**modelPropertySystemName** 分别为对应的唯一标识符。

语法 5.1: 属性元模型

```
1  Int Property propertySystemName annotation (localLabel="属性",unit = "kg",
    description="meta property");
```

语法 5.2: 图（或对象、点、关系、角色）属性

```
1  initial Property unique elementPropertySystemName =100 extends propertySystemName
    annotation (localLabel="重量");
```

语法 5.3: 模型属性

```
1  Property modelPropertySystemName= 500 refer graphPropertySystemName annotation
    (localLabel="食品重量", color="0,0,0", text=("Microsoft YaHei UI", 7, Bold, Italic));
```

其中，如图 5.8 所示，形式化属性元模型 $P_{mProperty}^{ip} = \langle Type, lab, unit, description \rangle$ 与属性元模型的具体语法形成映射，具体语法实现了对形式化表征模型的语言描述， ip 是唯一标识符，在语言中由 **propertySystemName** 代指的一串英文字母表示， $P_{mProperty}$ 是表示属性元模型，在具体语法中由以 **Property** 为关键词的该行语法表示， $Type$ 是属性类型，在具体语法中由 **Int** 为关键词的语法首词表示， lab 是该属性元模型的标签，或者称为名称，在具体语法中由 **localLabel=“属性”** 这样一段语法表示， $unit$ 是属性的单位，如具体语法中由 **unit=“kg”** 表示该属性的单位是千克， $description$ 是对该属性元模型的描述，具体语法由 **description=“meta property”** 表示，括号内可添加任意对于该属性的描述语句。

该具体语法实现了对形式化表征的映射，因此通过该具体语法可实现对可视化模型的描述。



图 5.8 形式化表征与具体语法的映射

5.4.2 角色元模型及模型具体语法

角色类分为角色元模型类和角色模型类：

(1) 角色元模型类定义对象类型如何连接关系类型，由关键词 **Role** 标识，关系

将在角色上间接关联对象。角色元模型类通过唯一标识符 **roleSystemName** 进行实例化, *RType*映射为角色的方向, 关系通过声明两端角色的类型及方向, 定义数据的流动方向或者关系的传递方向, 如表 5.2 所示的方向分为两类。

表 5.2 角色方向

方向	含义
Import	始端, 包含 Import 的角色是数据的入口。
Export	终端, 包含 Export 的角色是数据的出口。

同时 A_{mm} 中定义的 *shape* 映射为角色元模型类的形状, 角色的含义根据所属的建模语言不同而不同, 这里只介绍可以表达的角色形状如表 5.3 所示。

表 5.3 角色形状固定语法

关键词	意义
solid arrow	实心箭头
dotted arrow	空心箭头
solid diamond	实心菱形
dotted diamond	空心菱形
arrow shape	箭形
ring	实心圆形
solid circle	空心圆形
none	无形状

$V_{mRole}^{ir} = \langle RType, P_{mProExt}, A_{mm} \rangle$ 映射的角色元模型的语法如语法 5.4 所示。

语法 5.4: 角色元模型

```

1  Export Role roleSystemName //角色元模型定义
2      initial Property unique address1="连接" extends address0 annotation (localLabel="输出端属
    性"); //角色元模型属性
3      anotation (
4          localLabel="输出角色", //角色元模型名称
5          description="角色的描述", //角色元模型描述
6          shape="solid arrow", //角色元模型形状
7          mult= (0,10), //多重性
8      );
9  end roleSystemName;
```

(2) 角色模型类是在模型中的角色元模型的实例化, 通过关键词 **refer** 继承。角色模型类通过唯一标识符 **roleModelSystemName** 进行实例化, 除了实例化属性之外,

还可进行剖视，通过关键词 **explode** 剖视为一个新的模型，如剖视为 **model2**，并可在 **visualizationList** 中定义剖视模型的位置顺序。角色模型类中的形状类型与元模型相同，可进行变更，但原则上遵循元模型类定义的形状。 $V_{role}^i = \langle P_{property}, A_m, ref \rangle$ 及剖视约束映射的具体语法如语法 5.5 所示。

语法 5.5: 角色模型

```

1  Role roleModelSystemName refer roleSystemName //定义角色模型
2      Property address11= "每日" refer address1 annotation (localLabel="间断连接"); //角色模型
      的属性
3      this.explode(model2); //角色的剖视，可添加多个剖视图
4      annotation (
5          localLabel="角色",
6          shape="ring", // 角色的形状为实心圆
7          visualizationList="model2:(0,0,100,200)", //剖视图位置
8      );
9  end roleModelSystemName;
```

5.4.3 关系元模型及模型具体语法

关系具体语法分为关系元模型类和关系模型类具体语法：

(1) 关系元模型类是对象元模型类之间的连接关系，其通过关键词 **Relationship** 标识，是结构化模型的重要部分。在元模型中，定义关系元模型类是为了制定一种对象模块和对象模块之间的连接关系，如 SysML 语言模块定义图中的关系：关联、泛化、依赖等。关系元模型类通过唯一标识符 **relationshipSystemName** 进行实例化。

从形式化表征 $E_{mRelationship}^{ie} = \langle EType, P_{mProExt}, V_{mRole}, A_{mm} \rangle$ 中，关系元模型类中包含角色，没有角色的关系不能完整表示对象模块和对象模块之间的关系，一般认为，关系的方向为与始端角色相连接的对象模块指向与终端角色相连接的对象模块，但在语法中角色元模型类不能重复定义，因此关系中的 V_{mRole} 使用声明方式 “**Role roleSysgemName**” 表示。同时，关系元模型类定义不同的 **shape** 表达不同的含义，如在 SysML 中泛化关系是通常会在模块定义图中显示的一种关系，这种关系表示两种模块之间的继承：即一个更加一般化的元素，叫做超类型，以及一个更具体的元素，叫做子类型。使用泛化在系统模型中创建分类树。泛化的形状是一条是

实线，因此，完整表达一个泛化关系需要一条实线形状，以及用形状为空心三角箭头的终端角色表示超类型，和无形状的始端角色表示子类型。这种关系从子类型向超类型读作“……是一种……”。

关系元模型类的形状见表 5.4。

表 5.4 关系形状固定语法

关键词	意义
solid	实线
dash	虚线
dot	点线
dash dot	点划线

$E_{mRelationship}^{ie} = \langle EType, P_{mProExt}, V_{mRole}, A_{mm} \rangle$ 映射的关系元模型类的语法如语法 5.6 所示。

语法 5.6: 关系元模型

```

1  Relationship relationshipSystemName //定义关系元模型
2      initial Property unique relationshipColor extends colors annotation (localLabel="颜色"); // 关
    系元模型属
3      Role deriveTo; //角色声明
4      Role deriveFrom; //角色声明
5      annotation (
6          localLabel="关系 relat1", //关系名称
7          description="描述", //关系元模型描述
8          icon="Exchange.png", //关系元模型的角标
9          shape="solid", //关系元模型的形
10     );
11 end relationshipSystemName

```

(2) 关系模型类定义一种实例化的关系，通过关键词 **refer** 继承唯一标识符为 **relationshipSystemName** 的关系元模型，关系模型类中除了实例化属性之外，通过在图元模型类中声明分解剖视，在模型中使用关键词 **decompose** 进行分解、**explode** 进行剖视，关系模型类必须并且只能包含两个角色模型类。关系模型类中的形状类型与元模型相同，可进行变更，但原则上遵循元模型类定义的形状。其次，关系模型类还可定义线宽、字体、折点坐标等固定语法属性，形式化表征 $E_{rela}^{jr} = \langle P_{property}, V_{role}, A_m, ref \rangle$ 及分解剖视约束映射的语法如语法 5.7 所示。

语法 5.7: 关系模型

```

1  Relationship relaModelSystemName refer relationshipSystemName //关系模型
2  Property relationshipColor1= "red" refer relationshipColor annotation (localLabel="地址 1");
   //关系模型属性
3  RoleInstanceClass;      //这里 RoleInstanceClass 没有具体的定义，代指角色模型
4  this.decompose (model2); //关系分解
5  this.explode (model3,model4); //关系剖视
6  annotation (
7      localLabel="特征组合",      //关系名称
8      shape="solidLine",          //关系的形状
9      endLocation= (1,1;1,1),      //关系的端点坐标
10     polylineLocation="1,12;3,41", //关系上各折点坐标
11     color="0,0,0",              //关系的颜色
12     labelDisplay=true,          //关系上名称显示的开关
13     text= ("Microsoft YaHei UI", 9, Normal, Normal), //关系的字体设置
14     smoothness="None",          //平滑度设置
15     avoidObstructionsRouting=false, // 是否避开障碍
16     closestDistanceRouting=false, //是否使用最短路径
17     jumpLinkStatus="None",      //线的跳转连接状态
18     jumpLinkType="Semicircle",  //线的跳转连接类型
19     reverseJumpLink=false,      //线的反向跳转连接
20     visualizationList="model2:(0,0,100,200); model3:( 10,20,100,200); model4:
      (10,10,300,400)",          //分解、剖视图位置
21 );
22 end relaModelSystemName;

```

5.4.4 点元模型及模型具体语法

点的具体语法定义一组或一类点。点可以为对象如何通过关系连接提供额外的语义或约束。点元模型除自身需要定义之外，还需在对象元模型中声明。点的抽象语法中有三个层次，分为点元模型类、对象点类和点模型类。点的三个层次在实例化过程中需具备以下性质：

1) 依赖关系

点元模型可自定义，但点模型必须依赖对象模型存在，当删除对象模型时，该对象模型上的点模型也一并删除。

2) 多样性

对象模型上可以添加多个点模型，且点模型的方向不受限制。

3) 多重性

对象模型上的每一个点模型都可以与多个角色模型绑定，或者说每一个对象模型都可以通过点模型与多个关系模型连接。

(1) 形式化表征 V_{mPoint}^{it} 映射的点元模型类定义端口的通用属性，通过唯一标识符 **pointSystemName** 进行实例化，定义时不具有方向性。其映射的点元模型类的具体语法如语法 5.8 所示。

语法 5.8: 点元模型

```

1  Point pointSystemName //点元模型定义
2  initial Property unique number1 = 5 extends number0 annotation (localLabel="允许个数"); //
   点元模型属性
3  annotation (
4    localLabel="端口", //点元模型的名称
5    description="描述", //点元模型的内容
6    shape="circle", //点元模型的形状
7  );
8  end pointSystemName;
```

(2) 对象点类是点元模型类通过关键词 **extends** 在对象元模型中的继承。对象点作为对象自身与关系连接的接口，具有方向性，这里定义了三种类型的方向：**Input** 表示输入方向，与终端角色绑定；**Output** 表示输出方向，与始端角色绑定；**Undirected** 表示没有方向性，可与始端、终端均可绑定。形式化表征 $V_{mPoExt}^{ite} = \langle PType, ext, A_{mm} \rangle$ 映射的对象点的具体语法如语法 5.9 所示。

语法 5.9: 对象点

```

1  Output Point outPoint extends pointSystemName annotation (localLabel="输出点");
```

(3) 点模型类定义一种实例化的点，需包含关键词 **refer** 以及继承的对象点的唯一标识符，点模型类中除了实例化属性之外，还可进行分解或剖视为一个新的模型。点模型类中的形状类型与元模型相同，可进行变更，但原则上遵循元模型类定义的形状。点模型类可通过复制为多个同源点，各点具有相同的属性，但实例化为不同

的实体。其次，点模型类还定义了点位置、点字体、点颜色，点连接的点或对象等固定语法属性，形式化表征 $V_{point}^{po} = \langle P_{property}, A_m, ref \rangle$ 及分解剖视约束映射的点模型类定义的语法如语法 5.10 所示。

语法 5.10: 点模型

```

1  Point point211 refer inPoint1    //点模型的定义
2  Property address2111= "闭合" refer address2 annotation (localLabel="开关"); //点模型属性
3  this.decompose (model1);        //点的分解
4  this.explode (model3,model4);   //点的剖视
5  annotation (
6    localLabel="点",              //点模型名称
7    initialLocation=(10,0,0,0),   //点的位置坐标
8    shape="circle",               //点模型的形状
9    color="113,113,113;121,121,121;220,220,220", //点的填充、边框、文本颜色
10   text = ("宋体",20,Bold,Italic), //点名称的字体设置
11   visualizationList="model2:(0,0,100,200); model3:( 10,20,100,200); model4:
    (10,10,300,400)",              //分解、剖视图位置
12 end point211;
```

5.4.5 对象元模型及模型具体语法

对象定义了一组或一类具有相同特征（例如属性）的元素。通常，对象类如在 SysML 语言中用例图的用户模块、执行者等作为建模域的主体结构，与关系模型如扩展用例等构成模型的实体，对象具体语法包含对象元模型具体语法和对象模型具体语法。

（1）对象元模型类是结构化模型的重要部分，对象元模型类包含属性和固定语法，该对象是如 SysML 语言需求图中的对象：需求。 $V_{mObject}^{io}$ 映射的元模型对象通过唯一标识符 **objectSystemName** 进行实例化。

对象元模型除了自身属性及固定语法的定义，还可定义对象点，对象点的内容在点的具体语法中阐述，这里需要强调的是，对象可添加点，也可不添加点，即对象对于点依赖性。

对象元模型类的形状与其模式有关，模式确定视图的可视化显式或隐式特征，对象包含了两种模式：显示属性的容器模式：**property mode**，不显示属性的节点模

式: **blank property mode**。不同的模式, 对象的形状如表 5.5 中所示。

表 5.5 对象形状具体语法

	关键词	意义
属性显示模式	rounded rectangle	圆角矩形
	rhombus	菱形
属性隐藏模式	rectangle	矩形
	circle	圆
	rhombus	菱形
	ellipse	椭圆
	human	人形
	package	包图
	bar node	条形节点
	rounded rectangle	圆角矩形
	rhombus	菱形

$V_{mObject}^{io} = \langle OType, P_{mProExt}, V_{mPoExt}, A_{mm} \rangle$ 映射及扩展增量的对象元模型类的具体语法如语法 5.11 所示。

语法 5.11: 对象元模型

```

1  Object objectSystemName //对象元模型定义
2  initial Property unique celltestPro = " " extends celltest annotation (localLabel="身高"); //对
   象元模型属性
3  Output Point outPoint1 extends outPoint annotation (localLabel="输出点"); //定义对象点
4  annotation (
5    localLabel="需求", //对象元模型名称
6    description="描述", //对象元模型描述
7    icon="Entity.png", //对象的角标
8    visualizedMode= property mode, //对象显示模式
9    shape="circle", //对象的形状
10   selfShape="XXX", //对象自定义形状
11 );
12 end objectSystemName;
```

(2) 对象模型类通过 **refer** 对元模型类进继承, 实例化为以 **objectModelSystemName** 为唯一标识符的实体, 对象模型类具体语法中除了实例化属性之外, 还包含点、分解、剖视等。对象模型具体语法的形状类型与元模型相同, 可进行变更, 但原则上遵循元模型定义的形状。基于形式化表征 $V_{object}^{jo} = \langle$

$P_{Property}, V_{point}, A_m, ref >$, 分解剖视约束及增量定义的对象位置、对象颜色、名称字体、图片地址等, 其对象模型类的具体语法如语法 5.12 所示。

语法 5.12: 对象模型

```

1  Object objectModelSystemName refer objectSystemName //对象模型定义
2      Property celltestProIns= "[1,2,3;4,5,6;7,8,9]" refer celltestPro annotation (localLabel="矩阵");
    //对象的属性
3      PointInstanceClass; //这里 PointInstanceClass 没有具体的定义, 代指点模型
4      this.decompose(model2); //对象的分解
5      this.explode(model3); //对象的剖视
6      annotation (
7          initialLocation=(425,15,183,118), //对象模块的位置和大小
8          localLabel="车辆特征", //对象的名称
9          shape="rectangle", //对象的形状
10         color="0,0,0;0,0,0;0,0,0;194,239,255;255,255,255", //对象的三组颜色
11         screenMode=propertyMode, //对象的显示模式
12         imageAddress="6.png ", //对象的背景图片
13         text=("微软雅黑",8,Normal,Normal), //对象文本字体设置
14         iconDisplay=true, //对象图标显示开关
15         frame=(1,"solid"), //对象边框的设置
16         visualizationList=" model2:(0,0,100,200); model3:( 10,20,100,200)", //分解、剖视图位置
17     );
18 end objectModelSystemName;
```

5.4.6 图元模型具体语法

图的具体语法定义图元模型类, 定义图元模型是生成一种建模语言视图的过程, 如 SysML 建模语言中的需求图、活动图、状态机图的生成。包含对对象、关系的组织, 对绑定的定义, 以及分解、剖视模式的定义等。

关键词 **Partial** 用于标识图元模型, 以区别于实例化的 **Model**。通过声明对象元模型类和关系元模型类, 确定该图的基本组成; 关键词 **Constraint()**表示约束, 允许对元模型元素的附加条件定义, 这些条件必须在建模期间或之后被满足。其中包含绑定约束: **bind connector()**建立对象和关系的约束关系, 以及分解和剖视的指定: **decompose**表示分解, 支持对象、点、关系元素的细化, 将一个元素展开为一个图进

行详细说明。**explode** 表示剖视，支持对象、点、关系、角色元素，从多个新的视图模型的角度进行展示。

同时图具体语法还需对图模式进行了分类，以展示不同类型的视图。具体分类见表 5.6。

表 5.6 图模式具体语法

关键词	意义
diagram	普通图
sequenceDiagram	时序图
crossTable	交叉表格
tree	树形展示

形式化表征 $V_{mm}^{iv} = \langle Mtype, V_{mObject}, V_{mPoint}, V_{mRole}, E_{mRelationship}, P_{mProperty}, A_{mm} \rangle$ 映射及增量的图的具体语法如语法 5.13 所示。

语法 5.13: 图元模型

```

1  partial Graph diagramSystemName //图元模型定义
2      initial Property unique graphProperty1= "" extends height1 annotation (localLabel="重量"); //
    图属性
3      Object requirement1; //图中声明组成该图的对象
4      Relationship relat1; //图中声明组成该图的关系
5      Relationship relat2;
6      Constraint ( //图中约束的集合
7          bind connector(requirement1.inPoint1, relat2.satisfyTo), //对象中的点与角色绑定
8          bind connector(requirement1.outPoint1, relat2.satisfyFrom),
9          bind connector(requirement1, relat1.deriveFrom), //对象与关系下的角色绑定
10         requirement1.decompose(Diagram1, Diagram2), //对象的分解
11         requirement1.explode(Diagram2, Diagram3), //对象的剖视
12         relat1.deriveTo.explode(Diagram3), //角色的剖视
13     );
14     annotation (
15         localLabel="图",
16         description="描述",
17         type = diagram, //图模式选择
18     );
19 end diagramSystemName;

```

5.4.7 模型具体语法

模型类具体语法是描述具有相同属性的一组或一类模型。标识模型的关键词是 **Model**，通过唯一标识符 **modelSystemName** 进行创建，并通过关键词 **instanceof** 实例化图元模型 **graphSystemName** 的所有元素，如通过 SysML 的需求图创建需求模型，通过状态机图创建有限状态机模型。模型类可包含多个属性模型类、多个对象模型类、多个关系模型类，并通过关键词 **Connect** 定义 **connector** 进行绑定，每个 **connector** 是一个连接器，始于角色类，定义一种绑定类型，如角色与对象的绑定、角色与对象中的点绑定。此外，模型中允许在固定语法中增量添加笔记 **NodePlug-in**、链接 **LinkModelPlug-in**、表单 **TablePlug-in**、布局 **LayoutPlug-in**、甘特图 **GanttChartPlug-in** 等伪对象（具体语法见附录 B）作为模型的插件，这些伪对象是模型的一部分，但没有元模型，只是作为模型的标记、补充要素。

形式化表征 $M_{modIns}^j = \langle Mtype, V_{object}, E_{rela}, P_{property}, A_m, ix, lab, ref \rangle$ ，绑定约束映射的模型及增量的具体语法如语法 5.14 所示。

语法 5.14: 模型

```

1  Model modelSystemName instanceof graphSystemName //定义模型
2  Property graphProperty112 = test1 refer graphProperty1 annotation (localLabel="地址 1"); //
   模型属性
3  ObjectInstanceClass; //这里 ObjectInstanceClass 没有具体的定义，代指对象模型
4  RelationshipInstanceClass; //这里 RelationshipInstanceClass 没有具体的定义，代指关系模
   型
5  Connect(connector(req1.point11,r1.role2),connector(req2,r1.role1)); //绑定关系
6  annotation (
7    localLabel="System 建模", //模型名称
8    NodePlug-in, //这里 NodePlug-in 没有具体的定义，代指笔记插件
9    LinkModelPlug-in, //这里 LinkModelPlug-in 没有具体的定义，代指链接插件
10   TablePlug-in, //这里 TablePlug-in 没有具体的定义，代指矩阵插件
11   LayoutPlug-in, //这里 LayoutPlug-in 没有具体的定义，代指布局插件
12   GanttChartPlug-in, //这里 GanttChartPlug-in 没有具体的定义，代指甘特图插件
13 );
14 end modelSystemName;
```

其中各插件具备以下的功能，作为模型信息的非图形视图，这些视图通常对大

表 5.7 主要类之间的二元关系

	元素属性	角色元模型	角色模型	关系元模型	关系模型	点元模型	对象点	点模型	对象元模型	对象模型	图元模型	模型
元素属性		包含		包含		包含			包含		包含	
角色元模型	包含于		继承	声明			绑定					
角色模型		泛化			包含			绑定		绑定		
关系元模型	包含于	被声明			继承						声明	
关系模型			包含于	泛化								包含
点元模型	包含于						继承					
对象点		绑定				被继承		继承	包含			
点模型			绑定				泛化			包含		
对象元模型	包含于	绑定					包含于			继承	声明	
对象模型			绑定					包含于	泛化			包含
图元模型	包含于			包含于					包含于			继承
模型					包含于					包含于	泛化	

量信息的显示非常有效。

(1) 笔记插件可作为模型可选的注释，可以附加在模型视图框架的内部，用于支持建模人员提取图的额外信息，信息可包括用户自定义的内容：

1) 版本——图版本号。

2) 完成状态——作者关于视图完成状态的说明，既可以包括一种陈述（如“进程中”、“草案”、“完成”等），也可以包括途中未有的信息的特定描述。

3) 描述——对视图内容或目的的文本描述。

笔记也可作为模型元素的注解标识，通过一条虚线附件到任何模型元素（对象模型、点模型、关系模型）或者元素集的标识。注解标识为模型提供附文本信息，可包括参考文档的超级链接。

(2) 链接插件是对视图其他信息的说明，或者用于辅助导航，提供对其他图的超级链接。

(3) 表单是一种高效且有表现力的信息表达方式，允许应用表单注释作为图注释的替代方案来表示包含在建模语言模型库中的建模信息。表格式也可以表示模型元素特征和/或模型元素间的关系。

(4) 布局就是把视图中的元素按照某种规律摆放到指定的位置。在 SysML 规范中布局起到图框架的作用，用以封装图的内容。图框架对应于某个模型元素，提供了针对图内容的情境。

(5) 甘特图通过条状图来显示项目、进度和其他时间相关的系统进展，直观表明计划何时进行，便于管理者弄清项目的剩余任务，评估工作进度。

表 5.7 中对语言中的主要类和类之间的二元关系进行了梳理，以更为清晰语言规范的结构和类之间的约束。

5.4.8 项目及语言类具体语法

$T_{project}^{it} = \langle L, lab \rangle$ 作为形式化表征顶层模型，在具体语法中也作为最外层总体，将所有具体语法包含其中，项目类由关键词 **Project** 标识项目的开始，由 **end** 关键词标识结束，**projectSystemName** 为该项目的唯一标识符，注释 **annotation** 是固定语法的集合，项目的 **annotation** 仅有名称 **localLabel**，表示该项目的名称。项目下是可由多个语言类组成，表示创建如 SysML、UPDM 等建模语言。 $T_{project}^{it}$ 映射的具体语法如语法 5.15 所示。

语法 5.15: 项目

```

1  Project projectSystemName
2    LanguageInstances; //语言的代码段
3    annotation( localLabel="名称");
4  end projectSystemName;

```

$L^{il} = \langle Ltype, V_{mm}, M_{modIns}, M_{package}, E_{ext}, lab \rangle$ 映射的语言类是对一种建模语言的元模型及其模型等的集合，由关键词 **Language** 标识语言的开始，通过唯一标识符 **languageSystemName** 进行实例化，语言包含了 V_{mm} 映射的元模型、 M_{modIns} 映射的模型、 M_{modIns} 映射的包等具体语法内容。其具体语法可表示如语法 5.16 所示。

语法 5.16: 语言

```

1  Language languageSystemName
    //以下在语言中不是具体的语法定义，代指元模型及模型具体语法
2    PropertyMetaModels;
3    GraphMetaModels;
4    ObjectMetaModels;
5    RelationshipMetaModels;
6    RoleMetaModels;
7    PointMetaModels;
8    ModelInstances;
9    Packages;
10 annotation( localLabel="名称");
11 end languageSystemName;

```

通过基于形式化表征的抽象语法类抽取，本节定义了各主要类概念以及类之间的关系；然后通过具体语法，对每一个关键类进行了类展开，确定其具体的语法结构，通过对现有建模语言的抽象和具象，形成一套完整的多视图统一建模语言规范，以满足各个建模语言的规范要求。

如系统建模语言 SysML 的活动图的统一建模语言如附录 D 所示。多视图统一建模语言对于复杂产品生命周期建模的满足性将在 6.4 节，通过智能电动汽车架构建模过程进行验证。

5.5 本章小结

本章对多视图统一建模语言进行了研究，主要是进行了抽象语法和具体语法的定义。其中的抽象语法指定了语言的类和类之间的关系，通过定义元模型和模型的抽象语法，明确语言的类组成及包含、继承等类之间的关系，形成文本语言的框架；再通过具体语法丰富语言，具体语法指定了抽象语法框架中的项目、语言、元模型及模型等的显式表达，每个可表示的抽象语法概念都扩展为一个具体的语法规范结构。最终实现满足面向复杂系统生命周期多阶段建模的多视图统一建模语言规范的构建。

第 6 章 复杂系统架构多视图集成表达验证平台设计与验证

6.1 引言

多视图统一建模语言需要一个可视化环境才能展示建模，本章设计并开发基于模型系统工程的复杂系统多视图集成表达验证平台，该平台的开发针对北京理工大学“十四五”装备预研共用技术项目“多系统建模与设计空间智能决策技术”等项目的具体需求。基于该平台，通过多语言对比和复杂装备典型案例建模和分析，开展分层抽象模型构建、基于图论有向图的模型统一形式化表征、多视图统一建模语言表达三个多视图统一建模关键技术的验证。

6.2 多视图集成表达验证平台设计

6.2.1 平台应用场景

多视图集成表达验证平台定位为面向基于模型系统工程的多视图统一表达支持平台，其核心任务是为产品研发人员提供建模环境，不同的研发人员需要的环境配置不同。复杂产品的研发涉及多领域、多阶段，需要不同建模语言的支持，本文的平台满足各种建模语言的构建，如体系设计使用 UPDM 建模语言，系统设计使用 SysML 建模语言，流程设计使用 BPMN 建模语言，以及汽车领域的嵌入式系统架构设计使用 EAST-ADL 特定域建模语言等。因此，本文将平台用户分为两类，领域专家及架构设计师。

领域专家。领域专家指的是拥有丰富的专业知识和研发经验的研发人员，其主要工作场景是领域建模语言的开发。当面临全新的系统设计，全新的设计流程，全新的领域，现有的建模语言无法满足建模需求时，由该领域的领域专家总结领域特征，设计领域对象符号、属性、关联关系、约束关系等，形成该领域的建模语言。为支持领域专家的特定域建模语言设计，多视图集成表达验证平台提供丰富的元模型构建环境。

架构设计师。架构设计师指的是负责使用工具进行建模的一类人员，可以是具有丰富经验的体系构建人员、系统总体设计人员、子系统设计人员等。架构设计师主要是使用建模语言如 SysML、BPMN、DMN、OPM、UML、UPDM、DSML 等在工具画布上构建相应的架构模型，依据专业知识和语言使用规范，实现目标体系、系统、

子系统等的模型构建。同时当设计需求发生变化时，架构设计师能够重用模型，快速的进行迭代设计。为支撑架构设计师的建模需求，平台需要提供如 SysML、BPMN、DMN、OPM、UML、UPDM 等丰富的建模语言元模型库、领域专家设计的特定域建模语言元模型库，以及清晰、适用、便利的建模环境。

两类用户分工不同存在不同的工作场景，建模语言设计场景和架构模型构建场景。建模语言设计场景包含两种基本的元模型类型：按照现有建模语言标准规范构建的建模语言元模型和特定域建模语言元模型。其中按照现有建模语言标准规范构建的建模语言可以元模型库的形式存在，以便使用时直接调用，也可在原有建模规范的基础上扩展需要的节点；特定域建模语言设计是领域专家的工作场景，领域专家结合领域知识和相应的规范和约束，按照 GOPRR 元元模型顺序构建 GOPRR 元模型，完成特定域建模语言的构建，如基于 UML、SysML 和 AADL 的概念和汽车需求，并符合 AUTOSAR（汽车开放系统架构）开发汽车嵌入式系统的架构描述语言 EAST-ADL，涵盖了包括车辆特性、功能、要求、可变性、软件组件、硬件组件和通信在内的抽象元素。架构模型构建场景是各个架构设计师的工作场景，调用已有的建模语言，针对各自的复杂系统开展架构建模，如体系架构设计师在进行体系设计时直接调用 UPDM 的元模型库进行体系架构建模，流程架构设计师在进行流程制定时直接调用 BPMN 的元模型库进行流程架构建模等。

6.2.2 平台框架设计

为实现以上应用场景，多视图集成表达验证平台采用了层次化的框架，如图 6.1 所示。框架共分为四层：用户界面层、业务逻辑层、核心驱动层和数据层。

(1) 用户界面层。用户界面层是用户和平台之间交互的接口，为用户的正常工作提供界面。用户通过工具栏向导进行工作，领域专家通过工具栏中的创建项目、创建语言、配置元模型构建建模语言，体系、系统、子系统等架构总体设计师通过工具栏中的创建模型进行建模。

(2) 业务逻辑层。业务逻辑层是平台的核心，体现了平台主要执行的功能和业务范围。该平台的主要功能模块包括需求管理、体系架构定义、系统架构定义、特定域架构定义，以及多视图统一建模语言定义，服务于系统生命周期中的使命任务分析，体系、系统、子系统等架构设计阶段。平台可进行基于多种方法论的架构建模，如基于 MagicGrid^[132]流程的需求、行为、结构、参数架构建模，如基于使命、运行、特征/

功能、逻辑、物理架构的架构建模等。在实现该过程或领域的建模过程中，涉及到特定领域建模时，需根据领域场景自定义图配置，定义对象、关系、点、角色、熟属性等构建特定域建模语言，以支持系统架构设计。

(3) **核心驱动层**。核心驱动层是功能实现的基础，为各功能模块的实现提供后台支撑。平台的核心驱动包括语言解释器、代码生成器、元模型驱动器、模型驱动器和插件驱动器。

(4) **数据层**。数据层的作用是为建模语言和模型的持久化提供支撑。本文将数据层分为两类，一类是规范化的建模语言，作为元模型库的形式存在于数据层，以便架构设计师直接使用建模，这类建模语言包括 SysML、BPMN、DMN、OPM、UML、UPDM 等；一类是建立的各类模型，其数据以多视图统一建模语言的统一格式进行保存。

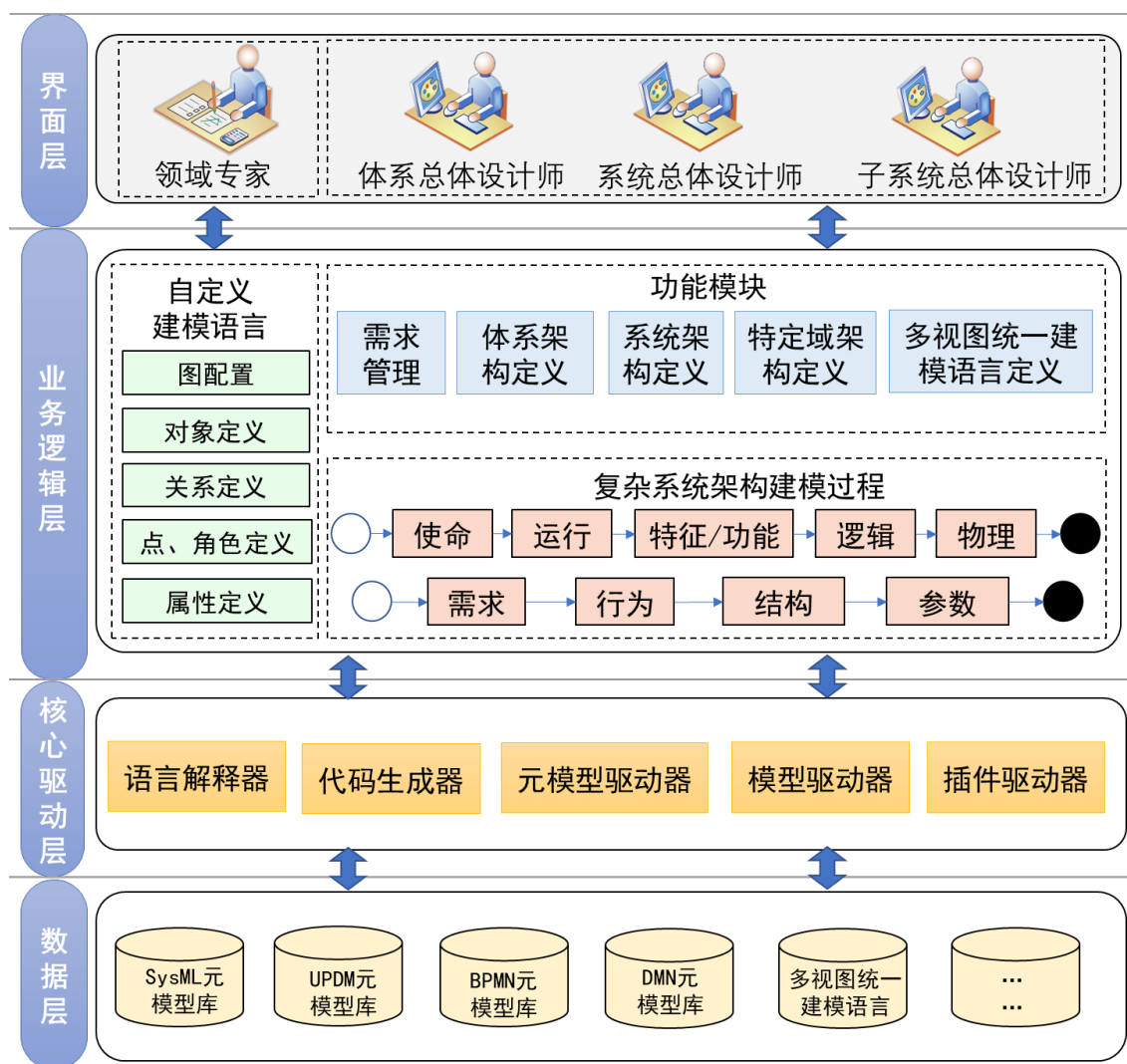


图 6.1 平台技术框架

6.2.3 平台实现的技术基础

多视图集成表达验证平台基于 Eclipse 平台架构技术完成开发，可打包成 exe 可执行程序方便用户的安装和使用。从技术层面看，包含了界面框架层（实现图 6.1 用户界面层和业务逻辑层）和工具控制层（实现图 6.1 核心驱动层）。两个层次所采用的具体技术框架如下。

界面层：界面层通过 SWT、JFace、Eclipse GMF、Sirius、WindowBuilder 技术实现。SWT（The Standard Widget Toolkit）是一个用于 Java 的开源小部件工具包，旨在提供对实现它的操作系统的用户界面设施的高效，可移植访问。JFace 是建立在 SWT 之上的 UI 部件，是 Eclipse 组织为了开发 Eclipse IDE 环境所编写的一组底层图形界面 API，其底层实现为 SWT。它扩展了 SWT 并能与 SWT 交互操作。JFace 工具箱提供了一组功能强大的用户界面组件，开发人员可以在独立应用程序中利用这些组件，以简化常见的 UI 编程任务。Eclipse GMF 是图形编辑器领域的模型驱动架构方法，通过定义工具、图形和映射模型定义，可以在 Eclipse 中生成功能齐全的图形编辑器。Sirius 是一个 Eclipse 项目，允许通过利用 Eclipse 建模技术（包括 EMF 和 GMF）创建 MetaGraph 的图形建模工作台。WindowBuilder 由 SWT Designer 和 Swing Designer 组成，可以非常轻松地创建 Java GUI 应用程序，而无需花费大量时间编写代码。使用所见即所得的可视化设计器和布局工具创建简单的表单到复杂的窗口并生成 Java 代码。WindowBuilder 构建一个抽象语法树（AST）来导航源代码，并使用 GEF 来显示和管理可视化表示。支持自由格式的代码编辑和用户重构。通过以上技术，构建了用户交互界面的相关内容。

工具控制层：工具控制层包含工具的运行框架和建模语言的解释和代码生成器。运行框架主要包括了 Eclipse OSGi、Eclipse P2 updates、Eclipse Plug-in Register 等 Eclipse 控制开发技术，OSGi 是 Open Services Gateway initiative 的缩写，叫做开放服务网关协议，允许动态的启动，停止，安装，删除项目的模块而无需重启服务；Eclipse P2 updates 是 Eclipse 平台为 Eclipse 插件提供的安装和更新机制；Eclipse Plug-in Register 提供了工具的插件控制管理机制。建模语言的解释和代码生成器主要是基于 ANTLR 进行词法分析和语法分析，并按照 Java 语言生成分析器代码，用于识别和解析多视图统一建模语言，并基于 StringTemplate 模板引擎实现代码生成，具体的建模语言的解释和代码生成器将在下一节中介绍。

6.2.4 多视图统一建模语言解释器及代码生成器

语言解释器的目的是对语言进行解释，使采用文本式多域一体化建模语言构建的模型能够被自动解析生成图形化的模型，语言代码生成器的目的是由图形化建模方式构建的图形化多域模型能够自动解析生成多视图统一建模语言表达的文本式模型。

如图 6.2 所示为语言解释器和代码生成器的作用效果。

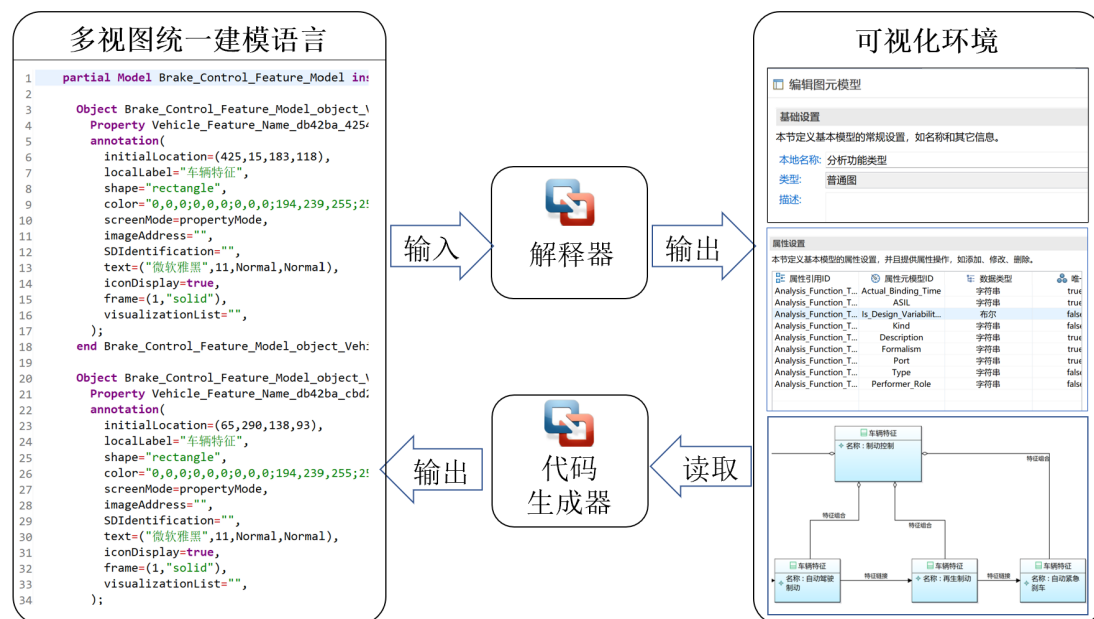


图 6.2 语言解释器和代码生成器

6.2.4.1 多视图统一建模语言解释器

语言由一系列有意义的语句组成，语句由词组组成，词组是由更小的子词组和词汇符号组成。一般来说，如果一个程序能够分析计算或者“执行”语句，我们就称之为解释器。多视图统一建模语言解释器的构建如图 6.3 所示，首先根据 ANTLR 元语言抽象出多视图统一建模语言的语法文件，然后通过 ANTLR 生成语法分析器和词法分析器，语法分析器会建造一种名为语法分析树（parse tree）或者句法树（syntax tree）的数据结构，提供针对语言的操作接口，再通过编程进行语义解析映射，最后通过前端画布的映射形成可视化的模型。

ANTLR 是一款强大的语法分析器生成工具，可用于读取、处理、执行和翻译结构化的文本或二进制文件，本文通过 ANTLR 来处理文本式多视图统一建模语言。在 ANTLR 的使用环境中，首先需要构建语法文件，这里的语法是一门语言的正式描述，基于 ANTLR 语法本身遵循的一种专门用来描述其他语言的语法——ANTLR 元语言进行构建。一条语法由语法申明和紧随其后的若干条规则组成，规则又分为词法分析器

规则和语法分析器规则，语法分析器规则以小写字母开头，词法分析器规则以大写字母开头。词法分析器将输入的字符序列分解成一系列词法符号。词法符号是语言的基本词汇符号，可以代表如“标识符”一类的符号，也可以代表一个单一的运算符，或者代表一个关键字。多视图统一建模语言的部分语法文件见附录 C。

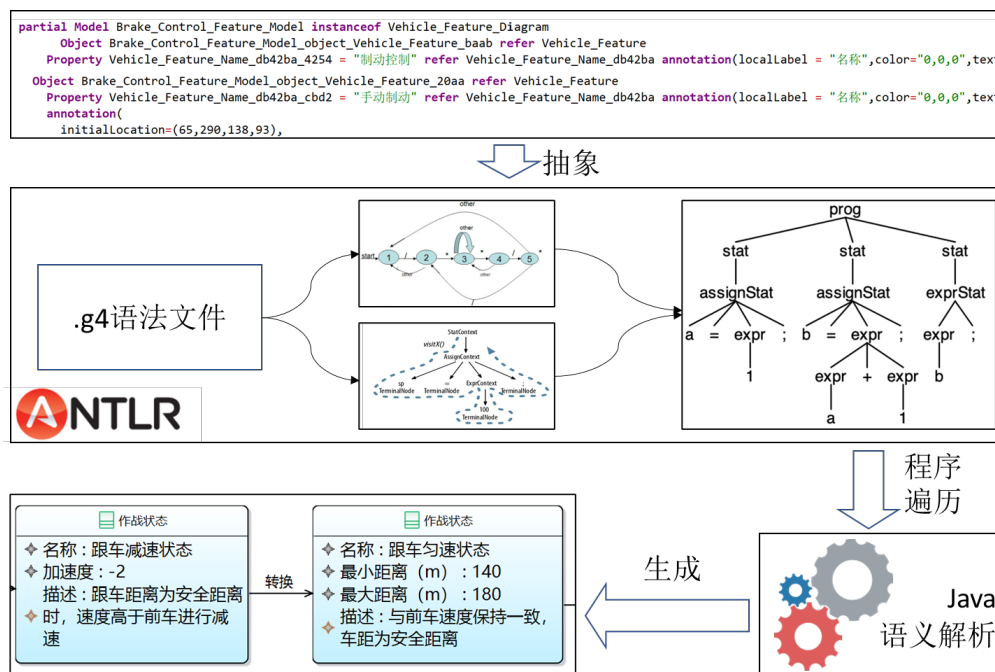


图 6.3 多视图统一建模语言解释器

以形成语法文件为基础，通过 ANTLR 工具解析，形成具有接口功能的词法分析器和语法分析器，并自动建立语法分析树，语法分析树是一种描述语法与输入文本匹配关系的数据结构，如图 6.4 所示为输入语法 6.1 中多视图统一建模语言所做出的匹配，如其中的 String 匹配到语法分析器规则 dataType 的子规则 primitiveType 下的规则 textualType，而如 Project、annotation 等关键词则以模板的形式固定在语法的相应位置。

语法 6.1: 多视图统一建模语言示例

- 1 **Project** projectSystemName **annotation** (localLabel="项目一")
- 2 **Language** languageSystemName **annotation** (localLabel="语言一")
- 3 **String Property** address00 **annotation** (localLabel="属性 1",description="meta property");
- 4 **end** languageSystemName;
- 5 **end** projectSystemName

在代码层，ANTLR 生成的词法分析器和语法分析器，包含以下几个文件：

- (1) 文件 FormalizeParser.java 包含的是语法分析器的类定义；

- (2) 文件 `FormalizeLexer.java` 包含的是词法分析器的类定义;
- (3) 文件 `FormalizeVisitor.java` 是语法分析树访问器;
- (4) 文件 `FormalizeBaseVisitor.java` 是接口的默认实现类, 为语法分析树访问器中的每个方法提供一个空实现;
- (5) 其他非 Java 文件如 `Formalize.tokens`, `FormalizeLexer.tokens`, 等存储数字形式的词法符号类型对应的关系。

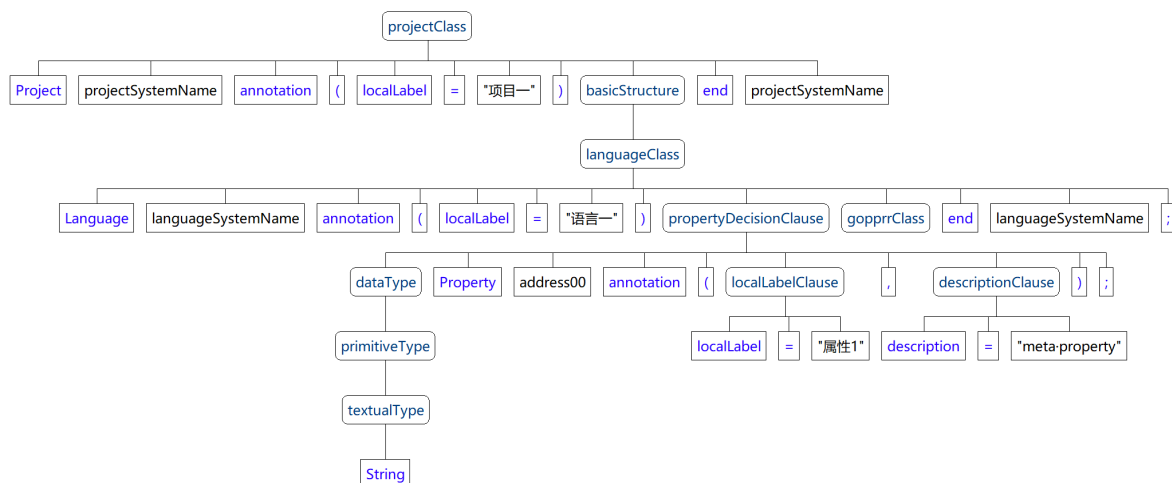


图 6.4 语法分析树

本文通过继承 `FormalizeBaseVisitor` 类, 构建 `xxxVisitor.java` 等文件, 对本文所关注的类方法进行重写来控制遍历顺序, 并与前端可视化环境进行语义映射, 实现对多视图统一建模语言的读取。

6.2.4.2 多视图统一建模语言代码生成器

通过建立图形化模型与多视图统一建模语言的映射机制, 语言代码生成器实现前端可视化模型向结构化的多视图统一建模语言生成的过程。如图 6.5 所示, 主要是元模型、模型元素的解析, 并将解析到的元素和数据映射到语言模板中, 通过组合形成结构化的多视图统一建模语言。

首先基于多视图统一建模语言规范, 定义语义模板, 根据语言数据结构, 定义项目、语言、(图、对象、关系、点、角色、属性) 元模型、(图、对象、关系、点、角色、属性) 模型、(笔记、链接、表单、布局、甘特图) 插件等节点的语义模板。语义模板采用 `StringTemplate` 模板引擎实现, `StringTemplate` 是一种基于 Java 的模板引擎库, 模板引擎是一个代码生成器, 使用模板创建文本, 这些模板实际上只是带有“洞”的文档, 我们可以在其中粘贴称为属性的值。属性可以是程序对象 (例如字符串或

VarSymbol 对象)、模板实例,也可以是包括其他序列的属性序列。

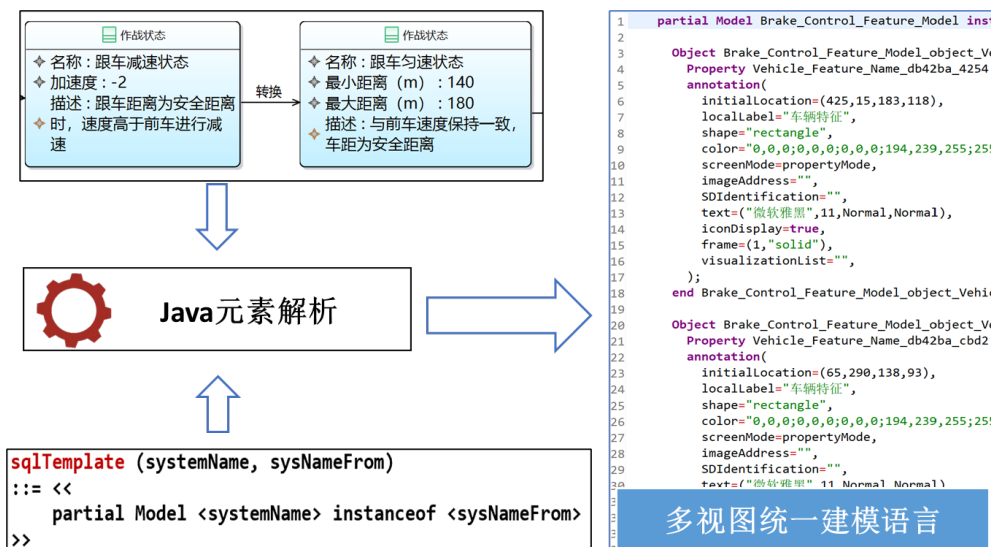


图 6.5 多视图统一建模语言代码生成过程

本文使用模板引擎是用于生成结构化文本的多视图统一建模语言, StringTemplate 将模板分解为文本和属性表达式块, 默认情况下, 它们包含在尖括号中, 如语法 6.2 所示代码块, 关键词 *sqlTemplate* 是 StringTemplate 模板的声明, *systemName* 和 *sysNameFrom* 是模板片段中的值, *partial*, *Model* 和 *instanceof* 作为多视图统一建模语言的关键词, 放在模板片段的固定位置。

语法 6.2: 语言生成模板

```

1 sqlTemplate (systemName, sysNameFrom)
2 ::= <<
3   partial Model <systemName> instanceof <sysNameFrom>
4   >>

```

将多视图统一建模语言的所有节点生成 StringTemplate 模板之后, 开发 Java 代码块实现元模型和模型元素的解析, 将解析识别得到的模型信息自动填充到所定义的模板中, 实现与建模信息一致的语言生成。

6.2.5 平台开发与运行环境

多视图集成表达验证平台是在开源的集成开发环境 Eclipse 中完成开发的基于 Eclipse 平台框架的客户端软件平台。开发过程中主要使用 Java 语言, 并使用了 SWT、JFace、Eclipse GMF、Sirius 等界面编程框架。平台开发完成后打包可执行文件.exe,

可在计算机上直接安装使用。由北京理工大学在“十四五”装备预研共用技术项目“多系统建模与设计空间智能决策技术”期间开发的多系统建模与设计空间智能决策系统和北京理工大学合作国家重点研发计划项目“复杂产品设计制造服务一体化系统建模理论”期间开发的设计制造服务一体化系统中，作为系统特定域建模语言的开发、方法论应用、建模过程实现的平台。

表 6.1 和表 6.2 分别对平台的开发和运行环境信息进行了概括。

表 6.1 平台开发环境

项目	内容
操作系统	Windows 10
开发工具	Eclipse、ANTLRWorks2、Gitee
开发语言	Java、JavaScript
界面生成器	WindowBuilder
工具运行框架	Eclipse Plug-in Register、Eclipse P2 updates、Eclipse OSGi

表 6.2 平台运行环境

项目	内容
操作系统	Windows 11、Windows 10、Windows 7
内存	8G 以上
硬盘	250G 以上
显示器	分辨率 1024×768、刷新率 60Hz 以上

6.2.6 平台主界面

登录多视图集成表达验证平台，导入建模语言之后，系统工程师使用的建模环境主要包含图 6.6 中的 5 部分，①是导航栏，领域专家和架构设计师可通过导航栏创建项目、语言、元模型和模型，进行保存、返回、编译等操作，并开展平台具备的功能实现见文献^{[[133]-[138]]}，包含架构驱动、代码生成、指标可满足性验证、行为仿真、模型服务化、DSM 设计结构矩阵设计等。同时提供了与 Simulink、OpenModelica 工具的接口插件功能，可直接调用工具进行仿真。②是浏览器，平台提供了三种模式的浏览器：资源浏览器、模型浏览器、包浏览器，其中资源浏览器树状展示结构展示了建模过程的所有元素，包含项目集合，GOPRR 元模型构建的元素，模型构建的元素，以及 icon、图片等资源；模型浏览器侧重于展示、查看各个图元模型实例化的模型；包浏览器旨在对模型进行分类，如分为需求包、功能包、逻辑包、物理架构包管理各模型，实现对模型的导航。③是大纲，即展示建模区画布上的缩略图，方便鸟瞰模型整体结构，并快速进入指定的模块位置。④是建模区，是平台最核心的功能区域，主要分为控制面板和画布两部分，控制面板提供了对象集合、关系集合、插件集合，用以

拖拽进画布进行建模。⑤是结果展示区，分为控制台、SMT 分析结果、属性三个子区域，其中控制台监控建模过程中后台的运行过程，当出现错误时提示报错信息，正常运行时显示各个模型创建成功；SMT 分析结果针对平台功能——指标可满足性验证，当使用该功能时，分析过程和结果会在该区域展示；属性区是画布中建模功能的延伸，画布中各对象模块、关系模块的元素信息展示在属性区域，如对象的元模型信息、对象 id、对象名称标签、对象的属性等，以支持精细化建模。

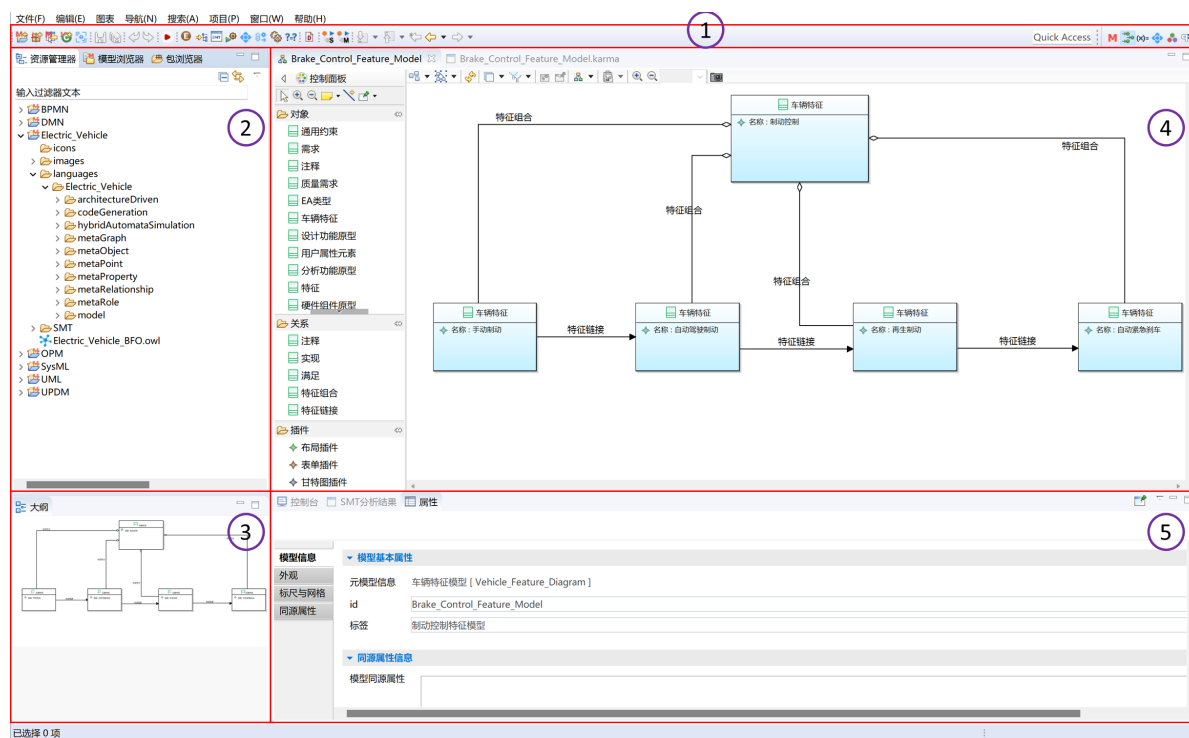


图 6.6 建模环境主界面

除了以上 5 部分建模相关区域外，平台设置顶端菜单栏区域，用于对平台本身进行设置，如创建文件、搜索文件、窗口首选项等。此外，平台中元模型的构建界面将在下一小结中展示。

6.3 分层抽象模型表达能力验证

本节通过多视图集成表达验证平台，对提出五种现有的 MBSE 语言进行定量和定性分析，验证统一分层抽象模型对复杂系统生命周期多种建模语言的满足性表达能力。

6.3.1 定量分析

定量分析是一种研究策略，侧重于数据收集和分析的量化。多视图统一建模语言

的原则就是通过 GOPRR-E 中的 6 种分层抽象模型，对建模语言进行分析和匹配，当建模语言的所有要素都可以通过本文的 6 种抽象模型进行描述，则可证明本文的分层抽象模型实现了对该建模语言的完全表达。使用现有的五种建模语言规范，包括 BPMN、DMN、OPM、UPDM 及 SysML，作为参考来提取元模型层抽象模型，实现抽象模型的满足性验证。

表 6.3 是几种建模语言的信息，其中，BPMN 支持霍尔模型过程 A₁₂，A₁₃，A₁₃，即体系、系统、子系统阶段的使命分析和运行分析；DMN 支持霍尔模型过程 A₂₆，A₃₆，即物理架构的决策过程；OPM 支持霍尔模型过程 A₂₃，A₃₃，A₂₆，A₃₆，即系统、子系统的运行分析、逻辑分析、物理架构分析过程；UPDM 支持霍尔模型过程 A₁₁，A₁₂，A₁₃，即体系设计阶段的需求、使命分析、运行分析过程；SysML 支持霍尔模型过程 A₂₃，A₂₄，A₂₅，A₂₆，A₃₃，A₃₄，A₃₅，A₃₆，即系统、子系统设计阶段的运行分析、逻辑分析、物理架构分析过程。

表 6.3 建模语言信息

建模语言	版本	维护机构	支持霍尔模型阶段	文献参考
BPMN	2.0.2	OMG	A ₁₂ ，A ₁₃ ，A ₁₃	[140]
DMN	1.2	OMG	A ₂₆ ，A ₃₆	[141]
OPM	19450:2015	ISO	A ₂₃ ，A ₃₃ ，A ₂₆ ，A ₃₆	[142]
UPDM	2.1	OMG	A ₁₁ ，A ₁₂ ，A ₁₃	[143]
SysML	1.6	OMG	A ₂₃ ，A ₂₄ ，A ₂₅ ，A ₂₆ ，A ₃₃ ，A ₃₄ ，A ₃₅ ，A ₃₆	[144]

抽象过程以 SysML 活动图为例，通过分析活动图包含的对象、关系等确定元模型。在 SysML 中活动是行为的表达形式，通过受控动作序列实现输入和输出转换。活动图是基于流行为建模的主要表示方式，类似于广泛用于系统行为建模的功能流图。活动提供了超越传统功能流图的增强功能，如表达活动与系统结构（块，组成）关系的能力以及建模连续流动行为的能力。

通过对 SysML 活动图动作、节点、流等分析和细化，映射为统一抽象元模型如表 6.4 所示，将 SysML 活动图分解为图元模型 1 个，对象元模型 24 个，点元模型 4 个，关系元模型 3 个，角色元模型 4 个，属性元模型 34 个。在附录 D 中，展示了活动图中图元模型形式化的多视图统一建模语言。

因为篇幅限制，展示 SysML 的多视图统一建模语言的在代码托管协作平台 gitCode 中展示（网址：<https://gitcode.net/bit01/sysml-language-specification.git>）。SysML 其他图也按照上述方式分解和抽象，共建立 SysML 图元模型 9 个，对象元模型

83 个，点元模型 12 个，关系元模型 31 个，角色元模型 31 个，属性元模型 92 个。

表 6.4 活动图抽象元模型

抽象元模型	元素			数量
图元模型	活动图			1
对象元模型	调用行为动作	调用运行动作	不透明动作	24
	中央缓冲区节点	数据存储节点	活动参数节点	
	发送信号动作	接收事件动作	接收时间事件动作	
	接收更改结构特性事件动作	初始节点	活动结束节点	
	流结束节点	决策节点	合并节点	
	分支节点	汇合节点	可中断的活动区域	
	结构化活动节点	扩展区域	条件节点	
	循环节点	顺序节点	泳道	
点元模型	输入引脚	值引脚	动作输入引脚	4
	输出引脚			
关系元模型	控制流	对象流	异常处理程序	3
角色元模型	来源	目标	保护节点	4
	异常输入			
属性元模型	名称	行为	待办事项	34
	运行	身体和语言	类型	
	选择	速率	排序	
	处于状态	是否控制类型	重定义节点	
	参数方向	参数	参数速率	
	多重性	事件类型	事件元素	
	触发器	是否解编	发生时间	
	是否相对	信号	处于端口	
	是否本地可重入	更改表达	中断边	
	端口	结构特性	决策输入流	
	保护	概率	重定义边	
	异常类型			

基于以上的方法，设计并统计抽象元模型实现以上建模语言如表 6.5 所示。

表 6.5 建模语言中抽象元模型数量

语言	图	对象	点	属性	关系	角色	对比建模工具
BPMN	3(3)	58(58)	10(0)	55(55)	3(3)	4(0)	Cameo
DMN	1(1)	4(4)	0(0)	21(21)	3(3)	6(0)	Camunda Modeler
OPM	1(1)	3(3)	0(0)	14(14)	15(15)	16(0)	Opcat
UPDM	52(52)	68(68)	5(0)	111(111)	24(24)	23(0)	Enterprise Architect
SysML	9(9)	83(83)	12(0)	92(92)	31(31)	31(0)	MagicDraw

其中，括号内数字为对比建模工具中该建模语言的元素数量统计，括号外数字为使用多视图集成表达验证平台构建的抽象元模型数量，表中点和角色出现括号内外数字不一致的情况，是因为在其对比建模工具中无点和角色的明确定义，在本文的抽象

模型中将点和角色元模型进行适配。表中结果表明, SysML、UPDM 等多种建模语言均可通过统一抽象元模型的描述, 实现对建模语言的完整性表达, 验证了多层抽象模型对多建模语言模型元素的表达能力。

6.3.2 定性分析

定性分析是从语言的性质、特点和变化对语言完整性的主观判断。本文从可视化元模型规范技术、连接器类型规范技术、符号规范技术、视觉表现力技术等^[139]方面对统一抽象模型做出定性分析, 并对比 BPMN、DMN、OPM、UPDM、SysML 等建模语言, 展示统一分层抽象模型的性能。

(1) 可视化元模型规范技术

表 6.3 中的所有建模语言规范都引入了一个表示建模语言语法的元模型。但是, 用于指定元模型的技术存在差异。大多数规范使用可视化元模型, 可视化元模型规范技术使用多种图形化方法来指定建模语言的语法规则, 其包含了以下几种元模型模式:

1) 切片元模型是指将完整的元模型分成多个切片, 每个切片至少包含一个复用元素。本文基于统一抽象元元模型, 从元元模型生成元模型, 再由元模型组成建模语言, 该过程中的元元模型即是最底层的切片, 所生成的元模型, 如属性元模型被对象元模型和关系元模型多次复用, 符合切片元模型的定义。

2) 引用元模型是指一种特殊的切片元模型, 切片元模型本质上会引入冗余, 为了缓和这种影响, 通过引用模式缓解冗余, 例如语法 6.3 中的属性引用, 使用关键词 `extends` 来实现对元模型的引用, 这些引用元模型使用的引用元素仅包含不同切片中引用的元模型元素的名称, 而省略了属性等附加信息。

语法 6.3:属性元模型及对象元模型中的属性

```
1  initial Property unique numberTest = 100 extends celltest annotation (localLabel="原属性");
   //对象元模型中引用 numberTest 生成的新的属性, 其初始属性值来自 numberTest。
2  Property gr1 lex1 extends numberTest annotation (localLabel = "引用属性");
```

3) 泛型元模型描述模型的泛化能力, 为建模语言引入结构。本文基于 GOPRR-E 抽象元元模型以及 M3-M0 建模框架, 具备标准的框架式语言结构, 可泛化为 BPMN、DMN 等建模语言以及特定域建模语言, 所有语言的元模型派生自通用的元元模型。

4) 表单元模型主要是用于收集系统设计中的相关数据。通过使用表单插件工具, 可以将系统设计中的数据进行记录、存储、处理和汇总。

结合文献^[139], 加入本文统一分层抽象模型显示化的多视图统一建模语言, 表 6.6

总结几种建模语言对可视化元模型规范技术的支持，其中多视图统一建模语言支持所有的可视化元模型规范技术。

表 6.6 可视化元模型规范技术的使用

	切片 元模型	引用 元模型	泛型 元模型	表格 元模型	使用的 数量
BPMN	√	√	×	√	3
DMN	√	√	×	√	3
OPM	×	×	×	√	1
UPDM	√	√	×	√	3
SysML	√	√	×	√	3
多视图统一建模语言	√	√	√	√	4

(2) 连接器类型规范技术

模型层的连接器是指两个或多个对象类型实例的连接结构，元模型层的连接器类型规范是指在两个对象类型之间使用连接线。根据 Dominik 对不同连接器规范的总结，具体的连接器类型规范有以下几种：

表 6.7 连接器类型分析

	连接器	连接器类型	自然语言	表单或矩阵	使用的数量
BPMN	×	√	√	√	3
DMN	×	√	√	√	3
OPM	×	×	√	√	2
UPDM	√	√	√	√	4
SysML	√	√	√	√	4
多视图统一建模语言	√	√	√	√	4

1) 通过一个连接器，其连接器类型由元模型中关联两个对象类型的连接器指定，这样指定的连接器通常没有属性，对象类型表示连接器类型的实例的起始和端点。在本文抽象元模型中，连接器用 `connector` 表示，没有属性的连接器指连接器中的 `Relationship` 及两端配置的 `Role` 没有属性。

2) 通过一个连接器类型，这里的连接器类型不同于 1) 中的连接器，类似于对象类型，连接器类型是通过 `Relationship`、`Role` 配置属性及展示相应的固定语法和自然语言名称实现。

3) 通过自然语言，该规范技术与元模型及元模型之间的关系无关，而是通过自然语言的方式描述模型之间的关系，因为自然语言的灵活性，可在无法直接表示图与图、对象与对象、图与对象关系等弱约束性，不需要建立可视化的连接关系时使用，本文的抽象模型通过 `Node`、`LinkModel` 等类中的自然语言编辑能力实现。

4) 通过表格或矩阵指定连接器类型被广泛使用, 因为矩阵可以紧凑的可视化给定对象类型之间的所有允许连接器类型, 使用户可以轻松快速定位和查看。

几种建模语言的连接器类型分析见表 6.7, 多视图统一建模语言支持了所有的连接器类型。

(3) 符号规范技术

符号规范是指在为系统建模者开发模型的具体语法时, 基于图形规范对元模型(对象、关系、点和角色)实例的图形可视化, 可视化对于系统建模者的沟通和理解起着重要的作用。基于多视图统一建模语言及适配建模环境的开发, 可以根据抽象元模型要求适配相应的符号。本文为统计和比较提供了四种符号规范:

1) 自然语言符号是指通过自然语言在建模构造中引入符号, 即声明“对象元模型 A 由黑色实心圆表示”。

2) 动态表示法是指对象模型可以根据当前状态(即属性值)更改模型符号形状的表示法, 本文抽象元模型可在 *selfShape* 中自定义多种形状, 以指定不同的状态。

3) 替代表示法是指相同的语法可以由可视化表示中的多个可选符号表示, 当针对特定领域的建模或向特定领域扩展建模时替代表示尤为重要, 由于统一抽象元模型较高的设计自由度, 可根据领域的不同选择适合但不唯一的符号作为固定语法。

4) 一致性水平用于遵守强制性符号指南, 即每种建模语言的软件支持工具都需要使用一致的符号。一致性水平对于旨在实现建模语言的软件工具供应商来说尤为重要。

如表 6.8 的总结分析, 多视图统一建模语言支持所有四种类型的符号规范。

表 6.8 符号规范技术

	自然语言符号	动态表示法	替代表示法	一致性水平	使用的数量
BPMN	√	√	√	√	4
DMN	√	×	√	√	3
OPM	√	√	×	√	3
UPDM	√	√	√	√	4
SysML	√	√	√	√	4
多视图统一 建模语言	√	√	√	√	4

(4) 视觉表现力技术

视觉表现力支持对建模结构的感知和判别。视觉表现力是通过可视化元模型的不同实例的能力来度量的。视觉表现力是一个不模糊的度量, 不受评估者的主观感受的

影响，并且大多数规范没有严格的符号规定来阻碍建模语言的目的或对语义的评估。

Bertins 视觉字母表^[145]包含 8 种可视化功能来衡量视觉表现力，包括颜色、形状、位置（水平/垂直）、亮度、大小、方向和纹理，考虑的变量越多，视觉表现力就越强，更高的视觉表现力促进建模构造的感知可辨别性，即构造应很容易相互区分。这 8 个变量在一定程度上是自描述的，相互之间没有关联。理论上，最大的视觉表现力是 8，最低的是 0。但是本文在基于模型系统工程领域，所列的语言都被认为是采用图表表示的概念建模语言，因此，所有语言都给出了位置变量，最低的表现力是 2。视觉字母表专注于对象类型，而不是连接器类型——例如，对象大小或纹理变量。视觉字母表的变量可以根据建模工具供应商的偏好来决定，但使用更多的需要系统设计人员对领域建模场景的理解和设置偏好，如颜色被用来区分框架的各个层级别，行为对象用圆框表示，主动和被动结构元素用方角框表示，动机对象通常用对角线框表示。本文的统一建模语言的分层抽象模型通过关键词 *color* 定义颜色，通过关键词 *shape* 定义形状，通过关键词 *initialLocation* 定义位置（水平/垂直）和大小，而目前的建模语言规范鲜有提到节点（对象）亮度、方向和纹理三种变量，例如，很少会出现在某种条件下描述用于抛出事件的节点必须有一个暗填充这样严格的标记准则。因此语言对于领域建模的表达并未做出相应支持。

在表 6.9 的总结中，多视图统一建模语言的视觉表现力最强。

表 6.9 建模语言符号可视化

	视觉表现力	变量
BPMN	4	颜色，形状，位置(水平/垂直)
DMN	3	形状、位置(水平/垂直)
OPM	4	颜色，形状，位置(水平/垂直)
UPDM	4	亮度，形状，位置(水平/垂直)
SysML	4	颜色，形状，位置(水平/垂直)
多视图统一建模语言	5	颜色，形状，大小，位置(水平/垂直)

以上规范技术定性分析多视图统一建模语言的抽象元模型或模型，展示了分层抽象模型对其他建模语言建模要素功能的覆盖能力，验证了复杂系统使用多视图统一建模语言进行生命周期多阶段建模的可能性。多视图统一建模语言的抽象元模型支持切片元模型、引用元模型、泛型元模型等可视化元模型规范技术的表达，可满足对 SysML、OPM 等建模语言可视化结构的要求；多种连接器类型是对建模过程的约束，不同的建模语言有不同的连接规范，多视图统一建模语言的抽象元模型灵活的支持多种连接器类型规范技术，可满足不同场景的建模；分层抽象模型包含丰富的对象形状，

以及形状自定义能力，对于基于模型系统工程领域的复杂系统建模要求，满足其符号规范技术和视觉表现力技术。

6.4 基于案例的形式化表征及多视图统一建模语言能力验证

智能电动汽车的设计并没有先验经验作为参考，本文采用 MBSE 的方法实现复杂装备智能电动汽车的正向设计，通过多视图统一建模语言满足电动汽车体系、系统、子系统等生命周期多阶段集成建模，验证形式化复杂生命周期能力及统一建模语言支持复杂建模过程的能力。

6.4.1 智能电动汽车架构设计方法

智能电动汽车的架构设计是一个复杂的基于模型系统工程，本文基于系统性思维方法，开展智能电动汽车的架构设计。如图 6.7 所示，通过以下四个步骤开展架构设计。

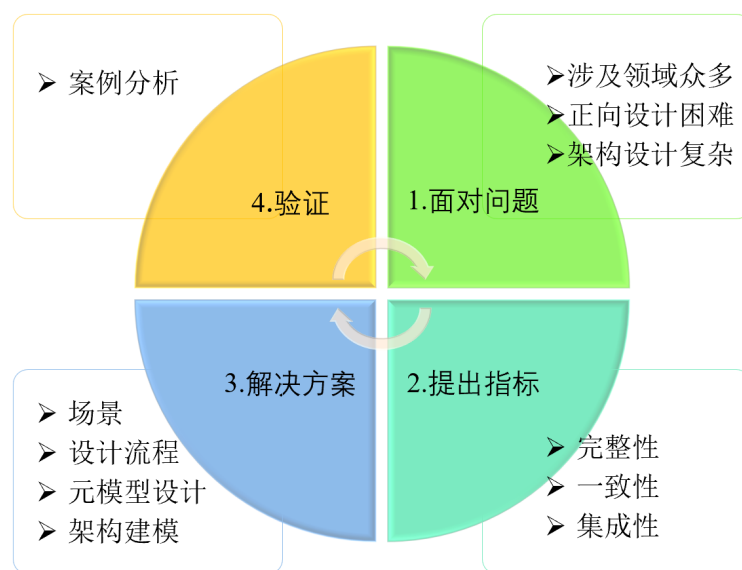


图 6.7 系统性思维方法

首先是面对智能电动汽车的适用环境和研发提出问题，如涉及的领域众多，正向研发设计困难，从使命任务开始的架构设计过程复杂。其次根据问题提出指标，如模型生命周期的完整性、表达的一致性、多阶段多领域集成性等。再次通过场景分析、设计正向设计流程、元模型设计，进行架构建模。最后验证阶段进行案例分析。

6.4.2 架构设计问题及需解决的指标

如图 6.8 的智能电动汽车，架构设计涉及路况、驾驶、环境、通信等多方面的影响，是一个复杂的系统设计过程。特别是在智能交通系统中，随着车辆增多、路网扩大、终端设备的增多，导致网络体系复杂、系统交互众多、系统不确定性增加，面临着车辆的正向架构设计越发复杂的问题。

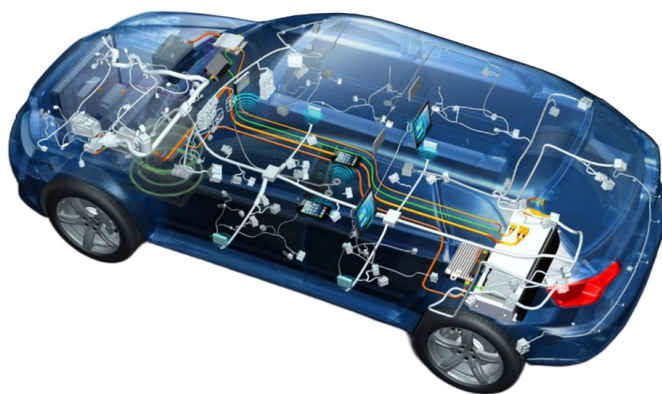


图 6.8 智能电动汽车

图片来源: <https://zhuanlan.zhihu.com/p/341142015>

因此，本文提出以下性能指标，以支持智能电动汽车的架构设计和验证：

(1) 完整性，智能电动汽车涉及从智能交通网的体系分析、整机系统设计、子系统设计等阶段，设计过程需保证建模过程的完整性，以提高模型质量。

(2) 一致性，智能电动汽车涉及多个领域，多个阶段过程，设计过程需要保证模型语义的一致性，以避免多领域多阶段同一含义术语表述不一致导致的歧义，以及数据能够在不同领域和阶段一致性传递。

(3) 集成性，架构建模需要建模语言的支持，目前单一建模语言很难满足建模过程的需求，若采用多种建模语言，则必然面临多语言的集成和数据交互的问题，因此需考虑设计过程中语言的集成性，以提高数据传递效率，从而提高设计效率。

基于以上性能指标开展的智能电动汽车的架构设计，可提高设计效率，保证模型质量。

6.4.3 智能电动汽车架构设计方案

6.4.3.1 智能电动汽车架构设计场景

针对上述问题，本文开展智能电动汽车的架构设计。智能交通系统包含的智能电动汽车、导航卫星、信号塔、控制中心等，通过借助互联网、物联网以及大数据技术

的优势，更多的引入高新技术设备，能够充分掌握车辆的行驶状况以及周边的交通运输环境。当车辆面临路面突发情况时，智能交通系统将第一时间发现危险情况，响应突发事件。智能交通系统体系复杂，各部分之间在运行过程中存在着众多交互，大大增加不确定性，在智能交通系统的顶层使命任务研发体系中，以智能电动汽车为对象的设计涉及到环境感知系统、自动驾驶系统、信号交互系统等多个系统，是一个复杂的过程。

本文将从智能交通系统中两辆车遇到障碍自动刹车的场景为使命任务出发，开展针对智能电动汽车的架构设计，场景如下：

（1）在单向道，两辆车前后起步发车。两辆车等待发车，前车出发到匀速行驶，后车发车行驶。

（2）两车同时在直线路段上行驶。前车正常速度行驶，后车在行驶过程中逐渐加速追上前车。后车开始减速，减速到与前车保持匀速行驶。

（3）控制中心检测到前车前方有障碍物，前车随即减速停车。前车同步障碍物信号到后车，后车减速停车。随后两辆车停止行驶。

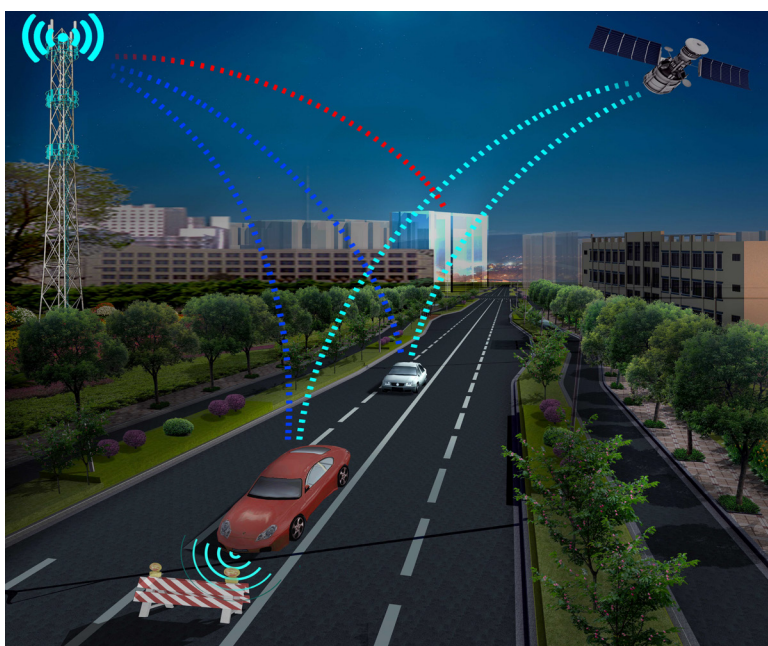


图 6.9 智能交通系统

在图 6.9 智能交通系统中，包括以下信息交互：

- （1）智能电动汽车与信号塔的双向通信；
- （2）智能电动汽车请求与接收通信卫星的 GPS 信号；

(3) 智能电动汽车向周围发送雷达波，并接收障碍物反射的雷达波；

(4) 控制中心接收信号塔信息，并发送控制指令。

针对以上场景，在 MBSE 的建模方法中，并没有从使命开始的产品生命周期正向设计方法，且并没有单一建模语言可以满足本场景下装备的架构设计。因此，采用本文研究的多视图统一建模语言，针对交通运输系统中智能电动汽车的架构设计问题，采用从使命任务出发的架构建模方法，构建智能电动汽车特定域建模语言，以满足该场景下的智能电动汽车架构正向设计。

6.4.3.2 智能电动汽车的架构建模流程

本文通过使命—运行—特征与功能—逻辑—物理架构（Mission、Operation、Feature/Function、Logic、Physical Architecture, MOFLP）的建模流程为智能电动汽车案例进行架构建模。

(1) 使命分析

通过系统顶层概念图完成智能交通系统场景的使命任务分析，分析出智能电动汽车、无线电导航卫星、信号塔、控制中心之间的交互关系，得到潜在的设计方案。

(2) 运行分析

针对智能交通系统中的智能电动汽车，使用状态转移图对作为目标系统的智能电动汽车进行运行概念分析，通过分析智能电动汽车系统的生命周期，确定系统分析的出发点，定义使命的候选运行概念解决方案。然后从运行行为中派生利益相关者需求：通过用例图对智能电动汽车利益相关者进行运行分析，并在子系统通过用例图精确定义设计者、测试人员、驾驶员等在各个运行阶段系统的预期行为。

(3) 特征和功能分析

通过车辆特征图进行智能电动汽车特征建模，包含了动力控制特征、感知特征、自动驾驶特征、横向控制特征、纵向控制特征等。根据特征映射为相应的功能分析建模，如动力总成控制功能分析、全自动驾驶控制功能分析、生成全局路径功能分析、横向控制功能分析、纵向控制功能分析等，实现对智能电动汽车行驶场景的速度控制、智能性等进行功能分析。

(4) 逻辑分析

通过逻辑架构描述刹车系统运行行为变化，通过时序图将刹车过程中涉及到的所有逻辑架构及相互之间的功能流和交互进行分析，确定抽象层次的物理视点模型，支持后续的物理架构设计。同时，通过流程图对车辆布线装配流程、内饰装配流程和车

身装配流程等进行智能电动汽车系统的装配流程进行定义。

（5）物理架构分析

通过明确实现逻辑模型中的对象对应的物理组件的及个数，确定物理组件之间的连接关系、物理组件的属性等，最终形成物理架构。其中可包含使用包图进行驾驶控制物理结构分解，使用内部模块图对其中的驾驶控制逻辑进行分析，然后通过模块定义图对驾驶控制的物理接口进行分析，最后使用故障树分析图对驾驶控制过程中的故障情况进行故障树分析和验证。

6.4.3.3 智能电动汽车特定域元模型设计

根据 6.4.3 节的智能电动汽车的架构建模流程，目前的单一建模语言没有能够与之匹配的建模规范，因此通过多视图统一建模语言开发智能电动汽车特定域建模语言是满足本案例建模的先决条件。

为实现模型的统一构建，需首先进行智能电动汽车特定域建模语言的元模型设计。为满足设计流程，如图 6.10 共构建系统顶层概念图、状态转移描述图、用例图、时序图、模块定义图、流程图、车辆特征图、故障树分析图等共 11 个图元模型。以下分别是根据 6.4.3.2 节的元模型构建方法构建的 11 个图的详情，因篇幅限制，仅对系统顶层概念图展示标识法和文本语言。

（1）系统顶层概念图描述使命任务及其分类，或想定。为展现智能电动汽车主要的运行概念和一些感兴趣或者独特的运行情况：描述主题架构和其环境间的相互作用，以及体系结构和外部系统的相互作用。其主要用途是帮助领域系统设计者进行沟通，意在表示共同决策者的想法。本案例的使命任务较为清晰，想定明确，因此在设计系统顶层概念图时精简 UPDM 图中的对象数量，设计针对智能电动汽车场景的系统顶层概念图如表 6.10，包含一个对象和一个关系，其中执行机构对象，由一个矩形图标识，其中在建模时执行机构的背景图可根据使命任务的环境进行更换，以便更为直观的图形化表达出相互之间的关系；执行机构包含两个属性，名称属性和执行机构角色属性，名称属性的属性值表达该对象的名称，执行机构角色属性的属性值表达该对象在使命任务场景中的角色，如名称为电动汽车，执行机构角色属性为运行终端。依赖关系，由一端带箭头的虚线标识，表示模型中的一种元素（客户端）依赖于模型中的另一种元素（提供者），更准确地说，依赖关系表示，当提供者元素发生改变时，客户端元素可能也需要改变。两种模型元素之间创建依赖关系，是为了建立两者之间的可跟踪性。

系统顶层概念图	用例图	时序图	模块定义图
对象 执行机构 关系 依赖	对象 执行者 执行器 边界系统 环境影响 外部系统 传感器 用户系统 用例 包 模块 系统上下文 关系 包含 扩展 关联 泛化 包图 对象 包 模型 关系 包导入 元素导入	对象 客户端端口 生命线 执行机构 关系 消息 内部模块图 对象 值属性 组成属性 引用属性 约束属性 流属性 参与属性 绑定引用 类行为属性 关系 连接器 绑定连接器 项目流连接器 项目流绑定连接器	对象 包 模块 接口模块 流规范 约束模块 域模块 子系统 外部模块 系统 系统上下文 值类型 量种类 单位 关系 接口实现 链接 关联块 定向关联 关联 定向聚合 聚合 定向组合 组合 泛化 使用 项目流
状态转移描述图	包图	内部模块图	关系
对象 分支节点 进入点 退出点 复合状态 最终状态 连接点引用 决定节点 集合节点 浅薄历史 初始状态 结束 悠久历史 状态 伪状态 子状态 关系 转换 自我转换	包图 对象 包 模型 关系 包导入 元素导入	内部模块图 对象 值属性 组成属性 引用属性 约束属性 流属性 参与属性 绑定引用 类行为属性 关系 连接器 绑定连接器 项目流连接器 项目流绑定连接器	关系 接口实现 链接 关联块 定向关联 关联 定向聚合 聚合 定向组合 组合 泛化 使用 项目流
流程图	车辆特征图	车辆功能类型图	故障树分析与验证图
对象 任务 服务任务 发送任务 接收任务 用户任务 手工任务 业务规则任务 脚本任务 子流程 事务子流程 特定子流程 调用活动 开始事件 信息开始事件 计时器开始事件 错误开始事件 关系 序列流 数据关联	对象 通用约束 需求 注释 质量需求 EA类型 车辆特征 设计功能原型 用户属性元素 分析功能原型 特征 硬件组件原型 关系 注释 实现 满足 特征组合 特征链接	对象 流出口 流入口 通用约束 需求 注释 客户端端口 电源端口 服务器端口 质量需求 用户属性元素 流入流出口 分析功能原型 关系 客户端服务器接口 注释 分配 实现 衍生需求 满足 电源 流	对象 故障中间事件 其他事件 危险事件分类 或门 故障底事件 与门 汽车完全完整性等级 故障顶事件 关系 参考 评估 约束

图 6.10 智能电动汽车特定域建模语言元模型集合

表 6.10 系统顶层概念图说明

元素	标识法	文本语言
系统顶层概念图		<pre> 1 partial Graph High_Level_Operational_Concept 2 Object Actuator; 3 Relationship Dependency; 4 Constraint (5 bind connector(Actuator,Dependency.Dependency_Incoming), 6 bind connector(Actuator,Dependency.Dependency_Outgoing), 7); 8 annotation(9 localLabel="顶层概念图", 10 type = diagram, 11 description="", 12); 13 end High_Level_Operational_Concept; </pre>
对象： 执行机构		<pre> 1 Object Actuator 2 initial Property Actuator Name ce2322 = "null" extends Name 3 annotation(localLabel ="名称"); 4 initial Property Actuator_Performer_Role_9a69d2 = "null" 5 extends Performer_Role annotation(localLabel ="执行机构角色"); 6 annotation(7 localLabel="执行机构", 8 description="", 9 icon="", 10 shape="rectangle", 11 visualizedMode=propertyMode, 12); 13 end Actuator; </pre>
关系： 依赖		<pre> 1 Relationship Dependency 2 Role Dependency_Outgoing; 3 Role Dependency_Incoming; 4 annotation(5 localLabel="依赖", 6 description="", 7 icon="", 8 shape="dottedLine", 9); 10 end Dependency; </pre>
角色： 输入角色		<pre> 1 Import Role Dependency Incoming 2 annotation(3 localLabel="依赖始端", 4 description="", 5 shape="no", 6); 7 end Dependency_Incoming; </pre>
角色： 输出角色		<pre> 1 Export Role Dependency_Outgoing 2 annotation(3 localLabel="依赖终端", </pre>

		<pre> 4 description="", 5 shape="arrowShape", 6); 7 end Dependency_Outgoing; </pre>
属性： 名称属性	◆ 名称：null	<pre> 1 String Property Name annotation (localLabel ="名称",unit = "null",description=""); </pre>
属性： 执行机构角色属性	◆ 执行机构角色：null	<pre> 1 String Property Performer_Role annotation (localLabel ="执行机 构角色",unit = "null",escription=""); </pre>

(2) 状态转移图描述活动对时间序列的相应，包含初始状态、状态、集合节点、分支节点等对象和转换、自我转换等关系，用于表达目标对象与外部的状态交互关系。

(3) 用例图描述特定的一系列动作，包括变种系列动作和错误系列动作，系统、子系统或者类可以通过于外部对象交互来执行，以提供值的服务。本案例中针对智能电动汽车的系统建模，设计用例图元模型包含执行者、执行器、用例、边界系统、外部系统等对象，以及包含、扩展、关联、泛化等关系，以表达智能电动汽车设计过程中与外部的所有交互。用例图的使用包含以下关键点：1) 用例是系统将会执行的一种服务或一种行为；2) 系统所执行的所有行为并非都是用例，用例只是系统行为的子集，是外部执行者能够直接触发或者参与的行为；3) 执行者可以是一个人或者一个外部系统；4) 触发用例的执行者叫做主执行者；5) 每个用例都因该代表主执行者的目的；6) 用例名称不会传达大量信息。用例图是系统的一种黑盒视图，因此也很适合作为系统的情境图。

(4) 包图描述系统模型的组织方式，系统模型的组织方式由包的层次关系决定，而包的层次关系则将模型中的元素分配到逻辑上紧密相关的图中。本文通过给包对象设置分解或剖视模式，可将包对象直接关联到下一层级的图中，换言之，包是一系列命名对象的容器。本案例设计的包图包含包和模型两个对象，以及包导入和元素导入两个关系，用以描述智能电动汽车系统中的结构关系。

(5) 时序图描述系统动态行为信息，说明了随着时间推移而发生的行为和时间的序列，模块的各个部分会通过操作调用和异步信号彼此交互，以产生浮现式的行为。本案例的时序图包含客户端端口、生命线、执行机构等对象和消息关系。我们可以使用生命线元素，为系统行为中的执行机构建模，然后使用生命线之间的消息，为那些执行机构之间的交互建模。

(6) 内部模块图描述了指定单个模块的内部结构，是系统或者系统一个组成部

分的静态视图，内部模块图不包含模块对象，而是用来显示对某一个模块的合法配置——模块属性之间特定的一系列连接。内部模块图包含值属性、组成属性、约束属性、引用属性等对象和连接器、绑定连接器等关系。内部模块图会表达模块的组成部分必须如何组合才能够创建有效实例，还会显示模块的实例必须如何与外部实体连接，以在整体上创建系统的有效实例。

(7) 模块定义图描述系统的结构，包含模块、流说明、系统、系统上下文、值类型等对象和关联、泛化、聚合等关系，我们需要把智能电动汽车的系统、子系统、设备等都当作一个个不同层级的模块等对象集合，再将各模块通过关联等连接方式进行组合，可形成智能电动汽车的整体架构。

(8) 流程图描述业务流程操作，是为了在创建业务流程模型时提供一个简单的机制，同时又能够处理来自业务流程的复杂性。本案例流程图参考 BPMN 2.0 规范，包含任务、服务任务、子流程、调用活动等对象和序列流、数据关联等关系，用于支持智能电动汽车的各个业务流程开发。

(9) 车辆特征图描述车辆级别汽车的特征，即通过各个特征表示“什么是车辆？”如车辆由“电机、车身、底盘、电气设备”等特征组合而成，车辆的各个变体可能具有或不具有的特征或特征。本案例车辆特征图参考 EAST-ADL 建模规范，包含车辆特征、通用约束、功能设计原型、用户属性元素等对象和特征组合、满足等关系，用以表达车辆动力控制、感知等方面的嵌入式电子架构开发。

(10) 车辆功能类型图描述车辆的功能分析架构，本案例车辆功能类型图参考 EAST-ADL 建模规范，包含流入口、流出口、分析功能原型、通用约束等对象和流、满足、注释等关系，用以表达对车辆特征模型的功能分析。

(11) 故障树分析与验证图描述系统失效问题，并且通过计算找到故障原因以降低风险。本案例故障树分析与验证图包含故障顶事件、故障低事件、或门、与门等对象和评估、参考等关系，通过对制动失灵的分析，获得故障模式所对应的故障影响。

图 6.11 是以上智能电动汽车特定域建模语言构建的 11 个图的文件资源，通过图中各元模型的构建，共同形成的智能电动汽车特定域建模语言。通过该元模型库的构建，提供了从使命任务到物理架构的建模环境，保证了对生命周期多建模语言的集成性。



图 6.11 多视图统一建模语言资源

6.4.4 基于统一建模语言智能电动汽车架构设计

本节基于 6.4.3.2 节的架构建模方法和 6.4.3.3 节的元模型，开展智能电动汽车的架构设计。通过如图 6.12 所示的流程构建智能电动汽车相关的体系、系统、子系统架构模型。

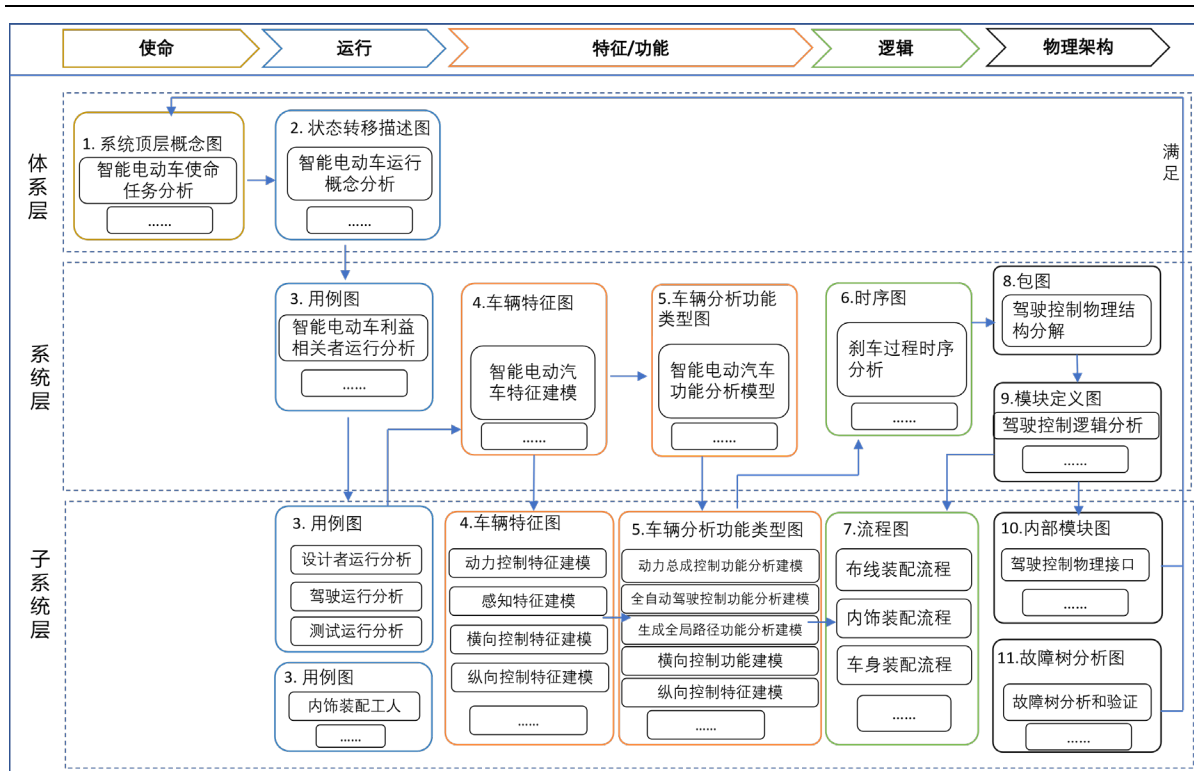


图 6.12 智能电动汽车架构模型构建流程

6.4.4.1 使命分析

两辆车遇到障碍自动刹车的使命场景描述如下：

- 在单向道，两辆车前后起步发车。
- 前车出发到匀速行驶，后车发车行驶，两车同时在直线路径上行驶。
- 前车正常速度行驶，后车在行驶过程中逐渐加速追上前车。
- 后车开始减速，减速到与前车保持匀速行驶。
- 控制中心检测到前车前方有障碍物，前车随即减速停车。前车同步障碍物信号到后车，后车减速停车。随后两辆车停止行驶。

根据以上使命场景的描述，建立如图 6.13 所示的使命任务，包含了 6 个执行机构，其中电动汽车 A 和电动汽车 B 与卫星存在 GPS 信号请求和 GPS 信号接收两个关系，完成与卫星的通信；同时两辆电动汽车之间会有雷达的信号接收和反射；电动汽车与信号塔之间传输电动汽车反馈的雷达和 GPS 信号，信号塔传输对电动汽车的控制指令，电动汽车 B 与障碍物之间关系是电动汽车 B 发射雷达信号，经过障碍物反射，接收雷达信号；信号塔与控制中心之间的关系是信号塔发射这两状态信息到信号塔，控制中心反馈车辆控制指令到信号塔。在整个使命任务过程中，电动汽车 B 与卫星、电动汽

车 A、障碍物、信号塔共 4 个对象存在信息的交互。

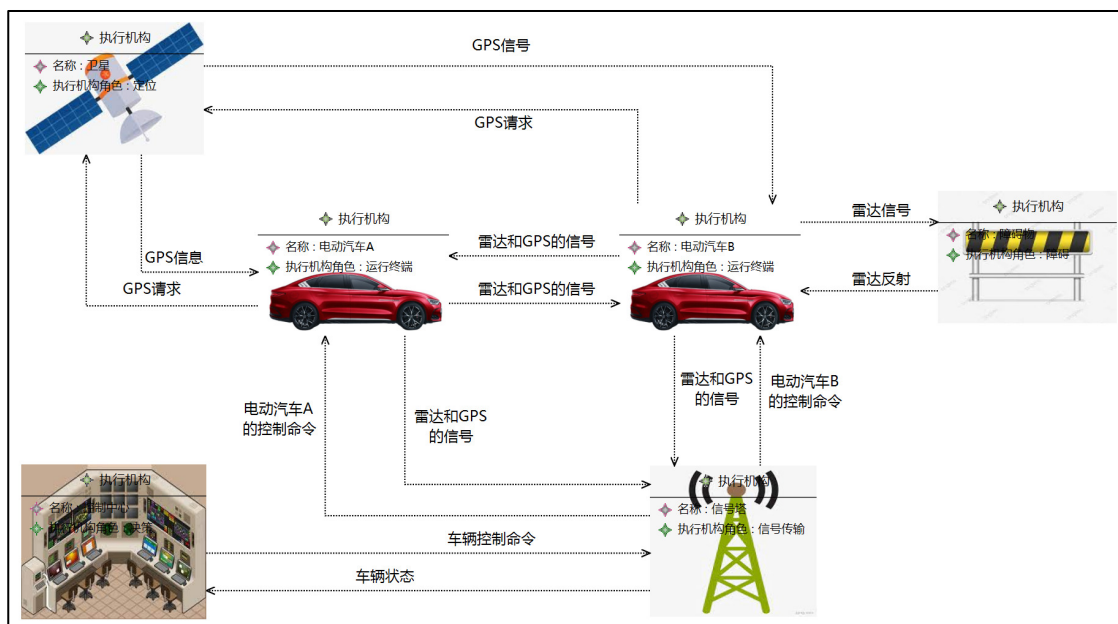


图 6.13 使命任务

6.4.4.2 运行分析

选取电动汽车 B 为研究对象，研究其与外部的状态交互关系。如图 6.14 使用状态转移图构建运行概念模型来描述电动汽车将如何响应场景中的事件，如图 6.15 和图 6.16 使用用例图构建利益相关者运行分析、驾驶运行分析等描述涉众和电动汽车系统之间的交互。

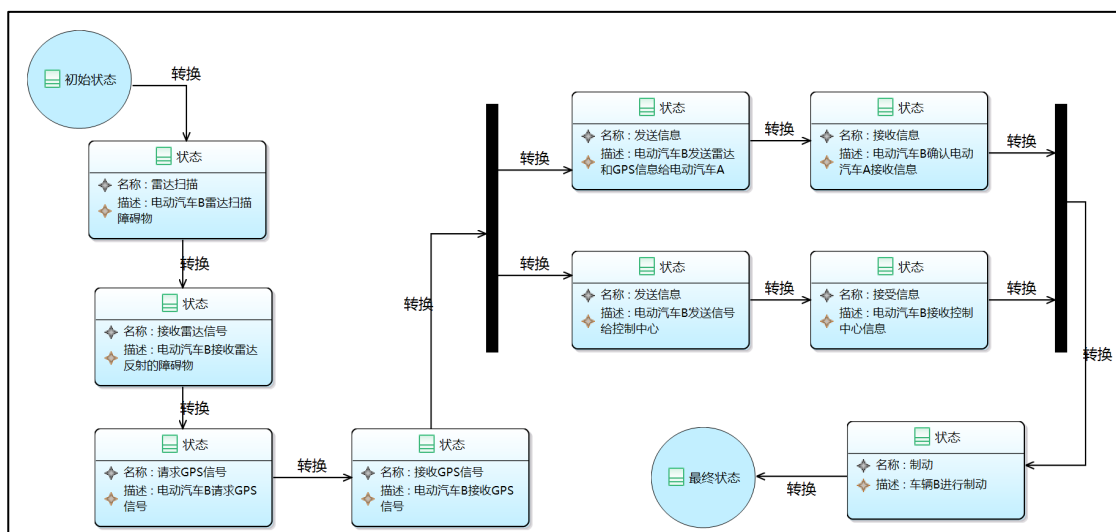


图 6.14 运行概念模型

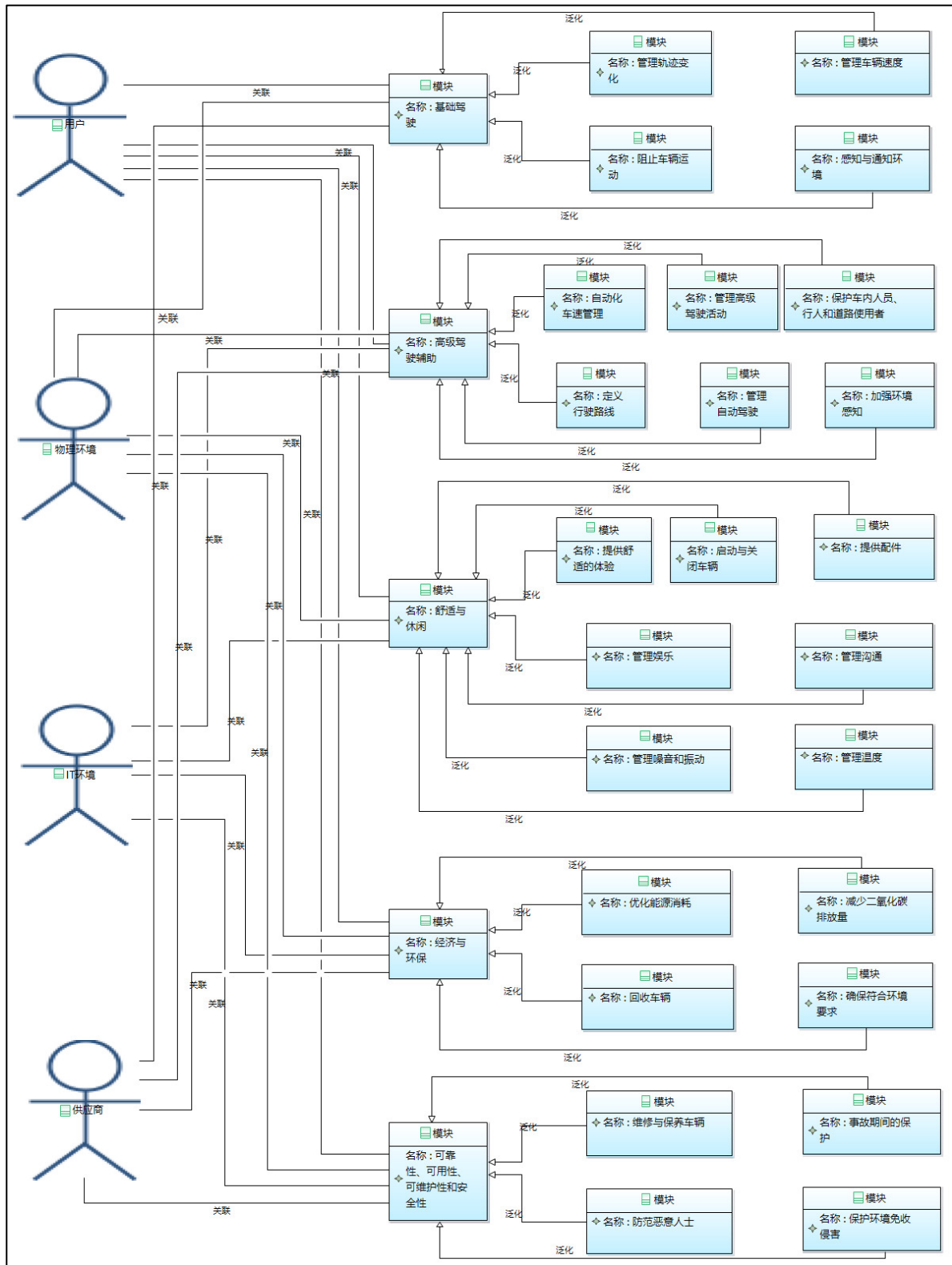


图 6.15 利益相关者运行分析

运行概念模型中选取电动汽车 B 为对象，初始状态为在车道上电动汽车 B 匀速行

驶, 然后电动汽车 B 雷达扫描障碍物, 当接收雷达反射的障碍物信息之后, 电动汽车 B 请求获得 GPS 定位信息, 在接收到 GPS 信息之后, 电动汽车 B 的状态通过分支节点转换为两种状态, 一方面传递信息, 发送雷达信号和 GPS 信息给电动汽车 A, 并接收电动汽车 A 的确认接收信息, 另一方面请求决策, 发送信息到控制中心, 并接收控制中心反馈的指令, 然后电动汽车 B 采取制动措施, 最终在障碍物之前停车。

在运行概念模型中可分析出四类利益相关人, 用户、物理环境、IT 环境和供应商, 每个利益相关人与多个模块产生关联, 如基础驾驶模块、高级驾驶辅助模块、舒适与休闲模块、经济与环保模块、靠性、可用性、可维护性和安全性模块, 每个模块又可泛化为多个子模块, 如基础驾驶模块泛化为管理轨迹变化、管理车辆速度、阻止车辆运动、感知与通知环境等。这里的用户, 也可进一步的细化为驾驶员、控制中心人员、软件测试人员、硬件测试人员、会计、软件工程师、硬件工程师等, 分别负责电动汽车的设计、测试、成本等环节。其中的每一类用户又关联多个电动汽车模块, 如驾驶员关联驾驶、定位、导航、通讯、信息分析等模块。通过对利益相关人的分析, 我们可以明确电动汽车涉及到的特征模块。

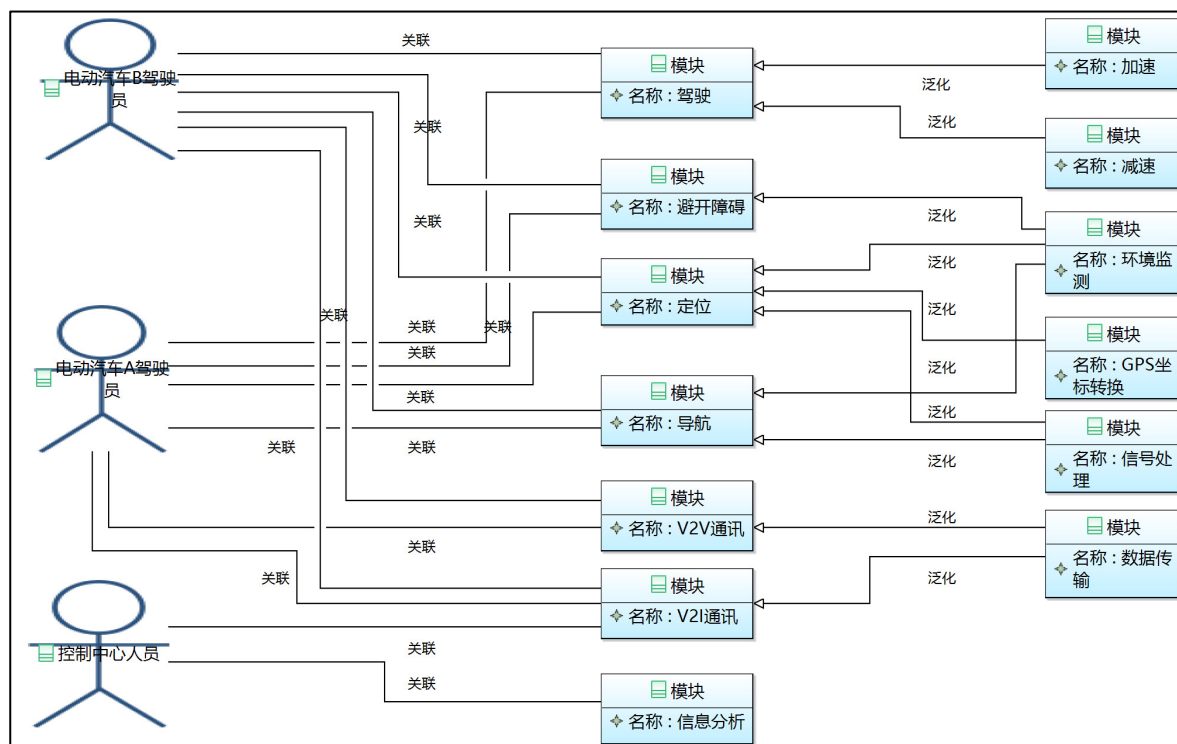


图 6.16 驾驶运行分析

6.4.4.3 特征和功能分析

如图 6.17 所示, 特征是面向用户的一类相关功能的综合, 车辆特征可扩展为车辆

的功能，功能能够实现对特征的全覆盖。针对电动汽车的安全和保障方面，本案例通过使命任务模型、运行概念模型、用例模型的提炼，描述出智能车 76 个与安全和保障有关的特征。

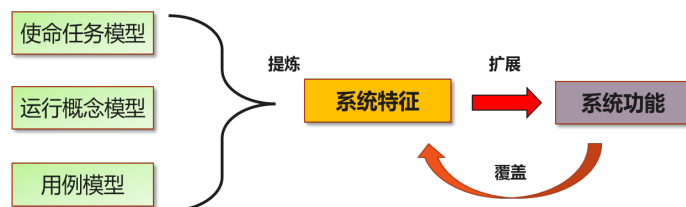


图 6.17 特征和功能关系

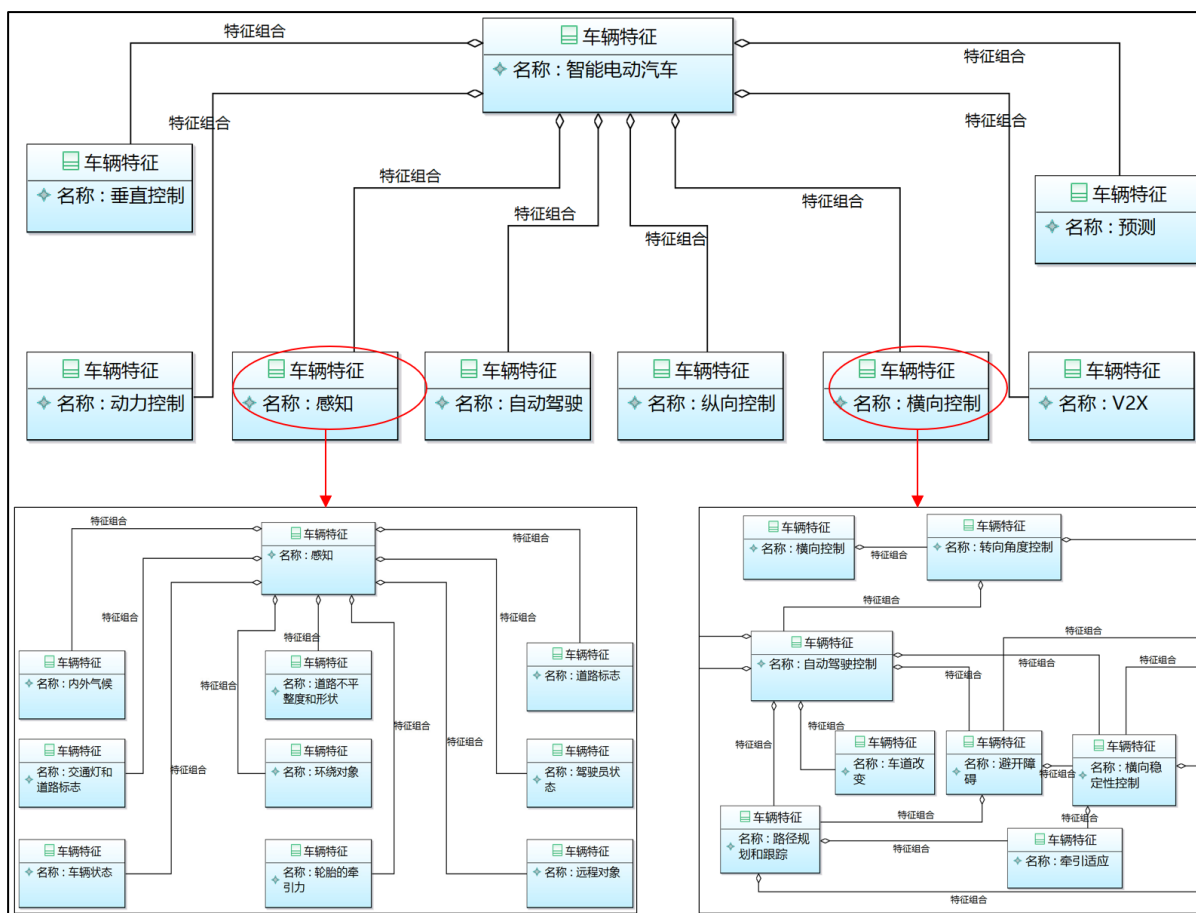


图 6.18 电动汽车安全和保障相关特征

如图 6.18 所示，智能电动汽车特征分为八组，分别是垂直控制特征、动力控制特征、感知特征、自动驾驶特征、纵向控制特征、横向控制特征、预测特征、V2X（vehicle to everything，即车对外界的信息交换）特征。其中如对象感知特征，通过分解模式，到达下一层级感知的分解图，可更详细的阐述感知特征，包含内外气候、交

通灯和道路标识、车辆状态、道路不平度度和形状、环绕对象、轮胎牵引力、道路标志、驾驶员状态、远程对象共 9 个子特征。横向控制特征可转化为转向角度控制特征，转向角度控制特征则分为自动驾驶控制特征和手动驾驶控制特征，其中自动驾驶控制特征可继续分解为更小单元的控制特征，如自动泊车、车道保持、车道改变、路径规划和跟踪、避开障碍、横向稳定性控制等；手动驾驶控制特征可分解为线控转向、避开障碍和横向稳定性控制。

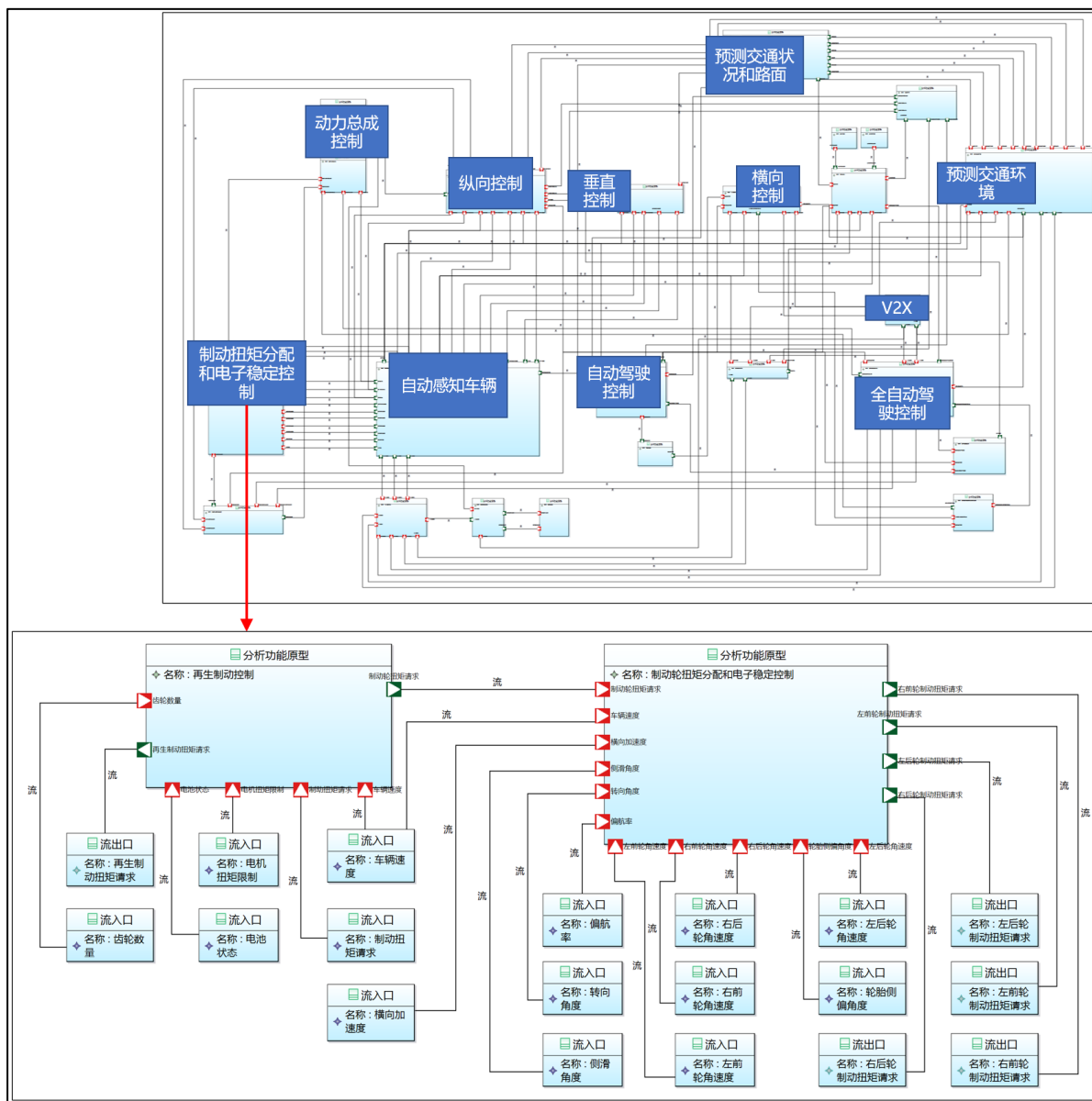


图 6.19 智能电动车功能分析模型

每个特征都由一个或多个功能实现，如图 6.19 所示，与安全和保障有关的智能电动汽车的八组特征所对应的功能之间存在着复杂的关系，如感知特征对应的自动感知

车辆功能，通过电机转速、电机扭矩限制、转向角度、齿轮数量等端口与纵向控制相关联，通过自主感知端口与自动驾驶控制相关联，通过俯仰角、车辆垂直加速度、车辆底盘高度、车辆速度、侧倾角等端口与垂直控制功能相关联，通过车辆横向速度、转向角度、偏航率、车辆速度等端口与横向控制功能相关联，通过电池状态端口与动力总成控制功能相关联，通过侧滑角、轮胎滑移角、车轮角速度、横向加速度、齿轮齿数等端口与制动扭矩分配和电子稳定控制功能相关联。

其中制动扭矩分配和电子稳定控制功能可细化为图 6.19 中所示，制动扭矩分配和电子稳定控制功能通过流关系（Relationship）与再生制动控制功能、右后轮角速度、左后轮角速度、右前轮角速度、左前轮角速度、车辆速度、制动扭矩、横向加速度、转向角度等子功能产生联系，其中的再生制动控制子功能则通过流关系（Relationship）与电机扭矩限制、车辆速度、制动扭矩请求等功能产生联系。

从特征分析到功能分析，共构建了 356 个从传感器到执行器的完整功能架构。在完成了功能建模之后，将根据系统功能列出实现此功能的对象，并通过时序图描述此功能的实现过程，完成逻辑架构设计。

6.4.4.4 逻辑分析

逻辑分析是通过建模，明确参与实现某项功能的系统对象的逻辑过程，如针对电动汽车安全和保障相关特征和功能的功能分析，通过时序图分析，获得电动汽车刹车的逻辑过程。

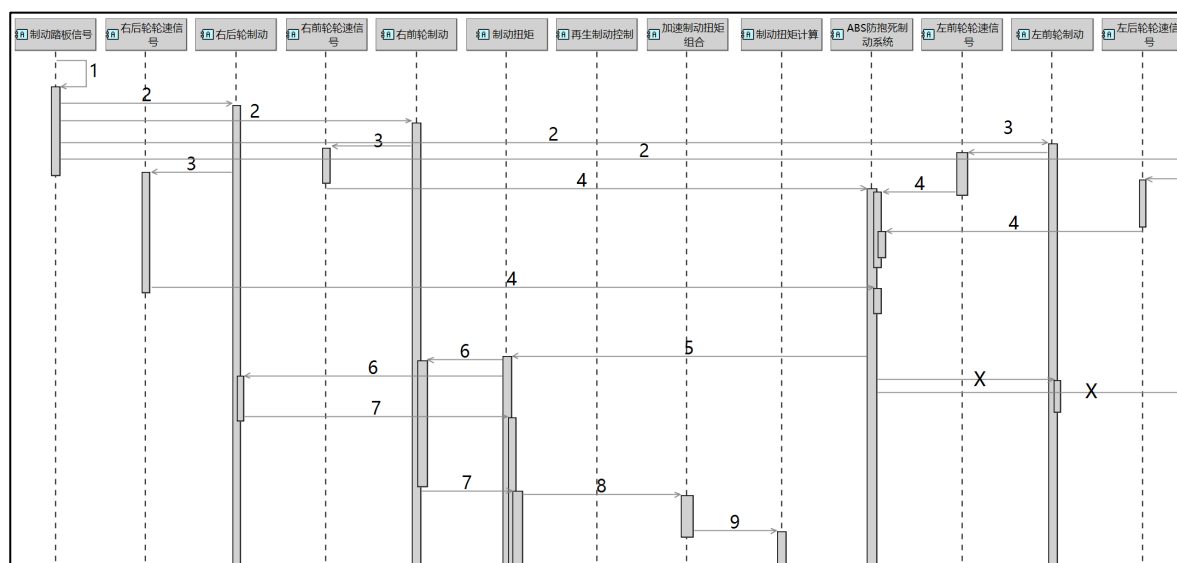
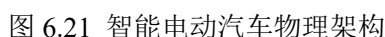


图 6.20 具有 ABS 功能的刹车时序图

物理架构分析是根据逻辑模型中的对象,通过经验或数据库的支持,明确对应的物理组件及个数等,并确定物理组件之间的接口关系、物理组件的属性等,如制动踏板信号可映射为物理组件中的刹车踏板和刹车踏板传感器。



129

系统、四轮刹车系统、四轮悬挂系统、转向舵等，其中的电机接收电源系统供电，并通过制动过程的再生制动系统产生电力，传输给电源系统，同时电机通过齿轮箱将动力传输给四个车轮，车前轮又通过转向舵控制方向，各车轮传感器固定在车轮中，车轮通过悬架系统与车架相连，以此实现物理架构模型构建。

此外针对制动过程故障问题，案例中可进行故障树分析的建模和分析，建立简单的二态故障树，通过使命任务获得故障顶事件层，通过逻辑架构和物理架构获得中间事件层和底事件层。

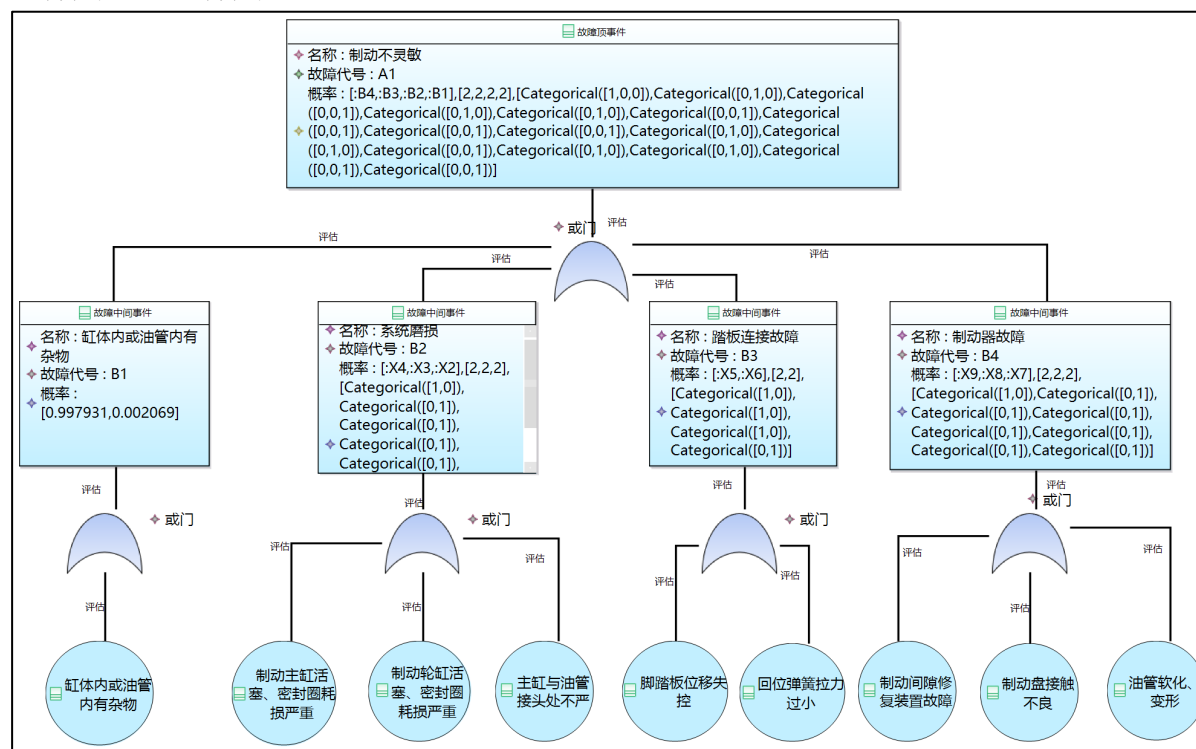


图 6.22 故障树建模

如图 6.22 中，顶事件是 A1 制动不灵敏，导致系统故障的故障中间事件 B1 缸体内或油管内有杂物，B2 系统磨损，B3 踏板连接故障，B4 制动器故障，表明系统故障根本原因的故障底事件 X1 缸体内或油管内有杂物，X2 制动主缸活塞、密封圈损耗严重，X3 制动轮缸活塞、密封圈损耗严重，X4 主缸与油管接头处不严，X5 脚踏板位移失控，X6 回位弹簧拉力过小，X7 制动间隙修复装置故障，X8 制动盘接触不良，X9 油管软化、变形。每一层之间通过或门进行选择，这里的每一个对象都包含概率属性，而每个对象节点都只有两种状态，发生和不发生，如对象 X5 脚踏板位移失控的发生概率是 0.992509，不发生概率 0.007491，对象 X6 回位弹簧拉力过小的发生概率是 0.986591，不发生概率 0.013409，因此，通过或门，对象 B3 踏板连接故障的发生概率为 X5 和 X6

任意一个发生的概率。通过计算引擎的求解，可以得出制动不灵敏的发生概率。

本案例通过更多场景的构建，以及使命分析、运行分析、特征和功能分析、逻辑分析、物理架构分析的多轮迭代，最终可实现智能电动汽车的架构设计，验证了多视图统一建模语言构建复杂系统的模型一致性和完整性。同时，该建模过程各阶段的设计师在同一环境中构建模型，保证了唯一数据源，数据之间通过一致的标准进行传递，便于各阶段模型交互和架构设计师之间的协同，验证了复杂系统生命周期多阶段架构模型的集成性，提高设计效率。

6.4.5 案例分析

6.4.5.1 集成性分析

在 MOFLP 的建模流程中，如表 6.11 所示，智能电动汽车的架构设计应用到系统顶层概念图、状态转移描述图、用例图、时序图、模块定义图、流程图、车辆特征图、故障树分析图等共 11 个图元模型，分别来自于 UPDM、SysML、EAST-ADL、BPMN 及 FTA 等建模语言规范。其中，工具 EA 支持 UPDM 标准规范，进行 DoDAF 和 MoDAF 的建模；工具 MagicDraw 支持 SysML 标准规范，可进行 SysML 的建模；工具 Cameo 支持 BPMN 标准规范；MetaEdit+ 支持 EAST-ADL 标准规范；工具 Systemweaver 支持汽车领域的 FTA 建模。

表 6.11 元模型集成

序号	元模型	语言规范	支持建模工具	集成建模工具	建模语言
1	系统顶层概念图	UPDM	Enterprise Architect	多视图集成表达验证平台	多视图统一建模语言
2	状态转移描述图	UPDM			
3	用例图	SysML			
4	时序图	SysML			
5	包图	SysML	MagicDraw		
6	内部模块图	SysML			
7	模块定义图	SysML			
8	流程图	BPMN	Cameo		
9	车辆特征图	EAST-ADL			
10	车辆功能类型图	EAST-ADL	MetaEdit+		
11	故障树分析图	FTA			
			Systemweaver		

虽然通过集成 EA、MagicDraw、Cameo、MetaEdit+及 Systemweaver 可完成整体的建模，但存在建模环境风格迥异、模型割裂、数据异构、相互之间语义不统一、模型关联复杂等问题，严重影响模型准度和建模效率。基于本文研究的多视图统一建模语言和建模环境开展的建模，通过在同一建模环境中实现所有元模型和模型的构建，集成 UPDM、SysML、EAST-ADL 等各建模语言规范。通过基于使命—运行—特征与功能—逻辑—物理架构建模方法，实现了从使命任务开始的生命周期架构设计过程，满足了体系、系统、子系统等多生命周期阶段，提高了设计完整度。同时建模过程避免了多建模工具建模导致的数据异构及交互的困难，统一了模型语义和数据，从而保证了模型的质量。因此多视图统一建模语言和建模环境能够实现模型的集成，大大提高设计效率，降低研发成本。

6.4.5.2 建模完整性分析

基于模型的复杂系统霍尔模型从多个维度介绍了复杂装备的研制过程，是解决复杂系统研制有效程序。如表 6.12 所示，在基于模型的霍尔模型中，本案例主要形式化表征了机械、电气、控制等学科领域的架构设计，建模方面侧重从安全性的系统工程视角展开，在基于模型的霍尔模型中系统维开展生命周期体系、系统及子系统阶段架构设计，在逻辑维开展使命、运行、特征和功能、逻辑、物理架构分析，完成了表 2.1 中 A₁₂-A₁₃，A₂₃-A₂₆，A₃₃-A₃₆ 的建模流程，共构建图元模型 11 个，模型 28 个。对霍尔模型中各维度的表达验证了架构建模过程的完整性。

表 6.12 智能电动汽车架构设计总体介绍

学科领域维	系统工程视角	系统维	逻辑维	图元模型个数	模型数
机械、电气、控制	安全性	体系、系统及子系统架构设计	使命、运行、特征和功能、逻辑、物理架构分析	11	28

智能电动汽车的特定域建模语言设计，形式化了霍尔模型中时间维生命周期的多阶段，其中，基于 UPDM、SysML、BPMN 等建模标准，通过多视图统一建模语言形式化了系统顶层概念图、状态转移描述图、用例图、时序图等共图元模型 $|V_{mm}^{iv}| = 11$ ，如表 6.13 所示，包含对象元模型 $|V_{mObject}^{io}| = 142$ 、关系元模型 $|E_{mRelationship}^{ie}| = 44$ 、点元模型 $|V_{mPoint}^{it}| = 56$ 、角色元模型 $|V_{mRole}^{ir}| = 88$ ，以及属性元模型 $|P_{mProperty}^{ip}| = 156$ 。形式化表征的函数数量统计，表明对于系统顶层概念图、状态转移描述图、用例图等图统一建模语言实现是准确的，论证了架构建模过程的完整性。

表 6.13 特定域形式化元模型统计

V_{mm}^{iv}	系统顶层概念图	状态转移描述图	用例图	车辆特征图	车辆功能类型图	时序图	流程图	包图	内部模块图	模块定义图	故障树分析图	总计
$V_{mObject}^{io}$	1	15	11	11	12	3	54	2	8	17	8	142
$E_{mRelationship}^{ie}$	1	2	4	5	8	1	2	2	4	12	3	44
V_{mPoint}^{it}	0	0	4	17	6	0	10	0	15	4	0	56
V_{mRole}^{ir}	2	4	8	10	16	2	4	4	8	24	6	88
$P_{mProperty}^{ip}$	2	10	9	24	11	6	41	12	19	14	8	156

图 6.23 元模型元素

通过基于标准建模语言的元模型形式化，如基于 DoDAF OV-6b：状态转移描述开发的元模型状态转移描述图（STDD, State Transition Description Diagram）， $V_{mm}^{ztzy001} = \langle STDD, V_{mObject}, V_{mPoint}, V_{mRole}, E_{mRelationship}, P_{mProperty}, A_{mm} \rangle$ ，通过状态、子状态、复合状态等对象模型元素 $|V_{mObject}|$ 共 15 个，能够覆盖体系视角下研究对象汽车的状态转换情况，通过元模型元素的定制，确保了智能电动汽车描述的完整性。

在图 6.23 中，为了更准确的表达特征，模型元素——车辆特征（VC, Vehicle Characteristics）， $V_{mObject}^{cltz001} = \langle VC, P_{mProExt}, V_{mPointExt}, ext, A_{mm} \rangle$ ，定义了实际绑定时间、基数、特征参数等共有属性 $|P_{mProExt}| = 11$ ，捕获车辆特性以定义正在开发的系统；模型元素——硬件组件原型（HCP, Hardware Component Prototype）， $V_{mObject}^{yzjyx001} = \langle VC, P_{mProExt}, V_{mPointExt}, ext, A_{mm} \rangle$ ，提供了通信、电源、IO 等可选的硬件端口共 $|V_{mPointExt}| = 9$ ，为信息交互提供准确的输入输出引脚，捕获车辆嵌入式子系统特征。其中，视图、模型元素等各个要素的 ID 唯一， $V_{mm}^{iv} \rightarrow V_{mObject}^{io} \rightarrow V_{mPoint}^{it}$ 等各要素的

包含关系严谨，展示了形式化对复杂系统生命周期系统维多阶段的表征能力，提高了模型表达的完整性。

6.4.5.3 表达一致性分析

智能电动汽车的架构设计形式化了霍尔模型中逻辑维多阶段过程，如表 6.14 所示，在智能电动汽车的架构建模阶段，面向智能汽车运行场景，通过使命任务分析、运行分析、特征和功能分析、逻辑分析、物理架构分析，共形式化模型 $|M_{modIns}^j| = 28$ ，如车辆运行概念模型（CMVO, Conceptual Model of Vehicle Operation）实例化了状态转移描述图 M_{modIns}^{ztzy01} (CMVO) instance $V_{mm}^{ztzy001}$ (STDD)，包含对象实例 $|V_{object}^{jo}| = 459$ ，如感知特征模型（PFM, Perceptual feature model）实例化了车辆特征图 V_{object}^{gztz01} (PFM) instance $V_{mObject}^{cltz001}$ (VC)，以及点实例 $|V_{point}^{jpo}| = 536$ 、关系实例 $|E_{rela}^{jre}| = 668$ 、角色实例 $|V_{role}^{jro}| = 1336$ 、属性实例 $|P_{property}^{jpr}| = 505$ ，通过实例化元模型要素，继承元模型特征，完成逻辑维从智能汽车使命场景到物理架构的模型演进过程，实现基于场景的智能电动汽车架构设计，展示了形式化对复杂系统生命周期逻辑维多阶段的表征能力，提高了模型表达的一致性。

表 6.14 形式化架构模型统计

V_{mm}^{iv}	使命分析	运行分析		特征与功能分析		逻辑分析		物理架构分析				总计
	系统顶层概念图	状态转移描述图	用例图	车辆特征图	车辆功能类型图	时序图	流程图	包图	内部模块图	模块定义图	故障树分析图	
M_{modIns}^j	1	1	4	7	6	1	1	4	1	1	1	28
V_{object}^{jo}	6	13	80	77	89	15	24	77	29	30	19	459
V_{point}^{jpo}	0	0	0	24	292	0	0	0	94	126	0	536
E_{rela}^{jre}	14	13	101	96	189	28	26	73	47	63	18	668
V_{role}^{jro}	28	26	202	192	378	56	52	146	94	126	36	1336
$P_{property}^{jpr}$	12	18	80	82	89	0	22	125	29	1	47	505

6.5 本章小结

本章首先基于多视图统一建模语言，设计并开发基于模型系统工程的多视图集成表达验证平台，该平台提供了一种良好的建模语言构建环境和建模环境，能够有效的支持设计人员进行复杂系统生命周期多阶段、多领域的建模。然后基于验证平台开展多视图统一建模关键技术验证。通过对比 BPMN、DMN、OPM、UPDM 和 SysML 五种建模语言，定量及定性的分析验证了多视图统一建模的分层抽象模型对于各建模语言的模型要素、各元模型及模型规范技术的满足性；通过智能电动汽车的架构设计，验证了多视图统一建模语言对于装备复杂系统多阶段架构建模的支持，保证了唯一数据源，提高了模型集成能力和设计效率；通过对过程模型的形式化表征分析，验证了对装备复杂系统生命周期架构模型集成性、完整性和一致性表达的支持。

第 7 章 结论与展望

7.1 总结

面向装备复杂系统的研制任务，前期阶段的系统设计对于降低成本、提高效率非常重要。基于模型的系统工程以形式化的建模方法有效支持装备复杂系统生命周期的前期需求、设计、分析、验证和确认活动，已经成为工业界和学术界的共识。论文在综合研究基于模型的装备复杂系统设计过程及技术应用的情况下，针对复杂系统在生命周期多阶段建模时，面临模型统一描述，准确、一致性表达等问题，结合科研项目的具体需求，以解决系统工程师通过模型集成，提高设计效率和设计准确性为目标，开展复杂系统架构多视图统一建模关键技术研究。论文的主要研究工作和结论概括如下。

(1) 回顾了研究复杂系统设计的核心内容——从基于模型的复杂系统设计过程中提出架构模型一致性对于提高设计准确性和设计效率的重要性，从建模语言的综述中提出开发具备集成各种建模语言规范能力的多视图统一建模语言的必要性，从建模语言形式化表征方法中提出使用图论的数学理论定义复杂系统抽象模型的形式化表征的合理性。

(2) 提出了复杂系统架构多视图统一建模框架。为满足装备复杂系统生命周期多阶段架构模型实现统一描述和集成，保证唯一权威数据源，制定了基于霍尔模型的多视图统一建模框架，从系统维、逻辑维、领域维概括了复杂装备的研制过程，并从使命分析、运行分析、特征/功能分析、逻辑分析、物理架构分析的装备设计流程中总结出多视图统一建模需求，并提出分层抽象模型构建技术、基于图论有向图的抽象模型形式化表征技术、多视图统一建模语言表达技术三个层次递进的技术路线。

(3) 提出了基于 GOPPRR-E 元素的分层抽象模型。针对复杂系统架构设计过程中生命周期多阶段模型缺少通用抽象元素描述，不能够支持复杂系统架构模型统一表达的问题，提出 GOPPRR-E 元元模型，并结合 MOF 建模框架，在 M3-M1 层框架中定义一组新的抽象模型，表达各层的建模元素以及使用这些元素的规则，该抽象模型通过自底向上的层次映射关系实现模型表达，是定义复杂系统抽象装备模型形式化表征的基础。

(4) 提出了基于图论有向图的抽象模型形式化表征模型。针对表达复杂系统架

构模型缺少完整的形式化语义以支持理论分析，导致模型的精确度、一致性和完整性无法得到明确的解释的问题，基于图论有向图进行元模型层类型图、模型层实例图等形式化分析，并提出元元模型、元模型及模型形式化表征模型，以及扩展约束形式化表征模型，实现了数学逻辑对装备模型本质的精准定义，支持多视图统一建模语言的抽象语法构建。

(5) 提出多视图统一建模语言。针对支持复杂系统生命周期多阶段架构建模的多种建模语言底层数据异构、建模环境各异引起模型表达不一致、集成和交互困难等问题，提出多视图统一建模语言，主要是进行了抽象语法和具体语法的定义。其中的抽象语法指定了元模型、模型等元素和元素之间的关系，形成文本语言的框架；具体语法指定了抽象语法框架中的元模型及模型等如何显式的表示，每个可表示的抽象语法概念都扩展为一个具体的语法规则结构，最终形成支持多视图统一建模的语言规范。

(6) 建立了多视图集成表达验证平台并进行了关键技术验证。该平台提供了一种良好的建模语言构建环境和建模环境，能够有效的支持设计人员进行复杂系统生命周期多阶段、多领域的建模。基于验证平台开展了多视图统一建模关键技术的验证，首先通过对比 BPMN、DMN、OPM、UPDM 和 SysML 五种建模语言，定量及定性的分析验证了多视图统一建模的分层抽象模型对于各建模语言的模型要素、各元模型及模型规范技术的满足性；然后通过多视图统一建模语言重用设计领域的架构设计规范及框架，如 UPDM 和 SysML 等进行智能电动汽车的架构设计，验证了多视图统一建模语言对于装备复杂系统多阶段架构建模的支持，保证了唯一数据源，提高了模型集成能力和设计效率；并通过对模型的形式化表征分析，验证了对复杂系统生命周期架构模型准确性、完整性和一致性表达的支持。同时，架构设计过程的元模型及模型通过重用，可提高后续装备的设计效率，提高语言的应用价值。

7.2 主要创新点

在面向装备复杂系统架构建模过程中，本文主要提出以下三个创新点：

(1) 建立了复杂系统架构统一表征的抽象模型

针对复杂系统生命周期架构建模过程中，各阶段架构模型特征不同导致模型的语义难以一致性解释的问题，建立了架构模型统一表征的抽象模型。该抽象模型凝练了能够全面抽象复杂系统架构描述的元元模型 GOPRR-E，包括图 (Graph)、对象 (Object)、属性 (Property)、点 (Point)、角色 (Role)、关系 (Relationship)、扩展

(Extension) 共七种基本元素集, 明确了元素及元素之间的关联和约束关系, 确定了抽象过程的继承机制, 进而支持架构模型中组织、活动、时序、状态等视图, 任务、行为、功能、逻辑、物理等过程的统一描述, 保证了复杂系统架构建模过程中模型语义的一致性。

(2) 建立了基于有向图的复杂系统架构视图量化表示的数学模型

针对复杂系统架构模型异构且语义表达不一致, 导致复杂系统生命周期过程架构建模时难以实现各架构模型特征一体化描述的问题, 首次建立了基于有向图的复杂系统架构视图量化表示的数学模型。该数学模型支持统一构建系统生命周期设计过程要素和设计过程架构模型, 支持系统更多领域不同架构特征的描述, 准确的揭示架构模型中系统级、子系统级各要素之间的关联关系, 明确架构模型的拓扑表达, 验证了系统生命周期架构视图描述的完整性, 提高了复杂系统架构视图的统一表征能力。

(3) 研发一种支持复杂系统多阶段架构模型集成的建模语言

针对装备复杂系统生命周期多阶段多领域的架构模型数据异构, 导致模型集成和数据交互困难的问题, 提出面向装备复杂系统的多视图统一建模语言, 该建模语言是国际上首个支持异构架构模型集成的语义式元语言, 通过开发系统架构建模软件系统, 支持基于模型的体系设计、系统设计等系统生命周期过程, 提供了唯一的权威数据源, 促进了设计过程架构模型的集成, 支撑了架构模型的综合验证。以智能电动汽车架构设计验证表明, 该系统能够有效提升复杂系统正向总体设计效率。

7.3 展望

论文以提高系统工程师的装备架构建模效率为目的, 以统一建模语言底层数据实现复杂系统多视图统一表达为目标, 从基于 GOPPRR-E 元元模型元素出发, 围绕 MOF 元建模机制和图论形式化表征进行统一建模语言规范的设计和开发开展了一系列相关研究。为提高复杂系统架构多视图统一建模能力还需要开展以下研究:

(1) **提高对装备复杂系统多领域要素的表达能力。**下一步将继续挖掘更多的工业应用场景和装备研发流程, 从中抽象并总结出建模要素, 提高多视图统一建模语言的可描述能力, 以便全面的、系统的、完整的描述系统。

(2) **提高多视图统一建模语言的约束表达能力。**目前的多视图统一建模语言对于约束的表达仍不够完善, 需结合 OCL 语言 (Object Constraint Language, 对象约束语言) 更精确的约束模型或系统中对一个或多个值的限制。通过 OCL 的语法, 规范多

视图统一建模语言在模型执行操作前应满足的条件及在操作执行后必须满足的条件等。

(3) **提高复杂系统架构模型的共享能力。**随着装备系统复杂性的增加，模型的体量越发庞大，会导致模型之间的关系越来越复杂，数据关联、传递和追溯越来越困难，因此提高系统模型向本体模型的转化能力，通过本体模型实现数据关联、查询和共享，并通过 OSLC (Open Services for Lifecycle Collaboration, 开放式生命周期协作服务) 技术，打通多视图统一建模语言与模型和服务平台的接口，将本地的数据文件通过适配器上传至云端，共享给多个协作用户，并实现数据的版本管理及增删改查。

参考文献

- [1] Vaneman W K. Enhancing model-based systems engineering with the Lifecycle Modeling Language. 2016 Annual IEEE Systems Conference (SysCon) [C]. Piscataway: IEEE, 2016: 1-7.
- [2] Eisenmann H, Miro J, de Koning H P. MBSE for European Space-Systems Development[J]. INSIGHT, 2009, 12(4): 47-53.
- [3] Barritt, Brian, et al. Integrated Approach to Architecting, Modeling, and Simulation of Complex Space Communication Networks. SpaceOps 2010 Conference Delivering on the Dream Hosted by NASA Marshall Space Flight Center and Organized[C]. Reston: AIAA, 2010: 1941.
- [4] Friedenthal S, Griego R, Sampson M. INCOSE model-based systems engineering (MBSE) initiative. INCOSE 2007 symposium[C]. Hoboken: Wiley, 2007, 11: 1-29.
- [5] Gacek C, Abd-Allah A, Clark B, Boehm B. On the definition of software system architecture. In Proceedings of the First International Workshop on Architectures for Software Systems[C]. New York: ACM, 1995, 24 :85-94.
- [6] ISO/IEC/IEEE DIS 15288-2008, Systems and software engineering-System life cycle processes[S]. Berlin: ISBN, 2008.
- [7] 周永华,李瑞敏,刘冲,陆化普.基于 UML 的多视图集成建模方法[J].交通运输工程学报,2005(02):99-105.
- [8] Schwenn, Ronald E., et al. Defense Acquisitions: Assessments of Selected Weapon Programs[R]. WASHINGTON DC: GOVERNMENT ACCOUNTABILITY OFFICE WASHINGTON DC, 2011.
- [9] Farr, John V. Systems life cycle costing: economic analysis, estimation, and management[M]. Boca Raton: CRC Press, 2011:40-45.
- [10] Mayk and A. M. Madni, The Role of Ontology in System-of-Systems Acquisition[R]. San Diego: Command and Control Research and Technology Symposium, 2006.
- [11] International Council on Systems Engineering (INCOSE). Systems engineering vision 2020[R]. Seattle, WA: International Council on Systems Engineering, 2007: 5-32.
- [12] 王文跃,侯俊杰,毛寅轩,靳捷,卢志昂.面向复杂产品研制的 MBSE 体系架构及其发展趋势研究[J].控制与决策,2022,37(12): 3073-3082.

- [13] Lemazurier L, Chapurlat V, Grossetête A. An MBSE approach to pass from requirements to functional architecture[J]. IFAC-Papers OnLine, 2017, 50(1): 7260-7265.
- [14] Masior J, Schneider B, Kürümlüoglu M, et al. Beyond Model-Based Systems Engineering towards Managing Complexity[J]. Procedia CIRP, 2020, 91: 325-329.
- [15] Morkevicius A, Aleksandraviciene A, Mazeika D, et al. MBSE Grid: A Simplified SysML - Based Approach for Modeling Complex Systems. INCOSE International Symposium[C]. Hoboken: Widly, 2017, 27(1): 136-150.
- [16] Davis R. ARIS design platform: advanced process modelling and administration[M]. Berlin: Springer Science & Business Media, 2008: 20-25.
- [17] Scheer A W, Nüttgens M. ARIS architecture and reference models for business process management[M]. Berlin: Business process management, Springer, 2000: 376-389.
- [18] Biermann E, Ehrig K, Köhler C, et al. Graphical definition of in-place transformations in the eclipse modeling framework. International Conference on Model Driven Engineering Languages and Systems[C]. Berlin: Springer, 2006: 425-439.
- [19] Buchmann T, Schwägerl F. On a-posteriori integration of Ecore models and hand-written Java code. 2015 10th International Joint Conference on Software Technologies (ICSOT) [C]. Piscataway: IEEE, 2015, 2: 1-8.
- [20] Davis J. GME: the generic modeling environment. Companion of the 18th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications[C]. New York: ACM, 2003: 82-83.
- [21] Bézin J, Brunette C, Chevrel R, et al. Bridging the generic modeling environment (GME) and the eclipse modeling framework (EMF). Proceedings of the Best Practices for Model Driven Software Development [C]. San Diego: OOPSLA, 2005, 5: 1-9.
- [22] Özgür T. Comparison of Microsoft DSL Tools and Eclipse Modeling Frameworks for Domain-Specific Modeling in the context of Model-Driven Development[J]. Digitala Vetenskapliga Arkivet 2007: 56.
- [23] Bézin J, Hillairet G, Jouault F, et al. Bridging the ms/dsl tools and the eclipse modeling framework. Proceedings of the International Workshop on Software Factories[C]. San Diego: OOPSLA. 2005, 5: 1-19.

- [24] Dimitrieski V, Čeliković M, Ivančević V, et al. A comparison of Ecore and GOPRR through an information system meta modeling approach. Joint Proceedings of colocated Events at the 8th European Conference on Modelling Foundations and Applications (ECMFA 2012) [C], Berlin: Springer. 2012: 217-228.
- [25] Seidewitz E. What models mean[J]. IEEE software, 2003, 20(5): 26-32.
- [26] 王崑声, 袁建华, 陈红涛, 蒲洪波. 国外基于模型的系统工程方法研究与实践[J]. 中国航天, 2012(11): 52-57.
- [27] 郝彬彬, 鲁金直, 李军, 等. 基于特定域建模方法的航空发动机 MBSE 实施方法[J]. 科技导报, 2019, 37(7): 88-95.
- [28] Lucas F J, Molina F, Toval A. A systematic review of UML model consistency management[J]. Information and Software technology, 2009, 51(12): 1631-1645.
- [29] Budgen D, Burn A J, Brereton O P, et al. Empirical evidence about the UML: a systematic literature review[J]. Software: Practice and Experience, 2011, 41(4): 363-392.
- [30] Genero M, Fernández-Saez A M, Nelson H J, et al. Research review: a systematic literature review on the quality of UML models[J]. Journal of Database Management (JDM), 2011, 22(3): 46-70.
- [31] Munthe I R, Rambe B H, Pane R, et al. UML Modeling and Black Box Testing Methods in the School Payment Information System[J]. Jurnal Mantik, 2020, 4(3): 1634-1640.
- [32] Assunção W K G, Vergilio S R, Lopez-Herrejon R E. Automatic extraction of product line architecture and feature models from UML class diagram variants[J]. Information and Software Technology, 2020, 117: 106198.
- [33] Holt J. UML for Systems Engineering: watching the wheels[M]. England & Wales: IET, 2004:101-130.
- [34] Schütz D, Legat C, Vogel-Heuser B. On modelling the state-space of manufacturing systems using UML[J]. IFAC Proceedings Volumes, 2012, 45(6): 469-474.
- [35] Christoulakis F, Thramboulidis K. IoT-based integration of IEC 61131 industrial automation systems: The case of UML4IoT. 2016 IEEE 25th International Symposium on Industrial Electronics (ISIE) [C]. Piscataway: IEEE, 2016: 322-327.
- [36] Mhenni F, Choley J Y, Penas O, et al. A SysML-based methodology for mechatronic systems architectural design[J]. Advanced Engineering Informatics, 2014, 28(3): 218-231.

- [37] Hampson K. Technical evaluation of the systems modeling language (SysML)[J]. *Procedia Computer Science*, 2015, 44: 403-412.
- [38] Friedenthal S, Moore A, Steiner R. A practical guide to SysML: the systems modeling language[M]. San Francisco : Morgan Kaufmann, 2014:20-31.
- [39] dos Santos Soares M, Vrancken J L M. Model-Driven User Requirements Specification using SysML[J]. *JSW*, 2008, 3(6): 57-68.
- [40] Capasso C, Hammadi M, Patalano S, et al. A multi-domain modelling and verification procedure within MBSE approach to design propulsion systems for road electric vehicles[J]. *Mechanics & Industry*, 2017, 18(1): 107.
- [41] Linhares, Marcos V, et al. Introducing the modeling and verification process in SysML. 2007 ieee conference on emerging technologies and factory automation[C]. Piscataway: IEEE, 2007: 334-351.
- [42] Bassi L, Secchi C, Bonfe M, et al. A SysML-based methodology for manufacturing machinery modeling and design[J]. *IEEE/ASME transactions on mechatronics*, 2010, 16(6): 1049-1062.
- [43] 石福丽,黄炎焱,朱一凡,杨峰.基于 SysML 的无人侦察机需求描述方法研究[J].*计算机仿真*,2007(06): 53-56.
- [44] Pétin J F, Evrot D, Morel G, et al. Combining SysML and formal methods for safety requirements verification. 22nd International Conference on Software & Systems Engineering and their Applications[C]. Lyon: HAL, 2010: CDR0M.
- [45] Jarraya Y, Soeanu A, Debbabi M, et al. Automatic verification and performance analysis of time-constrained sysml activity diagrams. 14th Annual IEEE International Conference and Workshops on the Engineering of Computer-Based Systems (ECBS'07) [C]. Piscataway: IEEE, 2007: 515-522.
- [46] David P, Idasiak V, Kratz F. Reliability study of complex physical systems using SysML[J]. *Reliability Engineering & System Safety*, 2010, 95(4): 431-450.
- [47] Chouali S, Hammad A. Formal verification of components assembly based on SysML and interface automata[J]. *Innovations in Systems and Software Engineering*, 2011, 7(4): 265-274.
- [48] Pedroza G, Apvrille L, Knorreck D. AVATAR: A SysML environment for the formal verification of safety and security properties. 2011 11th Annual International Conference on New Technologies of Distributed Systems[C]. Piscataway: IEEE, 2011: 1-10.
- [49] Hause M. The Unified Profile for DoDAF/MODAF (UPDM) enabling systems of systems on many levels. 2010 IEEE International Systems Conference[C]. Piscataway: IEEE, 2010: 426-431.

- [50] Hause M. 4.5. 2 Model - Based System of Systems Engineering with UPDM. INCOSE International Symposium[C]. Hoboken: Wiley, 2010, 20(1): 580-594.
- [51] Mavris D. A Review of the Proposed UML Profile for the Department of Defense Architecture Framework and Ministry of Defense Architecture Framework (UPDM). International Symposium[C]. Hoboken: Wiley, 2007: 1-25.
- [52] 陈超.基于 UPDM 的空天协同观测系统体系结构分析与建模[J].空天防御,2018,1(02): 46-52.
- [53] 吴骞华.UPDM 下的武器装备体系建模方法分析[J].通讯世界,2019,26(02): 241-242.
- [54] 孙岩,王光伟,何新华,范兆军.基于 UPDM 的武器装备体系建模[J].装甲兵工程学院学报,2013,27(02): 66-69.
- [55] Papke B. Enabling Agile Security with MBSE and UPDM[J]. INSIGHT, 2016, 19(2): 18-20.
- [56] Sanduka I, Obermaisser R. Model-based development of systems-of-systems with real-time requirements. 2014 12th IEEE International Conference on Industrial Informatics (INDIN) [C]. Piscataway: IEEE, 2014: 188-194.
- [57] Recker J C. BPMN modeling—who, where, how and why[J]. BPTrends, 2008, 5(3): 1-8.
- [58] Zarour K, Benmerzoug D, Guermouche N, et al. A systematic literature review on BPMN extensions[J]. Business Process Management Journal, 2020, 26(6): 1473-1503.
- [59] 蔡章利, 易树平. 基于 BPMN 的业务流程一体化建模方法[J].计算机集成制造系统,2010,16(03): 551-557.
- [60] White S A. Process modeling notations and workflow patterns[J]. Workflow handbook, 2004: 265-294.
- [61] Simpson R C. An XML representation for crew procedures[J]. NASA Summer Faculty Fellowship Program, 2005,1:1-8.
- [62] Object Management Group, Business Process Model and Notation (BPMN) Version 2.0.2[S], 2018, URL www.omg.org/spec/BPMN/2.0.2/ (accessed 22 November 2018).
- [63] 王刚. 基于 BPMN 规范的工作流引擎设计与实现[D]. 成都: 电子科技大学, 2020.
- [64] Kozma D, Varga P, Larrinaga F. Data-driven Workflow Management by utilising BPMN and CPN in IIoT Systems with the Arrowhead Framework. 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA) [C]. Piscataway: IEEE, 2019: 385-392.

- [65] Fernandes J, Reis J, Melão N, et al. The role of Industry 4.0 and BPMN in the arise of condition-based and predictive maintenance: A case study in the automotive industry[J]. *Applied Sciences*, 2021, 11(8): 3438.
- [66] Bazhenova E, Zerbato F, Oliboni B, et al. From BPMN process models to DMN decision models[J]. *Information Systems*, 2019, 83: 69-88.
- [67] Rodríguez A, Fernández-Medina E, Piattini M. A BPMN extension for the modeling of security requirements in business processes[J]. *IEICE transactions on information and systems*, 2007, 90(4): 745-752.
- [68] Kelly S, Tolvanen J P. Domain-specific modeling: enabling full code generation[M]. Hoboken: John Wiley & Sons, 2008, 30-45.
- [69] 江涛. 特定领域元建模语言的形式化及其模型一致性验证研究[D]. 昆明: 云南大学, 2010.
- [70] 杨志斌, 皮磊, 胡凯, 顾宗华, 马殿富. 复杂嵌入式实时系统体系结构设计与分析语言: AADL[J]. *软件学报*, 2010, 21(05): 899-915.
- [71] Gauffillet P, Heim S, Bonnin H, et al. ITEA SPICES AADL experimentation at airbus. Proc. of the 14th Int'l Conf. on Reliable Software Technologies, Ada-Europe[C]. Washington: IEEE Computer Society Press. 2009: 1-10.
- [72] Farail P, Gauffillet P, Canals A, et al. The TOPCASED project: a toolkit in open source for critical aeronautic systems design. Conference ERTS'06[C]. Lyon: HAL, 2006: 1-9.
- [73] Lewis B, Feiler P. Multi-Dimensional model based engineering using AADL. 2008 The 19th IEEE/IFIP International Symposium on Rapid System Prototyping[C]. Piscataway: IEEE, 2008: xv-xviii.
- [74] Debruyne V, Simonot-Lion F, Trinquet Y. EAST-ADL—An architecture description language[C]. IFIP World Computer Congress, TC 2 Piscataway. Boston: Springer, 2004: 181-195.
- [75] Cuenot P, Frey P, Johansson R, et al. 11 the east-adl architecture description language for automotive embedded software. Dagstuhl Workshop on Model-Based Engineering of Embedded Real-Time Systems[C]. Berlin: Springer, 2007: 297-307.
- [76] Sobol G, Dori D. Improving OPM Conceptual Models by Incorporating Design Structure Matrix. 2020 IEEE International Symposium on Systems Engineering (ISSE) [C]. Piscataway: IEEE, 2020: 1-8.

- [77] Dori D, Kohen H, Jbara A, et al. OPCloud: An OPM Integrated Conceptual - Executable Modeling Environment for Industry 4.0[J]. *Systems Engineering in the Fourth Industrial Revolution*, 2019: 243-271.
- [78] Brisacier-Porchon L, Ensta O H, Boutemy R. Modeling A UAV in Practice: A Comparison between Rhapsody and Capella. 2021 IEEE International Symposium on Systems Engineering (ISSE) [C]. Piscataway: IEEE, 2021: 1-8.
- [79] Xu R, Fang W, Yin W. Practice of ARCADIA and Capella in Civil Radar Design[M]. Berlin: *Complex Systems Design & Management*. Springer, 2021: 487-487.
- [80] Ma J, Wang G, Lu J, et al. Systematic Literature Review of MBSE Tool-Chains[J]. *Applied Sciences*, 2022, 12(7): 3431.
- [81] Kruse B, Blackburn M. Collaborating with OpenMBEE as an authoritative source of truth environment[J]. *Procedia Computer Science*, 2019, 153: 277-284.
- [82] Gianni D, Lindman N, Fuchs J, et al. Introducing the european space agency architectural framework for space-based systems of systems engineering[M]. Heidelberg: *Complex Systems Design & Management*, Springer, 2012: 335-346.
- [83] Oditia T A. Application of Model-Based Systems Engineering Using MADe, and MagicDraw Modeling Tools to Ascertain Reliability, Risk, and Safety of Aerospace Components[D]. Morgan State University, 2020.
- [84] Lindsey N J, Alimardani M, Gallo L D. Reliability analysis of complex NASA systems with model-based engineering. 2020 Annual Reliability and Maintainability Symposium (RAMS) [C]. Piscataway: IEEE, 2020: 1-8.
- [85] Shah A , Kerzhner A, Schaefer D, et al. Multi-view modeling to support embedded systems engineering in SysML[M]. Berlin: *Graph transformations and model-driven engineering*, Springer, 2010: 580-601.
- [86] Godart P, Gross J, Mukherjee R, et al. Generating real-time robotics control software from sysml. 2017 IEEE Aerospace Conference[C]. Piscataway: IEEE, 2017: 1-11.
- [87] Gery E, Harel D, Palachi E. Rhapsody: A complete life-cycle model-based development system. International Conference on Integrated Formal Methods[C]. Berlin: Springer, 2002: 1-10.

-
- [88] Arnold A, Beleani M, Ferrari A, et al. An Application of SMC to continuous validation of heterogeneous systems[J]. EAI Endorsed Transactions on Industrial Networks and Intelligent Systems, 2017, 4(11): e2-e2.
 - [89] Kalawsky R S, O'Brien J, Chong S, et al. Bridging the gaps in a model-based system engineering workflow by encompassing hardware-in-the-loop simulation[J]. IEEE systems Journal, 2013, 7(4): 593-605.
 - [90] 海晓航. 基于 MBSE 的民机起落架系统建模与仿真分析[D]. 南京: 南京航空航天大学, 2018.
 - [91] 张绍杰, 李正强, 海晓航, 等. 基于 MBSE 的民用飞机安全关键系统设计[J]. 中国科学: 技术科学, 2018, 3: 299-311.
 - [92] Roques P. MBSE with the ARCADIA Method and the Capella Tool. 8th European Congress on Embedded Real Time Software and Systems (ERTS 2016) [C]. Lyon: HAL, 2016: 1-11.
 - [93] Hause M C, Day R L. Frenemies: Opm and SysML together in an MBSE model. INCOSE International Symposium[C]. Hoboken: Widly, 2019, 29(1): 691-706.
 - [94] Törmänen M, Häggglund A, Rocha T, et al. Integrating Multi-Disciplinary Optimization into the Product Development Process using Model-Based Systems Engineering (MBSE). NAFEMS World Congress[C], Knutsford: NAFEMS. 2017: 287-300.
 - [95] Hause M C, Day R L. Frenemies: Opm and SysML together in an MBSE model. INCOSE International Symposium[C]. Hoboken: Widly, 2019, 29(1): 691-706.
 - [96] Casebolt J M, Jbara A, Dori D. Business process improvement using Object - Process Methodology[J]. Systems engineering, 2020, 23(1): 36-48.
 - [97] 李必信, 李必信, 廖力, 王璐璐等. 软件架构理论与实践[M]. 北京: 机械工业出版社, 2018: 50-53.
 - [98] Ng M Y, Butler M. Towards formalizing UML state diagrams in CSP. First International Conference on Software Engineering and Formal Methods[C]. Piscataway: IEEE, 2003: 138-147.
 - [99] Rossi C, Enciso M, De Guzmán I P. Formalization of UML state machines using temporal logic[J]. Software & Systems Modeling, 2004, 3(1): 31-54.
 - [100] Li L. Generating CCS model from scenarios. 2015 12th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP) [C]. Piscataway: IEEE, 2015: 135-145.
 - [101] Hall A. Using Z as a specification calculus for object-oriented systems. International Symposium of VDM Europe[C]. Berlin: Springer, 1990: 290-318.

- [102] France R B, Bruel J M, Larrondo-Petrie M, et al. Exploring the semantics of UML type structures with Z[M]. Boston: Formal Methods for Open Object-based Distributed Systems, Springer, 1997: 247-257.
- [103] Abrial, Jean-Raymond. From Z to B and then Event-B: Assigning proofs to meaningful programs. Integrated Formal Methods: 10th International Conference[C]. Berlin: Springer, 2013:1-15.
- [104] Lano K, Clark D, Androutsopoulos K. UML to B: Formal verification of object-oriented models. International Conference on Integrated Formal Methods[C]. Berlin: Springer, 2004: 187-206.
- [105] Levis AH. Generation of architectures for distributed intelligence systems. Proceedings of the IEEE International Conference on Control and Applications[C]. Piscataway: IEEE, 1989: 1-7.
- [106] Wang LC. An Integrated object-oriented Petri Net paradigm for manufacturing control systems. Int J Comp Int Manuf[C]. London: Taylor& Francis, 1996, 9(1): 73-87.
- [107] Huang Edward, McGinnis L F, Mitchell S W. Verifying SysML activity diagrams using formal transformation to Petri nets[J]. Systems Engineering, 2020, 23(1): 118-135.
- [108] Andries M A A. Graph rewrite systems and visual database languages[D]. Rijksuniversiteit te Leiden, 1996: 40-65.
- [109] Kleppe A, Rensink A. On a graph-based semantics for UML class and object diagrams[J]. Electronic Communications of the EASST, 2008, 10: 1-16.
- [110] Kang S, Kim H, Baik J, et al. Transformation rules for synthesis of UML activity diagram from scenario-based specification. 2010 IEEE 34th Annual Computer Software and Applications Conference[C]. Piscataway: IEEE, 2010: 431-436.
- [111] Bardohl R, Ehrig H, Lara J, et al. Integrating meta-modelling aspects with graph transformation for efficient visual language definition and model manipulation. International Conference on Fundamental Approaches to Software Engineering[C]. Berlin: Springer, 2004: 214-228.
- [112] 段海波. 从霍尔模型技术系统的发展进化看传统系统工程到现代系统工程的演变[J]. 智能制造, 2016 (2): 88-95.
- [113] Yang Z, Du H, Liu Y, et al. Use the Harmony-SE Approach to Extend the Advantages of MBSE. 2021 IEEE 16th Conference on Industrial Electronics and Applications (ICIEA) [C]. Piscataway: IEEE, 2021: 223-227.
- [114] Jiang D, Zhao Y, Jiang T. Multi-level and Multi-phased Maturity Evaluation Method for MBSE Based Complex Product Design Stage. 2018 2nd IEEE Advanced Information Management,

- Communicates, Electronic and Automation Control Conference (IMCEC) [C]. Piscataway: IEEE, 2018: 2208-2212.
- [115] Friedenthal S, Moore A, Steiner R. A practical guide to SysML: the systems modeling language[M]. Burlington: Morgan Kaufmann, 2014: 42-53.
- [116] Waseem M, Sadiq M U. Application of model-based systems engineering in small satellite conceptual design-A SysML approach[J]. IEEE Aerospace and Electronic Systems Magazine, 2018, 33(4): 24-34.
- [117] Aleksandraviciene A, Morkevicius A. MagicGrid® Book of Knowledge-A Practical Guide to Systems Modeling using MagicGrid from No Magic[J]. Inc, Allen Texas, USA, 2018: 18-25.
- [118] Model. MBSE 建模工具之 MagicDraw [EB/OL]. URL: <https://modelbaba.com/mbse/243.html>.
- [119] 孙胜楠,鲁金直,陈金伟,黄百乔,兰小平,张会生.多架构建模方法在飞机娱乐系统的实现及应用[J].科技导报,2020,38(21): 177-186.
- [120] Estefan J A. Survey of model-based systems engineering (MBSE) methodologies[J]. In cose MBSE Focus Group, 2007, 25(8): 1-12.
- [121] Czarnecki K. Generative programming: Methods, techniques, and applications tutorial abstract. International Conference on Software Reuse[C]. Berlin: Springer, 2002: 351-352.
- [122] Alanen M, Porres I. A relation between context-free grammars and meta object facility metamodels[M]. TURKU: Turku Centre for Computer Science, 2004:104-120.
- [123] Schauerhuber A, Wimmer M, Kapsammer E. Bridging existing Web modeling languages to model-driven engineering: a metamodel for WebML. Workshop proceedings of the sixth international conference on Web engineering[C]. New York: ACM, 2006: 5-es.
- [124] Čeliković M, Luković I, Aleksić S, et al. A MOF based meta-model and a concrete DSL syntax of IIS* Case PIM concepts[J]. Computer Science and Information Systems, 2012, 9(3): 1075-1103.
- [125] Poernomo I. The meta-object facility typed. Proceedings of the 2006 ACM symposium on Applied computing[C]. New York: ACM, 2006: 1845-1849.
- [126] OM Group. OMG Meta Object Facility (MOF) Core Specification[J]. 2015.
- [127] Kern H, Hummel A, Kühne S. Towards a comparative analysis of meta-metamodels. Proceedings of the compilation of the co-located workshops on DSM'11, TMC'11, AGERE[C]. New York: ACM, 2011: 7-12.

- [128] Ma J, Wang G, Lu J, et al. Semantic Modeling Approach Supporting Process Modeling and Analysis in Aircraft Development[J]. *Applied Sciences*, 2022, 12(6): 3067.
- [129] Van Mierlo S, Van Tendeloo Y, Meyers B, et al. Domain-specific modelling for human–computer interaction[M]. Cham: *The Handbook of Formal Methods in Human-Computer Interaction*, Springer, 2017: 435-463.
- [130] Lu J, Ma J, Zheng X, et al. Design ontology supporting model-based systems engineering formalisms[J]. *IEEE Systems Journal*, 2021, 16(4): 5465 - 5476.
- [131] Kleppe A, Rensink A. On a graph-based semantics for UML class and object diagrams[J]. *Electronic Communications of the EASST*, 2008, 10: 1-16.
- [132] Xia Z, Fu R. Study on Space-Borne Remote Sensing Imaging System by MagicGrid Methodology[M]. Singapore: *Signal and Information Processing, Networking and Computers*, Springer, 2021: 280-287.
- [133] Lu J, Wang G, Ma J, et al. General modeling language to support model - based systems engineering formalisms (part 1). *INCOSE International Symposium*[C]. Hoboken: Wiley, 2020, 30(1): 323-338.
- [134] Chen J, Wang G, Lu J, et al. Model-based system engineering supporting production scheduling based on satisfiability modulo theory[J]. *Journal of Industrial Information Integration*, 2022, 27: 100329.
- [135] Guo J, Wang G, Lu J, et al. General modeling language supporting model transformations of mbse (part 2). *INCOSE International Symposium*[C]. Hoboken: Wiley, 2020, 30(1): 1460-1473.
- [136] She S, Lu J, Wang G, et al. Model-Based Systems Engineering Supporting Integrated Modeling and Optimization of Radar Cabin Layout. *IFIP International Conference on Advances in Production Management Systems*[C]. Cham: Springer, 2021: 218-227.
- [137] Li H, Wang G, Lu J, et al. Cognitive twin construction for system of systems operation based on semantic integration and high-level architecture[J]. *Integrated Computer-Aided Engineering*, 2022 (Preprint): 1-19.
- [138] Wu S, Lu J, Hu Z, et al. Cognitive Thread Supports System of Systems for Complex System Development. 2021 16th International Conference of System of Systems Engineering (SoSE) [C]. Piscataway: IEEE, 2021: 82-87.
- [139] Bork D, Karagiannis D, Pittl B. A survey of modeling language specification techniques[J]. *Information Systems*, 2020, 87: 101425.

- [140] Object Management Group, Business process model and notation (BPMN) specification version 2.02[S], 2014, URL <http://www.omg.org/spec/BPMN/2.0.2/PDF>. (Accessed 13 May 2019).
- [141] Object Management Group, Decision modeling notation (DMN) specification version 1.2[S], 2019, URL <https://www.omg.org/spec/DMN/1.2/PDF>.(Accessed 13 May 2019).
- [142] International Organization for Standardization, Object process methodology (OPM) specification version 522[S], 2014, URL <https://www.iso.org/standard/62274.html>. (Accessed 12 May 2019).
- [143] Object Management Group, Unified Profile for DoDAF and MODAF (UPDM) specification version 2.1[S], 2013, URL <http://www.omg.org/spec/UPDM/2.1>. (Accessed 4 August 2013).
- [144] Object Management Group, Systems Modeling Language (SysML) specification version 1.6[S], 2019, URL <https://www.omg.org/spec/SysML/1.6/>.(Accessed 1 November 2019).
- [145] Bertin, Jacques. Semiology of graphics[D]. University of Wisconsin press, 1983.

附录 A 属性单位表

名称	单位	名称	单位	名称	单位	名称	单位
角度	deg	能量密度	J/m ³	线性电流密度	A/m	摩尔内能	J/mol
弧度角	rad	功率	W	磁通量密度	T	摩尔热容	J/(mol · K)
立体角	sr	质量流量	kg/s	磁通量	Wb	摩尔速率	mol/s
长度	m	体积流量	m ³ /s	磁矢势	Wb/m	分子数密度	1/m ³
面积	m ²	温度	K	电感	H	浓度	mol/m ³
体积	m ³	温度斜率	K/s	磁导率	H/m	化学势	J/mol
时间	s	线性温度因数	1/K	电磁矩	A · m ²	分子极化率	C · m ² /V
角速度	rad/s	二次温度因数	1/K ²	磁偶极矩	Wb · m	扩散系数	m ² /s
角加速度	rad/s ²	压力因数	Pa/K	电阻	Ohm	法拉第常数	C/mol
速度	m/s	压缩率	1/Pa	电阻率	Ohm · m	离子强度	mol/kg
加速度	m/s ²	热通量	W/m ²	传导率	S/m	电导率	S/m
频率	Hz	导热	W/(m · K)	磁阻	1/H	摩尔电导率	S · m ² /mol
波数	1/m	热传递系数	W/(m ² · K)	磁导	H	旋磁系数	A · m ² /(J · s)
循环波数	rad/m	热阻	K/W	电导	S	活度	Bq
分贝	dB	热绝缘	m ² · K/W	视在功率	VA	比活度	Bq/kg
质量	kg	热导率	W/K	虚功率	var	角截面	m ² /sr

北京理工大学博士学位论文

密度	Kg/m ³	热扩散率	m ² /s	跨导特性	A/V ²	光谱截面	m ² /J
	g/cm ³	热容	J/K	反转电位	1/V	光谱角截面	m ² /(sr · J)
系数	1	比热容	J/(kg · K)	电力常数	N/A	粒子注量	1/m ²
比容	m ³ /kg	熵流量	J/(K · s)	光谱辐射能密度	J/m ⁴	粒子注量率	1/(s · m ²)
线密度	kg/m	焓密度	kg · s ² /m ⁵	辐射强度	W/sr	能通量	J/m ²
面密度	kg/m ²	压力密度	s ² /m ²	辐射亮度	W/(sr · m ²)	能通量率	W/m ²
动量	kg · m/s	温度密度	kg/(m ³ · K)	发光强度	cd	粒子电流密度	1/(m ² · s)
冲量	N · s	压力焓	J · m · s ² /kg ²	光通量	lm	质量衰减系数	m ² /kg
角动量	kg · m ² /s	密度能量	J · m ³ /kg	光量	lm · s	线性功率	J/m
角冲量	N · m · s	压力能量	J · m · s ² /kg	亮度	cd/m ²	总原子阻止力	J · m ²
转动惯量	kg · m ²	电流	A	光出射	lm/m ²	总质量制动力	J · m ² /kg
力	N	电流斜率	A/s	照度	lx	动力	m ² /(V · s)
刚度	N/m	电荷	C	曝光量	lx · s	中子源密度	1/(m ³ · s)
平动阻尼常数	N · s/m	电荷体密度	C/m ³	发光效率	lm/W	传递的比能	Gy
力矩	N · m	电荷面密度	C/m ²	摩尔吸收系数	m ² /mol	剂量当量	Sv
电力矩	N · m/A	电场强度	V/m	声阻抗	Pa · s/m ³	吸收剂量率	Gy/s

北京理工大学博士学位论文

旋转弹簧常数	$\text{N} \cdot \text{m}/\text{rad}$	电势	V	比声阻抗	$\text{Pa} \cdot \text{s}/\text{m}$	照射	C/kg
旋转阻尼常数	$\text{N} \cdot \text{m} \cdot \text{s}/\text{rad}$	电势斜率	V/s	响度级	phon	照射率	$\text{C}/(\text{kg} \cdot \text{s})$
压力	Pa	电容	F	响度	sone	光谱浓度	s/m^3
	bar	电容率	F/m	物质数量	mol	状态密度	$1/(\text{J} \cdot \text{m}^3)$
面积二阶矩	m^4	电偶极矩	$\text{C} \cdot \text{m}$	摩尔质量	kg/mol	阻尼系数	1/s
能量	J	电流密度	A/m^2	摩尔体积	m^3/mol	对数减量	1/S
衰减系数	1/m						

附录 B Plug-in 具体语法

```

1  //NodePlug-in 笔记插件
2  Node nodeSystemName ( //关键词 Node,通过唯一标识符 nodeSystemName 定义笔记
3      content = "需要的描述内容", //笔记插件填写的描述内容
4      text = ("宋体",22,Bold,Italic), //笔记中可根据利益相关人需求定义文字的属性
5      color = "255,255,255;0,0,0;0,0,0", //定义笔记本身的可视化
6      initialLocation = (100,200,10,20), //定义笔记的位置及尺寸坐标
7  ),
8  //LinkModelPlug-in 链接插件
9  LinkModel linkSystemName ( //关键词 LinkModel, 通过唯一标识符 linkSystemName 定义链接
10     link (modelSystemName),
11     content = "需要的描述内容", //链接插件填写的描述内容
12     text = ("宋体",22,Bold,Italic), //链接中可根据利益相关人需求定义文字的属性
13     color = "255,255,255;0,0,0;0,0,0", //定义链接本身的可视化
14     initialLocation = (100,200,10,20), //定义链接的位置及尺寸坐标
15 ),
16 NodeAttachment attachmentSystemName (
17     Connection (linkSystemName, modelSystemName), //链接与对象键值对
18     polylineLocation="1,12;3,41", //连线的各折点坐标,
19 ),
20 //TablePlug-in 表单插件
21 Table tableSystemName ( //关键词 Table,通过唯一标识符 tableSystemName 定义表单
22     localLabel= "名称", //表单插件命名的名称
23     initialLocation = (100,200,10,20), //表单插件的位置及尺寸坐标
24     value = "[1,1,2;4,5,6]", //表单中内容以矩阵形式表示
25     |value = propertyValueSystemName, //表单值可关联模型的任一属性值
26 ),
27 //LayoutPlug-in 布局插件
28 Layout layoutSystemName ( //关键词 Layout, 通过唯一标识符 layoutSystemName 定义布局
29     localLabel = "名称", //布局插件命名的名称
30     diagramKind = bdd, //声明该图的类型, 如 SysML 语言的模块定义图
31     modelElementType = block, //声明模型元素类型, 每个图都可代表已经在系统模型中某处定义的元素, 这里
    声明该元素的类型, 如 SysML 语言中模块图定义图中的块元素。
32     modelElementName = "刹车系统", //声明该元素的元素名
33     initialLocation = (100,200,10,20), //布局插件的位置及尺寸坐标
34 ),
35 //GanttChartPlug-in 甘特图插件
36 GanttChart gantt ( //关键词 GanttChart, 通过唯一标识符 gantt 定义甘特图
37     localLabel = "甘特图", //甘特图名称
38     initialLocation = (600,400,300,400), //甘特图位置和尺寸坐标
39     color = "255,255,255;0,0,0;0,0,0", //甘特图背景颜色

```

```
40  time = ("年"; 2; 22; 34), //时间单位, 时间间隔, 起始时间, 终止时间
41  "task1" ( //甘特图任务名称
42  gantt1("甘特图 1";60;90;"255,255,0;0,0,0;0,0,0"), //任务要素定义
43  gantt2("甘特图 2";30;60;"255,0,255;0,0,0;0,0,0"),
44  ),
45  ),
```

附录 C 多视图统一建模语言 g4 语法文件

```
1  /**
2   Define a grammar called Formalize
3   */
4   grammar Formalize;
5   options{
6   language = Java;
7   }
8   @header {
9   package org.fc.mdt.core.antlr;
10  }
11  /**Grammar root for language */
12  prog
13  :
14  //定义项目
15  'Project' IDENT 'annotation' '(' 'localLabel' '=' String ')' basicStructure
16  'end' IDENT
17  ;
18  basicStructure
19  :
20  //项目下包含语言、模型转化、模型求解相关代码段
21  (
22  languageClass
23  | modeltoTextUnit
24  | modeltomodelUnit
25  | z3SolverClass
26  )*
27  ;
28  //定义语言
29  languageClass
30  :
31  'Language' IDENT 'annotation' '(' 'localLabel' '=' String ')'('extends' IDENT)?
32  propertyDecisionClause*
33  gopprClass
34  modelClass*
35  packageClass*
36  'end' IDENT ';'
37  ;
38  //定义 GOPRR 元模型
39  gopprClass
40  :
```

```

41  (
42  graphClass
43  | objectClass
44  | pointClass
45  | roleClass
46  | relationshipClass
47  )*
48  ;
49  //定义属性元模型
50  propertyDecisionClause
51  :
52  dataType 'Property' IDENT 'annotation' '(' localLabelClause (',' unitClause)? (',' descriptionClause)? ')' ';
53  ;
54  //定义图元模型
55  graphClass
56  :
57  'partial'? 'Graph' IDENT ('extends' IDENT)?
58  (
59  metapropertyReferenceClause
60  | objectRefClause
61  | relationshipRefClause
62  | concreteSyntaxClause
63  | constraintClause
64  )* 'end' IDENT ';
65  ;
66  //定义对象元模型
67  objectClass
68  :
69  'partial'? 'Object' IDENT ('extends' IDENT)?
70  (
71  metapropertyReferenceClause
72  | pointRefClause
73  | decompositionDefClause
74  | explosionDefClause
75  | concreteSyntaxClause
76  )* 'end' IDENT ';
77  ;
78  //定义关系元模型
79  relationshipClass
80  :
81  'partial'? 'Relationship' IDENT ('extends' IDENT)?
82  (
83  metapropertyReferenceClause

```

```
84 | roleRefClause
85 | explosionDefClause
86 | decompositionDefClause
87 | concreteSyntaxClause
88 )* 'end' IDENT ';'
89 ;
90 //定义角色元模型
91 roleClass
92 :
93 roleType? 'Role' IDENT ( 'extends' IDENT)?
94 (
95 metapropertyReferenceClause
96 | explosionDefClause
97 | decompositionDefClause
98 | concreteSyntaxClause
99 )* 'end' IDENT ';'
100 ;
101 //定义点元模型
102 pointClass
103 :
104 'Point' IDENT ( 'extends' IDENT)?
105 (
106 metapropertyReferenceClause
107 | explosionDefClause
108 | concreteSyntaxClause
109 | decompositionDefClause
110 )* 'end' IDENT ';'
111 ;
112 //定义模型
113 modelClause
114 :
115 (
116 propertyClause
117 | objectInstance
118 | relationshipInstance
119 | connectClause
120 | concreteSyntaxClause
121 | sVGPartClause
122 | externalFunctionDeclareClause
123 )*
124 ;
125 //定义对象模型
126 objectInstance
```

```
127 :
128 ('Group'| 'State')? 'Object' IDENT ('refer'| 'clone'| 'copy') IDENT('IDENT)?
129 (
130 propertyClause
131 | pointInstance
132 | explosionClause
133 | decompositionClause
134 | concreteSyntaxClause
135 )*
136 'end' IDENT('IDENT)? ';'
137 ;
138 //定义属性模型
139 propertyClause
140 :
141 ('Property'| 'global') IDENT '=' literalClause ('refer'| 'clone') IDENT
142 'annotation'
143 '(' propertyAnnotation
144 ')' ';'
145 ;
146 // 定义属性模型固定语法
147 propertyAnnotation
148 :
149 ( localLabelClause
150 |colorClause
151 |textClause
152 )*
153 ;
154 // 定义名称
155 localLabelClause
156 :
157 'localLabel' '=' String ','?
158 ;
159 //词法规则定义字符串: String
160 String
161 :
162 ""
163 (
164 ESC
165 |.
166 )*? ""
167 ;
168 /**将一条规则声明为 fragment 可以告诉 ANTLR, 该规则本身不
169 *是一个词法符号, 它只会被其他的词法规则使用。
```



```
170 *  
171 **/  
172 fragment  
173 ESC  
174 :  
175 '\\ [btnfr""\\]  
176 ;
```

附录 D 多视图统一建模语言描述 SysML 活动图

```

1  partial Graph Activity_Diagram
2  initial Property Activity_Diagram_Property_To_Do_4f5d74 = "null" extends Property_To_Do annotation(localLabel
   = "待办事项");
3  initial Property Activity_Diagram_Property_Context_2475bb = "null" extends Property_Context
   annotation(localLabel = "上下文");
4  Object Object_Call_Behavior_Action;
5  Object Object_Call_Operation_Action;
6  Object Object_Opaque_Action;
7  Object Object_Central_Buffer_Node;
8  Object Object_Data_Store_Node;
9  Object Object_Activity_Parameter_Node;
10 Object Object_Send_Signal_Action;
11 Object Object_Accept_Event_Action;
12 Object Object_Accept_Change_Structural_Feature_Event_Action;
13 Object Object_Initial_Node;
14 Object Object_Activity_Final_Node;
15 Object Object_Flow_Final_Node;
16 Object Object_Decision_Node;
17 Object Object_Merge_Node;
18 Object Object_Fork_Node;
19 Object Object_Join_Node;
20 Object Object_Interruptible_Activity_Region;
21 Object Object_Structured_Activity_Node;
22 Object Object_Expansion_Region;
23 Object Object_Conditional_Node;
24 Object Object_Loop_Node;
25 Object Object_Sequence_Node;
26 Object Object_Swimlanes;
27 Relationship Relationship_Control_Flow;
28 Relationship Relationship_Object_Flow;
29 Relationship Relationship_Exception_Handler;
30 Constraint (
31  bind connector(Object_Opaque_Action, Relationship_Control_Flow.Role_Source),
32  bind connector(Object_Initial_Node, Relationship_Control_Flow.Role_Source),
33  bind connector(Object_Structured_Activity_Node, Relationship_Control_Flow.Role_Source),
34  bind connector(Object_Accept_Event_Action, Relationship_Control_Flow.Role_Source),
35  bind connector(Object_Decision_Node, Relationship_Control_Flow.Role_Source),
36  bind connector(Object_Fork_Node, Relationship_Control_Flow.Role_Source),
37  bind connector(Object_Conditional_Node, Relationship_Control_Flow.Role_Source),
38  bind connector(Object_Sequence_Node, Relationship_Control_Flow.Role_Source),

```

```

39  bind connector(Object_Merge_Node,Relationship_Control_Flow.Role_Source),
40  bind connector(Object_Send_Signal_Action,Relationship_Control_Flow.Role_Source),
41  bind connector(Object_Call_Behavior_Action,Relationship_Control_Flow.Role_Source),
42  bind connector(Object_Call_Operation_Action,Relationship_Control_Flow.Role_Source),
43  bind
    connector(Object_Accept_Change_Structural_Feature_Event_Action,Relationship_Control_Flow.Role_Source),
44  bind connector(Object_Join_Node,Relationship_Control_Flow.Role_Source),
45  bind connector(Object_Expansion_Region,Relationship_Control_Flow.Role_Source),
46  bind connector(Object_Loop_Node,Relationship_Control_Flow.Role_Source),
47  bind connector(Object_Opaque_Action,Relationship_Control_Flow.Role_Target),
48  bind connector(Object_Structured_Activity_Node,Relationship_Control_Flow.Role_Target),
49  bind connector(Object_Accept_Event_Action,Relationship_Control_Flow.Role_Target),
50  bind connector(Object_Activity_Final_Node,Relationship_Control_Flow.Role_Target),
51  bind connector(Object_Decision_Node,Relationship_Control_Flow.Role_Target),
52  bind connector(Object_Fork_Node,Relationship_Control_Flow.Role_Target),
53  bind connector(Object_Flow_Final_Node,Relationship_Control_Flow.Role_Target),
54  bind connector(Object_Conditional_Node,Relationship_Control_Flow.Role_Target),
55  bind connector(Object_Sequence_Node,Relationship_Control_Flow.Role_Target),
56  bind connector(Object_Merge_Node,Relationship_Control_Flow.Role_Target),
57  bind connector(Object_Send_Signal_Action,Relationship_Control_Flow.Role_Target),
58  bind connector(Object_Call_Behavior_Action,Relationship_Control_Flow.Role_Target),
59  bind connector(Object_Call_Operation_Action,Relationship_Control_Flow.Role_Target),
60  bind connector(Object_Accept_Change_Structural_Feature_Event_Action,Relationship_Control_Flow.Role_Target),
61  bind connector(Object_Join_Node,Relationship_Control_Flow.Role_Target),
62  bind connector(Object_Expansion_Region,Relationship_Control_Flow.Role_Target),
63  bind connector(Object_Loop_Node,Relationship_Control_Flow.Role_Target),
64  bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Input_Pin_a04eff,Relationship_Object_Flow
    w.Role_Source),
65  bind connector(Object_Accept_Event_Action,Relationship_Object_Flow.Role_Source),
66  bind connector(Object_Decision_Node,Relationship_Object_Flow.Role_Source),
67  bind
    connector(Object_Opaque_Action.Object_Opaque_Action_Point_Output_Pin_cb47a3,Relationship_Object_Flow.Role
    le_Source),
68  bind connector(Object_Fork_Node,Relationship_Object_Flow.Role_Source),
69  bind
    connector(Object_Conditional_Node.Object_Conditional_Node_Point_Output_Pin_9feaa8,Relationship_Object_Flow
    w.Role_Source),
70  bind connector(Object_Conditional_Node,Relationship_Object_Flow.Role_Source),
71  bind connector(Object_Central_Buffer_Node,Relationship_Object_Flow.Role_Source),
72  bind connector(Object_Merge_Node,Relationship_Object_Flow.Role_Source),
73  bind connector(Object_Send_Signal_Action,Relationship_Object_Flow.Role_Source),

```

```

74  bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Input_Pin_e6c513,Relationship_Object_Flow.Role_Source),
75  bind connector(Object_Call_Behavior_Action,Relationship_Object_Flow.Role_Source),
76  bind connector(Object_Accept_Change_Structural_Feature_Event_Action,Relationship_Object_Flow.Role_Source),
77  bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Action_Input_Pin_e07395,Relationship_Object_Flow.Role_Source),
78  bind connector(Object_Join_Node,Relationship_Object_Flow.Role_Source),
79  bind
    connector(Object_Call_Operation_Action.Object_Call_Operation_Action_Point_Output_Pin_a75664,Relationship_Object_Flow.Role_Source),
80  bind connector(Object_Expansion_Region,Relationship_Object_Flow.Role_Source),
81  bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Action_Input_Pin_7d44d9,Relationship_Object_Flow.Role_Source),
82  bind connector(Object_Opaque_Action,Relationship_Object_Flow.Role_Source),
83  bind
    connector(Object_Call_Behavior_Action.Object_Call_Behavior_Action_Point_Output_Pin_9cfe4f,Relationship_Object_Flow.Role_Source),
84  bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Value_Pin_c2d5a2,Relationship_Object_Flow.Role_Source),
85  bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Action_Input_Pin_01bb02,Relationship_Object_Flow.Role_Source),
86  bind connector(Object_Structured_Activity_Node,Relationship_Object_Flow.Role_Source),
87  bind
    connector(Object_Accept_Event_Action.Object_Accept_Event_Action_Point_Output_Pin_2949b1,Relationship_Object_Flow.Role_Source),
88  bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Input_Pin_00deab,Relationship_Object_Flow.Role_Source),
89  bind connector(Object_Activity_Parameter_Node,Relationship_Object_Flow.Role_Source),
90  bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Output_Pin_474d09,Relationship_Object_Flow.Role_Source),
91  bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Output_Pin_1b376f,Relationship_Object_Flow.Role_Source),
92  bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Expansion_Node_007a41,Relationship_Object_Flow.Role_Source),

```

```

93  bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Output_Pin_1719c9,Relationship_Object_Flow.Role_Source),
94  bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Action_Input_Pin_82ecf1,Relationship_Object_Flow.Role_Source),
95  bind connector(Object_Data_Store_Node,Relationship_Object_Flow.Role_Source),
96  bind connector(Object_Sequence_Node,Relationship_Object_Flow.Role_Source),
97  bind
    connector(Object_Accept_Change_Structural_Feature_Event_Action.Object_Accept_Change_Structural_Feature_Event_Action_Point_Output_Pin_a4b949,Relationship_Object_Flow.Role_Source),
98  bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Value_Pin_6d407f,Relationship_Object_Flow.Role_Source),
99  bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Output_Pin_8bdd1c,Relationship_Object_Flow.Role_Source),
100 bind connector(Object_Call_Operation_Action,Relationship_Object_Flow.Role_Source),
101 bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Value_Pin_8197a2,Relationship_Object_Flow.Role_Source),
102 bind connector(Object_Loop_Node,Relationship_Object_Flow.Role_Source),
103 bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Value_Pin_b7140a,Relationship_Object_Flow.Role_Source),
104 bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Input_Pin_8f74f4,Relationship_Object_Flow.Role_Source),
105 bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Input_Pin_a04eff,Relationship_Object_Flow.Role_Target),
106 bind
    connector(Object_Call_Behavior_Action.Object_Call_Behavior_Action_Point_Action_Input_Pin_2de96c,Relationship_Object_Flow.Role_Target),
107 bind
    connector(Object_Opaque_Action.Object_Opaque_Action_Point_Input_Pin_eddb86,Relationship_Object_Flow.Role_Target),
108 bind connector(Object_Decision_Node,Relationship_Object_Flow.Role_Target),
109 bind connector(Object_Fork_Node,Relationship_Object_Flow.Role_Target),
110 bind
    connector(Object_Call_Behavior_Action.Object_Call_Behavior_Action_Point_Input_Pin_d1c3dc,Relationship_Object_Flow.Role_Target),
111 bind connector(Object_Conditional_Node,Relationship_Object_Flow.Role_Target),

```

```

112 bind connector(Object_Central_Buffer_Node,Relationship_Object_Flow.Role_Target),
113 bind connector(Object_Merge_Node,Relationship_Object_Flow.Role_Target),
114 bind connector(Object_Send_Signal_Action,Relationship_Object_Flow.Role_Target),
115 bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Input_Pin_e6c513,Relationship_Object_Flow.Role_Target),
116 bind connector(Object_Call_Behavior_Action,Relationship_Object_Flow.Role_Target),
117 bind
    connector(Object_Opaque_Action.Object_Opaque_Action_Point_Action_Input_Pin_d0880c,Relationship_Object_Flow.Role_Target),
118 bind
    connector(Object_Call_Operation_Action.Object_Call_Operation_Action_Point_Input_Pin_38d962,Relationship_Object_Flow.Role_Target),
119 bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Action_Input_Pin_e07395,Relationship_Object_Flow.Role_Target),
120 bind connector(Object_Join_Node,Relationship_Object_Flow.Role_Target),
121 bind connector(Object_Expansion_Region,Relationship_Object_Flow.Role_Target),
122 bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Action_Input_Pin_7d44d9,Relationship_Object_Flow.Role_Target),
123 bind
    connector(Object_Send_Signal_Action.Object_Send_Signal_Action_Point_Input_Pin_c12aa2,Relationship_Object_Flow.Role_Target),
124 bind connector(Object_Opaque_Action,Relationship_Object_Flow.Role_Target),
125 bind
    connector(Object_Send_Signal_Action.Object_Send_Signal_Action_Point_Action_Input_Pin_375cb5,Relationship_Object_Flow.Role_Target),
126 bind
    connector(Object_Call_Operation_Action.Object_Call_Operation_Action_Point_Action_Input_Pin_4d115a,Relationship_Object_Flow.Role_Target),
127 bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Action_Input_Pin_01bb02,Relationship_Object_Flow.Role_Target),
128 bind connector(Object_Structured_Activity_Node,Relationship_Object_Flow.Role_Target),
129 bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Input_Pin_00deab,Relationship_Object_Flow.Role_Target),
130 bind connector(Object_Activity_Final_Node,Relationship_Object_Flow.Role_Target),
131 bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Expansion_Node_007a41,Relationship_Object_Flow.Role_Target),
132 bind connector(Object_Flow_Final_Node,Relationship_Object_Flow.Role_Target),

```

```

133 bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Action_Input_Pin_82ecf1,Relationship_Object_Flow.Role_Target),
134 bind connector(Object_Data_Store_Node,Relationship_Object_Flow.Role_Target),
135 bind connector(Object_Sequence_Node,Relationship_Object_Flow.Role_Target),
136 bind connector(Object_Call_Operation_Action,Relationship_Object_Flow.Role_Target),
137 bind connector(Object_Loop_Node,Relationship_Object_Flow.Role_Target),
138 bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Input_Pin_8f74f4,Relationship_Object_Flow.Role_Target),
139 bind connector(Object_Opaque_Action,Relationship_Exception_Handler.Role_Protected_Node),
140 bind
    connector(Object_Call_Behavior_Action.Object_Call_Behavior_Action_Point_Output_Pin_9cfe4f,Relationship_Exception_Handler.Role_Protected_Node),
141 bind connector(Object_Structured_Activity_Node,Relationship_Exception_Handler.Role_Protected_Node),
142 bind
    connector(Object_Accept_Event_Action.Object_Accept_Event_Action_Point_Output_Pin_2949b1,Relationship_Exception_Handler.Role_Protected_Node),
143 bind connector(Object_Accept_Event_Action,Relationship_Exception_Handler.Role_Protected_Node),
144 bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Output_Pin_474d09,Relationship_Exception_Handler.Role_Protected_Node),
145 bind
    connector(Object_Opaque_Action.Object_Opaque_Action_Point_Output_Pin_cb47a3,Relationship_Exception_Handler.Role_Protected_Node),
146 bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Output_Pin_1b376f,Relationship_Exception_Handler.Role_Protected_Node),
147 bind
    connector(Object_Conditional_Node.Object_Conditional_Node_Point_Output_Pin_9feaa8,Relationship_Exception_Handler.Role_Protected_Node),
148 bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Output_Pin_1719c9,Relationship_Exception_Handler.Role_Protected_Node),
149 bind connector(Object_Conditional_Node,Relationship_Exception_Handler.Role_Protected_Node),
150 bind connector(Object_Sequence_Node,Relationship_Exception_Handler.Role_Protected_Node),
151 bind
    connector(Object_Accept_Change_Structural_Feature_Event_Action.Object_Accept_Change_Structural_Feature_Event_Action_Point_Output_Pin_a4b949,Relationship_Exception_Handler.Role_Protected_Node),
152 bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Output_Pin_8bdd1c,Relationship_Exception_Handler.Role_Protected_Node),
153 bind connector(Object_Send_Signal_Action,Relationship_Exception_Handler.Role_Protected_Node),

```

```

154 bind connector(Object_Call_Behavior_Action,Relationship_Exception_Handler.Role_Protected_Node),
155 bind connector(Object_Call_Operation_Action,Relationship_Exception_Handler.Role_Protected_Node),
156 bind
    connector(Object_Accept_Change_Structural_Feature_Event_Action,Relationship_Exception_Handler.Role_Protect
        ed_Node),
157 bind
    connector(Object_Call_Operation_Action.Object_Call_Operation_Action_Point_Output_Pin_a75664,Relationship_
        Exception_Handler.Role_Protected_Node),
158 bind connector(Object_Expansion_Region,Relationship_Exception_Handler.Role_Protected_Node),
159 bind connector(Object_Loop_Node,Relationship_Exception_Handler.Role_Protected_Node),
160 bind
    connector(Object_Send_Signal_Action.Object_Send_Signal_Action_Point_Input_Pin_c12aa2,Relationship_Exception_
        n_Handler.Role_Exception_Input),
161 bind
    connector(Object_Call_Behavior_Action.Object_Call_Behavior_Action_Point_Action_Input_Pin_2de96c,Relationsh
        ip_Exception_Handler.Role_Exception_Input),
162 bind
    connector(Object_Send_Signal_Action.Object_Send_Signal_Action_Point_Action_Input_Pin_375cb5,Relationship_
        Exception_Handler.Role_Exception_Input),
163 bind
    connector(Object_Call_Operation_Action.Object_Call_Operation_Action_Point_Action_Input_Pin_4d115a,Relation
        ship_Exception_Handler.Role_Exception_Input),
164 bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Action_Input_Pin_01bb02,Relationship_Exception_Handl
        er.Role_Exception_Input),
165 bind
    connector(Object_Opaque_Action.Object_Opaque_Action_Point_Input_Pin_eddb86,Relationship_Exception_Handl
        er.Role_Exception_Input),
166 bind
    connector(Object_Loop_Node.Object_Loop_Node_Point_Input_Pin_00deab,Relationship_Exception_Handler.Role_
        Exception_Input),
167 bind
    connector(Object_Call_Behavior_Action.Object_Call_Behavior_Action_Point_Input_Pin_d1c3dc,Relationship_Exc
        eption_Handler.Role_Exception_Input),
168 bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Expansion_Node_007a41,Relationship_Exc
        eption_Handler.Role_Exception_Input),
169 bind
    connector(Object_Expansion_Region.Object_Expansion_Region_Point_Action_Input_Pin_82ecf1,Relationship_Exc
        eption_Handler.Role_Exception_Input),
170 bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Input_Pin_e6c513,Relationsh
        ip_Exception_Handler.Role_Exception_Input),

```



```

171 bind
    connector(Object_Opaque_Action.Object_Opaque_Action_Point_Action_Input_Pin_d0880c,Relationship_Exception
        _Handler.Role_Exception_Input),
172 bind
    connector(Object_Call_Operation_Action.Object_Call_Operation_Action_Point_Input_Pin_38d962,Relationship_Ex
        ception_Handler.Role_Exception_Input),
173 bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Action_Input_Pin_e07395,Relationship_Exception
        n_Handler.Role_Exception_Input),
174 bind
    connector(Object_Sequence_Node.Object_Sequence_Node_Point_Input_Pin_8f74f4,Relationship_Exception_Handl
        er.Role_Exception_Input),
175 bind
    connector(Object_Structured_Activity_Node.Object_Structured_Activity_Node_Point_Action_Input_Pin_7d44d9,R
        elationship_Exception_Handler.Role_Exception_Input),
176 Object_Conditional_Node.explode(Activity_Diagram),
177 Object_Loop_Node.explode(Activity_Diagram),
178 Object_Swimlanes.explode(Activity_Diagram),
179 Object_Call_Behavior_Action.explode(Activity_Diagram),
180 Object_Interruptible_Activity_Region.decompose(Activity_Diagram),
181 Object_Structured_Activity_Node.decompose(Activity_Diagram),
182 Object_Expansion_Region.decompose(Activity_Diagram),
183 Object_Sequence_Node.decompose(Activity_Diagram),
184 );
185 annotation (
186     localLabel="活动图",
187     type = diagram,
188     description="",
189 );
190 end Activity_Diagram;

```

攻读学位期间发表论文与研究成果清单

- [1] **Ma J**, Wang G, Lu J, et al. Systematic Literature Review of MBSE Tool-Chains[J]. Applied Sciences, 2022, 12(7): 3431 (SCI, 论文第 2、3 章)
- [2] **Ma J**, Wang G, Lu J, et al. Semantic Modeling Approach Supporting Process Modeling and Analysis in Aircraft Development[J]. Applied Sciences, 2022, 12(6): 3067. (SCI, 论文第 5 章)
- [3] **Ma J**, Lu J, Wang G, et al. Multi-architecture Modeling Language for Supporting Model-based Systems Engineering Formalism[J]. IEEE Systems Journal. (评审中) (SCI, 论文第 4 章)
- [4] **Ma J**, Wang G, Lu J, et al. Multi-architecture Modeling Approach Supporting Architecture Modeling and Analysis of Intelligent Electric Vehicles[J]. Advanced Engineering Informatics. (评审中) (SCI, 论文第 6 章)
- [5] **马君达**,王宏伟,鲁金直,等. 需求驱动的无人平台架构设计方法[C].第四届体系工程学术会议论文集. 2022: 55-69.
- [6] Lu J, **Ma J**, Zheng X, et al. Design ontology supporting model-based systems engineering formalisms[J]. IEEE Systems Journal, 2021. (SCI)
- [7] Lu J, Wang G, **Ma J**, et al. General modeling language to support model - based systems engineering formalisms (part 1) [C]. INCOSE International Symposium. 2020, 30(1): 323-338.

攻读学位期间参与的科研项目

- [1] 2017-2019 年. 国防基础科研重点项目“XXX 平台机电协同设计技术”，（子课题主要完成人）
- [2] 2015-2018 年. 国家部委基础科研重大项目“基于知识工程和通用信息模型的 XXX 协同设计技术”。（参与）
- [3] 2020-2023 年. 科技部国家重点研发计划项目“复杂产品设计制造服务一体化系统建模理论”。（子课题主要负责人）
- [4] 2021-2023 年. 民用航天重大项目“基于 MBSE 的 XZHJ 体系设计与验证技术”（子课题主要负责人）
- [5] 2021-2024 年. 装备预先研究共用技术项目“多系统建模与设计空间智能决策技术”（子课题主要负责人）

攻读学位期间参与的竞赛

- [1] 马君达,栗寒,胡振超,等. 2021 第七届中国国际“互联网+”大学生创新创业大赛（全国总决赛）, 国赛入围奖。
- [2] 马君达,栗寒,陈婧琦,等. 2021 第七届中国国际“互联网+”大学生创新创业大赛（北京赛区复赛）, 北京市一等奖。
- [3] 马君达,胡振超,郭江敏,等. 2020 第六届中国国际“互联网+”大学生创新创业大赛（北京赛区复赛）, 北京市一等奖。
- [4] 马君达,陈婧琦,栗寒,等. 2021 年北京地区高校大学生优秀创业团队评选, 北京市一等奖。
- [5] 鲁金直,马君达. 第十五届“春晖杯”中国留学人员创新创业大赛高技术服务组, 省部级优胜奖。
- [6] 胡振超,马君达,吴俊鹏,等. 第二届“启航杯”MBSE 建模大赛, 三等奖。

致谢

四季更替，花落又花开，弹指一挥已面临毕业，期间的学习生活经历将使我受益终生。在即将完成博士论文之际，回想匆匆五年，感触颇多，今天的这份成就离不开老师、师兄弟、亲人及朋友的支持、帮助和鼓励，在此向他（她）们致以最诚挚的感谢。

首先感谢我的导师阎艳教授，阎老师清晰的科研思路，帮我在研究上做出规划，并且为我的研究提供了很大的支持；阎老师对学生深切的关怀和帮助，也让我非常感动，每当生活遇阻，总能得到阎老师的宽慰和鼓励，同时，阎老师待人接物的处事态度给我留下深刻印象，值得我一生去学习。

感谢王国新教授，王老师对我有知遇之恩，当年能踏入北理的校门，离不开王老师的忙前忙后。王老师具体负责指导我的研究，五年多的时间里，王老师对于我的培养付出了很多心血，他高瞻远瞩，总能在研究的重要拐点给予我指导和帮助，无论是我博士前期对于理论研究茫然时的悉心指导，相互交流，还是我博士中期更换研究方向之后与我的共同研讨，总能在王老师身上看到那种积极和专注的态度。博士研究期间跟王老师各地“征战”，全国多地的调研、交流、做项目、开学术会，学到了很多，王老师在我心中早已亦师亦友，最后也感谢王老师对于我博士论文写作的帮助。

特别感谢鲁金直博士，鲁博士是我人生中的贵人，博士期间我们展开了非常好的合作，在我的科研上帮助很多，是我科研方向的领路人。在鲁博士的大力支持下，我们北理工系统工程团队不断的优化、实践我们的研究成果，在国内得到学术界和科研单位的认可。鲁博士虽人在国外，但五年多的时间里，与鲁博士大会小会几百次，早已成为最熟悉的“陌生人”，期待鲁博士回国后开展更多的合作，继续深耕系统工程领域。也再次感谢鲁博士对我毕业论文的大力指导。

还要感谢郝佳老师，在我博士前两年还处于探索研究时期时，为我提供了很多帮助，帮我做研究规划，寻找突破点。郝老师科研能力很强，又为人谦逊，是我学习的榜样。同时感谢明振军老师、王儒老师、黄思翰老师，三位老师都是我求学期间的优秀师兄，感谢对我博士期间的指点和帮助。在课题研究和论文撰写过程中也得到了北理工工业与智能系统工程研究所胡耀光、宫琳、张发平、薛庆、敬石开、姚丽娅、张静楠等老师的帮助，在此向他们表示感谢。

最让人难以忘怀的，是一起奋斗过的小伙伴，感谢系统工程团队的吴绶玄博士，

王宏伟、郭江敏、余士严、靳益利、栗寒、陈婧琦、丁洁、李子航、董梦如、陈锐等硕士，以及我们系统工程国际联合团队的上海交通大学胡振超博士，成都电子科技大学的杨兆瑞、曹宇茜硕士，瑞典皇家理工大学的付铄硕士，中科蜂巢科技有限公司的吴俊鹏、陈洪涛、郝卫亚、孙天宇等开发人员，与你们在一起的日子总是充满欢声笑语，感谢对于我成长的莫大帮助。

感谢知识工程组的贾良跃、牛红伟、张则焕、叶文斌、黄禹、冯睽睽、张云贺、万斯来、曹家滨、李传浩、Waqas Ahmad等博士和硕士，以及室友李铭浩博士，跟我一起渡过愉快的求学生涯。

最后我要感谢我的家人，正是他们毫无保留的支持，让我坚定了读博的信念和决心，他们的关怀让我随时感受到温暖，更加坚强的面对求学期间的种种困难，感谢我哥博士期间为我提供的精神和物质上的支持，无论发生什么，他总是一如既往的相信和鼓励。最后的最后感谢我的爱人邵亚楠女士对我的理解和陪伴，她的辛苦付出和我无微不至的照顾让我更加专注于学业，博士求学的完成离不开她背后的默默付出。

感谢各位专家于百忙之中抽出时间评阅本文。